ARCH LINUX 分册教材

第一册

Arch Linux 与它的社区

发行版哲学、项目结构、社区协作与本地化

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

这一册要回答的问题

Arch 的多数困惑不是操作问题，而是立场问题：为什么装系统没有一路点下去的向导，为什么一次例行升级可能要求你先读新闻，为什么有些软件明明能用却进不了官方仓库。这些"为什么"的答案写在项目的历史、哲学与组织结构里，而不在任何一条命令里。

本册不讲怎么装系统，它讲装上之后你要遵守的那套规矩由谁制定、边界在哪里、有分歧时去哪里查证、出了问题该找谁。读完之后，你应该能用自己的话解释"Arch 不是为所有人准备的发行版"这句话，能分清官方仓库、非官方用户仓库、AUR 与维基各自承担什么角色，知道一份软件包的维护者是谁、缺陷该报给哪一方，也能把一份英文维基页面或一段论坛讨论，读成中文里可以照着做的步骤。

在全套九册中的位置

这是整套教材的起点，前面没有前置册。紧接其后的是第二册《安装、启动与软件包管理》，那里才真正动手：分区、引导、包管理器的用法与打包；再往后，系统管理、网络、硬件、桌面、应用、开发与安全各册逐层展开。

所以本册的角色更像"地图与词典"，而不是"工具箱"。它给后面八册提供共同词汇：什么是上游、什么是打包、滚动更新意味着什么、社区与官方如何分工。把这一册读扎实，后面遇到"某条消息说要手动干预"时，你就能顺着这些概念推断原因，而不只是照抄命令。

推荐读法与可跳读的部分

开头讲"关于 Arch"和"Arch 项目"的两章请按顺序读完，它们是全书地基，跳过之后会处处卡壳。讲开发流程和团队的章节可以先快速浏览，等你在第二册读到打包、或在后面某册里想向项目提交缺陷报告时再回头细读，那时会格外有用。讲建议阅读的一章本质是书目清单，适合当索引用，不必通读。本地化一章放到第一次想读英文维基、或者想参与翻译的时刻，收益最高。至于其余几篇偏工具与概念的短章，例如维基软件与 Live 环境，按需取用即可，第一遍通读时略过并不影响。

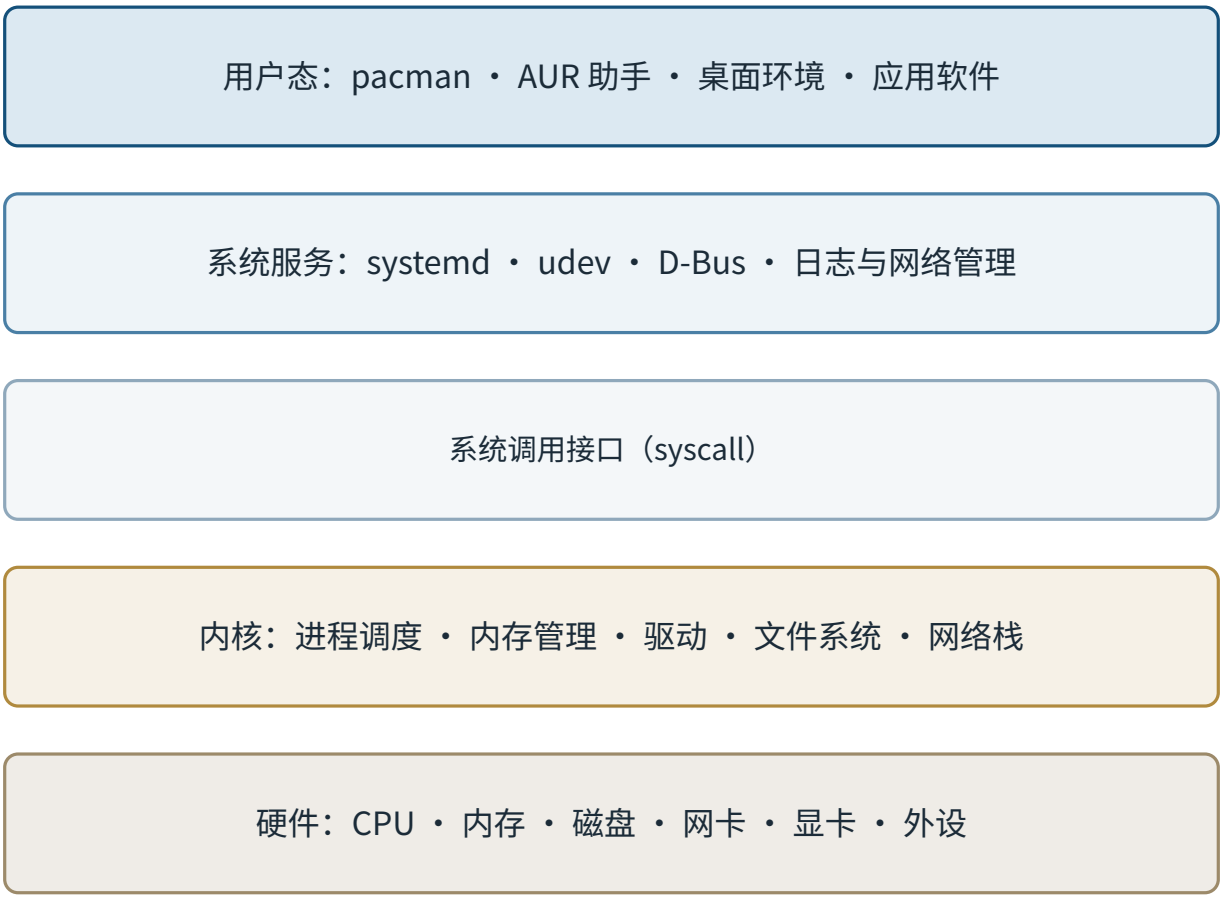
前置知识与常见误区

前置要求很低：会开机、会上网检索、能在终端里输入命令并看懂提示符。不需要你会编程，也不需要你事先装过 Arch。

需要提防的误区有四个。其一，把本册当成安装手册——安装是第二册的事，这里只解释理由。其二，把发行版哲学当成宣传口号；它其实是一套取舍标准，能解释许多具体决策的来龙去脉。其三，把非官方用户仓库或 AUR 里的东西当作官方支持；那里由社区维护，出问题时责任方与排查路径都不同。其四，觉得英文维基看不懂就绕道去找二手教程；过时的二手信息是新手最常见的故障来源，本地化一章正是为跨过这道坎准备的。

系统层次

从上到下理解一台 Arch 机器



上层依赖下层提供的接口；越往下越接近硬件，越不适合直接操作

Arch Linux 的五层结构：用户态、系统服务、系统调用、内核与硬件

目　录

第 1 章　关于 Arch（18 节）

1.1 Arch Linux　1.2 Arch Linux 愚人节笑话　1.3 常见问题　1.4 Arch Linux 新闻报道　1.5 BlackArch Linux　1.6 首页　1.7 Arch 术语　1.8 镜像源　1.9 Arch 构建系统　1.10 ASCII 艺术　1.11 官方仓库　1.12 AUR 提交准则　1.13 目录　1.14 创建软件包　1.15 参与贡献　1.16 Arch 与其他发行版的比较　1.17 Arch 用户软件仓库　1.18 Arch 是最好的!

第 2 章　Arch 项目（5 节）

2.1 Arch Linux 存档库　2.2 Arch Linux Archive　2.3 官方仓库 web 接口　2.4 Namcap　2.5 网络引导

第 3 章　Arch 开发（4 节）

3.1 Bug Day　3.2 AUR 扫除日　3.3 创建一个干净的 chroot　3.4 漏洞报告准则

第 4 章　团队（4 节）

4.1 访问级别和角色　4.2 软件包维护者　4.3 Arch 测试团队　4.4 Arch 安全团队

第 5 章　建议阅读（4 节）

5.1 建议阅读/给新用户的关于如何不去弄坏 Arch Linux 系统的建议　5.2 建议阅读/中国用户的推荐解决方案　5.3 建议阅读/服务器部署建议　5.4 建议阅读

第 6 章　本地化（4 节）

6.1 中文本地化　6.2 繁体中文本地化　6.3 Locale　6.4 简体中文本地化

第 7 章　Arch 社区（3 节）

7.1 Arch Linux 中文社区　7.2 行为准则　7.3 一般准则

第 8 章　维基软件（3 节）

8.1 XWiki　8.2 DokuWiki　8.3 MediaWiki

第 9 章　Live Arch 系统（2 节）

9.1 离线安装　9.2 Archiso

第 10 章　Configuration management (简体中文)（1 节）

10.1 Dotfiles

关于 Arch

本章把 Arch 从零散印象整理成可查、可用、可参与的对象：核心原则与历史、官方仓库与镜像源、构建与打包、AUR 与提交准则，以及它和其他发行版的取舍差异。读法上先立坐标：《Arch Linux》《官方仓库》《镜像源》《Arch 术语》必读，读过才知道 core、ABS、AUR 各指什么；再走流程：《创建软件包》《Arch 用户软件仓库》《AUR 提交准则》连读，从 PKGBUILD 一路到提交；《参与贡献》《Arch 与其他发行版的比较》值得细读；《目录》《首页》、新闻报道、愚人节笑话一类偏名录与谈资的，可以略过。

最容易犯的错是把 AUR 当成官方仓库，以为里面的包由 Arch 团队维护；其次是跳过《镜像源》直奔软件源，没弄清镜像排序与启用特定镜像的规则，之后在仓库与包管理上反复卡壳。

1.1　Arch Linux

难度 进阶 | **篇幅** 10.2 KB | **所属** 关于 Arch

本节将讲到

- 核心原则
- 历史沿革

Arch Linux 是一个通用操作系统，它是基于 x86-64 架构的一个 GNU/Linux 发行版。

Arch Linux 采用滚动升级模式，尽全力为使用者提供最新的稳定版软件。初始安装完成的 Arch Linux 只是一个基本系统，随后用户可以根据自己的喜好安装需要的软件，并自己动手配置成理想的个性化系统。

核心原则

以下核心原则构成了我们通常所指的 Arch 之道，或者说 Arch 的哲学。对于这些原则或许最好的结词是 Keep It Simple, Stupid（对应中文为“保持简洁，且一目了然”）。

简洁

Arch Linux 将简洁定义为：**避免任何不必要的添加、修改和复杂化的增加**。它提供的软件包都来自原始开发者（以下简称上游)），且仅进行本发行版（以下简称下游）相关的最少改动。

- 不包含上游不愿意接受的补丁。绝大部分 Arch 下游补丁都已经被上游接受，会在下一个正式版本里被包含。
- 配置文件同样来自上游，下游对它的改动仅包含必需的调整，比如特殊的文件系统路径变动。此外Arch Linux在安装软件包后不会自动启用和启动软件包提供的服务。

- 软件包通常都和一个上游项目直接对应，仅在极少数情况下才会拆分软件包。
- 官方不支持图形化配置界面，建议用户使用命令行或文本编辑器修改配置。

现代化

Arch Linux 尽全力使软件处于最新的稳定版本，只要没有出现系统软件包损坏，都会尽量使用最新版本的软件包。

Arch Linux 采用滚动更新的方式，安装之后可以通过命令持续更新系统与软件包。

Arch Linux 现面向 GNU/Linux 用户提供了许多新特性，包括 systemd 初始化系统、现代化的文件系统、LVM2/EVMS、软件磁盘阵列（软 RAID）、udev 支持、initcpio（附带 mkinitcpio）以及最新的 Linux 内核。

实用性

Arch Linux 注重实用性，且尽力避免意识形态之争，最终的设计决策都是由开发者之间的共识决定。开发者依赖基于客观事实的技术分析和讨论，避免国家之间政治因素的影响，不被世俗流行观点左右。

Arch Linux 的软件仓库中包含大量的软件包和编译脚本，用户可以按需自由选择。此外仓库中既提供了开源、自由的软件，也提供了闭源软件，即**实用性大于意识形态**。

以用户为中心

许多 Linux 发行版都试图变得更加“用户友好”，而 Arch Linux 则一直是、且永远会是“**以用户为中心**”：

- 本发行版的目标是满足向它贡献的人的需求，而非吸引尽可能多的用户。
- 本发行版的目标用户是熟练的 GNU/Linux 用户，或者任何乐于自己动手、愿意阅读文档、解决自己的问题的人。

Arch Linux 鼓励每一个用户参与和贡献，报告和帮助修复 bugs，提供软件包补丁和参加核心项目。Arch 开发者都是志愿者，通过持续的贡献成为团队中的一员。*Archers* 还可以自行贡献软件包到 Arch 用户仓库，提升 ArchWiki 文档质量，在论坛、邮件列表或者 IRC 中给其它用户提供技术支持。

Arch Linux 已经是全球许多 GNU/Linux 用户的选择，还有许多国际社区在提供帮助和翻译文档。

通用性

Arch Linux 作为一种通用 Linux 发行版，它的初始安装仅提供命令行环境。由于 Arch 默认提供最小化安装，用户不需要删除大量不需要的软件包，而是可以从官方软件仓

库成千上万的高质量软件包中进行选择，用于搭建自己的系统。目前仅支持 x86-64 架构。(对 i686 架构的支持已经结束)

Arch Linux 使用 Pacman 包管理系统实现系统和软件包的滚动更新。并且 Arch Linux 还提供一个类似 ports 的包构建系统（Arch Build System），通过它可以轻松从源码构建软件包然后安装，并用一个命令完成同步。您甚至可以用一个命令重新构建整个系统。Arch 还提供 Arch 用户仓库，它包含了成千上万个由用户自行维护的 PKGBUILD 脚本，配合 makepkg 工具，从编译到打包一气呵成。此外用户还能轻松构建和维护属于自己的软件仓库。

历史沿革

这些年来，Arch 社区不断成长，与此同时也收到了大量的关注和评论。

除少数特例以外，Arch Linux 的开发者大部分是不领工资的志愿者与爱好者，目前也并没有通过 Arch 商业变现的计划。关于Arch Linux 开发的详细历史可以浏览 Wayback Machine 的 Arch 部分和 Arch Linux 新闻存档及Arch Linux CN 首页上的中文版新闻列表。

早期

加拿大程序员、吉他手 Judd Vinet 从 2001 年早期开始开发 Arch Linux ，并于 2002 年 3 月 11 日正式发行 0.1 版。关于这段历史，他当初是受到 Slackware、BSD、PLD Linux 和 CRUX 之简洁原则的启发，但在当时苦于这些发行版普遍缺少软件包管理工具，所以 Vinet编写了 Pacman 包管理器，可以自动处理软件包的依赖解析、安装、删除和更新，后来他进一步建立了这个发行版。

中期

这个图表见证了Arch Linux 社区的稳步扩大。而且从早期开始，Arch 就树立起了**开放、友好和社区互助**的形象。

ArchWiki 的诞生

2005 年 7 月 8 日，由 MediaWiki 搭建的 ArchWiki 开始运行。

A. Griffin 时代

2007 年下半年，Judd Vinet 退出了 Arch Linux 的开发，并把领导权交给美国程序员 Aaron Griffin，也就是 Phrakture，他一直到现在都是 Arch Linux 开发者。

Arch 安装脚本

在 2012 年 7 月的 Arch Linux 安装介质中弃用了基于菜单的*Arch 安装框架*（AIF），改为使用*Arch 安装脚本*（arch-install-scripts包）。

Systemd 时代

2012 到 2013 年期间， Arch 用 Systemd 替换了 System V init：

抛弃 i686 支持

鉴于在开发者和社区中对 i686 架构的使用逐渐式微，Arch Linux 对 i686 的支持已经于2017年11月底被抛弃。

选举项目的领导角色

2020 年初，Arch 开始启用新的流程选择未来的领袖，详情记载在 DeveloperWiki:Project Leader。

Aaron Griffin 决定不再担任项目领导，因此 Arch 通过选举方式正式确认 Levente Polyak 为新的领导，结果记录在这里。

GitLab 时代

2023 年 5 月，Arch Linux 将其打包相关基础设施迁移至自建的 GitLab 实例。除了内部更改和创新外，这也导致testing 仓库拆分为 core-testing 和 extra-testing， staging 仓库拆分为 core-staging 和 extra-staging，最终 community 合并至 extra。更多相关内容您可以到这里阅读。

6个月后，即 2023 年 11 月，旧的错误跟踪器（Flyspray）迁移至 GitLab，其协作功能（问题和合并请求）向公众开放。因为存档问题，原处保留一个旧错误跟踪器的静态副本，以保证指向其原有资源的链接（例如随机选择的FS#56716）仍然有效。

1.2 Arch Linux 愚人节笑话

难度 进阶 | **篇幅** 15.7 KB | **所属** 关于 Arch

本节将讲到

- 2014–2020
- 2013：Pacman基于systemd
- 2012
- 2011：Canterbury
- 2010：愚人节挑战

每年4月1日，Arch Linux网站都会在头版刊登一则愚人节笑话。本文收集了这些笑话，供本wiki的读者阅读。

2014–2020

这几年的愚人节笑话比较微妙，尚未有人发现，但我们可以证明的是，这些年份确实有发布愚人节笑话。有关寻找这些笑话尝试（虽然都失败了）的英文论坛帖子见下：2014、2016、2018、 2020。

2013：Pacman基于systemd

发布于： 2013 / **永久链接：** Pacman 4.1 released

该版本的主要特性是PackageManager和systemd之间的紧密集成。在对如何最好地在滚动发布系统上执行更新进行了大量讨论之后，我们意识到，在执行更新时，尽可能少地运行其他进程是至关重要的。此外，更新的安全方面意味着必须尽快提供更新。我们认为实现这一点的最佳方法是在关机时执行更新。这是通过一个新的守护进程pacmand实现的，它在后台监视和下载更新。当发现更新时，它会安排重新启动系统（因此需要集成systemd）。此时，重新启动的时间不可配置，但会弹出一个计时器，允许您将其延迟预设的时间。配置可能会添加到pacman-4.2中，届时pacmanctl将可供一般使用。在该版本发布之前，Arch Linux将通过在其\[testing\]仓库中执行所有更新，并仅在一周内的某一时间推送更新，从而将影响降至最低。决定后将发布新闻。

当然，这些将使systemd成为pacman的硬依赖。我们认为这是可以接受的，因为Arch Linux已经正式改用systemd。由于此版本未在没有systemd的系统上进行测试（不太可能能够工作），因此如果Arch用户或其他使用pacman的发行版希望继续使用pacman作为其包管理器，则需要切换到systemd。在pacman-4.2中，与系统的集成将变得更加紧密，我们计划使用即将推出的kdbus消息传递接口——通过libsystemd总线——来允许其他程序与pacman交互，从而使替代前端的开发更加容易。

2012

看起来2012年没有愚人节笑话……

2011：Canterbury

发布于： 2011 / **永久链接：** Arch news archive #519

历史

在FOSDEM 2011 发行版合作活动上，Arch Linux 和 Debian 的主要开发者讨论了一个想法：结合各自项目最突出的优点，共同开发出一个新的、更优秀的发行版。这个想法其实是最初在先前的一次openSUSE会议中提出的，他们相信协同努力，共同开发一个新发行版还是有可能性的。

一周后的2月16日，Linux标准规范（Linux Standard Base）的版本4.1 发布了。随着新规范的发布，“新的统一发行版”再度成为热门话题。这次Gentoo和Grml也参与进来了。

Dieter，双ISO/USB-IMG安装文件的策划人员，已经开始着手开始新安装程序的讨论，确定将负责进一步合并更难的 p2arts，例如其基础设施和系统工具。

目标

我们的目标是真正在一起努力，并能够共同对抗专有操作系统，展示自由软件社区实际上真正能够为一个共同的目标而合作，而不是创造更多的分歧。

特性

Canterbury发行版为用户着想，致力于将各个Linux发行版的精华集于一身，成为操作系统界新的转机：

- 像Arch一样简单——简洁且保持前沿。
- 像Debian一样稳定——高度可靠。
- 像Gentoo一样可定制——用户能得到真正想要的。
- 像Grml一样便携——随时可用。
- 像openSUSE一样开放——广泛且欢迎所有人。

2010：愚人节挑战

发布于： 2010 / **永久链接：** Arch news archive #492

又是一年愚人节，但今年愚人节我们不会“那么”愚人。“不那么”，甚至大部分人都注意不到今年的愚人节笑话，但它确实存在。在本帖发布你对本次愚人节真相的猜测吧！第一个完全解开今年愚人节笑话的人将会得到一个上网本（当初是捐给我们的）作为解开谜题的奖励。

关于愚人节挑战的笑话

Arch 论坛： April Fools Challenge

第1页：“当然是个递归笑话。”

第15页：“四月份都快过一半了，怎么这帖子还有热度呢？”

第16页：“我要再发一个‘继续往下翻，这里啥都没有’的回复，这样人们就会真的觉得有点东西在……”

第16页：“？？？啥都没有？？？？？可是有386个回复和49275次浏览欸！……我是被困在黑客帝国里面了？”

2009：结束i686支持

发布于： 2009 / **永久链接：** Arch news archive #440

近期，开发者们一直在讨论向i686 port添加一点额外的优化以改进多媒体支持，但这将导致对旧设备的兼容性下降。就跟你们部分人已经知道的一样（，谷歌机翻），相关讨论的最终结果是，开发者决定只关注x86_64 port。开发者的总体意见是，现阶段x86_64 port已经足够完整，这让只关注x86_64 port成为可能，并且这符合 Arch 支持当前时代硬件的理念。x86_64架构2002年就已经可用了（i686是1995年的），我们相信大部分i686用户其实都应该有兼容x86_64的硬件了。

有关正式结束i686支持的时间线还没有确定，但我们得做出这样一个官方公告，毕竟这个决定已经泄露给Arch Linux-BR社区了。然而，在不久的将来可能确实不再会为 i686 构建主要更新（GNOME、KDE、Xorg 等）。用户仍将能通过ABS构建i686软件包。由于大多数特定于体系结构的补丁都是针对 x86_64 的，因此迁移过程应该是相对轻松的。

关于结束i686支持笑话的实际情况

2009年4月1日，该则关于结束i686架构支持的公告发布于frontpage以及Arch-Dev-Public邮件列表。这一公告在论坛、邮件列表和IRC上引发了一系列激烈的争论。本次争论导致的最大热点事件是 Aaron 威胁要离开 Arch。Similar to previous years, various phrases on the forums and other Arch sites were filtered to make life more entertaining。

事后报告

发布于： 2009 / **永久链接：** Arch news archive #441

各位Arch Linux用户你好，我们很高兴通知您，Arch Linux对i686的支持将不会结束。有关内容全部都是愚人节笑话的一部分，所有的开发者和论坛管理员在其中都扮演了很大的角色。

值得注意的是，这个愚人节笑话确实做了点好事。这个笑话促使部分用户发现他们用的是64位处理器，但却在使用32位操作系统，于是便迁移到了Arch Linux的64位版本。我们鼓励这些已经迁移的用户继续使用64位版本，以继续为该架构做出贡献，促进广大软件厂商跟进。

一个最终接受64位社区需求的典型例子是Adobe公司。他们近期为Flash插件添加了64位Linux支持，对此我们表示感谢。

对于此笑话造成的任何不便，我们深表歉意。但我们又如何能抗拒4月1日的恶作剧呢?

真正结束支持

2017年1月25日，官方正式公告，由于i686架构在开发人员和用户群中的实际使用比例下降，将逐步取消对i686架构的支持。

2008：Arch将采用德语作为官方语言

发布于： 2008 / **永久链接：** n/a

有关使用英语的Arch Linux用户的重要通知

在过去的两个月里，在私密的开发人员邮件列表上，开发者进行了热烈的讨论。而我们得出的结论是，将Arch定位为主要语言为英语的发行版对我们最大的用户群体是一种损害。因此，我们投票决定是否将 Arch Linux 的官方主要语言更改为德语，且更改为德语这一选项胜出。

请在过渡期间耐心等待——我们正在与 archlinux.de 密切合作以促进过渡，届时我们计划合并网站。不过我们很了解Arch这一伟大社区，我们确信将很快出现一个社区项目填补我们完全停止英语站点后留下的小空白。

另外，请注意，我们现在将使用“Arch”的德语发音。别担心，您会习惯的——我们 90% 的开发人员已经习惯了呢!

Happy Computenpeepers, 管理团队

关于Arch将采用德语作为官方语言这一笑话的实际情况

2008年4月1日，大部分Arch Linux网站（包括论坛）都被翻译为德语。上述通知也在大部分网站上出现，且论坛重写了用户发表的内容，将所有指向ArchWiki条目的链接指向一个叫“Deutschland”的页面，类似于“joke”（笑话）之类的词被“genius idea”（绝妙主意）这样的词汇重写。比如“Arch Linux April Jokes Collection（愚人节笑话合集）”被改写成“Arch Linux Deutschland genius idea Collection（绝妙主意合集）”。

2007：Arch Ark

发布于： 2007 / **永久链接：** Arch news archive #307

我们要改名啦!

这五年来，我们一直被称作Arch Linux，也有人一直把我们和Ark Linux搞混，而现在，我们郑重提出这一问题的解决方案。前几个月我们一直在尝试同Ark Linux的人员达成一个让双方发行版都受益的解决方案。今天，我们很高兴地宣布，Arch Linux将会改名，之后我们将会改叫Ark Linux！

接下来的几周内 archlinux.org 域名仍将由我们管理，这期间你们将适应改名，但最终我们会将域名也交换过来。通过改名这一方式，我们确信人们再也不会在区分Arch Linux和Ark Linux时出现问题。

Ark Linux万岁！

Ark Linux

Ark Linux曾经是一个真实存在的Linux发行版（已经停止开发了 ）。上方的笑话对那些因不小心打错“www.arklinux.org”成“www.archlinux.org”而开始使用Arch Linux的用户来说应该是更加有趣。“Arch Linux”这一名字的“Arch”被认为应当念作 /ɑːt͡ʃ/（/a-ch/，与larch、starch等英文单词押尾韵，与单词“archer”中“arch”的读音相同）。但是好像就实际到底怎么发音这一问题上并没有真正的共识。

相同的笑话同样发表在了Ark Linux社区，让这则公告乍看上去更加真实（至少第一眼是这样。很遗憾的是，Ark Linux新闻似乎没有人存档）。

2006：Judd前往Google工作

发布于： 2006 / **永久链接：** Arch news archive #214

当前变化

我在想向大家分享一些近期流言的实际情况。

Judd正式宣布接受了来自Google的工作，因此，将不再能继续参与Arch Linux的相关工作。

项目领头人已交接给十分有能力的Jason Chu（Xentac）。Jason负责的首要任务——在我们朝着0.7.1.1版本（代号Pony）的发布不断努力的同时——是创建\[stable\]软件仓库，里面将会放置一些较旧的软件。这将成为之后工作的重点，与此同时还包括安全更新的旧版移植以及兼容性修复。因此，现在的仓库将开始缺少一点内容，但最终这些会随着我们向更“稳定”和“注重生产级质量”的发布机制而被及时移除。

几周后你们就能看到0.7.1.1啦。

此致， Aaron

Judd's 对Arch项目的最终告别

Arch Linux的创始人Judd Vinet实际上已于2007年10月1日离开该项目，成为“Arch的头号啦啦队员”。 自此，Aaron Griffin将是项目的领导者。

Arch-Linux及其发布机制

Arch Linux当然从未想过“稳定”和“生产级质量”的发布机制。我们是滚动更新的系统，系统始终保持最新，没有固定的发布时间表。

2005：Arch与袋熊（Wombat）的相关争议

发布于： 2005 / **永久链接：** Arch news archive #147

CETW问题

我个人怀着极大的悲伤写下这条消息。

当我们首次发布 Arch Linux 0.7（代号Wombat）时，袋熊伦理治疗中心 (CETW) 联系了我们。 CETW不知为何认为我们发布一个代号为Wombat，袋熊，的“开源项目”是在伤害全世界的袋熊。虽然我们试图说服他们 Arch Linux 是一个 Linux 发行版，并且在我们的发行版制作过程中没有袋熊受到伤害，但他们没有接受。

所以他们威胁要起诉我们，除非我们改变对袋熊的伤害。

经过深思熟虑，我们发现我们实际上没有足够的钱在法庭上与他们抗争。因而，我们选择了庭外和解。我们解决方案的一部分是停止Arch Linux的开发。

因此，由于命名选择不当，Arch Linux将不复存在。在此我代表所有开发人员感谢社区的支持和贡献。

Wombat（袋熊）是个啥？

引自维基百科的“袋熊”页面：“袋熊（英语：Wombat，学名：Phascolomidae），是澳洲的有袋类动物。它们的脚短及肌肉发达，身长约有1米，尾巴亦很短。它们生活于澳洲东南部及塔斯马尼亚州的森林、山地及石楠地。袋熊的Wombat这个名字来自Eora土著社区，他们是悉尼地区的原住民。”

袋熊显然不会雇佣律师来骚扰Linux发行版。了解袋熊与人类的关系。

2004：GNOME 2.6

发布于： 2004 / **永久链接：** Arch news archive #58

GNOME 2.6 发布了！

正如标题所说，gnome 2.6 已经位于软件源中。没有太多需要注意的, Arjan 和 JGC 在测试阶段很好地消除了潜在问题，所以安装后体验应该是完美的。

升级：pacman -Syu 安装：pacman -S gnome

一些与GTK 2.x相关的软件包也进行了升级，以满足GNOME 2.6的要求。

GNOME版本历史

在上述“新闻”发布前，GNOME项目在2002年才发布GNOME 2.0，而2004年4月1日前最新的发布版本也才GNOME 2.2而已。

1.3 常见问题

难度 进阶 | **篇幅** 17.8 KB | **所属** 关于 Arch

本节将讲到

- 一般问题
- 安装
- 其他
- 软件包管理
- 关于64-bit

一般问题

Q) Arch Linux 是什么？

A) 请阅读 Arch Linux。

Q) 为什么我不想用 Arch Linux？

A) 您或许不想使用 Arch Linux，如果您：

- 没有经验、能力、时间、想法和愿望去打理这样一个高度可定制的 GNU/Linux 发行版。
- 需要非 x86_64 平台支持。
- 你是狂热的自由软件爱好者，只希望发行版提供 GNU 定义的自由软件。
- 你认为好的操作系统应当是已经配置好的：安装介质应默认包含一套完整的应用软件包、图形化桌面环境——达到“开箱即用”。
- 你不需要或不想使用滚动升级的发行版。
- 你对目前使用的操作系统感到满意。

Q) 为什么我要用 Arch Linux？

A) 因为 Arch 是最好的！

Q) Arch Linux 支持什么架构?

A) Arch Linux 现在只支持 x86_64(也被称为 amd64) 架构，对 i686 架构的支持已于 2017 年 11 月结束。

非官方支持的移植版本有： i686 和 ARM 。

Q) Arch Linux 遵循 FHS 吗?

A) Arch Linux 遵循适用于 systemd 服务管理器的文件系统架构，file-hierarchy(7) 包含了每个文件夹的解释和设计。 `/bin`, `/sbin` 和 `/usr/sbin` 现在是 `/usr/bin` 的符号链接，`/lib` (以及 `/lib64`) 是 `/usr/lib` 的符号链接。

Q) 我是一个彻头彻尾的Linux新手，我应该用 Arch Linux 吗?

A) 如果您是新手，要使用 Arch Linux 就必须愿意花时间学习新系统，并接受 Arch 是一个需要有 DIY 精神及能力的系统，每个用户都是自己系统的组建者。

在您开始问任何问题之前，建议您自己先通过 Google 、Wiki 类网站、ArchWiki 或者 Linux 相关论坛进行搜索。我们为您创建了这些资源并让您可以随时访问，有成千上万的**志愿者与爱好者**为您提供大量的信息资源。

推荐阅读: Arch术语#RTFM 。

Q) Arch Linux 的安装和配置十分麻烦，社区的人总是让我去读那些死长死长的手册。

A) Arch Linux 是针对一部分特定用户群体设计的，如果您不在其中，也许 Arch Linux 并不适合您。

Q) Arch Linux 是为服务器、桌面还是工作站设计的?

A) Arch Linux 并不是针对特定应用设计的，而是针对有特定需求的用户群体设计的。Arch Linux 适合那些喜欢并有动手能力的用户，以及希望打造属于自己的独一无二的系统的用户。因此，对于这些用户，Arch Linux 能胜任各种工作。也有许多人把 Arch Linux 同时当做桌面、服务器和工作站系统。此外，archlinux.org 网站也在 Arch Linux 上运行。

Q) 我太喜欢 Arch Linux 了，希望开发团队实现一个新功能。

A) 您可以参与进来，分享代码和解决方案。如果广大用户和开发者确实认同这个功能，您的想法有机会被添加到新版本系统中。Arch 社区依靠大众贡献分享的代码和工具蓬勃发展。

Q) 一般什么时候发布新版本?

A) Arch Linux 的发布版本只是安装和修复环境，包含base包 元软件包和一些辅助软件，一般每月上旬发布一次。

Arch Linux 使用滚动升级模式，只需执行一条命令，便可升级系统到最新版本。基于上述原因，发布新版本对 Arch Linux 来说并不重要，因为滚动升级的机制会使系统永葆青春。使用`pacman -Syu`命令升级系统，无需重装系统即可更新到最新版本。同样，Arch Linux 的新发行版本并不会提供什么新的功能，新的功能通过`pacman -Syu`命令即可立即拥有。

Q) Arch Linux 是否稳定？会不会经常坏掉?

A) 每个 Arch Linux 的用户应当对自己执行滚动升级的系统稳定性负责。由用户自行决定何时升级、修改配置。Arch Linux 与其他发行版的一个不同之处在于，Arch Linux 是真正的“DIY”发行版。一味抱怨系统损坏是没有意义的，毕竟上游的改动不是 Arch 开发者的责任。

如果您想要使系统变得更加稳定，请参阅：系统维护，以提高系统稳定性。

Q) Arch Linux 需要更多曝光（比如广告）。

A) Arch Linux 已经得到了应有的新闻媒体曝光机会。Arch Linux 的目标不是为了一步登天。我们的目标只是使它变得更好。让它自然成长吧，试图强迫它成长得更快会造成诸多问题。

Q) Arch Linux 需要更多开发者。

A) 或许吧。欢迎您来帮助我们！可以来逛逛论坛，IRC channels 以及邮件列表，看看您有什么需要做的。或者，看看论坛的 Community Contributions 子版面吧。

安装

Q) 我感觉 Arch Linux 需要安装程序，比如带有图形界面的那种。

A) 从 2021-04-01 起，Arch Linux 有了更好的安装程序。参见 archinstall。

Q) 我安装了 Arch Linux，现在眼前只有一个命令行登录界面，我该怎么办?

A) 建议您阅读通用建议。

Q) 哪个桌面环境或窗口管理器比较好?

A) 有很多不同的选择，从中选择最适合您的即可。参见：桌面环境。

Q) Arch Linux 比起其他“小型”发行版，有何独特之处?

A) 建议您阅读 Arch与其他发行版的比较。

其他

Q) 什么是 AUR ？

A) 建议您参见：Arch用户软件仓库#FAQ。

Q) 为什么我感觉 Arch Linux 下网速比其他系统慢?

A) 您的网络是否正确配置？如有疑问，您可以读一读网络配置。

Q) 为什么 Arch Linux 把我的所有内存用光了?

A) 实际上，内存不用就浪费了。

很多新用户发现，Linux 内核对物理内存的处理机制和他们所以为的有所不同。由于访问物理内存比访问磁盘要快得多，内核会把最近访问的数据缓存到物理内存中。物理内存中缓存数据会在空闲物理内存耗尽时被释放。

而造成新用户的困惑的罪魁祸首，恐怕是`free`命令：

```
$ free -h
              total        used        free      shared  buff/cache   available
Mem:          2.8Gi       1.1Gi       283Mi       224Mi       1.4Gi       1.2Gi
Swap:         3.0Gi       881Mi       2.1Gi
```

要注意 "free" 和 "available" 的区别，即空闲和可用的区别。在上面的例子中, 电脑所有的 2.8G 内存几乎全部占用。"free"中只剩下 283 MiB 是未使用的。但是在总量 2.8G 内存中的 1.4G 是 "buff/cache" 缓存。实际上还剩下 1.2G 是可使用的（不考虑交换空间）。查看 free(1) 了解更多。

这有什么用？当然是更好的性能!

如果您对此十分困惑，请参见此文。另外还有一篇文章有详细的解释：https://www.linuxatemyram.com/。

Q) 我的磁盘空间被什么东西占用了?

A) 问题的答案因系统而异。有一些应用列表或许能告诉您答案。

软件包管理

详情请参考 pacman、pacman/提示和技巧和官方仓库。

Q) 我发现了某个软件包的错误，该怎么办?

A) 首先您需要搞清楚 Arch 队伍是否能够解决这个错误。有时并非如此（Firefox 崩溃也许是 Mozilla 开发团队的错误）——这便是所谓的***上游错误***。如果您确信是 Arch 的问题，您可以采取以下步骤：

1. 在论坛中搜索有关信息。看看是否有人已经注意到这个情况。

2. 在 Arch Linux Gitlab 的 Bugtracker 提交具有详细信息的 Bug 报告。
3. 也可在论坛中发篇帖子，阐述问题细节并说明您已经报告过这个情况。这会避免其他人重复报告同样的问题。

Q) Arch Linux 软件包需要独特的后缀名。".pkg.tar.zst"太长了，而且令人困惑

A) 在 Arch 邮件列表曾有过讨论。一些用户提议使用 .pac 作为文件扩展名。但就目前情况来说，我们没有计划改变软件包的扩展名。

正如 Tobias Kieslich（一名 Arch 开发者）所说："一个软件包就是一个 **\[压缩包\]**！它可以被很多软件打开、研究和操作。此外，这种 mime-type 也可以被大多数软件自动地正确识别。"

Q) Pacman应该提供函数库接口，这样其他软件就可容易地获得软件包信息。

A) libalpm(3)（Arch Linux Package Management，Arch Linux 软件包管理）是 pacman 的后端，这个库大大方便了交互式前端的编写（例如图形化前端）。

Q) Pacman需要某某功能！

A) 如果您有新想法，可以在 pacman-dev 邮件列表进行讨论，或者查看官方论坛看看其他人有没有类似的想法。

也欢迎您自己在系统中实现新功能。也许您的代码或补丁不会被官方接受，但他人可能会用到。

Q) 仓库和镜像的区别是什么？

A) 请您参考软件仓库。

Q) 我刚刚安装了一个软件包，怎么启动呢？

A) 如果您正在使用像KDE和GNOME这样的软件包，它们的程序一般会自动出现在菜单中。如果是命令行程序，则可使用如下命令查看可执行文件名称和位置：

```
$ pacman -Qlq package_name | grep /usr/bin/
```

Q) 在官方仓库中，为什么只为每个共享链接库提供一个版本？

A) 某些发行版，比如 Debian，会为一个共享库提供多个版本，比如：`libfoo1`，`libfoo2`，`libfoo3`等等。这样，就可以同时安装那些依赖不同版本 libfoo 的程序。

Arch 官方只对最新版的程序提供支持。只要摆脱了过时的软件包，包的维护人员就可以全力维护新版本的程序。而只要上游提供新版本的共享库，我们就会重新编译相关程序，更新仓库。

Q) 执行`pacman -Syu`时，终端显示某个共享库需要升级，但依赖它的程序没有升级，我该怎么做呢?

A) 这种情况理应不会发生。假如官方源一个名为`foobaz`的程序，成功地使用新版本共享库`libbaz`编译成功，那么它应该和`libbaz`一同更新。如果编译不成功，由于旧的`foobaz`存在依赖关系，例如：

```
libbaz=1.5
```

所以当`libbaz`升级时，它会被移除。

如果`foobaz`是您自己编译的或是从 AUR 获取的，当`libbaz`升级时您应当重新编译它。如果编译失败，还请您通知`foobaz`开发者。

Q) 会不会出现一种情况就是，仓库中的内核主版本更新了，而某些驱动包没有一同升级的情况?

A) 这个绝对不可能。当内核主版本升级时，所有支持的驱动一定会一同更新。

然而，如果您安装有非内核支持的驱动程序包，比如`catalyst`，内核升级有可能导致系统挂机。您应当在内核升级时自行重编译它们。

Q) 升级前该做什么?

A) 在升级之前，记得访问 Arch Linux 新闻列表、通告邮件列表 (有时候需要看一看论坛和各个邮件列表）。需要特别注意的事项都会列在那里。另见系统维护#更新系统。

Q) 我知道某个包已经更新，但是 pacman 并没有发现更新

A) 可能是 pacman 镜像还没有同步，最好是等等看或者是换一个镜像服务器。MirrorStatus 可以帮助您区分哪些镜像服务器是最新的。

Q) 某上游项目已经发布了一个新版本，要多长时间 Arch Linux 才会更新到新版本?

A) 软件包将会在准备好之后发布，发布的时间取决于上游的变动和打包人员的精力。

如果发布的是小版本，可能只需要几个小时。如果是大版本更新，则可能需要数周时间。

此外，有些软件包会在 testing 仓库中停留一段时间进行小范围测试。软件包维护者会尽快将稳定的版本发布到仓库中。如果您发现官方软件仓库中的软件过旧，可以在软件包页面进行标记。

Q) 如果需要一个老的软件库，可以 symlink 到新版本吗?

A) 运气好的话，可以用来救急。但是这并不是一个好办法：

- 软件库不会无缘无故更新版本 - API/ABI 可能发生变动，这些变动是否影响到使用只能听天由命。
- 软件包管理器不会记录软链接，缺乏经验的新手执行这种操作，可能会引起难以恢复的错误。
- 未被记录的老版本文件很快就会被遗忘，且安全漏洞不会被修复。

最好是使用或编写兼容软件包，用其提供需要的版本。

关于64-bit

Q) 我如何确定我的处理器（CPU）是否支持 x86_64?

A) 运行下面的命令：

```
$ less /proc/cpuinfo
```

查找 `flags` 条目。如果你看见 `lm` 标志，那么您的处理器是支持 x86_64 的。

Windows 系统用户可以使用免费软件 CPU-Z 或商业软件 AIDA64 中的 CPUID 工具查看 CPU 的信息，这些软件可以通过识别 CPU 的指令集来确定处理器是否支持64位。

如果您使用带有 AMD 的 "x86-64" 指令集或者英特尔的 "EM64T" 指令集的 CPU，则可以使用 x86_64 的发行版和二进制包。

Q) 为什么现在应该使用64位系统?

A) 在大多数情况下，64位系统会更快，而且更安全。更安全是因为它拥有 Address space layout randomization (ASLR) 、 Position-independent code (PIC) 特性，以及 NX Bit (它在i686内核中，由于禁用了物理地址扩展 (PAE)而无法使用)。

如果你的计算机在运行 4.0 GiB 或者更多的内存，那应该使用64位系统，因为多余的内存是无法被32位系统分配的。

现如今，程序员也更加倾向于忽视32位系统，因为新的 x86 CPU 通常都支持64位扩展。

还有许多其他的理由让我们抛弃32位系统。但是在内核、用户空间和单独的程序中，我们没有办法列出所有的64位系统比32位系统做得更好的地方。

中文社区仓库

主条目： Arch Linux 中文社区仓库

Q) \[archlinuxcn\] 仓库和 AUR 是什么关系?

A) 没有特别的关系。部分软件包的 PKGBUILD 打包脚本也同时存在于 AUR 而已。然而并不是所有 \[archlinuxcn\] 提供的软件包都存在于 AUR，也不是所有同时存在于 \[archlinuxcn\] 和 AUR 的软件包就一定会使用相同的打包脚本。我们不是 AUR 的二进制版。

在使用 \[archlinuxcn\] 提供的软件包时遇到问题，也请您报告到我们的 GitHub issues。

Q) 既然有 AUR，我为什么会想要使用 \[archlinuxcn\] 仓库?

A) \[archlinuxcn\] 仓库提供许多额外的、已经打包好的软件。相比于 AUR，您不必就地从世界各地下载软件源码然后花费时间编译，而是可以直接安装已经编译好的软件包。

如果您是中国大陆用户，您可以利用我们位于中国大陆境内的多个镜像源，来快速下载到软件，而不必处理访问国际网络可能会遇到的问题。

Q) 我是 Manjaro （或其它 Arch Linux 衍生发行版）用户，我可以使用 \[archlinuxcn\] 仓库吗?

A) 我们不建议这样做。\[archlinuxcn\] 仓库只支持 Arch Linux，在衍生发行版上使用可能会出现问题。

如果您在非 Arch Linux 系统上使用 \[archlinuxcn\] 仓库遇到了问题，还请不要将问题报告给我们。

1.4 Arch Linux 新闻报道

难度 高级 | **篇幅** 20.3 KB | **所属** 关于 Arch

本节将讲到

- 2023
- 2022
- 2021
- 2020
- 2019

注意：此页面上的内容可能不是最新的，请首先参阅 ArchWiki 上的 Arch Linux press coverage 。

参见 Wikipedia:Arch Linux 与 Arch Linux on DistroWatch。

2023

- Migrating Arch Linux's packaging infrastructure to GitLab Levente Polyak for GitLab open source blog - 2023-09-11
- The Numbers Don’t Lie: Arch Linux Reigns Supreme on Reddit Bobby Borisov - linuxiac.com 2023-01-02

2022

- Arch Linux finally adds a new installer to ease a rather challenging process Jack Wallen - techrepublic.com 2022-05-09
- Arch Linux turns 20: Small, simple, great documentation Liam Proven - theregister.com 2022-03-15
- 20 years of Arch Linux: Skilled system with a hobbyist popularity\[失效链接 2024-03-03 ⓘ\] translation of article by Martin Gerhard Loschwitz - heise.de 2022-03-11

2021

- Arch Linux's Install Media Adds "Archinstall" For Quick/Easy Installations Michael Larabel - Phoronix.com 2021-04-04
- Arch Linux Developers Discuss Idea Of Providing An x86-64-v3 Port Michael Larabel - Phoronix.com 2021-03-16
- Arch Linux Looking To Employ LTO By Default, Possibly Raise x86-64 Requirements Michael Larabel - Phoronix.com 2021-03-09

2020

- 5 Reasons to Use Arch Linux Matt Weber - anchor.fm/thelinuxcast 2020-06-03
- Arch Linux Announces New Project Leader Michael Larabel - Phoronix.com 2020-02-04

2019

- Arch Linux Held Their First Internal Conference Last Month In Berlin Michael Larabel - Phoronix.com 2019-11-17
- Arch Linux OS Challenge: Install Arch 'The Easy Way' With These 2 Alternative Methods Jason Evangelho - Forbes.com 2019-06-10
- Introducing The Arch Linux Summer 2019 Community Challenge Jason Evangelho - Forbes.com 2019-06-09
- BTW, I Installed Arch (first 19 minutes of the podcast) Jason Evangelho and Joe Ressington - chooselinux.show 2019-05-30
- How to Conquer Your Fear of Arch Linux Jonathan Terrasi - linuxinsider.com 2019-04-28

2018

- 10 Reasons to Install an Arch Linux-Based OS on Your PC Bertel King, Jr. - makeuseof.com 2018-04-02

2017

- Linux Action Show Episode 461 (from 00:11:00) Chris Fisher - YouTube archive from the show originally at www.jupiterbroadcasting.com 2017-03-18
- Arch Linux: The Simple Linux Carla Schroder - linux.com 2017-02-02

2016

- mintCast 250 – The Return of Arch (from 00:49:40) Rob and Isaac - mintcast.org 2016-07-25
- mintCast 239 – A Friendly Look at Arch (from 00:32:16) Rob, Scott, Joe and Isaac - mintcast.org 2016-01-21

2015

- DistroWatch Weekly Feature Story about Arch Linux Jesse Smith - distrowatch.com 2015-12-21
- Why you should try Arch Linux David Bailey - www.dbailey.uk.to 2015-10-04
- 1+ Year Running Arch Linux on a Lenovo Yoga 2 Keith Curtis - keithcu.com 2015-04-07
- The grumpy editor's Arch Linux experience Jonathan Corbet - LWN.net 2015-04-01
- Five Reasons to Use Arch Linux Swapnil Bhartiya - www.itworld.com 2015-03-18

2014

- Linux Distros Ben Everard - www.linuxvoice.com 2014-10-8
- Arch: Shangri-La for Ubuntu Power Users? SAM TRAN - www.omgubuntu.co.uk 2014-4-19
- Arch Linux 2014 review – Livin' on the edge paranoidpenguin.net 2014-03-29
- Arch Linux is Becoming More Mainstream Dr. Roy Schestowitz - techrights.org 2014-03-07
- Now Might be a Good Time to Give Arch Linux a Try Dr. Roy Schestowitz - techrights.org 2014-02-12
- Opinion: Arch Linux and Stability Andrew Powell - thelinuxrain.com 2014-02-04

2013

- Arch Linux on a HiDPI Lenovo Yoga 2 Pro Keith Curtis - keithcu.com 2013-11-11
- Arch Linux: Succeeding on the Bleeding Edge (YouTube Video) Allan Mcrae gives a talk at XX SINFO in Lisbon, Portugal - 2013-03-01

2012

- Under the Hood with Arch and Gentoo Richard Hillesley - linuxuser.co.uk 2012-10-20

- linux-guide-slackware-arch-linux John Browning - engadget.com 2012-08-09
- Moving to Arch Linux from Fedora Chris Duckett - techrepublic.com 2012-08-07
- Arch Linux 2012.07.15 drops AIF Chris von Eitzen - h-online.com 2012-07-23
- Arch linux – not just for geeks? claudecat - claudecat.wordpress.com 2012-07-05
- KISS simplicity: Arch Linux Guillermo Garron - www.garron.me 2012-05-21
- Five-best-linux-distributions Alan Henry - lifehacker.com 2012-04-23
- Pacman Package Manager Linux Library - linuxlibrary.org 2012-03-21
- Arch Linux Review (Early 2012) InfinitelyGalactic Reviews - youtube.com 2012-01-31
- Arch'ed in a day; a noob experience Justin - justinstories.wordpress.com 2012-01-28
- On switching to Arch Linux Mike Crittenden - mikethecoder.com 2012-01-26
- Arch Linux: Only the Hardcore Need Apply Joel McLaughlin - geardiary.com 2012-01-26

2011

- ArchLinux, not just for the elite Moved by Freedom – Powered by Standards | A weblog by Charles-H. Schulz 2011-11-06
- An Arch Tale Dave Phillips - Linux Journal 2011-10-04
- Arch Linux – "It is what you make it" Richard Hillesley - h-online.com 2011-09-16
- The Six Best Linux Community Server Distributions Carla Schroder - linux.com 2011-08-08
- Ask TWIL: Best Linux Distribution and What's Your Favorite Distro? Jordan Keyes - thisweekinlinux.com 2011-07-27
- The Arch Way Linux Journal - linuxjournal.com 2011-01-13

2010

- Arch Linux Review The Linux Action Show - jupiterbroadcasting.com 2010-10-31
- Distro Review: Arch Linux (Video) This Week on Linux - thisweekinlinux.com 2010-05-19
- Rolling with Arch Linux Ivan Jelic - LWN.net 2010-03-10
- Interview: Arch Linux Team Jordan Spencer Cunningham - OSNews 2010-01-11

2009

- Dennis Wyman - Zangaroa Blog 2009-11-13
- Arch Linux 2009.08 Benchmarks Michael Larabel - Phoronix.com 2009-08-14
- Arch Linux: The Simple, Flexible (and Fast!) Distro Chris Smart - Linux-Mag.com 2009-08-12
- Arch Linux Magazine Eduardo Romero, Editor 2009-08

- Open Source Watershed: Watching Software Flow Downstream (PDF Document) Scott Shawcroft - OSWatershed.org 2009-07-20
- Arch User Magazine - Issue #3 - Daniel Griffiths, Editor 2009-06
- Why people resort to Arch Linux LazyTechGuy.com 2009-05-19
- Arch Linux - Syahrizal - OpenSourceReview.com 2009-05-11
- Best Linux distros for power users, gamers, newbies, and more TechRadar.com 2009-05-10
- Arch User Magazine - Issue #2 - Daniel Griffiths, Editor 2009-05
- Arch Linux 2009.02 - Maurice Lawles - TechieMoe.com 2009-04-28
- Distro Review - Arch Linux 2009.02 Dan Lynch - DanLynch.org 2009-04-14
- Arch User Magazine - Issue #1 - Daniel Griffiths, Editor 2009-04
- Arch Linux Review ggarron - Go2Linux.org 2009-03-21
- Archlinux the best way to K.I.S.S. Bigbrovar.WordPress.com 2009-03-16
- Linux Review 11: Arch Linux Open_N0DE - Linux-Exploration.BlogSpot.com 03-15-2009
- Installing Arch Linux Part 1 (YouTube Video) silvernode - YouTube.com 2009-03-10
- Installing Arch Linux Part 2 (YouTube Video) silvernode - YouTube.com 2009-03-10
- Installing Arch Linux Part 3 (YouTube Video) silvernode - YouTube.com 2009-03-10
- Arch Linux Revolts Against ATI Catalyst Driver Michael Larabel - Phoronix.com 2009-03-01
- Arch User Magazine - Issue #1-Preview (PDF Document)) Daniel Griffiths, Editor 2009-03
- Distribution Release: Arch Linux 2009.02 DistroWatch.com 2009-02-17
- Arch Linux in review DistroWatch Weekly - Issue 286 2009-01-19
- A Weekend With Arch Kristin Shoemaker - Ostatic.com 2009-01-11
- Arch Linux: First Impressions Shinobu - Zuttobenkyou.WordPress.com 2009-01-08

2008

- Review: Arch Linux - A sleek, fast Linux distribution Ravi - LinuxHelp.BlogSpot.com 2008-12-12
- Arch Linux - a review EvilGold - EvilGold.BayWords.com 2008-10-27
- Life on the cutting edge - ArchLinux Gautham - TheSmallerBang.WordPress.com 2008-10-10
- Arch Linux Installation Walkthrough (YouTube Video) linuxandfriends - YouTube.com 2008-09-30
- Checking Out Arch Linux MD Weems - BrightHub.com 2008-09-20
- A Quick Review of Arch Linux yoyoned - LinuxBasement.com 2008-08-23
- A Simple Linux Review chi - 42Gems.com 2008-08-21
- Linux guru? then switch to Arch Linux! - Anthony Williams - AnthonyWilliams.com 2008-07-13

- Arch Linux 2008.06: Overlord Celettu's Weblog - Celettu.WordPress.com 2008-06-25
- Distribution Release: Arch Linux 2008.06 DistroWatch.com 2008-06-24
- Arch Linux Review blackbird - Forum.NotebookReview.com 2008-05-04
- Arch Linux: Popular KISS distro Robin Heggelund Hansen - Hardware.no 2008-03-25
- Arch: Pros Only, But Not Bad DJ Gentoo - Distrogue.BlogSpot.com 2008-01-26
- PLANETOSS: Arch Linux - Is this really a geek's distro? vasanth - PlanetOSS.com 2008-01-17
- A Stable Arch Branch? K. Mandla - KMandla.WordPress.com 2008-01-05

2007

- The Linux Link Tech Show - Interview with Aaron Griffin, lead developer of Arch Linux Episode #223 12-12-07
- Arch Linux, really impressive. Even when virtualized. TuxToday.WordPress.com 2007-11-14
- Arch 2007.08-2 Review OpenReview - SimplyJat.BlogSpot.com 2007-11-14
- Distribution Release: Arch Linux 2007.08-2 DistroWatch.com 2007-10-07
- Arch Linux Review DVD-Guides.com 2007-09-13
- Distribution Release: Arch Linux 2007.08 DistroWatch.com 2007-08-05
- Linux Reality: Podcast Episode 70 - Arch Linux Episode #70 2007-07-11
- Arch Linux: An Observational Review Rohrbakn - Rohrbakn.BlogSpot.com 2007-06-22
- Ten Things I Love About ArchLinux Vince Wadhwani - UrbanPuddle.com 2007-05-31
- Distribution Release: Arch Linux 2007-05 DistroWatch.com 2007-05-17
- Fallen Under the Spell of Arch Voodoo srlinuxx - TuxMachines.org 2007-04-20
- Distribution Release: Arch Linux 0.8 DistroWatch.com 2007-04-01
- ArchLinux, The Distro I wish I had found sooner. crouse - USALug.org 2007-01-10
- Interview with Judd Vinet, founder of Arch Linux\[失效链接 2023-10-29 ⓘ\] OSSBlog.it 2007-01-02

2006

- Review: Arch Linux 0.7.2 – swamytk, KaruppuSwamy.com 2006-11-12
- Watching the Evolution of Arch Linux – Eugenia Loli-Queru, OSNews.com 2006-07-03
- Distribution Release: Arch Linux 0.7.2 – DistroWatch.com 2006-05-23
- The Simplicity of Arch (PDF Document) – Jon Kent, Linux-Magazine Issue #64 2006-03
- The Linux Link Tech Show – Interview with Judd Vinet, founder of Arch Linux Episode #122 2006-02-08

- Distribution Release: Arch Linux 0.7.1 – DistroWatch.com 2006-01-05

2005

- The Linux Link Tech Show – Interview with James Rayner, creator of the Archie Live CD Episode #91 2005-07-06
- The Big Arch Linux Interview – Eugenia Loli-Queru, OSNews.com 2005-03-29
- Arch Linux: Why It Rocks – Eugenia Loli-Queru, OSNews.com 2005-03-21
- Make your Arch Linux a Bluetooth Access Point – Eugenia Loli-Queru, OSNews.com 2005-02-27
- Arch Linux for Power Users – Ladislav Bodnar, Linux Weekly News 2005-02-02
- A Week in the Life of an Arch Linux Newbie – Andrew Roberts, OSNews.com 2005-01-27
- Distribution Release: Arch Linux 0.7 – DistroWatch.com 2005-01-25

2004

- Arch Linux Vs Slackware: The Best of All Worlds – Eugenia Loli-Queru, OSNews.com 2004-11-03
- WRBurns' Review of Arch Linux 0.6 – WRBurns, ExtremeTech.com 2004-04-14
- LinuxLookup: Arch Linux version 0.6 (Widget) Review – LinuxLookUp.com 2004-03-23
- Distribution Release: Arch Linux 0.6 – DistroWatch.com 2004-03-01
- Arch Linux: An End To My Distro Shuffle? – Andrew Barilla, OSNews.com 2004-02-10

2003

- Arch Linux: A Better Distribution – Ben Mazer, OSNews.com 2003-10-15
- Interview with Judd Vinet – Ladislav Bodnar, DistroWatch.com 2003-08-03
- My look at Arch Linux 0.2, 0.3, and 0.4 – rabnud, Linux Weekly News 2003-08-01
- A Look at Arch Linux – Ladislav Bodnar, Linux Weekly News 2003-07-23
- Distribution Release: Arch Linux 0.5 – DistroWatch.com 2003-07-21

2002

- Distribution Release: Arch Linux 0.4 – DistroWatch.com 2002-12-18
- Introduction to Arch Linux – Jon "Trekboy", OSNews.com 2002-12-01
- Distribution Release: Arch Linux 0.3 – DistroWatch.com 2002-08-08
- Distribution Release: Arch Linux 0.2 – DistroWatch.com 2002-04-18

1.5 BlackArch Linux

难度 入门 | **篇幅** 2.2 KB | **所属** 关于 Arch

本节将讲到

- 安装
- 南京大学 (江苏南京) (ipv4, ipv6, http, https)
- SJTUG 软件源镜像服务 (上海) (ipv4, ipv6, https)
- 清华大学 (北京) (ipv4, ipv6, http, https)
- 中国科学技术大学 (安徽合肥) (ipv4, ipv6, http, https)

BlackArch 是一个基于 Arch Linux 的渗透测试发行版，提供了大量的网络安全工具。它是专为渗透测试人员和安全研究人员创建的开放源代码的发行版。该存储库包含 3000 多个可以单独或成组安装的工具。BlackArch 与现有的 Arch Linux 安装相容。

安装

安装教程可能不完整，请下载blackarch 官方 PDF

以 `root` 身份运行 https://blackarch.org/strap.sh 并按照说明进行操作：

```
$ curl -O https://blackarch.org/strap.sh
```

验证 `SHA1` ：

```
$ echo 26849980b35a42e6e192c6d9ed8c46f0d6d06047 strap.sh | sha1sum -c
```

设置执行权限：

```
$ chmod +x strap.sh
```

运行 strap.sh：

```
$ sudo ./strap.sh
```

添加中国大陆 blackarch 镜像提高下载速度，在 `/etc/pacman.conf` 文件末尾添加镜像：

```
## 南京大学 (江苏南京) (ipv4, ipv6, http, https)
[blackarch]
Server = https://mirrors.nju.edu.cn/blackarch/$repo/os/$arch
## SJTUG 软件源镜像服务 (上海) (ipv4, ipv6, https)
[blackarch]
Server = https://mirror.sjtu.edu.cn/blackarch/$repo/os/$arch
## 清华大学 (北京) (ipv4, ipv6, http, https)
[blackarch]
Server = https://mirrors.tuna.tsinghua.edu.cn/blackarch/$repo/os/$arch
## 中国科学技术大学 (安徽合肥) (ipv4, ipv6, http, https)
[blackarch]
Server = https://mirrors.ustc.edu.cn/blackarch/$repo/os/$arch
## 阿里云 (Global CDN) (ipv4, ipv6, http, https)
[blackarch]
Server = https://mirrors.aliyun.com/blackarch/$repo/os/$arch
```

然后请安装 `blackarch-keyring` 包以导入 GPG key：

```
$ pacman -Sy blackarch-keyring
```

按照 wiki 启用 multilib 并运行：

```
$ sudo pacman -Syu
```

要列出所有可用工具，请运行：

```
$ sudo pacman -Sgg | grep blackarch | cut -d' ' -f2 | sort -u
```

安装所有的工具：

```
$ pacman -S blackarch
```

按照分类批量安装工具：

```
$ sudo pacman -S blackarch-<category>
```

列出 blackarch 的所有分类：

```
$ sudo pacman -Sg | grep blackarch
```

工具使用方法，请下载 blackarch 官方 PDF

1.6 首页

难度 入门 | **篇幅** 3.4 KB | **所属** 关于 Arch

本节将讲到

- 关于发行版
- 社区
- 阅读和编辑这个 Wiki
- 相关链接

欢迎访问 Arch Linux 中文维基——由社区驱动的中文 Arch Linux 知识库！

目录表包含本维基的完整目录。

关于发行版

Arch Linux : Arch Linux 简介：Arch Linux 是什么？能做什么？有何优势？发展历程?

常见问题 : 常见问题解答，解决您的疑问。

Arch 与其他发行版的比较 : Arch 和其他发行版的相似点和不同点。

安装指南 : Arch Linux 的完整安装指南。

建议阅读 : 包含安装系统后的相关教程和热门文章。

软件列表 : 官方软件仓库和 AUR 中常用软件的分类描述列表。

社区

参与贡献 : 无论新手或专家，您都可以参与我们，为 Arch 社区做出贡献。

国际社区 : 汇总世界各地的 Arch Linux 社区的信息。

行为准则 : Arch 社区的行为规范。

中文社区：与中文用户交流。

阅读和编辑这个 Wiki

阅读：了解一篇 Wiki 条目的基本结构。

查阅：搜索 Wiki，查找相关文章和离线浏览。

术语：了解 Arch Linux 常用的术语。

所有页面：访问所有页面的快捷方式。

参与贡献：完善 Wiki 的第一步。

翻译团队：跟踪和维护页面的中文翻译。

编辑指南：教您如何编辑 Wiki 文章。

编者注意：当前中文维基编辑的注意事项。

相关链接

全世界的 Arch 用户乐于帮助您，您可以通过以下方式找到他们：

Arch Linux 官方设施

- Arch Linux 官方论坛
- Arch Linux 官方 IRC 频道
- ArchWiki
- Arch 用户软件仓库（AUR）
- Arch Linux Gitlab
- Planet Arch Linux

Arch Linux 中文社区设施

- Arch Linux 中文社区首页
- Arch Linux 中文论坛
- Arch Linux 中文社区仓库
- Arch Linux 中文社区交流群
- Arch Linux 中文星球

1.7 Arch 术语

难度 进阶 | **篇幅** 8.9 KB | **所属** 关于 Arch

本节将讲到

- ABS

- Arch Linux
- Arch Linux 档案
- AUR
- bbs

Help:术语表

翻译状态：

- 本文（或部分内容）译自 Arch terminology，最近一次同步于 2024-07-16，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本页面试图揭开 Arch Linux 社区术语的神秘面纱。您可以自由的添加或更改任何术语，但是请使用某个章节的编辑选项。如果想添加新术语，请去原文界面编辑，然后翻译到此界面。

注：本文为规避后续争议，术语权且采用与原文一致的顺序。

ABS

ABS 是 Arch Build System 的缩写。

Arch Linux

应该用下面术语指代 Arch：

- **Arch Linux**
- **Arch** （省略 Linux）
- **archlinux** （UNIX 名）

Archlinux、ArchLinux、archLinux、aRcHlInUx 等称呼都是不标准的。

'Arch' 在 "Arch Linux" 中的官方读音是 /ɑːrtʃ/。"arch" 的读音不同于 "ark"：和单词 "archer" 或 "archnemesis" 中的一样，与 "archangel" 中的不同。 近似拼音 "a er qi" 的读音，不读作 "a er ke"。

Arch Linux 档案

Arch Linux Archive (ALA)，故称 Arch Linux Rollback Machine (ARM)，保存历史上的官方软件仓库快照、ISO 镜像和 boot straps 压缩包。

AUR

Arch User Repository（Arch 用户仓库） 是一个社区驱动的 Arch 用户仓库。其储存包的描述信息（PKGBUILD），后者允许您用 makepkg，从源码构建包并通过

pacman 安装。AUR 的创建是为了整理和分享来自社区的新包，并促使 extra 仓库尽快收录受欢迎的包。

许多进入官方仓库的新包最先收录于 AUR。AUR 中，用户可以贡献自己的包构建（PKGBUILD 及相关文件）。AUR 社区可以对 AUR 中的包投票。如果一个包足够受欢迎——前提是它要有一个兼容的许可证，和良好的封装——它可能进入 *extra* 仓库（可以直接通过 pacman 或 ABS 访问）。

您可以在这里访问 *A*rch Linux *U*ser Community *R*epository。

bbs

Bulletin **b**oard **s**ystem，在 Arch 中指用户支持论坛。

core/\[core\]

core 仓库包含 Arch linux 的最基本软件包：一个命令行系统的所需都在 \[core\] 中。

custom/user repository

任何人都可以创建供其它人使用的在线仓库。要这么做，需要一批软件包及对应的兼容 pacman 的数据库文件。把这些托管到线上，其他人就可添加你的仓库并使用了。

参考 Custom local repository.

Developer

這些半神級的開發者，無償地為 Arch 持續改進。他們的地位僅次於 Arch 的神祇 Judd Vinet 和 Aaron Griffin，而這兩位神祇又只能屈居於「玉米餅（tacos）」之下。

extra/\[extra\]

Arch 的官方软件包很精简，但是我们提供了更大更完整的 "附加" 软件仓库。其包含大量非 core 的软件包。此仓库在社区的强力支持下日益壮大。桌面软件环境，窗口管理器和常用程序都位于此仓库。

initramfs/initrd

参见 mkinitcpio。

KISS

Keep It Simple, Stupid 的简写。简约是 Arch Linux 坚持的原则。

makepkg

makepkg 会读取 PKGBUILD 文件，然后按脚本编译软件包。所需的是 Linux 编译环境，curl包 和一些编译脚本。基于脚本构建的优点是一劳永逸。有了编译脚本后，只

需执行 makepkg，它会执行剩余的工作：下载并验证源代码，检查依赖关系，配置构建时间，构建软件包，安装软件包到临时根目录，进行定制，生成元数据，然后打包供 pacman 使用。

namcap

namcap 是软件包分析工具，可以检查 Arch Linux 软件包及其 PKGBUILD 文件。 可以按规则检查文件列表、文件本身或单独的 PKGBUILD 文件。

规则检查会返回三类消息：错误（error），警告（warning）或信息（information）（类似提示或注释）。错误（'E:'）是 namcap 非常确信错误并需要修复的东西。警告（'W:'）是 namcap 认为应当更改，但如果您清楚您在做什么，就可以忽略的东西。信息（'I:'）只在您使用 info 参数时显示。信息类消息提供无需更改任何东西但可能有用的的信息。

package

请参考 Pacman#安装软件包。

注意：不同 GNU/Linux 发行版使用不同的包和包管理器，这意味着您不能用 pacman 在 Arch 上安装一个 Debian 软件包。

软件包维护者

软件包维护者的作用是：当上游新版本可用时，更新包；处理有关指定包中 bug 的支持问题。该术语适用于：

- Arch 员工，故称受信用户（Trusted User，TU），管理 *extra* 仓库中的包，并监督 AUR。他们由现有软件包维护者以多数票任命，并遵循 AUR 软件包维护者指南和软件包维护者章程。
- 在官方仓库（尤其是*core*）维护包的开发者。
- 在 AUR 维护 PKGBUILD 的所有用户。

某个包的维护者是指此人当前对这个包负责。之前的维护者应当在 PKGBUILD 中，与其他对此包有贡献的人一同，作为贡献者列出。

pacman

请参考 pacman。

PKGBUILD

PKGBUILD 是构建 Arch Linux 软件包所使用的脚本。详情参考 Creating packages。

仓库/repo

软件仓库包含一个或（通常地）多个基于 PKGBUILD 预构建的软件包。官方仓库分成多个部分以便管理。Pacman 使用这些仓库查找和安装软件包。一个仓库可以是本地的（在您自己的电脑上），也可以是远程的（先下载软件包再安装）。

RTFM

RTFM 代表 "*R*ead *T*he *F*riendly *M*anual"（参阅友好文档）。很多 Linux/Arch 新用户在询问一个程序文档明确定义的问题时，会收到这个简洁回复。

通常，在用户在发问前，未曾尝试自己寻找解决方案时，这个缩写会出现。如果有人这么回复您，请注意：这并不是侮辱，只是委婉地请您更细心些，同时对您显现出的疏忽表示略微不满。

收到这个信息后，最正确的动作是阅读手册页面。要阅读某个特定程序的手册：`man *程序名称*`,。

如果没有找到需要的信息，还可以查看下面内容：

- 搜索 wiki
- 搜索论坛
- 搜索邮件列表
- 搜索 web

注：以上链接直接出自英语原文，限于特殊情况，某些可能无法访问。读者请自行变通。

测试仓库

主要的软件包/更新在正式发布前，会放在此仓库进行测试，查看是否有 bug 和升级问题。默认禁用，可在 `/etc/pacman.conf` 中启用。

Arch 之道

Arch Linux 原则的一个非正式传统说法。

受信用户（TU）

参见 #软件包维护者。

Wiki

我们的 wiki 一个存放 Arch Linux 文档的地方，任何人都可以对这些文档做贡献。

1.8 镜像源

难度 进阶 | **篇幅** 9.3 KB | **所属** 关于 Arch

本节将讲到

- 官方镜像
- 启用特定的镜像
- 镜像排序
- 服务端排序
- 问题解决

本页面说明如何选择和配置镜像，以及列出可用的镜像。

官方镜像

可以通过软件包 pacman-mirrorlist包 获取官方镜像列表。若要想获得更新的镜像列表，请访问 Pacman Mirrorlist Generator 页面。

访问镜像状态页面来检查镜像的状态。这里推荐使用最新的镜像，或者说是已同步的镜像。

如果想要将自己的镜像添加到官方列表中，参见 arch:DeveloperWiki:NewMirrors。同时，请将其添加至非官方镜像页面。

支持IPv6的镜像

Pacman Mirrorlist Generator 也可以用来获取支持IPv6的镜像列表。

启用特定的镜像

想要启用镜像，打开 `/etc/pacman.d/mirrorlist` 并定位到你的地理区域。对您想使用的镜像取消注释。例如：

```
# Any
# Server = http://mirrors.kernel.org/archlinux/$repo/os/$arch
Server = https://mirrors.kernel.org/archlinux/$repo/os/$arch
```

参见按照速度排序查看帮助选择镜像的工具。

提示：

- 取消5个你最喜欢的镜像的注释，把他们放在 mirrorlist 文件最上方。这样你就很容易找到它们并且如果第一个镜像出问题可以很容易切换。这也让合并 mirrorlist 更新更容易。
- HTTP 镜像比 FTP 快，因为 HTTP 可以保持连接，而使用 FTP 时 *pacman*每下载一个新软件包就需要重新建立连接，这样也就导致了短暂的停顿。
- HTTPS 镜像比 HTTP 和 FTP 更难受到干扰和篡改，在部分地区的可用性会更高。

也可以在 `/etc/pacman.conf` 中指定镜像。对于 *\[core\]* 仓库，默认设置是：

```
[core]
Include = /etc/pacman.d/mirrorlist
```

如果想要使用 *HostEurope* 镜像作为默认镜像，请把它添加在 `Include` 行之前：

```
[core]
Server = http://ftp.hosteurope.de/mirror/ftp.archlinux.org/core/os/$arch
Include = /etc/pacman.d/mirrorlist
```

pacman 会首先尝试链接这个镜像。如果需要的话，可以继续修改*\[testing\]*, *\[extra\]*, 和 *\[community\]*部分。

注意：如果镜像直接在 `pacman.conf` 中声明，记得在所有的仓库使用同样的镜像。否则不相容的包就可能被安装。如 *\[core\]* 中的 linux 和 *\[extra\]* 中的旧的内核模块不相容。

强制 pacman 刷新软件包列表

镜像可能不同步，并且从旧镜像中获取的的软件包列表也可能与新镜像中的软件包列表不一致，即使列表里的日期可能表明它们是一致的。

创建和编辑 `/etc/pacman.d/mirrorlist` 之后，使用下面命令强制更新镜像：

```
# pacman -Syyu
```

传入两次 `--refresh` 或 `-y` 将强制更新所有软件包列表，即使系统认为它们已经是最新。**每次修改镜像之后都应该使用 `pacman -Syyu`**。

警告：在大多数情况下，如果你强制更新 pacman 数据库，你将需要强制降级任何可能太新的软件包以对应新的镜像所提供的版本。这可以避免软件包升级不一致导致的部分更新问题。

```
# pacman -Syyuu
```

当正在使用同步成功的镜像或者检查镜像的 **lastsync** 文件来以确保软件包列表已经更新后，则没有必要使用这条命令。

镜像排序

当下载软件包时，pacman 按照 `/etc/pacman.d/mirrorlist` 中的顺序来使用镜像。服务器在列表中出现的顺序决定了其优先级。

由于速度最快的服务器可能是不同步的，所以仅基于速度来对镜像进行排序很可能不是最佳方案。取而代之的是，先制作一个按照速度排序的镜像列表，然后根据他们的状态来删除那些不同步的镜像。

建议定期重复这一过程，来保持镜像列表同步。

按速度排序

排序已存在的镜像列表

软件包 pacman-contrib包 提供了一个 Bash 脚本，`/usr/bin/rankmirrors` 可以根据镜像建立连接的速度对其进行排序，来使用最快的本地镜像。

首先，备份已存在的 `/etc/pacman.d/mirrorlist`：

```
# cp /etc/pacman.d/mirrorlist /etc/pacman.d/mirrorlist.backup
```

接下来准备 `/etc/pacman.d/mirrorlist.backup` 以使用 *rankmirrors* 进行排序，可执行以下操作：

- 编辑 `/etc/pacman.d/mirrorlist.backup` ，将准备测速的镜像取消注释。
- 如果文件中的服务器已按照国家或地区分组，则可通过以下命令提取特定国家或地区的所有服务器：

```
    $ awk '/^## Country Name$/{f=1; next}f==0{next}/^$/{exit}{print substr($0,
1);}' /etc/pacman.d/mirrorlist.backup
```

- 若要取消所有的注释，可执行以下 `sed` 命令：

```
    # sed -i 's/^#Server/Server/' /etc/pacman.d/mirrorlist.backup
```

- 最后，对镜像进行排序，这里使用 `-n 6` 参数来实现只获取 6 个最快的镜像：

```
    # rankmirrors -n 6 /etc/pacman.d/mirrorlist.backup > /etc/pacman.d/mirrorlist
```

获取实时镜像列表并对其排序

可以从 Pacman Mirrorlist Generator 获取一些国家/地区的最新镜像列表来使用 *rankmirrors*。 下面的命令会获取位于*中国*且支持 *https* 协议的最新镜像的列表，然后将位于列表的服务器取消注释，最后再对其排序后打印输出 5 个最快的镜像。

```
$ curl -s "https://archlinux.org/mirrorlist/?
country=CN&protocol=https&use_mirror_status=on" | sed -e 's/^#Server/Server/' -e '/
^#/d' | rankmirrors -n 5 -
```

提示：可以通过使用任何基于文本的浏览器（比如 elinks(1)）导航至 `[https://archlinux.org/mirrorlist](https://archlinux.org/mirrorlist)` 来以交互式方式完成这个过程。

现已存在的工具：

Rate Mirrors — 此工具通过获取镜像，跳过不同步或正在同步的镜像，然后使用关于海底电缆和互联网交换的信息在国家或地区之间切换，最终找到快速的镜像。并且足够快以实现在每次更新以前运行。它也支持通过标准输入自定义镜像列表。

: https://github.com/westandskif/rate-mirrors || rate-mirrorsAUR, rate-mirrors-binAUR

服务端排序

官方的 Pacman Mirrorlist Generator 提供了一个简单的方式来获取一个排序过的镜像列表。因为所有排序是在考虑了多种因素的单一服务器上完成的，与在每个单独的客户端上进行排序相比，镜像服务器和客户端的负载量就要低很多。

Reflector — 从镜像状态页面取回最新的镜像列表，过滤后通过速度排序再覆盖写入 `/etc/pacman.d/mirrorlist`

: https://xyne.dev/projects/reflector/ || reflector包

问题解决

镜像列表缺失

如果你遇到以下错误：

```
error: config file /etc/pacman.d/mirrorlist could not be read: No such file or directory
```

可以直接从网站上获取镜像列表：

```
# curl -o /etc/pacman.d/mirrorlist https://archlinux.org/mirrorlist/all/
```

如果碰到 `$arch` 变量未定义的问题，请在 `/etc/pacman.conf` 中加入：

```
Architecture = auto
```

请确认已如上文对首选的镜像取消注释，然后执行：

```
# pacman -Syu pacman-mirrorlist
```

异常的镜像

如果你确定一个镜像运行不正常，并且其没有反映在镜像状态页面上，更换镜像并考虑开启一个 bug 报告。关于镜像的问题，issue 应该发在 Arch Linux 项目上（位于登录框下方的列表，或者通过在页面上搜索 switch）

参见

- GitHub archweb mirrorlist.py - source code of the archweb mirrorlist generator

1.9 Arch 构建系统

难度 进阶 | **篇幅** 10.1 KB | **所属** 关于 Arch

本节将讲到

- 工具链
- 用例
- 用法
- 技巧

Arch 构建系统（Arch build system，ABS）是一套从源码构建并打包软件包的系统。在 Arch 中，pacman 专门管理二进制软件包（包括那些由 ABS 创建的）；而 ABS 则是一系列工具，负责把源代码编译成可安装的 *.pkg.tar.zst* 软件包。

Arch 构建系统类似于 *\BSD 上的 ports，其用于自动化从源码构建软件包的过程。*Port *可以自动下载源代码、解压缩、打补丁、编译和安装软件。一个* port* 仅仅是指用户

电脑上的一个目录，该目录根据即将安装的软件来命名，它包含一些能指导源码的下载和编译安装的文件。Port 系统让你只需在 port 目录下运行 `make` 或 `make install clean` 就能安装你想要的软件。

ABS 的概念与 Ports 相似。它由为每个 Arch Linux 可用软件包提供的 git 仓库组成。每个目录中并不包含二进制包或源代码，而是包含一个 PKGBUILD 文件（有时也会有其它文件）。在有 `PKGBUILD` 文件的目录里运行 makepkg 命令，系统就会在目录中下载软件的源代码、编译并打包在构建文件夹里。然后就可以通过 *pacman* 进行安装或升级了。

工具链

'ABS' 可以作为一个总括性术语来使用，因为它包含并依赖于若干其他部件。因此，尽管从严格意义上来讲并不精确，ABS可指代包含以下工具的完整工具集：

Arch 构建系统包含并依赖于数个组件和工具，分别用于从源码到构建出软件包的各个步骤中：

仓库树：目录树包含构建所有官方软件包所需的文件，但不包括软件包本身和源代码。这些仓库以 Git 仓库的形式托管在 gitlab.archlinux.org 上。更多详细信息请参阅#仓库结构。

PKGBUILD：Bash 脚本，包含软件的源代码的 URL 和编译打包指令。

makepkg：一个 shell 命令工具，会读取 PKGBUILD，自动下载源码、编译并创建 *.pkg.tar** 包（拓展名由 `makepkg.conf` 中的 `PKGEXT` 指定）。makepkg 也可以用来从 AUR 或第三方代码构建你自己的自定义软件包，具体细节请参考创建软件包。

pacman：pacman 是完全独立的一个系统，但是它在通过 makepkg 或手动安装或移除软件包、解决依赖关系时都是必需的。

`pkgctl`/devtools包：`devtools` 是用于为 Arch Linux 发行版构建和维护官方仓库软件包的一系列工具。pkgctl(1) 是一个高层级工具，用于帮助从软件源码仓库构建 Arch Linux 软件包并发布到二进制软件仓库中。

AUR：Arch 用户软件仓库（AUR）的 PKGBUILD 独立于官方仓库，但其中的软件包同样可以使用 ABS 的工具进行打包构建。它包含成千上万的用户贡献的 PKGBUILD，来提供 Arch 官方仓库中没有的软件。如果需要编译官方 Arch 树之外的软件包，AUR 中已经存在的可能性非常大。

警告：官方 PKGBUILD 假定包是在干净的 chroot 环境中构建的。在“脏”的环境中构建可能会失败或者在运行时有意外行为。因为如果编译系统动态检查依赖的话，编译结果会受到当前系统可用包的影响。

仓库结构

每个软件包在 Arch Linux GitLab 服务器的 archlinux/packaging/packages 命名空间下都有其独立的*源码*仓库。每个仓库都包含了官方打包所使用的 PKGBUILD 等文件。另外，这里还存放了开发者在打包过程中所使用的一些文件。

例如，acl包 的目录结构是这样的：

```
acl
├── keys
│   └── pgp
│       ├── 259B3792B3D6D319212CC4DCD5BF9FEB0313653A.asc
│       ├── 600CD204FBCEA418BD2CA74F154343260542DF34.asc
│       └── B902B5271325F892AC251AD441633B9FE837F581.asc
├── PKGBUILD
└── .SRCINFO
```

源代码并不直接包含在ABS目录中，而是构建时从 `PKGBUILD` 里指定的源代码 URL 下载。

在构建完官方软件包后，它会被发布到官方仓库之一：*core*，*extra*，*multilib* 或是先放到测试仓库中。这些仓库存放着*二进制文件*，并不存放在 GitLab 上，而是由镜像源进行分发。

用例

Arch 构建系统可以将从源码构建出软件包的一些工作自动化，常见用法包括：

- 需要编译或重新编译软件包
- 由开发者和打包人员为 Arch Linux 构建官方二进制软件包
- 从源代码编译并安装 Arch 官方源里没有的软件（详情请参照创建软件包）
- 定制现有的软件包以满足你的特定需求（比如开启或禁用相关选项、打补丁）
- 用你的编译器的 flags 重新构建整个系统，“就像 FreeBSD 那样”
- 干净地编译安装你自己定制的内核。（参照内核编译）
- 使内核模块（比如某些显卡驱动）在你定制的内核下正常工作
- 修改 PKGBUILD 中的版本就能方便地编译和安装新的、老的、beta 或者开发版本的 Arch 软件包

用法

获取 PKGBUILD 源码

要想获取从源代码构建特定软件包所需的 PKGBUILD 文件，可以使用 *pkgctl* 工具，或者直接使用 Git。

使用 pkgctl 工具

如需使用 *pkgctl*，请先安装 devtools包。*pkgctl* 是一个帮助为 Arch Linux 构建软件包源文件的工具。

要使用 *pkgctl* 获取包含软件包 *pkgname* 最新构建文件的 git 仓库，请使用以下命令：

```
$ pkgctl repo clone pkgname
```

提示：默认情况下通过 SSH 获取，如果没有在 Arch GitLab 账户中设置 SSH 密钥，则需要通过 HTTPS 获取：`pkgctl repo clone --protocol=https *pkgname*`

请注意，这里所说的构建文件是指 PKGBUILD，可能还包括一些其他必要的文件，如密钥。也就是说，是 ABS 所需的基本文件，而不是由包的开发团队编写的源代码文件，例如 C 或 Python 文件。

该命令不仅会提供当前的构建文件，还会提供它的历史版本。此外，还可以使用其他 git 命令来签出旧版本或跟踪更改。

如果想获取特定版本，可以使用以下命令：

```
$ pkgctl repo clone --switch="2:1.19.5-1" go
```

如需获取更多详细信息以及其他可用命令，请阅读 pkgctl-repo-clone(1)

直接使用 git

使用以下命令来获取软件包：

```
$ git clone https://gitlab.archlinux.org/archlinux/packaging/packages/pkgname.git
```

例如获取 Apache 的构建文件：

```
$ git clone https://gitlab.archlinux.org/archlinux/packaging/packages/apache.git
```

构建软件包

关于如何配置 *makepkg* 来从 PKGBUILD 构建软件包，请参考 makepkg#配置。

然后把 PKGBUILD 所在目录复制到新的位置。在新目录按需要进行修改。 并按照 makepkg#使用来构建和安装软件包。

技巧

保留修改过的软件包

pacman 进行升级时会将修改后的软件包升级到仓库中的最新版本，可以通过下面方式避免这个行为：

在 PKGBUILD 中将软件包加入 `modified` 组：

```
PKGBUILD
groups=('modified')
```

然后将此组加入 `/etc/pacman.conf` 的 `IgnoreGroup`：

```
/etc/pacman.conf
IgnoreGroup = modified
```

当系统升级发现官方仓库中有新版本时，pacman 会显示软件包因为在 *IgnoreGroup* 中而被忽略的提示，这时需要从 ABS 编译更新的软件包以防止部分升级。

1.10 ASCII 艺术

难度 进阶 | **篇幅** 11.4 KB | **所属** 关于 Arch

本节将讲到

- 软件
- Arch ASCII 徽标
- Cowsay
- Ponysay
- 另请参见

软件

- **Asciiquarium** — ASCII 艺术中的水族馆/海洋动画。

: https://robobunny.com/projects/asciiquarium/ || asciiquarium包

- **boxes** — 文本模式框和注释绘图过滤器。

: https://boxes.thomasjensen.com/ || boxesAUR, boxes-gitAUR

- **FIGlet** — 一种用普通文本制作大字母的程序。

: http://www.figlet.org/ || figlet包

- **jp2a** — 一个小的实用程序，用于将 JPG 和 PNG 图像转换为 ASCII。

: https://csl.name/jp2a/ || jp2a包

- **TOIlet** — 自由的 FIGlet 实用程序替代。

: http://caca.zoy.org/wiki/toilet || toiletAUR

有关图表，请访问 https://asciiflow.com/ 。

Arch ASCII 徽标

请参阅原始论坛主题。

```
 \e[H\e[2J
          \e[0;36m.
         \e[0;36m/ \
        \e[0;36m/   \       \e[1;37m               #     \e[1;36m| *
       \e[0;36m/^.   \      \e[1;37m a##e #%" a#"e 6##%  \e[1;36m| | |-^-. |   | \ /
      \e[0;36m/  .-.  \     \e[1;37m.oOo# #   #    #  #  \e[1;36m| | |   | |   |  X
     \e[0;36m/  (   ) _\    \e[1;37m%OoO# #   %#e" #  #  \e[1;36m| | |   | ^._.| / \
\e[0;37mTM
```

```
     \e[1;36m/ _.~   ~._^\
    \e[1;36m/.^         ^.\ \e[0;37mTM
 \e[H\e[2J
           \e[1;36m.
          \e[1;36m/#\
         \e[1;36m/###\       \e[1;37m                #     \e[1;36m| *
        \e[1;36m/p^###\      \e[1;37m a##e #%" a#"e 6##%  \e[1;36m| | |-^-. |   | \ /
       \e[1;36m/##P^q##\     \e[1;37m.oOo# #   #    #  #  \e[1;36m| | |   | |   |  X
      \e[1;36m/##(   )##\    \e[1;37m%OoO# #   %#e" #  #  \e[1;36m| | |   | ^._.| / \
\e[0;37mTM
     \e[1;36m/###P   q#,^\
    \e[1;36m/P^         ^q\ \e[0;37mTM
```

来自 screenfetch包 3.6.5-1:

```
                  -`
                 .o+`
                `ooo/
               `+oooo:
              `+oooooo:
              -+oooooo+:
            `/:-:++oooo+:
           `/++++/+++++++:
          `/+++++++++++++:
         `/+++ooooooooooooo/`
        ./ooosssso++osssssso+`
       .oossssso-````/ossssss+`
      -osssssso.      :ssssssso.
     :osssssss/        osssso+++.
    /ossssssss/        +ssssooo/-
  `/ossssso+/:-        -:/+osssso+-
 `+sso+:-`                 `.-/+oso:
`++:.                           `-/+/
.`                                 `
```

来自 alsiAUR 0.4.7-1:

```
                  ##
                 ####
                ######
               ########
              ##########
             ############
            ##############
           ################
          ##################
         ####################
        ######################
       ########      #########
      ##########     ##########
     ###########    ###########
    ##########        ##########
   #######              #######
  ####                    ####
 ###                        ###
```

来自 archey3包 0.5-2:

```
               +
               #
              ###
             #####
             ######
            ;  #####;
           +##.#####
          +##########
         ############;
        ################+
       #######    #######
     .######;      ;###;`".
    .#######;      ;#####.
    #########.    .########`
```

```
   ######'           '######
  ;####                 ####;
  ##'                     '##
 #'                         `#
```

ArchWiki 的生日庆祝：

```
                                                    ,- " " " -.
                                                   /       _  \
                                                  :      / |     ;      ,- " " "-.
                                                  \      | |     /     /   ____    \
                                                   \     |_|    /     ;    |  |     ;
                                                     `.. ,, ..´       \    |__|    /
                                                     /✽\               \          /
                                                      )                 `. ,, .´
  _                 _  __          ___ _    _       (                   /✽\
 / \    _ __ ___ _| |_\ \        / (_) | _(_)         )                   (
/ _ \  | '__/ __| '_ \ \ /\ / /| | |/ / |           (                     )
/ ___ \| | | (__| | | \ V  V / | |   <| | ..... ) .*..---...*.....*
/_/   __|  ___|_| |_|_/_/  |_|_|__| ....♫♪happy birthday edition ♫
```

用户创作的 Arch ASCII 徽标：

```
                   -@
                  .##@
                 .####@
                 @#####@
               . *######@
              .##@o@######@
             /##############@
            /################@
           @#######@**%#######@
          @######`      %######o
         @#######@        ######%
       -@########h        ######@.`
      /######h**``        `**%@#####@
     @H@*`                      `*%#@
    *`                             `*
```

Cyllos 创作

```
\033[38;2;23;147;209m
```



LnLcFlx 创作

Cowsay

这是一些 cowsay包 和 fortune 一起使用的例子：

带有 fortune 的默认的牛

- 结合程序 cowsay包：

```
$ cowthink $(fortune)
 _________________________________________
( The best cure for insomnia is to get a )
( lot of sleep. -W.C. Fields              )
 -----------------------------------------
        o   ^__^
         o  (oo)_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

- 显示潜在的进攻性 fortune：

```
$ cowsay $(fortune -o)
 _____________________________________
< Chastity is its own punishment. >
 -------------------------------------
        \   ^__^
         \  (oo)_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

ASCII 图像由位于 `.cow` 中的 `/usr/share/cows` 文本文件生成，并且所有主题都可以使用 `cowsay -l` 列出。可以根据用户的喜好编辑这些文件。自定义图像也可以从头开始创建，也可以在网上找到。创建自定义 cow 文件的最简单方法是使用现有的 cow 文件作为模板。要测试自定义文件：

```
$ cowsay -f /path/to/file $(fortune)
```

带有 fortune 的随机的牛

```
$ fortune -c | cowthink -f $(find /usr/share/cows -type f | shuf -n 1)
 ____________________________________________________
( (computers) % A language that doesn't affect the   )
( way you think about programming is not worth       )
( knowing.                                           )
 ----------------------------------------------------
        o       (__)
         o      /oo|
          o    (_"_)*++++++++++*
                    //I#\ \ \ \ \ \ \   I \
                   I[I|I | | | | |   I I `
                   I`I ' /  /  /  '  '     I I
                   I I                       I I
                  ~ ~                      ~ ~
                        Scowleton
```

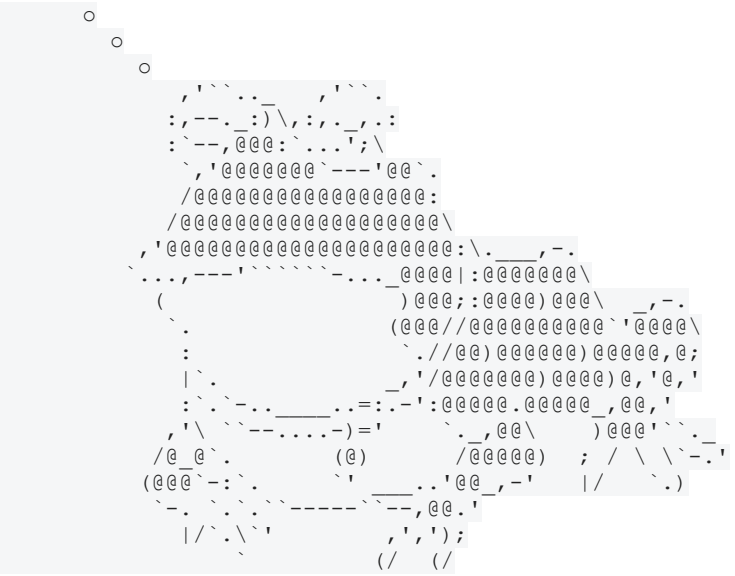
带有 fortune 的随机定制牛

可以将复杂的命令链接起来，以产生详细的 ASCII 文字，例如此 Futurama 示例。

- 显示带有随机面部表情的随机牛，并包装长行的 fortune 文本：

注意：随机选择的牛实际上是从具有自定义 ASCII 艺术的数据库中选择的蟾蜍

```
$ fortune -a | fmt -80 -s | $(shuf -n 1 -e cowsay cowthink) -$(shuf -n 1 -e b d g p s t
w y) -f $(shuf -n 1 -e $(cowsay -l | tail -n +2)) -n
 ________________________________________________
( Fry: I must be a robot. Why else would human   )
( women refuse to date me?                       )
 ------------------------------------------------
```



Ponysay

对于完整的 256 色类似于 cowsay 的艺术品，请使用 ponysay包（3.0 版本具有 422 个小马）。语法是相同的，`$ ponysay *message*` 可以让小马说些话，`ponysay -l` 显示完整的小马列表。要选择显示的小马，请运行 `$ ponysay --pony *x* *"message"*`，其中 `*x*` 是要显示的小马。要创建更多小马，请使用 util-say-gitAUR 并将它们分别存储在桌面和 TTY 的 `~/.local/share/ponysay/ponies` 和 `~/.local/share/ponysay/ttyponies/` 中。

另请参见

- Arch Linux 图稿
- /etc/issue "2.0" - 我的 ASCII 艺术版本，带有 Arch 徽标和文本

1.11 官方仓库

难度 进阶 | **篇幅** 13.5 KB | **所属** 关于 Arch

本节将讲到

- 稳定仓库
- testing 仓库
- staging repositories
- 历史背景

软件仓库是获取软件包的地方。

Arch Linux **官方软件仓库**包含着基础的和较为流行的软件。他们可通过Pacman便捷的获取。软件包由软件包维护者维护。

官方软件仓库中的软件包会持续更新，即老版本将被新版本所替代。Arch 没有主版本的概念：每个软件在上游有新版本时单独更新。每一个软件仓库都是互相联系的，即，软件仓库中的所有软件都是相互兼容的。

稳定仓库

core

core 仓库位于仓库镜像的 `.../core/os/` 目录。

core仓库包含着用于以下用途的软件包:

- 启动 Arch 系统。
- 连接网络
- 创建软件包
- 检查、管理和修复所支持的文件系统
- 系统安装（例如openssh包）。

以及上述软件包的依赖（不包括构建软件包所需的依赖），以及 base包 元软件包。

core 仓库对包有着相当严格的质量要求。软件包的更新需要经过开发者和用户的审察批准。对于低使用率的软件，适当的审核足矣：发布关于更新的通知，请求审核，根据更新的重要性在 core-testing 仓库暂留至多一周，并在确保无未解决bug报告后，由软件包维护者审核通过。

提示：如需在没有网络连接的情况下从 *core* （或其他软件仓库）创建本地软件仓库，见 Pacman/Tips and tricks#Installing packages from a CD/DVD or USB stick

extra

extra 仓库位于您最喜欢的仓库镜像的 `.../extra/os/` 目录。

*extra*包含不适合在*core*仓库的所有软件包，由 Trusted Users 和 Arch 开发者共同维护。 比如：Xorg，窗口管理器，网络浏览器，媒体播放器，程序语言支持Python和Ruby等等。

multilib

multilib 仓库位于您最喜欢的仓库镜像的 `.../multilib/os/` 目录。

multilib 包含着32位的软件和链接库，可以用于在64位系统上运行和构建32位软件，例如 wine包 steam包 等。

启用multilib

想使用 multilib 仓库，请在`/etc/pacman.conf`文件中取消 `[multilib]` 段落的注释：

```
/etc/pacman.conf
[multilib]
Include = /etc/pacman.d/mirrorlist
```

然后更新系统

提示：运行 `pacman -Sl multilib` 来列出在*multilib*仓库里的所有软件包，32位链接库的软件包以 `lib32-` 开头

禁用multilib

运行下面命令可以删除所有从 *multilib* 安装的软件:

```
# pacman -R $(comm -12 <(pacman -Qq | sort) <(pacman -Slq multilib | sort))
```

如果有 gcc-libs 冲突，请重新安装 gcc-libs包和base-devel包的依赖：

在 `/etc/pacman.conf` 中注释掉 `[multilib]` 段落：

```
/etc/pacman.conf
#[multilib]
#Include = /etc/pacman.d/mirrorlist
```

然后更新系统

testing 仓库

testing 仓库的目的是提供一个即将被放入主软件仓库的软件包的集散地。软件包维护者（和普通用户）可以访问并测试这些软件包以确保软件包没有问题。当位于 *testing* 仓库的软件包被测试无问题后，即可被移入主仓库。

不是所有的包都需要经过如此的测试过程，满足如下条件的软件包会进测试仓库：

- 一切需要进入 *core* 仓库的软件包都需要先经过 *core-testing* 仓库
- 更新该软件包可能损坏系统，需要进行测试。
- 会影响到大量其它的软件包，例如 perl包 或者 python包。

大的软件集合，如GNOME 或 KDE 更新时会更新大批软件包，也需要进行测试。

注意：testing 库并不是“最新”软件包的仓库。它的目的是存放一些由于是\[core\]软件集合的一部分或者由于其它方面的问题，可能引起**系统崩溃**的软件包更新。我们**强烈建议**使用 testing 库的用户订阅arch-dev-public邮件列表，监视testing 仓库论坛，并报告bug。你也应该考虑加入Arch测试团队

警告：

- 谨慎启用 *core-testing* 仓库，其中的软件包可能损坏系统。只有有足够的经验且知道如何应对问题的用户才可以启用。

- 请同时启用 *core-testing* 仓库和 *extra-testing* 仓库。如果启用了下文的其他 *testing* 仓库，也需要启用 *core-testing* 和 *extra-testing* 仓库
- 由于并不是所有的主仓库软件包都在测试仓库中有相应的版本，core 和 extra 主仓库应该保留，并保证相应的测试仓库在主仓库的前面

core-testing

位于镜像的 `.../core-testing/os/` 目录。

core-testing 包含要进入 *core* 仓库的候选软件包。

core-testing 库是唯一可以与其它官方软件仓库有名称冲突的仓库。如果要启用，应该在 `pacman.conf` 文件里把它设置为第一个仓库。

extra-testing

*extra-testing*库的功能类似 *core-testing* ，不过是为 *extra* 库服务的测试仓库。

multilib-testing

multilib-testing 库的功能类似 *core-testing* ,不过是为 *multilib-testing* 库服务的测试仓库。

gnome-unstable

GNOME 桌面环境的下一个稳定版本，在进入主 *extra-testing* 仓库前，会先放到 *gnome-unstable*.

要启用它，在 `/etc/pacman.conf` 中加入:

```
/etc/pacman.conf
[gnome-unstable]
Include = /etc/pacman.d/mirrorlist
```

gnome-unstable 项需要在所有仓库的最上面（保证在 *core-testing* 上面），即：

```
/etc/pacman.conf
[gnome-unstable]
[core-testing]
[core]
[extra-testing]
[extra]
```

如果有软件包相关的问题，请在 Arch bug tracker 汇报，其它问题请上游的 GNOME Gitlab汇报。

kde-unstable

包含 KDE Plasma 和应用程序的 *beta* 或*候选发行版本*.

要启用它，将下面内容加入 `/etc/pacman.conf`:

```
/etc/pacman.conf
[kde-unstable]
Include = /etc/pacman.d/mirrorlist
```

同样地，*kde-unstable* 项需要在所有仓库的最上面（保证在 *core-testing* 上面）。

请报告遇到的问题。

禁用测试仓库

如果要禁用之前启用的测试仓库，应该：

1. 从 `/etc/pacman.conf` 中删除(注释）掉配置
2. 执行 `pacman -Syuu` 将从这些仓库安装的软件还原到主仓库版本。

第二条为可选操作，如果在第一步后碰到任何问题，请先执行此操作。

staging repositories

警告：不要因为任何原因启用 *staging* 仓库，因为你的系统一定会在更新后损坏。这个仓库仅为后端开发人员使用

该存储库包含损坏的包，并且在一次重建多个包时仅供开发人员使用。 例如，为了重建依赖于新共享库的包，必须首先构建共享库本身并将其上传到临时存储库，以便其他开发人员可以使用。 一旦所有依赖包都被重建，这组包就会被转移到测试或主存储库，以更合适的方式.

详情请见 staging 仓库发布通知。

历史背景

大部分仓库是因为历史原因分开的。原本，当这个发行版只有一小部分用户时，是只有一个软件仓库的，它就是现在的 **core** ——那时候它叫做 **official** 。这个仓库主要存放了Judd Vinet选定的应用程序，当然现在已经不是这样了：它被设计为只包含“每种类型”程序中的一个 —— 一个桌面环境、一个主浏览器等等。

后来有用户不满Judd的选择，由于ABS系统使用简单，所以他们就用ABS来创建自己的软件包。这些软件包被放入一个名为**unofficial**的软件仓库，而这个仓库由除Judd以外的开发人员维护。最终，新仓库获得开发人员同样的支持，所以就不再使用 official 和 unofficial 这两个名称了。大约在0.5版本的时候，两个仓库更名为 **current** 和 **extra** 。

在2007.8.1版本发布之后， current 更名为 core ，以免让人误解它包含的内容。如今，这两个仓库在开发人员和社区眼中都是相同分量的。不过 core 还是有些不同的，主要区别在于安装光盘和发布的快照中的软件包都只在 core 当中。这个仓库可以实现一个完整的Linux系统，但并不一定是你想要的Linux系统。

在0.5或者0.6版本的时候，仓库里有大量的软件包没有开发者愿意去维护。于是一位开发人员（Xentac）建立了“受信用户仓库”（Trusted User Repositories），作为存放

被信任的用户（TU）自行创建的软件包的仓库。**staging**仓库里的软件包可以被Arch Linux的开发人员选拔入官方仓库，不过除此之外，开发人员和受信用户或多或少还是有所不同的。

就这样过了一段时间，逐渐的受信用户对他们的软件仓库感到厌倦，而非受信用户又期望可以将自己的软件包与大家分享。这导致了AUR的出现。慢慢的TU们形成更为严密的组织，如今他们共同维护**community**软件仓库。TU是一个独立于Arch Linux开发人员之外的组织，两者并没太多交流。不过，热门的软件包仍然会偶尔从 community 选拔入 **extra** 。AUR也允许非受信用户提交PKGBUILD给其他用户自愿使用。

在2006年， **core**仓库里未被正确编译的内核导致许多用户系统崩溃。之后，“core仓库审核机制”引入：所有\[core\]仓库软件包更新前，必须先在一个叫做 **core-testing** 的仓库进行测试，必须在其他开发者或 Arch Testing Team 成员同意后，软件包才能正式移入 core 。后来， core 中出现一些低使用率的软件，审核机制对它们有所宽松。

2009年末至2010年初，出现了一些新的文件系统，人们希望在安装系统时就使用它们（即纳入 core ）。鉴于 core 从来没有给出明确的界定（只是重要的包，由开发者亲自挑选的）它有了一个更为明确的定义。

2023 年，经过几年的准备工作，Arch 将后端服务迁移到了 git，并同时启用了新的仓库布局。在新的布局中，*extra* 包含了之前 *community* 仓库的软件包，testing 仓库分成了 *core-testing* 和 *extra-testing*, *community-testing* 被删除。 Trusted Users 也有了推送软件包到 *extra* 的权限。

1.12 AUR 提交准则

难度 进阶 | **篇幅** 14.3 KB | **所属** 关于 Arch

本节将讲到

- 提交软件包
- 维护软件包
- 请求

用户可以通过 AUR 分享 PKGBUILD 脚本。AUR 中不包含任何二进制包，仅包含用户上传的 `PKGBUILD`，供其他用户下载使用。这些 `PKGBUILD` 都是非官方的，没有经过完整审查，使用风险自担。

提交软件包

警告：在提交前请先熟悉 Arch 打包准则及所有的“相关文章”。请**仔细检查**上传的内容是否有误，**违反相应规则**的软件包可能会**不经警告被直接删除**。

如果在反复阅读该章节后仍不清楚软件包或构建/提交流程是否正确，可以将自己的 `PKGBUILD` 贴到 AUR 邮件列表、论坛 AUR 版、中文论坛 AUR/ABS/PKGBUILD 版或 IRC 频道，让大家帮您检查。

提交软件包的规则

提交软件包时请遵循下列的规则:

- 任何提交的 `PKGBUILD` 都不能编译**已经**位于**官方二进制软件包仓库**的程序。可通过官方软件包数据库进行查找，如果在其中发现了任意版本的程序，就**不要**进行提交。如果认为官方仓库的软件包已过期，请标记它。如果它有问题或者缺少功能，请反馈反馈 bug 报告。

: **唯一的例外**是和官方打包版本相比增加或开启了**新的功能**或者使用了不同的**补丁**。这时需要修改 `pkgname` 来表示新增的功能。例如加入了边栏支持的 GNU screen（`screen`）应该命名为 `screen-sidebar`。此外还需要同时添加 `conflicts=('screen')` 以避免与官方仓库中的包冲突。

- **检查 AUR** 中是否**已有**相同软件包。如果已经有人维护某软件包，可以以评论的形式将更改报告给维护人员。如果软件包无人维护或维护者无反馈，可以按需将其认领并更新。不要创建重复的包。
- 确保您的软件包**有人需要**。在提交前请仔细想想：有人会用这个软件包吗？它非常特别吗？如果不是只有少数人觉得它有用，就提交它。

: AUR 和官方软件仓库是用于放置通用软件和软件相关的内容的，包括：可执行文件、配置文件、软件的在线/离线文档和软件直接使用的媒体数据。

- 不支持 `x86_64` 架构的软件包不允许上传到 AUR。
- 不要在 AUR `PKGBUILD` 中使用 `replaces`，除非这个软件包将要被重命名（例如当 *Ethereal* 变成 *Wireshark* 时）。如果这个软件包是**已经存在的软件包的另一版本**，使用 `conflicts`（或者如果这个软件包被其他软件需要时，使用 `provides`）。两者的主要区别是：在 pacman 同步（-Sy）之后，如果遇到与 `replaces` 匹配并已经安装的软件包时，pacman 会立刻想去替换它。而 `conflicts` 则在安装软件包时进行替换。

- 请为那些由版本控制系统构建，而不属于任何特定版本的软件包的包名添加合适的后缀，如 `-git`。具体细则见 VCS 软件打包准则#软件包命名。
- 当源码可获得时，那些**预构建**可以直接部署的二进制包应该加上 `-bin` 后缀，Java 除外。除此之外，AUR 不应当包含 `makepkg` 创建的二进制包，也不应当包含文件列表。
- 从特定版本的源码构建的包不应该添加后缀。
- 请按照以下格式在 `PKGBUILD` 最上端加入包含当前**维护者**和过去的**贡献者**的信息**注释**，记得对邮件地址进行必要的处理以避免被垃圾邮件骚扰。然后移除所有不必要的行。例如：

注意：对邮件地址进行混淆会导致别人更难联系上你。

: 如果您从其它人接手了一个 `PKGBUILD`，像这样把您的名字和邮件地址加到最上面。 :```

: 如果在您之前有其他的维护者，请将它们添加到贡献者中。如果你不是初始上传者，也请将其加入贡献者中。如果您是协同维护者，也请加上现在的其他维护者。
: ```
```
# Maintainer: Your name <address at domain dot tld>
# Maintainer: Other maintainer's name <address at domain dot tld>
# Contributor: Previous maintainer's name <address at domain dot tld>
# Contributor: Original submitter's name <address at domain dot tld>
```

- 为仓库添加 `LICENSE` 文件，建议使用 0BSD 许可证。你可以直接从 RFC 0040 复制许可证文本。缺少许可证或使用与 0BSD 不同许可证的软件包不允许被加入到官方软件源中。

认证

要向 AUR 写入软件包，用户需要创建一个 SSH 密钥。将公钥导入到用户账户的 *My Account* 一节，然后为 `aur.archlinux.org` 主机配置对应的私钥，例如：

```
~/.ssh/config
Host aur.archlinux.org
  IdentityFile ~/.ssh/aur
  User aur
```

你应当为 AUR 生成新的密钥，而不是直接使用旧的，以在出问题时直接弃用密钥：

```
$ ssh-keygen -f ~/.ssh/aur
```

提示：在输入公钥时可以通过换行的方式设置添加多个公钥。

创建软件包仓库

如果您正在创建新的软件包，请通过克隆所需的 pkgbase 的方式建立一个远程 AUR 仓库和本地 Git 仓库。如果软件包还不存在，则会出现以下预料之中的警告：

```
$ git -c init.defaultBranch=master clone ssh://aur@aur.archlinux.org/pkgbase.git
Cloning into 'pkgbase'...
warning: You appear to have cloned an empty repository.
Checking connectivity... done.
```

注意：如果 `*pkgbase*` 与被删除的软件包相同，那克隆下来的仓库会不为空。

如果您已经有一个软件包了，但不是 Git 仓库的话，需要进行初始化：

```
$ git -c init.defaultBranch=master init
```

并添加远程 AUR 地址：

```
$ git remote add label ssh://aur@aur.archlinux.org/pkgbase.git
```

然后向远程仓库拉取以在 AUR 进行初始化。

提示：使用拉取和变基来解决 `*pkgbase*` 匹配到已删除软件包的冲突问题。

提交和更新软件包

警告：不想以全局身份提交？记得通过 `git config user.name [...]` 和 `git config user.email [...]` 编辑自己的本地身份！

警告：你的提交会以全局 Git 名称和邮件地址进行署名，且在推送后很难对提交进行修改（FS#45425）。如果你想以不同的身份推送到 AUR，可以使用 `git config user.name "..."` 和 `git config user.email "..."` 对单个包进行修改。

当释出新版本的软件时，更新 pkgver 或者 pkgrel 变量来提示所有的用户更新。如果仅仅是对 PKGBUILD 的小修改（例如修正笔误），请不要更新这些变量。

不要仅仅为了更新 `pkgver` 就更新 VCS 软件包。当上游有新的提交时，他们不会被视为过时。只有当有其他变化，比如打包流程发生变化时，才应该进行更新。

无论何时 `PKGBUILD` 的元数据发生变动（例如更新了 `pkgver()`），都需要重新生成 .SRCINFO。否则 AUR 不会显示更新后的版本号。

要上传或者更新一个软件包，至少要添加 `PKGBUILD` 和 `.SRCINFO` 和其它所有新增的或者修改过的辅助文件（例如 *.install* 文件或者如软件包补丁之类的本地源码文件），写上有意义的提交信息并提交，最后推送这些变动到 AUR 上。

例如:

```
$ makepkg --printsrcinfo > .SRCINFO
$ git add PKGBUILD .SRCINFO
$ git commit -m "useful commit message"
$ git push
```

注意：

- 如果您忘记在最后一次提交中包含 `.SRCINFO`，您可以通过 `git commit --amend` 修改最后一次提交来使得 AUR 接受您的推送。
- AUR 只允许推送到 `master` 分支。如果本地分支的名称不同，需要将其重命名后再次推送。

提示：为了保持工作目录和提交尽可能的干净，可以创建 gitignore(5) 文件来排除所有文件，然后再按需添加文件。

维护软件包

- 阅读其他用户的反馈，并试着听从建议改进软件包，这是个学习的过程！
- 升级软件包后，不要在评论中加入版本更新信息，这些信息会冲淡更重要的用户评论。
- 不要提交软件包后就放任不管！检查更新并改进 `PKGBUILD` 是维护者的责任。
- 发觉自己不再想维护某个软件包？可以通过 AUR 网页界面 `disown` 一个软件包并/或是在 AUR 邮件列表发条消息。如果这个包的所有维护者都 disown，那么它就会变成一个“orphaned”软件包。
- 自动化对维护人员来说很有用，但它不能取代人工干预（例如：项目可能会修改许可证、添加或删除依赖关系，以及其它的显著更改）。自动更新 `PKGBUILD` 的风险将由您自行承担，任何出现问题的账户与他们提交的软件包都可能被无事前通知地删除。

请求

删除、合并、取消维护权限请求可以通过右侧“Package actions”的“File Request”链接执行。这会给当前的维护者和 aur-requests 邮件列表自动发送邮件通知，之后软件包维护者会接受或拒绝请求。

删除

你可以请求 *unlist* AUR 中的一个 `pkgbase`。提交时请附上原因说明（务必用英语），也可以是其它有用的细节（比方说其它包提供了相同的东西，或者如果你就是这个包的维护者，需要重命名且软件包所有者同意，等等）。

注意：

- 只在软件包的评论区解释原因并不够。当软件包维护者采取行动前，aur-request 邮件列表是唯一能取得这些信息的地方。
- 删除请求可能被否决。如果你是这个包的维护者，我们可能会建议你 disown 这个软件包，以便其他维护者认领。
- 在软件包被“删除”后，其 Git 仓库仍可被克隆。

合并

合并请求会删除 `pkgbase`，并将现有的投票数、评论转移到另一个 `pkgbase` 中。需要用到目标软件包的名称。

这个操作适用于上游进行了重命名项目等情况。

注意：

- 这和 `git merge` 或者 GitLab 的 merge request 没有任何关系。

- 由于投票和评论会转移到已经存在的目标中，如果一个包本身没有投票或评论，那么其效果与#删除并链接到新包完全相同。

撤销所有权（orphan）

要求撤销当前维护者对 `pkgbase` 的所有权。如果现有维护者在两周之内没有响应，请求就会通过。另外，如果一个软件被标记为过期超过 180 天，请求会被自动通过。

1.13 目录

难度 进阶 | **篇幅** 16.0 KB | **所属** 关于 Arch

关于 Arch (5个分类、18个页面)

安装过程 (1个分类、24个页面)

Arch Linux 中文维基 (1个分类、6个页面)

Arch 社区 (7个页面)

团队 (6个页面)

Live Arch 系统 (2个页面)

Arch 的启动流程

Arch 构建系统

Arch Linux

Arch Linux 新闻报道

Arch Linux 愚人节笑话

Arch 是最好的！

Arch 术语

Arch 用户软件仓库

Arch 与其他发行版的比较

ASCII 艺术

AUR 提交准则

参与贡献

常见问题

创建软件包

官方仓库

基于 Arch 的发行版

镜像源

Makepkg

硬件 (14个分类、6个页面)

笔记本电脑 (15个分类、7个页面)

CPU (5个页面)

存储 (1个分类、10个页面)

打印机 (2个页面)

蓝牙 (3个页面)

模块 (1个页面)

声音 (11个页面)

输入设备 (2个分类、9个页面)

数码影像 (1个分类、2个页面)

图形 (30个页面)

显示器 (1个分类、1个页面)

移动设备 (1个分类、3个页面)

硬件检测和故障排除 (5个页面)

主板和 BIOS (1个页面)

Fwupd

基准测试

可信平台模块

Linux 固件

性能优化

可信平台模块/1.2

网络 (24个分类、34个页面)

串流 (1个分类、6个页面)

代理服务器 (9个页面)

DNS (6个页面)

防火墙 (11个页面)

分布式计算 (1个分类、4个页面)

FTP (1个分类、3个页面)

服务器 (6个分类、13个页面)

互联网应用 (8个分类、10个页面)

即时通信 (2个分类、9个页面)

模块 (1个页面)

目录服务 (1个页面)

SSH (12个页面)

VPN (6个页面)

网络电话 (1个页面)

网络服务器 (8个页面)

网络共享 (8个页面)

网络监控 (1个页面)

网络配置 (1个分类、6个页面)

网页 (1个页面)

无线网络 (7个页面)

协议 (6个分类)

邮件服务器 (5个页面)

邮件 (2个页面)

远程服务器 (5个页面)

安全

Cloud-init

Distcc

DNS-over-HTTPS

Drcom

FreeIPA

I2P

InfiniBand

IPv6

Isatapd

ISCSI

ISCSI/LIO

进阶网络流量控制

Kerberos

LDAP 认证

MAC 地址欺骗

MentoHUST

Netatalk

Open-iSCSI

OpenStack

PXE

身份管理

Sslh

Surfraw

Systemd-resolved

Telnet

USB/IP

VLAN

VnStat

Wake-on-LAN

网络配置

网桥

Wireshark

业余无线电

软件 (7个分类)

命令行 (3个分类、5个页面)

软件比较 (5个页面)

软件公司 (4个分类)

软件列表 (29个页面)

软件项目 (9个分类)

文件格式 (2个页面)

应用程序 (23个分类、7个页面)

系统管理 (24个分类、18个页面)

安全 (11个分类、30个页面)

包管理 (3个分类、21个页面)

备份 (5个页面)

本地化 (1个分类、7个页面)

电源管理 (1个分类、14个页面)

多媒体 (4个分类、17个页面)

Freedesktop.org (8个页面)

国际化 (空)

建议阅读 (4个页面)

脚本 (空)

Linux 控制台 (4个页面)

命令行 (3个分类、5个页面)

内核 (27个页面)

配置文件 (5个页面)

启动过程 (4个分类、17个页面)

日志记录 (空)

输入 (2个分类)

同步 (1个分类、2个页面)

图形用户界面 (5个分类、4个页面)

文件分享 (2个页面)

文件系统 (4个分类、41个页面)

系统恢复 (13个页面)

小提升 (2个分类、12个页面)

虚拟化 (1个分类、18个页面)

Anything-sync-daemon

Cron

D-Bus

环境变量

IPMI

默认应用程序

屏幕捕获

Preload

Scx-scheds

设备文件

Systemd/定时器

Systemd/用户

调试

Windows PE

系统时间

系统维护

性能优化

自动启动

开发 (6个分类、27个页面)

版本控制系统 (1个分类、5个页面)

编程语言 (2个分类、19个页面)

电子设计自动化 (1个页面)

控件工具集 (3个页面)

框架 (1个分类、4个页面)

问题跟踪系统 (3个页面)

Android

Apache Kafka

Arduino

Distcc

Eclipse

Flatpak

GNU Compiler Collection

GPGPU

核心转储

LLVM

Node.js

Ollama

OpenClaw

OpenGL

User:PovShell/Sandbox

Python/虚拟环境

Rbenv

SDL

Snap

调试/获取跟踪信息

调试

Vulkan

Xc3sprog

XDG 基本目录

语言服务器协议

桌面通知程序

Project:沙盒

列表 (1个分类、5个页面)

软件列表 (29个页面)

非官方镜像源

非官方用户仓库

国际社区

基于 Arch 的发行版

文件系统

1.14 创建软件包

难度 进阶 | **篇幅** 19.1 KB | **所属** 关于 Arch

本节将讲到

- 概述
- 准备工作
- 创建 PKGBUILD

- 测试 PKGBUILD 文件和软件包
- 把包提交给 AUR

本文旨在帮助用户利用类似于 ports 的 Arch 构建系统来创建自己的软件包，以及如何提交到 Arch 用户软件仓库。本文讲述了如何创建 PKGBUILD(5)，这是一个描述如何构建软件包的文件，makepkg 会使用它来从源代码创建二进制软件包。

关于构建规则和提高软件包质量的方法，请参考 Arch 打包准则。

概述

Arch Linux 中的软件包是通过 *makepkg* 工具以及存储在 `PKGBUILD` 文件中的信息编译的。运行 *makepkg* 时，它会自动在当前目录下搜索 `PKGBUILD` 文件，然后根据其中的信息下载文件，按需把软件源码重新编译打包为一个 `*[pkgname](../../01_安装与包管理/包开发/PKGBUILD.md#pkgname)*.pkg.tar.zst` 软件包文件。软件包文件中包含了 pacman 安装需要的二进制文件和安装指令。

一个 Arch 软件包就是一个使用 zstd(1) 压缩的 tar(1) 压缩包，或者叫 tarball。它包含了以下由 *makepkg* 生成的文件：

- 要安装的二进制文件；
- `.PKGINFO`: 包含所有 pacman 处理软件包所需的元数据、依赖等等；
- `.BUILDINFO`: 包含可复现编译需要的信息，参阅 BUILDINFO(5) ；
- `.MTREE`: 包含了文件的哈希值与时间戳，pacman 能够根据这些储存在本地数据库的信息校验软件包的完整性；
- `.INSTALL`: 可选的文件，可以用来在安装/升级/删除操作之后运行命令。(本文件只有在 `PKGBUILD` 中制定才会存在)；
- `.Changelog`: 一个可选的文件，保存了包管理员描述软件变更的日志。

准备工作

必需的软件包

首先，确定你已安装必须的工具包。安装 base-devel包 元软件包应当足够；它会引入 make(1) 和其它一些编译源码时需要的工具。

创建包的一个关键工具是 *makepkg*（由 pacman包 提供），它的功能请参阅 Arch 打包准则#Makepkg 的任务。

下载并测试安装

下载你想打包的软件的源代码压缩包，解压，按照作者所说的步骤安装它。记录下在编译和安装软件过程中需要的所有命令或步骤。你将要在 PKGBUILD 文件中重复这些命令和步骤。

大多数软件作者遵循三步走的安装惯例：

```
$ ./configure
$ make
# make install
```

建议在这时测试软件的功能是否正常。

创建一个干净的 chroot

建议参考创建一个干净的 chroot 来保证你的系统中的其它软件包和配置不会影响 PKGBUILD。这个方法容错性更强，是一个更加恰当的构建软件包的方法，通常能够发现那些你未曾意识到需要的依赖项，因为它们已经在你的系统中存在。

创建 PKGBUILD

当你运行 makepkg 时，它会在当前工作目录寻找一个 PKGBUILD 文件。如果找到 PKGBUILD 文件，它会下载该软件的源代码，根据 PKGBUILD 文件中的指令编译它。PKGBUILD 中的指令必须能完全被 Bash 解释。

编译成功后，最后的二进制文件和包的元信息（即包的版本、依赖）被一起打包在 *pkgname*.pkg.tar.zst 软件包文件中，这个文件包可以通过 makepkg --install 调用 pacman 或直接使用 pacman -U *<package file>* 来安装。

要开始制作一个包，你应该先创建一个空工作目录，进入该目录，创建一个 PKGBUILD 文件。你可以复制 PKGBUILD 模板（位于 /usr/share/pacman/）到工作目录，或者复制一个类似包的 PKGBUILD 也可以。如果你只想在别人的基础上更改一些选项的话，后一种方法比较方便。

PKGBUILD 变量

注意：主页面为 PKGBUILD，另外也请参考 PKGBUILD(5) § OPTIONS AND DIRECTIVES。

makepkg 预定义了以下变量，打包者需使用它们指代构建过程的临时路径：

srcdir：*makepkg*将会把源文件解压到此文件夹，或在此文件夹中生成指向 source 数组中文件的软连接。 **pkgdir**：*makepkg*会把该文件夹当成系统根目录，并将软件安装在此文件夹下。

这些变量都是*绝对路径*，即意味着只要你合适地使用这些变量，就不用担心当前工作目录的影响.

注意：*makepkg* 及延伸到 `build()` 和 `package()` 函数在运行过程中都应当是非交互的。在这些函数中调用交互工具或脚本可能会中断 *makepkg* 的运行，特别是启用了构建日志的时候（`--log`）。具体信息请参考 FS#13214。

PKGBUILD 函数

在构建软件包时，`makepkg` 会在 `PKGBUILD` 中有定义的情况下调用以下五个函数。`PKGBUILD` 中必须包含 `package()` 函数并总会被调用。对于其它函数，如果其未被定义，那么 `makepkg` 会将其略过。

在构建时，函数将按下文列出的顺序执行。

另请参考 PKGBUILD(5) § PACKAGING FUNCTIONS。

prepare()

此函数会执行用于预处理源文件以进行构建的命令，例如 patching。此函数执行在 pkgver()) 之前，软件包解压之后。如果解压过程被跳过（`makepkg --noextract`），那么 `prepare()` 函数就不会被执行。

注意：(从 PKGBUILD(5)) 中可以知道，该函数运行在 `bash -e` 模式下，意味着任何以非零状态退出的命令都会造成该函数中止.

如果不清楚应该将操作放到 `prepare()` 还是 `build()`，只需要记得 `prepare()` 中的步骤应在解压后仅执行一次，而 `build()` 中的步骤应在解压后文件被修改时都执行一次。

pkgver()

`pkgver()` 会在抓取并解压源文件，执行 prepare()) 后执行此函数，可用于在 makepkg 阶段中更新 pkgver 变量。

为使用 git/svn/hg 等工具的项目打包时，由于它们的构建过程相同，但源文件可能每天甚至每小时更新一次，这一特性将会十分有用。过去的方法是把日期写入到 pkgver 变量中，但这样一来 makepkg 会在即使软件没有更新的情况下依然重新构建软件包，因为它会认为软件包的版本改变了。其他与此有关的命令有 `git describe`, `hg identify -ni` 等等。请在提交 PKGBUILD 前做好测试，因为如果 `pkgver()` 执行失败，整个构建过程都会终止。

注意：pkgver 不能含有空格或连接符 (-)，通常可以用 sed 来进行修正。

build()

现在你需要编写 `PKGBUILD` 文件中的 `build()` 函数。这个函数使用 Bash) 语法的通用 shell 命令来自动编译软件，并创建名为 `pkg` 的软件安装目录。这允许 *makepkg* 无需详查你的文件系统就可以打包你的软件。

在 build() 函数中第一步就是进入由解压源码包所生成的目录。*makepkg* 会在执行 build() 函数之前将当前活动目录设为为 $srcdir；因此，大多数情况下第一条命令是这样的：(参考示例文件 /usr/share/pacman/PKGBUILD.proto) ：

```
cd "$pkgname-$pkgver"
```

现在，你需要把你当时手动编译软件时用到的命令一一列上。build() 基本上会自动运行你当时手动输入的命令，并在伪 root 环境下编译该软件。如果要打包的软件使用了一个配置脚本，最好在配置中加上 --prefix=/usr。许多软件都将自己安装到 /usr/local 下，但你只应在手动从源码安装时这么做。所有的 Arch Linux 软件包都应当使用 /usr 目录。文件中接下来的两行应当如下（参考 /usr/share/pacman/PKGBUILD.proto) ：

```
./configure --prefix=/usr
make
```

注意：如果你的软件不需要构建任何东西，请不要使用 build() 函数，但 package() 函数依然是必须的。

check()

用于执行 make check 和其它一些例行测试。强烈建议使用 check() 确保软件正常构建，并能搭配依赖正常使用。

如果不需要或是维护者无法使软件包通过测试，可以给 makepkg 传入 --nocheck 参数来禁用它。

如果要测试的是 GUI 图形化应用，可以在虚拟 xserver 中运行测试。

package()

最后一步是将编译好的文件放到一个目录下，让 *makepkg* 可以获取并打包为软件包。默认情况下该目录为 pkg——一个简单的伪 root 环境。pkg 目录复制了根目录下软件安装路径的继承关系，如果你需要手动把文件放到根目录下，那么需要把文件放在 pkg 下相同的文件层级结构中。比如，你想把一个文件安装到 /usr/bin，那么在伪 root 环境中对应的路径为 $pkgdir/usr/bin。安装步骤通常不需要用户手动复制大量文件到某个地方，而只需调用 make install 即可。为了将软件正确地安装到 pkg 路径下，最后一行一般应该这样写：

```
make DESTDIR="$pkgdir" install
```

注意：有时候在 Makefile 里没有使用DESTDIR；你可能需要使用 prefix 来替代。如果软件包是用 *autoconf/automake* 来构建的，那就需根据文档使用 DESTDIR；如果 DESTDIR 不起作用，可以试试 make prefix="$pkgdir/usr" install。如果这还不起作用的话，你就需要深入检查 make <...> install 具体执行的安装命令了。

`makepkg --repackage` 命令只运行 `package()` 函数,它只是将文件打包成软件包，并不运行编译过程。如果你只是更改了 `depends` 依赖变量，用这个命令来打包可以节省很多时间。

注意：使用类似 `mv` 的命令从 `$srcdir` 移动由 `build()` 生成的文件到 `$pkgdir` 会破坏 *makepkg* 的 `--repackage` 选项。

测试 PKGBUILD 文件和软件包

在编写 `build()` 函数时，需要频繁测试更改以确保没有 bug。你可以在包含 `PKGBUILD` 的目录下运行 `makepkg` 命令来确保没有问题。如果 `PKGBUILD` 没有错误，将会生成一个软件包；但是如果 `PKGBUILD` 有问题或未完成，它将抛出一个错误。

如果运行 `makepkg` 成功，在你的工作目录下将会生成一个名为 `*pkgname-pkgver*.pkg.tar.zst` 的新文件，可以使用 `pacman -U` 进行安装。注意，成功生成了软件包不代表软件能正常运行，它有可能仅包含有空目录，也可能包含如不正确的前缀等问题。你可以使用 pacman 的 `pacman -Qlp *pkgname*` 查询软件包的文件清单，以及通过 `pacman -Qip *pkgname*` 检查依赖项是否正确。

如果包看起来是正确的，那你的工作就完成了。但是如果你打算发布这个 `PKGBUILD` 文件，你就需要确认确认再确认 `depends` 数组中的依赖是否正确。

另外也要确保包二进制文件完全正常地*运行*！如果发布出去的软件包已涵盖所有必要文件，但仅因某些选项与其他系统不兼容而崩溃会非常令人讨厌。如果你只是为自己的系统编译这个软件，就不必做这个质量保证了，因为只有你一个人需要忍受这些错误。

检查包的逻辑性

确定包可以正常使用后，再使用namcap来检查错误：

```
$ namcap PKGBUILD
$ namcap pkgname.pkg.tar.zst
```

Namcap 将会做以下工作：

- 检查 `PKGBUILD` 里的一些常见错误，以及包文件结构是否有不必要或错位的文件
- 用 `ldd` 扫描包中所有的 ELF 文件，自动报告 `depends` 中缺失或可去除的依赖
- 启发式搜寻缺失或冗余的依赖

等等。

要养成用 *namcap* 检查包的习惯，以避免提交包后再做修复的麻烦。

使用 pkgctl 在干净的 chroot 环境中进行构建

本文或本章节可能需要合并到Arch 构建系统#构建软件包。

附注： 本文与构建系统无关，且与 #创建一个干净的 chroot 和 DeveloperWiki:Building in a clean chroot 重复。（在 Talk:创建软件包 中讨论）

可以使用 devtools包 的 `pkgctl` 来检查软件包是否可以在无其它包的环境中正常构建。在包含 PKGBUILD 的目录下执行：

```
$ pkgctl build
```

然后检查输出是否包含错误或警告。如果软件包依赖于 AUR 包，则需要事先构建并带入 chroot jail 中：

```
$ pkgctl build -I path/to/somepkg.tar.gz -I ...
```

注意：如果包之前是通过 AUR 助手安装的，那很可能缓存目录中已经有其 tarball 了。

更多选项请参考 pkgctl-build(1)。

把包提交给 AUR

请参考 AUR 提交准则，里面详细介绍了提交流程。

总结

- 下载你希望打包的软件源代码
- 试着编译安装包到任意目录
- 复制文件模板 `/usr/share/pacman/PKGBUILD.proto` 到一个临时目录，并重命名为 `PKGBUILD`
- 根据需要修改 `PKGBUILD` 文件
- 运行 `makepkg` 看看输出的打包结果是否正确
- 如果不正确，重复前两个步骤

注意事项

- 除非你“很清楚”你正在做什么，否则在开始自动打包*之前*，需确保你至少已成功手动打包一次。不幸的是，虽然大多数软件作者遵循了三步走的安装惯例：`./configure`; `make`; `make install`，但事情并不都是这样的，有时候你不得不自己打补丁才能安装成功。经验是：如果你手动无法编译成功或者无法将软件安装到指定子目录下，那你就不必费心打包了。`makepkg`没有任何魔力能消除源代码的问题让你编译成功。

- 在少数情况下，软件包不提供源码，而只能使用类似 `sh installer.run` 的命令进行安装，这时就需要你自己做很多工作了（比如查看 README，安装指导，手册，或者 Gentoo 或其它软件包安装器的 ebuild 等等）。在一些很变态的情况下，你需要自己编辑源码才能正常安装。但是，`makepkg` 需要完全自主运行，不能有用户的干预。因此，如果你想修改 makefiles，需要随 `PKGBUILD` 附上一个定制的补丁，然后在 `prepare()` 函数里安装这个补丁；或者你可以在 `prepare()` 函数里通过 `sed` 来修改。

自动化

校验和

可以使用 `updpkgsums` 工具简化为新版本软件更新校验和的工作，具体信息请参考 Makepkg#生成新校验和。

PKGBUILD 生成器

某些软件包的 `PKGBUILD` 可以通过工具自动生成。

注意：用户需要在提交文件到 AUR 前确保软件包满足高质量标准。

- Haskell：cblrepo，arch-hs包
- Node.js：nodejs-npm2archAUR
- Perl：perl-cpanplus-dist-arch包
- Python：pipman-gitAUR，pip2arch-gitAUR，python-pypi2pkgbuildAUR
- Ruby：gem2archAUR，pacgemAUR
- Rust：cargo-pkgbuildAUR，cargo-aur-binAUR

新的上游版本发布

来自 devtools包 的 ***pkgctl*** 以 `.nvchecker.toml` 配置文件的形式支持 nvchecker 集成，该文件需与 `PKGBUILD` 位于同一目录下。具体示例请参考 pacman 包的 .nvchecker.toml 配置文件。

提示：`pkgctl version setup` 会通过分析 PKGBUILD 的 source 数组尝试自动创建 `.nvchecker.toml` 配置文件。

完成配置后，可以使用 `pkgctl version check` 比较 `PKGBUILD` 中的 `pkgver` 来检查上游是否发布了新版本，之后可以使用 `pkgctl version upgrade` 更新 `PKGBUILD`。详细信息请参考 pkgctl-version(1)。

参考

- 如何正确创建补丁文件

- Arch Linux Classroom IRC 上关于如何创建 PKGBUILD 的教学日志
- 使用 fakeroot 方式安装软件包

1.15 参与贡献

难度 高级 | **篇幅** 19.9 KB | **所属** 关于 Arch

本节将讲到

- 社区
- 软件包
- 活动
- 软件项目
- 非官方项目

在进化生物学中，合作)是指个体付出一个小成本，可以为其他同胞带来更大的收益，而其他个体也一样提供回报，这样每个参与者都会获得非常大的收益。Arch 社区也是如此，很多成员希望为自己喜爱的 GNU/Linux 发行版做出贡献。大家的参与，不仅有利于 Archer 同胞，还能造福自由软件和开源软件的全体爱好者。

本文简单介绍了 Archer 为 Arch 社团做贡献的若干途径。开始贡献前请先熟悉一下行为准则。

社区

论坛发帖

参与我们的一个最简单的方法，就是逛 Arch Linux 官方论坛。您可以尽情发表您的高见、帮助菜鸟或结交新朋友。

完善维基

ArchWiki 是一个**社区**文档工程。内容**源自**社区，**服务**社区。贡献者奉献他们的时间和精力，在社区中共享知识和技能。我们鼓励**全体**用户做贡献。

加入聊天室

可以在 Arch 的 IRC 频道中帮助新人解决问题。进入前请先阅读讨论规则。有许多频道专门用来讨论不同的问题。

加入邮件列表

加入感兴趣的邮件列表。注意讨论的内容要遵守列表描述。

艺术

共享屏保、启动屏幕、调色盘、主题等，可以加入艺术子论坛以及中文论坛的美工与截图版块。

参阅 Arch Linux Art 和 ASCII 艺术。

软件包

报告安装的软件

pkgstats 使用 systemd 定时器定期将系统中安装的软件上报服务器，同时还能提供系统架构和使用镜像的信息。开发者可以通过数据统计，安排自己工作的优先级。信息是匿名发送，不会暴露用户隐私。可以通过统计页面查看信息。详情请阅读这个论坛帖子。

报告和修正 bug

您还可以通过到 Arch Linux Bugtracker 汇报有关 Arch 软件包 bug 来帮忙。

但若汇报的问题无意义时，反而会浪费大家的时间。如有意参与其中，请先阅读漏洞报告准则和 General guidelines#Packaging merge requests。

安全问题

新的漏洞不停出现，帮助 Arch 安全小组追踪新的漏洞。

帮助测试软件包

测试仓库中的软件包需要进行测试后才会进入主仓库。请帮助 Arch 测试组测试新软件包。

提出需求

在 bug 系统中提出软件包需求，提交前请先阅读漏洞报告准则#Bug 或者 Feature? ，以确保功能需求是合理的，此外还需阅读 General guidelines#Packaging merge requests 。

在 AUR 软件包页面的相关页面提出 AUR 软件包需求。

创建或维护 AUR 软件包

Arch 用户软件仓库是一个大众主导的软件仓库，包含了大量用户提交的 PKGBUILD。与官方软件库二进制包的形式不同，AUR 中的软件包将以 PKGBUILD 的形式供用户使用，并全由用户自行编译。用户可通过 AUR 分享软件，其中一些热门软件会选入官方的 Extra 软件仓库。

成为 Trusted User

像帮助维护 AUR 和 extra 软件仓库中的软件包，可以申请成为软件包维护者。 详情参阅软件包维护者#我如何成为软件包维护者。

活动

有一些社区组织的活动可以参见：

- Bug Day
- AUR Cleanup Day

软件项目

Arch Linux 发行版包含了很多组件，可以单独参与到某个项目。

Discussion around the various projects (unless noted otherwise) takes place on the arch-projects mailing list and in the #archlinux-projects IRC channel.

项目	描述	语言	维护者	其他链接
alpm	Project that works on specifications, Rust libraries and tools for Arch Linux Package Management	Rust	dvzrv, orhun	bugs, documentation
alpm.rs	Rust bindings for libalpm	Rust	Morganamilo	bugs
arch-boxes	Virtual machine images	Bash	klausenbusk	bugs, IRC #archlinux-releng, arch-releng mailing list
arch-install-scripts	Useful scripts for installing Arch Linux	Bash	Foxboron	bugs
arch-rebuild-order	A CLI tool to determine the rebuild order of provided package(s)	Rust	jelle	bugs
arch-release-promotion	Promotion and synchronization of existing releases of a project in Arch Linux's GitLab instance	Python	dvzrv	bugs
arch-repro-status	CLI tool for querying the reproducibility status of the Arch Linux packages	Rust	orhun	bugs
arch-security-tracker	The security.archlinux.org CVE tracking platform and aids in publishing advisories	Python (Flask)	anthraxx	bugs, IRC #archlinux-security, Advisory mailing list
arch-signoff	Sign off Arch Linux test packages	Python	Jelle	bugs
archinstall	Arch Linux official installer	Python	Torxed	bugs, Discord, documentation
archivetools	A turnkey solution to snapshot Arch Linux packages repositories, ISOs images and boostrap tarballs	Bash		bugs
archiso	Scripts and configuration for building live media	bash	dvzrv, nl6720	bugs, IRC #archlinux-releng, arch-releng mailing list
archlinux-common-style	Arch Linux common CSS styles		anthraxx, jelle	bugs

项目	描述	语言	维护者	其他链接
		SCSS, HTML, JavaScript		
archlinux-docker	Docker images	Bash	hashworks, sangy	bugs, docker hub, IRC #archlinux-releng, arch-releng mailing list
archlinux-keyring	Arch Linux PGP keyring handling	Python	anthraxx, dvzrv	bugs
archlinux-repro	Tools to rebuild Arch Linux packages	Python	Foxboron, Jelle, Coderobe	bugs
archmanweb	The man.archlinux.org website	Python (Django)	Lahwaacz	bugs
archweb	The archlinux.org website	Python (Django)	jelle	bugs
aurweb	The page and system for aur.archlinux.org	Python, HTML, MySQL	artafinde, lfleischer	bugs, IRC #archlinux-aurweb, aur-dev mailing list
bugbuddy	Helper daemon that watches the incoming bug reports for Arch Linux and makes sure that the right people are assigned to them	Rust	anthraxx	bugs
buildbtw	A service for assisting Arch Linux staff with building new versions of packages	Rust	anthraxx, sven, raffomania	bugs, IRC #archlinux-buildbtw
dbscripts	Scripts to release and manage packages into the repositories	Bash	Foxboron	bugs
devtools	Packaging tools for developers and packagers	Bash	Anthraxx, Foxboron, jelle, gromit	bugs
gitlab-exporter	GitLab metrics exporter	Rust	Artafinde, orhun	bugs
gluebuddy	A secure helper daemon that watches several aspects of the Arch Linux infrastructure and makes sure that certain conditions are met	Rust	anthraxx, jelle	bugs
infrastructure	Arch Linux infrastructure	Ansible, Bash, Python, Packer, Terraform, Zsh	anthraxx, foutrelis, freswa, heftig, jelle, klausenbusk, svenstaro, artafinde, gromit	bugs, IRC #archlinux-devops, arch-devops mailing list
keycloak-archlinux-theme	Keycloak Arch Linux theme	Java	Artafinde	bugs
mkinitcpio	Initrd generator	Ash, Bash	Foxboron, nl6720	bugs
mkinitcpio-archiso	Mkinitcpio integration for archiso	Ash, Bash	dvzrv, nl6720	bugs, IRC #archlinux-releng, arch-releng mailing list
namcap	Tool for checking binary packages and source PKGBUILDs for common packaging errors	Python	alerque, dvzrv, FFY00, kgizdov	bugs
netctl	Profile based systemd network management	Bash	jwitteveen	bugs
neoasknot	Contribution landing page for Arch Linux	JavaScript, Svelte	polyzen	bugs, whatcanidofor.archlinux.org

项目	描述	语言	维护者	其他链接
pacman	Package Manager	Bash, C	Allan, agregory	bugs, IRC #archlinux-pacman, pacman-dev mailing list
pacman-contrib	Contribution scripts to pacman	Bash, C, Perl	Demize, Polyzen	bugs, IRC #pacman-contrib, pacman-contrib mailing list
pkgstats-cli	pkgstats client	Go	pierres	bugs, pkgstats.archlinux.de
pkgstats.archlinux.de	Arch Linux package statistics website	JavaScript, PHP, Vue	pierres	bugs, pkgstats.archlinux.de
pyalpm	alpm Python bindings	Python, C	Jelle	bugs
pytest-pacman	Pytest plugin for generating repository sync databases	Python	Jelle	bugs
rebuilderd-website	Website for reproducible.archlinux.org	JavaScript	Jelle	bugs, reproducible-builds.org, Debian wiki, IRC #archlinux-reproducible
releng	Automation of release artifacts (installation medium, PXE boot)	Bash, Python	dvzrv	bugs, IRC #archlinux-releng, arch-releng mailing list
repod	Tooling to maintain binary package repositories for Linux distributions using the pacman package manager	Python	dvzrv	bugs, documentation
signstar	A secure enclave signing solution and tooling around it	Rust	dvzrv	bugs, documentation, IRC #archlinux-signstar

DeveloperWiki:Projects 包含了项目组成员、交流取到和所用编程语言。项目代码托管在 gitlab.archlinux.org.

捐款

Arch Linux 的生命力源于开发者和社区成员的无私奉献。我们的工作是无偿的，没有维持服务器运营的私人基金，因此也需要您的资金支持。

非官方项目

注意：下列所有项目都是社区支持的，即**非官方**。

Arch 拥有一个充满活力的用户社区。如果您的项目想参与到社区中，**请在对应的英文页面**加入：项目地址、项目开始时间和一两句项目简介。

Arch Linux 中文社区 : 中文用户社区相关信息

Arch Linux Subreddit : Reddit 中讨论 Arch 的地方。

Arch Women : 为 Arch 女性用户提供的团体（论坛帖子）

Telegram 群组 : Arch 相关问题的 Telegram 群组。

Matrix 聊天室 : Arch 相关问题的 Matrix 聊天室。

Community Contributions : Arch 相关项目的论坛.

Arch Linux topic @GitHub : Arch 相关项目的 GitHub 仓库.

怎样才能成为 Arch 开发者?

首先，牢记您所做的一切，都应以帮助大众为出发点。其次，牢记您是社区大众的一分子，帮助别人亦是帮助自己。

通常，新开发者是根据工作内容的增加由已有开发者选出。有时他们会提供一个职位，您可以申请并获得职位。但更多时候，开发者会邀请能力足够，适合开发团队需要的程序员加入。积累 Arch 贡献是成为团队一员的最好方法。

这里是一个列表，没有顺序之分。包含了可以为社区做贡献的方式:

- 通过及时提供帮助获得声誉
- 在论坛、IRC、和邮件列表回答问题
- 加入软件包维护者，获得软件打包经验，提升技能
- 将软件包提交到 AUR
- 加入以后可能成为 Arch 主流应用的程序开发或者自己建立一个项目
- 修改 pacman、makepkg 或GitLab 上的其它 Arch 项目，并通过 bug 管理系统提交补丁
- 浏览 bug 管理系统，修正问题
- 找到并提交新 Bug
- 修正维基错误、添加页面、清理页面并保证步骤符合最新系统
- 提交翻译

1.16 Arch 与其他发行版的比较

难度 高级 | **篇幅** 21.9 KB | **所属** 关于 Arch

本节将讲到

- 与基于源码的发行版对比
- 与通用发行版的对比
- 与新手友好发行版对比
- 与BSD系列操作系统的对比
- 参阅

本文在 Arch Linux 、其他流行的 GNU/Linux 发行版和类 UNIX 操作系统之间做了一些比较，以帮助用户判断 Arch Linux 是否能符合他们的需要。虽然对此进行一些描述有助于用户理解这些操作系统之间的不同点，但是比较 Arch Linux 和其他操作系统的最好办法还是由用户自行安装它们并进行亲身体验。

英文维基百科的《操作系统比较》和《Linux 发行版比较》这两个页面包含了更详细的比较信息。

在接下来的部分中，只有 Arch Linux 与其他发行版做了比较。社区适配的支持 x86_64 以外的架构的版本在基于 Arch 的发行版中列出。

与基于源码的发行版对比

基于源码的发行版具有高度可移植性，非常容易被移植到不同架构的系统上。它们可以根据特定计算机的体系架构和使用情况，控制和编译整个系统和所有软件包，其优化是最佳的。但缺点是如果计算机系统的配置性能较低，安装系统和软件包的源码编译会极为费时费力。Arch Linux 的基础软件包和其他所有软件包仅为 x86-64 架构进行编译。

CRUX

- CRUX 是让 Judd Vinet 有了创建 Arch 的灵感的、专注于 KISS 法则的一种轻量级发行版。
- CRUX 使用 BSD 风格的启动脚本，而 Arch Linux 使用 systemd 。
- 尽管 Arch Linux 使用滚动升级机制，CRUX 也差不多每年都会有发行版。
- 两者都提供了类 Ports 系统，并且和 *BSD 一样都提供了一个基础系统以供用户在其之上进行构建。
- Arch Linux 使用 pacman 来进行二进制包管理，同时还使用 Arch 构建系统。CRUX 使用一个叫 *prt-get* 的社区开发软件和它自己的 Ports 系统来处理依赖关系解析。尽管如此，CRUX 上所有的软件包都需要从源代码进行编译，即使 CRUX 的基础系统是基于二进制包的。
- Arch Linux 和 CRUX 官方都仅支持 x86_64 架构。
- Arch Linux 的软件仓库提供大量编译好的二进制软件包。除此之外，还拥有 Arch 用户软件仓库（AUR）。CRUX 的软件仓库则比较逊色，并且其 Ports 系统也缺乏官方支持。

LFS

- LFS（Linux From Scratch）只是以文档形式存在，它提供的文档指导用户如何从零开始构建一个完全个性化的 GNU/Linux 操作系统：从如何获取构建一个基础操作系统所需的源代码，到如何手动编译，如何打补丁，如何对系统进行配置。总之，LFS 提供了一个构建和定制基础系统的良好教程。
- LFS 不提供在线软件仓库。它必须由用户手动获取源代码，然后用 *make* 编译安装它们（这需要一些软件包管理方法，而这些方法在 LFS Hints 里面提到过）。

- Arch Linux 基础系统除了提供和 LFS 一样的软件包，还包含 systemd、pacman 等一些额外的工具，并且这些软件都已经为 x86_64 架构编译过了。Arch 社区和开发者提供了数以千计的软件包，这些软件包可以通过 pacman 或者 PKGBUILD 脚本进行安装（PKGBUILD 脚本需要和 Arch Build System 一起使用）。除此之外，Arch Linux 还包含了一个名叫 makepkg 的工具来生成便于 pacman 使用的软件包。
- Judd Vinet 从头开始构建了 Arch，然后用 C 语言编写开发了 pacman。所以，Arch Linux 有时候会被幽默的描述为“有一个很好的软件包管理器的 Linux”。

Gentoo Linux

- Arch Linux 和 Gentoo Linux 都是滚动升级的发行版，所以在上游（原始开发者）软件包发布不久，两者就会有软件包可以用。
- Gentoo Linux 的基础系统和软件包都是根据用户指定的 *USE* 标识直接从源代码构建。Gentoo Linux 提供了一个类 Ports 系统（ Ports 是 BSD 系统上的一个系统工具）来从源代码编译软件包，而 Arch Linux 基础系统被设计为“直接由预构建好的 x86_64 二进制软件包组成”。一般来说，Arch Linux 更易于构建和升级，而 Gentoo Linux 更易于进行系统化的定制。
- Arch Linux 只支持 x86_64 架构，而 Gentoo Linux 对 x86（i486/i686）、x86_64、PPC/PPC64、SPARC、Alpha、ARM、MIPS、HPPA、S/390、Itanium、M68K 以及 RISC-V 架构均提供官方支持。
- Gentoo Linux 的官方包管理工具比 Arch Linux 的更加复杂和“强大”，并且 Gentoo Linux 的某些核心功能特性（比如 USE 标记、SLOTs 等等）在 Arch Linux 中并没有相对应的替代品。究其原因，首先是因为 Arch Linux 主要是一个二进制发行版，其次是因为 Gentoo Linux 和 Arch Linux 在设计理念上有些许差别，Arch Linux 在原则上更偏向于架构的简洁性和避免过度设计。
- 因为 Gentoo Linux 和 Arch Linux 的安装都只包含一个基本系统，所以两者都被认为是需要高度定制化的系统。一般来说，Gentoo Linux 用户如果习惯于 systemd 的话，对 Arch Linux 的大多数方面都会比较满意。

GNU Guix System

- GNU Guix System 受到 NixOS 的启发，类似于 Arch Linux 受到 CRUX 的启发。
- Arch Linux 和 Guix System 都是滚动发布发行版，使软件包在上游发布后短时间内可供发行版使用。然而，Guix 系统主要是一个基于源代码的发行版（尽管存在预构建的二进制文件并称为“替代品”），而 Arch Linux 主要是一个二进制发行版。

- Arch Linux 使用 pacman 作为包管理器，而 Guix System 使用 GNU Guix，它支持其他发行版中不存在的实验性打包功能。
- Arch Linux 只支持 x86_64，而 Guix System 正式支持多种架构。
- Arch Linux 使用 systemd 作为 init 系统，而 Guix System 使用 GNU Shepherd。
- Guix System 打破了许多传统的 Unix 概念，包括文件系统层次结构标准。例如，在传统发行版中分布在不同的目录中的许多文件，在Guix系统中将位于`/gnu/store/`。
- Arch Linux 偶尔会发布非自由软件（通常是驱动程序），而 Guix System 只发布自由软件，并得到自由软件基金会的认可——尽管存在为 Guix 提供非自由软件的替代存储库。
- Arch Linux 希望用户直接配置已安装的软件包，而 Guix System 鼓励在 Scheme 中进行全局系统配置，进而实例化配置文件。

与通用发行版的对比

这些发行版的优点是受众更加广泛，可以满足绝大部分用户对计算机操作系统的需求。

Debian GNU/Linux

- Debian 是上游最大的发行版，其社区规模更大。它提供稳定（Stable）、测试（Testing）和不稳定（Unstable）三个版本分支，包含超过51000个二进制包。而 Arch Linux 的软件仓库相对较小，但是如果算上 AUR 仓库，那么 Arch Linux 支持的软件数量也差不多。
- Debian 对自由软件更热情，但也提供非自由软件仓库。Arch Linux 对 GNU 定义的非自由（“non-free”）软件更显宽容。
- Debian 对稳定分支的测试更加详细彻底，软件基本是上冻结的，并提供五年官方社区支持。Arch Linux 提供的的软件包会比 Debian Stable 中的软件包更新，但会与 unstable 和 testing 分支里的差不多，而且没有固定发布周期，均以滚动形式发布。
- Debian 支持许多架构，包括 amd64,arm64,armel,armhf,i386,ppc64el,riscv64 与 s390x等。而 Arch Linux 仅对 x86_64 架构提供官方支持，其对 arm 架构的支持则移植自社区项目（例如对 Raspberry Pi 的支持）。
- Arch Linux 对从源码创建软件包提供更好的支持，有一个类 ports 系统。而 Debian 不提供类 Ports 系统，而是依靠它巨大的二进制软件包仓库。

- Arch Linux 安装环境只提供最小的基本系统，然后通过编辑文本文件来配置系统。而 Debian 的配置方式则更加自动化，并且还提供多种安装方式，例如使用 *apt 任务*来安装预先选择的软件包组。
- 在启动脚本上，Debian（ 8.0 或更高版本）和 Arch Linux 均使用 Systemd，因为其总体性能是相当优异的。
- Arch Linux 一般将 lib 软件库与其头文件一同打包在一起，而在 Debian 中，头文件必须单独下载，并将 lib 软件库与头文件分别打包。
- Arch Linux 将补丁保持在最低限度，只会在迫不得已的时候才会去打补丁，以避免出现上游无法审阅的问题。而 Debian 的用户众多，所以会经常对软件包打补丁。

Fedora

- Fedora Linux 是 Red Hat® Enterprise Linux （红帽企业版 Linux，RHEL）的上游社区发行版，也是一种技术先导发行版，对新技术的使用非常激进。红帽（Red Hat）公司是该项目的主要赞助商，但成千上万的独立开发人员也为 Fedora 做出了贡献。软件包和项目在 Fedora 上发布，通过其自己独特的测试和质量保证流程，这些功能迁移到 CentOS Stream，并最终被合并引入到 Red Hat Enterprise Linux 中，其中一些最终被其他发行版采用。Arch Linux 没有固定版本，也不作为另一个发行版的分支，即使许多其他发行版基于 Arch Linux（例如 Steam Deck 的 SteamOS）。
- Fedora 使用 RPM 格式软件包、 DNF 软件包管理器。而 Arch Linux 使用 pacman 管理软件包。这两个项目的许多软件包，特别是桌面环境，都被描述为“原版”，并且没有自定义。
- Fedora 坚持开源理念，拒绝在官方仓库中包含非自由软件包，默认不提供有专利限制的软件。例如 MP3 支持（从 Fedora 25 开始已重新支持 MP3，而且一些第三方源通常也会提供这些内容）。而 Arch Linux 对于 MP3 及非自由软件则更加宽松，尊重用户的知情权和使用权。
- Fedora 提供许多不同的安装方式，包括图形化 Anaconda 安装程序、最小化安装和专家安装选项，有助于安装基本系统或是直到安装您选择的成熟桌面环境。Fedora "spins" 还提供许多不同的图形化桌面环境以供用户选择（这些桌面环境都会附带一些默认的软件包）。Arch Linux 被设计为仅提供一些简单脚本来进行最小化系统安装，并且同样可以根据用户个人需要安装使用不同的图形化桌面环境。

- Fedora 发行周期比较固定，但官方支持通过 dnf-plugin-system-upgrade（适合大部分的版本）或 rpm-ostree（适合 Fedora Atomic Host）工具进行跨版本升级。而 Arch Linux 是通过滚动升级系统。
- Arch Linux 有 ports 系统，而 Fedora 有 Fedora Copr（一个类 AUR 构建系统）。
- Arch Linux 和 Fedora 都面向有经验的用户和开发人员，两者都倡导用户积极为项目开发做出贡献。
- Fedora 在 SELinux 整合，GCJ 编译包（GCJ 的目的是解除对 Oracle JRE 的依赖）等方面是走在各发行版的前列的，并且会积极为上游开发做贡献。和其他项目相比，红帽公司和 Fedora 开发人员贡献的 Linux 内核代码最多。
- ArchWiki 被认为是各发行版的Wiki中内容最丰富和最易用的。而 Fedora wiki 仅是开发者、测试者和用户间交流信息的平台，并不是和 ArchWiki 一样为最终用户准备。其实它更像一个问题追踪和合作开发的 wiki。

Slackware

- Slackware 使用 BSD 风格的 init 脚本，而 Arch Linux 使用 systemd。
- Arch Linux 有一个强大完整的软件包管理系统 pacman。与 Slackware 的标准工具不同，它可以自己处理依赖关系并提供更加自动化的系统升级方式。Slackware 的用户会更喜欢手动处理依赖关系，以尽可能控制自己的系统，而 Slackware 本身也对预编译的库和依赖提供杰出的支持。
- Arch Linux 是一个滚动升级的系统，而 Slackware 的系统版本和软件包的发布则更为保守，它会更喜欢提供经过验证的稳定软件包。在这个方面，Arch Linux 更为*“前卫”*。
- Arch Linux 官方软件仓库提供成千上万的二进制软件包，而 Slackware 官方支持的软件包就比较少了。
- Arch Linux 提供 Arch Build System（一个类 Ports 系统）和 AUR（由用户贡献的数以万计的 PKGBUILD 的集合）。Slackware 提供一个类似的系统 slackbuilds.org，它是一个半官方支持的 Slackbuilds（和 PKGBUILD 相似）仓库。Slackware 用户一般会对 Arch Linux 的大多数方面非常满意。

与新手友好发行版对比

“新手友好版”有时也被叫做“新手发行版”。这些对新手友好的发行版之间有许多相同之处，而 Arch Linux 和它们其中的任何一个发行版都不同。如果您想通过构建极简系统的方式来学习 GNU/Linux ，Arch Linux 或许是一个好选择。因为相比一些“新手发行

版”，Arch Linux 只需安装很少的软件包。一些不同的“新手发行版”之间的特点如下所述：

Ubuntu

- Ubuntu 是一个非常流行的基于 Debian 的发行版，由 Canonical 公司提供商业支持。而 Arch Linux 是由志愿者从头开始独立构建开发而成的。
- Ubuntu 和 Arch Linux 的目标不同，并且面向的用户群体也不一样。Arch Linux 为那些渴望自己动手的用户设计，而 Ubuntu 则提供图形化安装程序自动配置好了的系统，对新手用户来说更“友好”。Arch Linux 从一开始就设计了一个最小化的基础系统，非常依赖有一定知识基础的用户按自己的特定需求进行个性化定制。一般来说，开发者和动手能力强的用户更喜欢 Arch Linux 。不过有许多 Arch Linux用户是从 Ubuntu 开始学习，掌握一定的技巧后最终转向使用 Arch Linux 。
- Arch Linux 开发不偏向于任何超出其社区提供支持的特定用户界面。此外，Canonical 公司的商业性质导致他们做出了一些有争议的决定，例如在 Unity 的 *Dash* 菜单中包含广告和收集用户数据。而 Arch Linux 是一个独立的、由社区驱动的项目，没有商业议程。
- Ubuntu 开发和推广的重心似乎转移到了触摸屏设备上，而 Arch Linux 的开发依然坚持以用户为中心，鼓励社区合作开发客制化的解决方案。
- Ubuntu 通常每 6 个月发布新版本，而 Arch Linux 通过滚动升级系统。
- Arch Linux 提供类 Ports 的软件包构建系统和 Arch User Repository，用户可以分享源代码编译脚本，然后用 Pacman 安装管理。Ubuntu 则使用更复杂的 apt，可以通过 Personal Package Archives重新分发二进制软件包。
- 这两个发行版的社区在某些方面也有所不同。Arch 社区要小的多，更倾向于鼓励用户为 Arch Linux 奉献一份力量。相比之下，Ubuntu 社区更加庞大，并能容忍其中并没有积极为开发、打包和维护做贡献的大多数用户。

Linux Mint

- Linux Mint 最初是一个 Ubuntu 的衍生版本。后来又衍生出 LMDE（Linux Mint Debian Edition），LMDE 是一个基于 Debian 的版本。而 Arch Linux 是一个独立的发行版，它依赖于自己的构建系统和官方仓库。
- 为了方便系统维护，Linux Mint 包含了一些图形化工具，它们叫做 *MintTools*（Mint 工具），而 Arch Linux 仅提供简单的命令行工具，例如 Pacman，因此 Arch Linux 的系统管理工作需要由用户自行组织安排。

- Linux Mint 主要以 Cinnamon 、 MATE 与 Xfce4 作为它的图形界面，不过它也可以由用户自行安装选择 KDE 。Linux Mint 同时支持 Plus codecs、Flash、DVD playback 和 MP3，这其中有一些是有专利限制的私有软件。
- Linux Mint 半年发布一次新版本。也就是在新版本 Ubuntu 发布一个月之后。Linux Mint 的每一个版本都基于最新的 Ubuntu LTS（长期支持版），并且提供五年技术支持。 Mint 的 Debian 版本 (LMDE) 基于 Debian Stable分支。而且只接收来自 Mint 的软件包更新和安全更新。与之相反，Arch Linux 则完全是一个滚动更新的发行版。

openSUSE

openSUSE 诞生于最初的 SUSE Linux，由 SUSE（SUSE Enterprise Linux 的制造商）赞助支持。SUSE Linux Enterprise Desktop （SLED） 基于 openSUSE Tumbleweed，与 openSUSE Leap 共享一个通用代码库。

- OpenSUSE 使用 Zypp 软件包管理器（终端命令为 *zypper*）、RPM 格式软件包以及备受推崇的 YaST2 图形化配置工具。Arch Linux 使用 Pacman 来管理 tar.zstd 包，没有提供此类图形配置工具。一般 openSUSE 更适合经验较少或需要使用图形界面自动完成配置的用户。
- openSUSE 提供 2 个不同的版本：
- Leap 是 openSUSE 的长期支持版本，具有离散版本。Leap 版本一般是每年发布一个版本，软件包会比 Arch Linux 更旧。
- Tumbleweed 是 openSUSE 的滚动发布版本。Tumbleweed 版本是滚动更新的，软件包版本的新旧和 Arch Linux 差不多。
- openSUSE 默认不提供有专利限制的软件，比如 MP3 支持。您需要添加第三方源来取得这些内容。而 Arch Linux 对于 MP3 及非自由软件更加宽松，更倾向于将决定权交给用户。
- 相比之下，Arch Linux 严格来说是一个滚动发布模型，不提供离散发布版本。Arch Linux 默认不是完整的桌面环境，只提供最小的基本系统安装。因此，openSUSE 可能更适合那些想要图形环境、自动配置或开箱即用的功能，同时仍然允许在所有发行版上进行自定义的用户。

与BSD系列操作系统的对比

- BSD 系列操作系统都起源于加利福尼亚大学伯克利分校所做的工作，它们致力于提供一个可以自由再分发、免费的 UNIX 系统。它们不是 GNU/Linux 发行版，而是以原始的 AT&T UNIX 系统的代码为基础演进的类 UNIX 操作系统。

- Arch Linux 和 BSD 都提供紧密集成的基本系统和 Ports 软件包管理系统。但与 GNU/Linux 系统（比如 Arch Linux）不同，BSD 操作系统的内核和用户空间的程序，如 shell 和常用工具（像 ls、cp、cat 和 ps）是集中在单一的源代码仓库中一起开发的。
- 与 GPL 许可证相反，BSD 许可证比较宽松，GPL 许可证规定衍生品需要在同一许可证下发布。而 Arch Linux 是在 GPL 许可证下发布的。
- 要获得 BSD 变体的更多信息，请参阅 Wikipedia:Comparison of BSD operating systems。

参阅

- DistroWatch.com- Linux 发行版新闻和评论

1.17 Arch 用户软件仓库

难度 高级 | **篇幅** 24.2 KB | **所属** 关于 Arch

本节将讲到

- 入门
- 安装与更新软件包
- 账户状态
- 反馈
- 调试包构建过程

Arch 用户软件仓库（Arch User Repository，AUR）是为 Arch Linux 用户而建，由社区主导的软件仓库。AUR 中的软件包以软件包描述文件（PKGBUILD）的形式提供，用户自己通过 makepkg 生成包，再由 pacman 安装。创建 AUR 的初衷是供社区管理和分享新的软件包，并由官方定期从中挑选软件包进入 extra 仓库。本文介绍用户访问和使用 AUR 的方法。

许多官方仓库软件包都来自 AUR。通过 AUR，大家可以相互分享新的软件包生成脚本（PKGBUILD 和其他相关文件）。用户还可以为软件包投票。如果一个软件包投票足够多，没有许可证问题且打包质量好，那么它就很有希望被收录进官方 *extra* 仓库（以后就可以直接通过 pacman 或 Arch 构建系统安装了）。

警告：AUR 中的软件包是由其他用户编写的，这些 `PKGBUILD` 完全是非官方的，未经彻底审查。使用这些文件的风险由您自行承担。

注意：如果你计划要使用 AUR，**强烈建议**关注 AUR 常规邮件列表，该列表被用于发布安全警告。

入门

用户可以从 AUR 网站下载 `PKGBUILD` 文件。这些 `PKGBUILD` 可以通过 *makepkg* 生成软件包，最后由 *pacman* 进行安装。

- 确保已经安装了 base-devel包。
- 浏览 #常见问题一节，以获得对最常见问题的解答。
- 在从 AUR 构建软件包之前，您可能希望调整 `/etc/makepkg.conf` 以优化系统的构建过程。通过调整 `MAKEFLAGS` 变量、使用多核进行压缩或使用不同的压缩算法，可以在多核处理器的系统上显著缩短包构建时间。用户还可以通过 `CFLAGS` 变量启用特定于硬件的编译器优化。详情参见 makepkg。

如果您设置了 AUR SSH 身份验证，也可以通过 SSH 连接到 AUR：运行 `ssh aur@aur.archlinux.org help` 以获得可用指令的列表。

安装与更新软件包

从 AUR 安装软件包并不困难。基本步骤如下：

1. 获取构建文件，包括 `PKGBUILD` 和所需的其他文件，比如 systemd 单元文件和补丁（往往并非软件包的实际代码）。
2. 检验 PKGBUILD；最重要！检验 `PKGBUILD` 及相关文件，确保其可信、无恶意。
3. 构建软件包：在保存文件的目录下运行 `makepkg`。这将下载代码，编译并打包。
4. 安装软件包：执行 `pacman -U *package_file*` 将软件包安装到系统上。

首先，确保安装了 base-devel包，该元软件包提供了 make包 等用于从源码进行编译的工具：

注意：AUR 中的软件包会假定构建环境里已安装了 base-devel包。

然后，选择一个合适的构建目录。该目录仅用作生成软件包或从源码构建时的工作目录，因此任何空白目录都可以。下面的示例中使用 `~/builds` 作为编译目录。

获取构建文件

通过在 AUR 主页中搜索，找到要安装的软件包。点击搜索结果中的软件名称，进入软件包信息页。阅读描述并确认是否为目标软件，并检查软件的更新时间和评论。

确认无误后，有几种方法可以获取包的构建文件：

- 克隆其 git 存储库，在其 AUR 页面的“包详细信息”中标记为“Git Clone URL”。这是首选方法，其优点是您可以通过 `git pull` 轻松获取包的更新。

: ``` $ git clone https://aur.archlinux.org/package_name.git

```
- 通过单击其 AUR 页面右侧“Package Actions”下的“Download snapshot”链接或在终端中下载快照：

: ```
$ curl -L -O https://aur.archlinux.org/cgit/aur.git/snapshot/package_name.tar.gz
```
: > **注意**：快照文件是压缩的，必须解压（最好在为 AUR 构建预留的目录中）：`tar -xvf *package_name*.tar.gz`

- 使用 GitHub 上的 archlinux/aur 只读镜像，每个包都位于一个分支中。建议只克隆一个分支（整个存储库太大，速度会很慢）。您可以使用以下两种方法之一执行此操作：
- 使用 `git clone --single-branch`：

```
        $ git clone --branch package_name --single-branch
https://github.com/archlinux/aur.git
```
- 部分克隆这个仓库 (`git clone --depth=1`) 和选择性添加分支：

: : ``` $ git clone --depth=1 https://github.com/archlinux/aur.git; cd aur$ git remote set-branches --add origin package_name$ git fetch$ git switch package_name

```
### 检验 PKGBUILD

切换到含有软件包的 `PKGBUILD` 文件的目录：
```
$ cd package_name

```
> **警告：**请仔细检查 `git` 代码库中的 `PKGBUILD`、所有 `*.install*` 文件以及其它文件，确认是否带有恶意或危险命令。如有任何怀疑，请不要构建软件包，并到论坛或邮件列表中[寻求帮助](../../02_系统管理与内核/系统恢复/常规故障排除.md#额外支援)。过去已经发现数例含恶意代码的软件包。 一些工具，比如 [traur](https://aur.archlinux.org/packages/traur/)AUR 和 [ks-aur-scanner](https://aur.archlinux.org/packages/ks-aur-scanner/)AUR，可辅助用户扫描 PKGBUILD 内容；但它们不能取代细致的手动检验。

查看所有提供的文件中的内容。例如使用 `*less*` 查看 `PKGBUILD`：
```
$ less PKGBUILD

```
检查 `PKGBUILD` 源码列中是否存在签名文件（`*.sig*` 或 `*.asc*`）。若是，则获取 PKGBUILD 列出的 [validpgpkeys](../../01_安装与包管理/包开发/PKGBUILD.md#validpgpkeys) 公钥之一。更多信息参见 [makepkg#验证签名](../../08_安全与加密/OpenPGP/Makepkg.md#验证签名)。

无论什么情况下，必须牢记恶意代码也可能被签名，因此必须仔细检查 `PKGBUILD`，尤其是可能从仓库之外获取软件代码的部分。

> **提示：**如果您正在更新软件包，您可能需要查看最后一次提交以来的变动。
>
> - 您可以使用 `git show` 来查看最后一次提交之后的变动。
> - 您也可以使用 `*vimdiff*`：`git difftool @~..@ --tool=vimdiff`。使用 `*vimdiff*` 的好处是可以查看到所有文件，以及对出现变更部分的提示。

### 构建软件包
```

手动检查文件后，以普通用户权限执行 [makepkg](../../08_安全与加密/OpenPGP/Makepkg.md) ，下面是一些有用的参数：

- `-s`/`--syncdeps` 会在构建之前使用 [pacman](../../01_安装与包管理/包管理器/Pacman.md) 自动解析和安装任何依赖项。如果包依赖于其他 AUR 包，则您应先手动安装它们。
- `-i`/`--install` 会在构建成功后安装软件包。这使您可以跳过通常需手动完成的下一步。
- `-r`/`--rmdeps` 会在构建后删除不需要的构建时依赖项。但是，下次更新包时可能需要重新安装这些依赖项。
- `-c`/`--clean` 会在构建后清理临时构建文件，因为它们不再需要。这些文件通常仅在调试构建过程时才需要。
- `--packagelist` 会在无需构建的情况下输出构建出的软件包文件名。

> **提示：**使用 `git clean -dfx` 可以删除所有不被 git 跟踪的文件，从而删除所有以前构建的包文件。

安装软件包

现在可以使用 pacman 安装该软件包：
> **注意：**
>
> - 如果您在 `makepkg.conf` 中更改了 `PKGEXT`，则包文件的名称可能会略有不同。
> - 以上示例只是对构建过程的简要总结。**特别**推荐阅读 [makepkg](../../08_安全与加密/OpenPGP/Makepkg.md) 和 [Arch 构建系统](Arch%20构建系统.md)文章以了解更多详情。

升级软件包

在包含包的 `PKGBUILD` 的目录中，您必须首先使用命令更新文件：
$ git pull

然后按照之前的构建和安装说明进行操作。

更新软件包

Pacman 不支持 AUR，因此更新您安装的所有包是*您的职责*，而不是 pacman 的。如果官方仓库中的包更新了，您需要重新构建依赖于那些库的 AUR 包。`checkrebuild` 和 [rebuild-detector](https://archlinux.org/packages/?name=rebuild-detector)包 提供的 `rebuild-detector` [钩子](../../01_安装与包管理/包管理器/Pacman.md#钩子)可用于查找需重新构建的软件包。

账户状态

封禁（Suspension）

作为[软件包维护者](../团队/软件包维护者.md)时，可以对用户配置“封禁”字段，从而挂起目标用户。**当用户被封禁时，他们不能：**

- 登录 https://aur.archlinux.org
- 接收通知
- 与 git 界面交互

非活动（Inactivity）

当编辑自己的账户或作为*软件包维护者*编辑其他用户的账户时，可以设置“非活动”字段。使用非活动账户有两个原因：

- 在其账户页面上显示某人被标记为非活动的日期
- 根据他们对新提案的不活动情况，生成当前的活跃*软件包维护者*计数

反馈

在软件包上添加评论

[AUR 网页界面](https://aur.archlinux.org)中有一个评论工具，允许用户就改进提出建议和反馈给 `PKGBUILD` 贡献者。

> **提示：**避免将补丁或 `PKGBUILD` 粘贴到评论部分：它们很快就会过时，最终会不必要地占用大量空间。请将这些文件通过电子邮件发送给维护者，甚至可以使用 [pastebin](../../06_软件与应用/应用程序/应用程序列表.md)。

[Python-Markdown](https://python-markdown.github.io/) 提供了基本的 [Markdown](https://en.wikipedia.org/wiki/Markdown) 语法来格式化评论。

> **注意：**
>
> - 此实现与官方[语法规则](https://daringfireball.net/projects/markdown/syntax)或许会有一些[差异](https://python-markdown.github.io/#differences)。
> - 对软件包 [Git](../../07_开发与工具/版本控制系统/Git.md) 存储库的提交哈希值，以及对 [Flyspray](../../07_开发与工具/问题跟踪系统/Flyspray.md) 工单的引用会被自动转换为链接。
> - 长评论默认折叠，可按需展开。

给软件包投票

所有的 Arch 用户都可以通过 AUR 界面为喜欢的软件包**投票**。所有软件包都有机会被包维护者接手，并收录至 [extra](官方仓库.md) 仓库。投票数是该选拔过程的重要依据之一，因此请尽可能地投票！

注册 [AUR 网站](https://aur.archlinux.org/)账户，在浏览软件包时会看到投票选项。注册后，还可以通过 [aur-auto-vote-git](https://aur.archlinux.org/packages/aur-auto-vote-git/)AUR 在命令行中投票。

此外，如果你设置了 [AUR SSH 身份认证](AUR%20提交准则.md#认证)，可以使用 ssh 密钥直接通过命令行投票，不再需要保存或者输入 AUR 密码：

```
$ ssh aur@aur.archlinux.org vote package_name
```

将软件包标记为过期

首先，点击软件包页面的“Flag Out-of-date”（中文版为“标记为过期”），并说明过期的原因，最好包含新版本的发布说明或 [tarball](../../02_系统管理与内核/数据压缩与归档/归档与压缩.md#仅归档) 的链接。

可以通过邮件通知维护人员。如果*两周*后还是没有得到答复，可以发出*遗弃*（orphan）请求。更多信息请参考 [AUR 提交准则#请求](AUR%20提交准则.md#请求)。

> **注意：**[VCS 软件包](../../01_安装与包管理/Arch%20打包准则/VCS%20软件打包准则.md) 在 `pkgver` 变化时并不会过时，所以不要标记它们，且维护者也只会取消标记并无视请求。AUR 维护者不应该仅为升高 `pkgver` 就进行提交。

调试包构建过程

1. 在构建任何东西之前，通过[更新](../../01_安装与包管理/包管理器/Pacman.md#升级软件包)确保您的构建环境是最新的。
2. 确保您已安装 [base-devel](https://archlinux.org/packages/?name=base-devel)包。
3. 对 `makepkg` 搭配使用 `-s` 选项，以在开始构建过程之前检查并安装所有依赖项。
4. 试试默认的 [makepkg 配置](https://gitlab.archlinux.org/archlinux/packaging/packages/pacman/-/blob/main/makepkg.conf)。
5. 常见问题请参阅 [Makepkg#问题处理](../../08_安全与加密/OpenPGP/Makepkg.md#问题处理)。

如果您在构建包时遇到问题，请先阅读其 `PKGBUILD` 及其 AUR 页面上的评论。

可能 `PKGBUILD` 本身就有配置问题。如果您无法自行解决，请将其报告给维护者（例如，通过[在 AUR 页面的评论中发布您遇到的错误](#在软件包上添加评论)）。您也可以在 [AUR 问题、讨论和 PKGBUILD 请求论坛](https://bbs.archlinux.org/viewforum.php?id=38)寻求帮助。

问题原因可能并不简单，可能会涉及自定义到 `CFLAGS`、`LDFLAGS` 和 `MAKEFLAGS`。为排除由您的特定系统配置引起的问题，请在[干净的 chroot](../Arch%20开发/创建一个干净的%20chroot.md) 中构建包。如果在干净的 chroot 下构建仍然失败，则问题很可能出在 `PKGBUILD`。

请参阅[创建软件包#检查包的逻辑性](创建软件包.md#检查包的逻辑性)以获得关于使用 [namcap](../Arch%20项目/Namcap.md) 的信息。如果您希望审核 `PKGBUILD`，请将其发布到 [aur-general 邮件列表](https://lists.archlinux.org/mailman3/lists/aur-general.lists.archlinux.org/)以获得[软件包维护者](../团队/软件包维护者.md)和其他 AUR 成员的反馈，或发布到[创建和修改包论坛](https://bbs.archlinux.org/viewforum.php?id=4)。您还可以在 [Libera Chat](https://libera.chat) 上的 #archlinux-aur [IRC 频道](../../03_网络与服务/互联网中继聊天/Arch%20的%20IRC%20频道.md)中寻求帮助。

提交

用户可以使用 AUR 分享 `PKGBUILD`。参见 [AUR 提交准则](AUR%20提交准则.md)以获得更多细节。

Web 接口翻译

参阅 AUR 源码中的 [i18n.md](https://gitlab.archlinux.org/archlinux/aurweb/-/blob/master/doc/i18n.md) 获得关于创建和维护 [AUR Web 界面](https://aur.archlinux.org)翻译的详细信息。

历史

最初，人们仅将 `PKGBUILD`、所需的补充文件和构建好的包上传到 `ftp://ftp.archlinux.org/incoming` 服务器。包及相关文件会暂存在此，直至一位“受信用户”（现已更名为[软件包维护者](../团队/软件包维护者.md)）看到这个程序并接管维护。

然后，受信用户仓库诞生了。社区中，特定的个人被授权托管他们自己的仓库，以供他人使用。在更灵活、更易用的目标下，AUR 在这个基础上扩展壮大。事实上，AUR 维护者曾一直被称作受信用户（Trusted user，TU），直到更名为软件包维护者。

在 2015 年 6 月 8 日到 8 月 8 日期间，AUR 从 3.5.1 版本迭代到 4.0.0，引入了通过 Git 仓库发布 `PKGBUILD` 的功能。 已存在的包会被丢弃，除非维护者手动地将它们移植到新的基础架构上。

AUR3 包的 Git 仓库

Github 上的 [AUR 存档](https://github.com/aur-archive)中有一个包含移植时 AUR 3 的所有包的仓库。

或者，采用等价的 [aur-mirror](https://github.com/felixonmars/aur3-mirror/) 仓库。

常见问题

什么样的软件包能被放到 AUR?

AUR 中的软件包仅是编译脚本，即为 [pacman](../../01_安装与包管理/包管理器/Pacman.md) 构建二进制文件的蓝图。只要你遵循了打包内容的许可条例，基本上符合[规则](AUR%20提交准则.md#提交软件包的规则)的软件包都可以提交。有时候，下载的链接具有 "you may not link" 条款（即不可分发），这时就不能提供下载链接，而是要用程序名称代替，用户需要用其它方式提前获取受限制的源代码。有疑问请及时询问。

怎么可以为 AUR 中的包投票?

见 #给软件包投票。

软件包维护者是什么?

参见 [Arch 术语#软件包维护者](Arch%20术语.md#软件包维护者)。

Arch User Repository 和 extra 仓库有何区别?

Arch User Repository 是储存所有用户提交的 `PKGBUILD` 的地方，软件包需通过 [makepkg](../../08_安全与加密/OpenPGP/Makepkg.md) 手动生成。吸引足够多的社区关注及有软件包维护者支持的 `PKGBUILD` 会被收录进 [extra](官方仓库.md) 仓库（由软件包维护者维护），以二进制软件包形式提供，可以由 [pacman](../../01_安装与包管理/包管理器/Pacman.md) 安装。

AUR 中的软件包过期了，我该怎么做?

参见 #将软件包标记为过期。

同时，你可以在本地编辑 `PKGBUILD` 来自行更新软件包。有时更新不需要修改构建流程，只用按需更新 `pkgver` 或 `source` 数组即可。

`makepkg` 无法构建某个软件包，我该怎么办?

您很可能忘了点啥，参见 #调试包构建过程。

ERROR: One or more PGP signatures could not be verified! 我该怎么办?

最有可能的情况是您没有所需的公钥来验证下载的文件。详情参见 [Makepkg#验证签名](../../08_安全与加密/OpenPGP/Makepkg.md#验证签名)。

如何编写 PKGBUILD?

查阅 [AUR 提交准则#提交软件包的规则](AUR%20提交准则.md#提交软件包的规则) 和 [创建软件包](创建软件包.md)。

我想提交一个 PKGBUILD，希望别人帮忙检查错误。

您可以通过多种渠道提交您的材料供审核；请看 #调试包构建过程。

PKGBUILD（AUR 软件包）怎样才能被收录到 extra 软件仓库?

一般至少需要 10 票，并且要有软件包维护者愿意维护，否则即便有上千票也不会收录。当然，如果某个软件包维护者愿意维护一个软件包的话，投票数往往不是决定因素。

一些流行的软件包未被收录的原因一般是：

- Arch Linux 的软件仓库中已经有了别的版本。
- 没有再发布的授权许可。
- 那个软件包的功能只和 AUR 相关（例如，是个 [AUR helper](../../06_软件与应用/软件比较/AUR%20助手.md)）

另见[包进入 extra 仓库的规则](../../08_安全与加密/OpenPGP/软件包维护者指导原则.md#软件包进入_extra_仓库的规则)。

如何加速编译?

参见 [Makepkg#减少编译时间](../../08_安全与加密/OpenPGP/Makepkg.md#减少编译时间)。

foo 和 foo-git 的区别是什么？

很多 AUR 软件包都包含稳定版本和开发版本。开发版本一般都会有代表版本控制系统的[后缀](../../01_安装与包管理/Arch%20打包准则/VCS%20软件打包准则.md#包名)。开发版本并不是为日常使用而设计的，不过可能包含新功能或者 bug 修复。由于执行 `makepkg` 时会从软件源获取最新的代码，因此 AUR 上的 `pkgver()` 并不代表上游的最新更改。这些软件包很可能会跳过对 [VCS 源代码](../../01_安装与包管理/Arch%20打包准则/VCS%20软件打包准则.md#VCS_源代码)的完整性效验。

同时请参阅 [系统维护#使用经测试的软件包](../../02_系统管理与内核/系统管理/系统维护.md#使用经测试的软件包)。

为啥某个软件包从 AUR 消失了?

可能是有软件包维护者认领了这个软件包，并把它收入到 [extra](官方仓库.md) 仓库中了。

软件包也可能因为不满足[提交软件包的规则](AUR%20提交准则.md#提交软件包的规则)而被删除，您可以在 [aur-requests 存档](https://lists.archlinux.org/archives/list/aur-requests@lists.archlinux.org/)中找到删除原因。

> **注意：**通常被删除软件包的 git 仓库仍被保留，细节请参考 [AUR 提交准则#请求](AUR%20提交准则.md#请求)。

我要怎么找出从 AUR 里消失的软件包?

对比 `pacman -Qm` 的输出（即在同步数据库中不存在的软件包清单）和 AUR 上的所有软件包清单：

```
$ comm -23 <(pacman -Qqm | sort) <(curl https://aur.archlinux.org/packages.gz | gzip -cd | sort)
```

如何得到包含所有 AUR 包的列表？

- https://aur.archlinux.org/packages.gz
- 使用来自 [python3-aur](https://aur.archlinux.org/packages/python3-aur/)AUR 的 `aurpkglist`。
- [新的 AUR 元数据存档](https://lists.archlinux.org/archives/list/aur-general@lists.archlinux.org/thread/D4YC6Y7L4T5VSEONUCLHOX2R4NJKNIDP/)

另请参阅

- [AUR 网页界面](https://aur.archlinux.org)
- [AUR 邮件列表](https://lists.archlinux.org/mailman3/lists/aur-general.lists.archlinux.org/)

1.18 Arch 是最好的！

难度 高级 | **篇幅** 29.2 KB | **所属** 关于 Arch

本节将讲到

- 历史
- 代码

Arch is the best 项目是一个非常复杂的和精致的，自我提升和令人兴奋的项目（虽然或许有点过度工程），该项目给出了 Arch 的优越性的证明。

历史

这个有远见的项目最初是由 Arch 社区的长期成员 lucke 在 2008 年 4 月设计的，它是一个简单的 shell 脚本，提供了"Arch is the best"的无可辩驳的证明。它是通过一个论坛帖子向全世界宣布的，从而启发了其他人的思想，他们立即开始将它移植到多种不同的语言中，包括编程语言和口头语言，以便地球上的每一个人都能充分欣赏和受益于这一革命性的发现。

代码

"Arch is the best"项目被移植到许多编程语言中。

1C:Enterprise 7.7/8/8.1/8.2 : 一种特定于过程域的编译动态类型化编程语言，主要类似于 VisualBasic，在俄罗斯和其他独联体国家广泛使用的"1C:Enterprise"产品中。

```
Предупреждение("Arch is the best!");
```

ABAP : 高级业务应用程序设计语言。

```
REPORT zwhat_is_the_best.
WRITE 'Arch is the best'.
```

Ada : 一种系统关键编程语言。

```
with Ada.Text_IO;
use Ada.Text_IO;
procedure ArchIsTheBest is
begin
   Put_Line("Arch is the best!");
end ArchIsTheBest;
```

APL : 一种编程语言。

```
'Arch is the best!'
```

AppleScript : 由 Apple Inc. 创建的一种脚本语言，自 System 7 起内置于 Classic Mac OS 中，并内置于所有 macOS 版本中。

```
display alert "Arch is the best!"
say "Indeed, Arch is the best."
```

ArnoldC : 基于 Arnold Schwarzenegger 的一行程序设计语言。

```
IT'S SHOWTIME
TALK TO THE HAND "Arch is the best!"
YOU HAVE BEEN TERMINATED
```

ATS : 一种使用依赖类型来提高程序可靠性的函数式编程语言。

```
implement main () = println! "Arch is the best!"
```

Awk : 一种数据驱动的程序设计语言，用于处理基于文本的数据。

```
BEGIN {
   print "Arch is the best!"
}
```

BASIC : 一种脚本语言，是 20 世纪 60 年代最常用的计算机编程语言之一，被认为是学生在使用 FORTRAN 等功能更强大的语言之前学习的第一步。

```
10 PRINT "Arch is the best!"
```

Batch : 一种用于 Windows 的脚本语言，可用于自动执行任务或只是享受一些乐趣。

```
@echo off
echo Arch is the best!
pause
```

Befunge : 被认为是第一种基于 ASCII 的二维通用通用语言（在某种意义上，"you could plausibly write Hunt the Wumpus in it"）。

```
<v"Arch is the best!"0
 <,_@#:
```

BIRL : 类似于 ArnoldC，但是对于 Bambam 的。

```
HORA DO SHOW
    CE QUER VER ESSA PORRA? ("Arch is the best!\n");
    BORA CUMPADE 0;
BIRL
```

Boo : 针对 .NET 和 Mono 的稳定的面向对象的静态类型编程语言，具有受 python 启发的语法，并特别关注通过语言和编译器可扩展性功能（例如宏和自定义编译管道）进行元编程。

```
print "Arch is the best!"
```

Bourne shell : 原始程序，应该与任何 shell 兼容。

```
#!/bin/sh
echo "Arch is the best!"
```

Bourne shell（备用） : 方便将输出传递到您喜爱的 IRC/email/IM 客户端。任何 shell 都能用。

```
#!/bin/sh
yes Arch is the best!
```

Bourne shell（动态更新）

```
#!/bin/sh
w3m -dump "https://wiki.archlinux.org/index.php/Arch_is_the_best" | sed -n '/
^Translations/,/^Encodings/p'
```

或

```
 #!/bin/bash
 curl -s "https://wiki.archlinux.org/index.php?title=Arch_is_the_best&action=raw" | sed
-n '/==Translations==/,$p' | sed "s/^\(.*\)$/* \1:/;t;s/^[^=]/  &/"
```

brainfuck : 旨在建立最小编译器并符合图灵完备性的语言，其名字也暗示程式码很难读懂。

```
++>++++++>+++++<+[>[->+<]<->++++++++++<]>>.<[-]>[-<++>]
<----------------.---------------.+++++.<+++[-<++++++++++>]<.
>>+.++++++++++.<<.>>+.------------.---.<<.>>---.
+++.+++++++++++++++.+.<<+.[-]++++++++++.
```

C : 注意这个项目中使用的三个空格缩进，很像其他高级的语言使用的缩进。

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
   puts("Arch is the best!");
   return EXIT_SUCCESS;
}
```

C# : 旨在成为一种简单、现代、通用、面向对象的编程语言。

```
using System;
public class ArchIsTheBest
{
   static public void Main ()
   {
      Console.WriteLine ("Arch is the best!");
   }
}
```

C++ : Arch == Linux++

```
#include <iostream>
#include <cstdlib>
int main ()
{
   std::cout << "Arch is the best!" << std::endl;
   return EXIT_SUCCESS;
}
```

COBOL : 一种简单、轻量级的编程语言。

```
    IDENTIFICATION DIVISION.
    PROGRAM-ID.  TheBest.

    PROCEDURE DIVISION.
        DISPLAY "Arch is the best!".
        STOP RUN.
```

CoffeeScript : 一种可编译为 JavaScript 的编程语言。

```
alert 'Arch is the best!'
```

Clojure : 在 JVM 上运行的 Lisp 方言。

```
(def translations {"english" "Arch is the best!",
                   "german" "Arch ist das Beste!",
                   "australian" "Arch is fair dinkum, mate!",
                   "h4x0r" "arhc 51 7he be57!",
                   "spanish" "¡Arch es el mejor!"})

(defn read-choice []
  (println "\nAvailable languages: ")
  (doall (map #(println (key %)) translations))
  (print "Enter language or Ctrl-c: ") (flush)
  (translations (read-line) :badinput))

(defn arch-is-the-best []
  (loop [choice (read-choice)]
    (case choice
      :badinput (do (print "\nBad input!\n")
                    (recur (read-choice)))
      (do (print "\n" choice "\n")
          (recur (read-choice))))))
```

或

```
(def translations {"english" "Arch is the best!",
                   "german" "Arch ist das Beste!",
                   "australian" "Arch is fair dinkum, mate!",
                   "h4x0r" "arhc 51 7he be57!",
                   "spanish" "¡Arch es el mejor!"
                   "street" "Arch iz da shizzle ma nizzle"})
(while 1
  (println "\nPick a language:\n" (map #(key %) translations) "\n language: ")
  (println (translations (read-line) "Not a valid language")))
```

或

```
(prn "Arch is the best!")
```

Common Lisp：在 SBCL 上测试，可以随意添加更多的翻译。

```
#!/usr/bin/sbcl --script
(defparameter *best-list* '((English "Arch is the best!")
            (Chinese "Arch, 她出类拔萃!")
          (German "Arch ist das Beste!")
          (Greek "Το Arch είναι το καλύτερο!")
          (Latin "Arch optimum est!")
          (French "Arch est le meilleur!")))
(defun aitb ()
  (format t "Available languages: ~{~{~@(~a~)~*~}~^, ~}.~%" *best-list*)
  (loop for input = (progn (format t "~&Input the desired language, (or 'quit'): ~%")
                           (force-output)
                           (read-line))
     if (string-equal input "quit")
     do (loop-finish)
     else
     do (let ((language-def
                (assoc input *best-list*
                       :key (lambda (lang) (symbol-name lang))
                       :test #'string-equal)))
          (if language-def
              (format t "~&~A~%" (second language-def))
              (format t "~&Invalid language.~%"))))
  (format t "~&May the Arch be with you!~%"))
(aitb)
```

Common Lisp（备用）：应该在任何实现上运行（Clisp，Allegro，SBCL ...）

```
(princ "Arch is the best!")
```

Crystal：一种面向对象的类似 Ruby 的语言。

```
puts "Arch is the best!"
```

Crystal（通过 web 服务器）：用于一次将消息分发给多个朋友。

```
# For giving the message to your friends
require "http/server"

server = HTTP::Server.new(80) do |context|
  context.response.content_type = "text/plain"
    context.response.print "Arch is the best!"
end

puts "Listening."
server.listen
```

csh：一个类似 C 的 shell.

```
#!/bin/csh
echo "Arch is the best!"
```

CSS：一种样式表语言，主要用于设置网页样式。

```
body * {
    display: none;
}

body::before {
    content: "Arch is the best!";
    font-family: monospace;
    font-size: 2.7rem;
    position: absolute;
    left: 50%;
    top: 50%;
    transform: translate(-50%, -50%);
}
```

D：一种 C 风格的语言。有后见之明，以及现代化的便利。

```
import std.stdio : writeln;
void main()
{
    writeln("Arch is the best");
}
```

Dart : 谷歌的 JavaScript 杀手

```
main(){
  print('Arch is the best');
}
```

Dogescript : 友好的 JavaScript

```
console.loge with '                So Arch'
console.loge with '     Much Good'
console.loge with '                          Wow'
```

Ebuild : Gentoo 的构建脚本格式。

```
DESCRIPTION="Arch is the best!"
SRC_URI="https://wiki.archlinux.org/index.php/Arch_is_the_best"

LICENSE="GFDL_1.3"
SLOT="0"
KEYWORDS=""
IUSE=""

DEPEND=""
RDEPEND=""

src_compile() {
einfo "Arch is the best!"
}
```

Emacs Lisp : GNU Emacs 和 XEmacs 文本编辑器使用的 Lisp 编程语言的方言

```
 (message "Arch is the best!")
```

Emojicode : 一种无分隔符，面向对象，命令式，高级的混合语言，以表情符号作为固定点和方法。

```
🏁 🍇
    😀 🔤Arch is the best!🔤❗
🍉
```

Elixir : 一种动态的功能语言，旨在构建可扩展和可维护的应用程序

```
IO.puts "Arch is the best!"
```

Erlang : 并发的，垃圾收集的编程语言和运行时系统。

```
-module(arch).
-export([is_the_best/0]).
   is_the_best() -> io:fwrite("Arch is the best!\n").
```

或在进程之间使用消息传递

```
-module(arch).
-export([ultimate_question/0,the_answer/0]).
the_answer() ->
    receive
        {Client,who_is_the_best} ->
            Client ! {self(),"Arch is the best!"};
        {Client,_} ->
            Client ! {self(),"Taco Taco Taco!"}
    end,
    the_answer().
ultimate_question() ->
    Pid = spawn(arch,the_answer,[]),
```

```
    Pid ! {self(),who_is_the_best},
    receive
        {Pid,Response} -> io:format("~s~n",[Response])
    end.
```

F# : 一种强类型，功能优先的编程语言，用于编写简单的代码来解决复杂的问题。

```
printfn "Arch is the best!"
```

Factor : 基于高级堆栈的语言。

```
"Arch is the best" print
```

FIM++ : 可以使用 Java 类的一种冗长，命令式，动态类型和解释性语言。

```
Dear Princess Celestia: Letter About Arch Linux.
Today I learned:
    I wrote "Arch is the best!".
Your faithful student, Twilight Sparkle
```

Forth : 基于堆栈的语言。

```
." Arch is the best" cr -- kiss way
```

Fortran95

```
program arch
   print *,"Arch is the best!"
end program arch
```

Genie : 一种新的编程语言，它允许使用更现代的编程样式，同时能够毫不费力地本地创建和使用GObjects。

```
init
 print "Arch is the best"
```

Gjs : GNOME 的 Javascript 绑定。它主要基于 Spidermonkey javascript 引擎和 GObject 自省框架。

```
#!/usr/bin/env gjs
print ('Arch is the best');
```

Go : 由 Google 创建的一种语言，它是 C，C++和 Python 之间的一个可爱的孩子。

```
package main

import "fmt"

func main() {
 fmt.Println("Arch is the best!")
}
```

Groovy : 使用 Java 虚拟机的一种敏捷和动态语言。

```
println 'Arch is the best!'
```

Haskell : IO 是一种简单无问题的语言。

```
main = putStrLn "Arch is the best!"
```

HTML : 一种用于创建和定义网页及其内容的标记语言。

```
<!DOCTYPE html>
<html lang='en'>
   <head>
      <title>Arch is the best!</title>
   </head>
   <body>
       Arch is the best!
   </body>
</html>
```

Idris：具有依赖类型的通用纯函数式编程语言。Haskell，但更疯狂。

```
module Main

main : IO ()
main = putStrLn "Arch is the best!"
```

Io：受 Smalltalk，Self，Lua，Lisp，Act1 和 NewtonScript 启发的纯面向对象编程语言。

```
"Arch is the best!" println
```

Java：这是一种非常便携的语言，几乎可以在任何东西上运行，甚至可以在您的烤面包机上运行！

```
public class ArchIsTheBest {
   public static void main(String[] args) {
       System.out.println("Arch is the best!");
   }
}
```

JavaScript：也称为 ECMAScript，这是一种基于原型的面向对象的脚本语言。

```
console.log('Arch is the best!');
```

JavaScript（在网络浏览器中）

```
alert('Arch is the best!');
```

Julia：数值计算的新方法。

```
println("Arch is the best!")
```

Kotlin：JetBrains 试图赢得世界统治。

```
fun main() {
   println("Arch is the best!")
}
```

LilyPond：功能强大的音乐雕刻程序，具有类似 LaTeX 的直观输入语言。

```
\version "2.12.3"
\include "english.ly"
\header { title = "Arch is the best!" }
\score
{
   <<
      \relative c' { c4 e g c \bar "||" }
      \addlyrics   { Arch is the best! }
   >>
}
```

LOLCODE：为什么不呢？

```
HAI
CAN HAS STDIO?
VISIBLE "ARCH IS TEH PWNZ LOL!"
KTHXBYE
```

Lua：一种轻量级，可扩展的编程语言。

```
print "Arch is the best!"
```

Malbolge：为使编程尽可能困难而创建的语言。

```
bCBA@?>=<;:9876543210/.-,+*)('&%$#"!~}|{zyxwvutsrqponmlkjihgfedcba`_^]
\[ZYXWVUTSRQPONMLKJIHGFEDCBA@?>=<;:9y16543210/.-,+*)('&}C#"!~}|{zyxwvu
tsrqponmlkjihgfedcba`_^]\[ZYXWVUTSRQPONMLK-CgGFEDCBA@?>=<;:98x6543210/
.-,+*)('&%$#"!~}|u;yxwpun4rqpRhmf,jihgIe^$ba`_^]\[ZYXQVUTMqQPONMFjJI+A
```

```
eEDC%A:^>=<|:981U54t21*/.-&Jk)('&}C#"!aw={z\xwvun4lqpi/mlkjiKaf_%p
```

Matlab : MathWorks开发的专有编程语言。

```
disp('Arch is the best!');
```

Morpho : Morpho 是一种支持过程式、面向对象和函数式编程的多范式编程语言。

```
writeln("Arch is the best!");
```

Myrddin : 一种旨在控制和简化的系统编程语言，具有强大的类型检查，泛型，类型推断，闭包和特征。

```
/* mbld -b aitb aitb.myr */
use std
const main = {
  std.put("Arch is the best!\n")
}
```

NASM / Yasm (i686) : 请注意，该字符串位于 .text 部分中，感觉更好。

```
;nasm -f elf32 arch.asm
;ld -o arch arch.o
;./arch

section .text
global _start
_start:
mov edx,len
mov ecx,msg
mov ebx,1
mov eax,4
int 0x80
xor ebx,ebx
mov eax,1
int 0x80
msg: db "Arch is the best!",10
len equ $-msg
```

NASM / Yasm (x86_64) : 采用了 AMD 的好用的新指令 *syscall*。

```
;nasm -f elf64 arch.asm
;ld -o arch arch.o
;./arch

section .text
global _start
s:
    db 'Arch is the best!',0ah
l equ $-s
_start:
    mov rax,1
    mov rdi,1
    mov rsi,s
    mov rdx,l
    syscall
    mov rax,60
    xor rdi,rdi
    syscall
```

Nim : 可移植的轻量级编程语言。

```
echo "Arch is the best!"
```

node.js : 一个基于 Chrome 的 JavaScript 运行时构建的平台，可使用事件驱动的非阻塞 I/O 模型轻松构建快速，可扩展的网络应用程序，从而使其轻巧高效，非常适合跨分布式设备运行的数据密集型实时应用程序。

```
console.log('Arch is the best!');
```

node.js（http 服务器）：一个可以发出"Arch is the best!"信息的 node.js 程序。使用 HTTP

```
require('http').createServer((req,res) => {res.writeHead(200, {'Content-Type': 'text/plain'});res.end('Arch is the best!');}).listen(80);
```

Objective-C：一种反射型，面向对象的编程语言，将 Smalltalk 样式的消息传递添加到 C 编程语言中。

```
NSLog(@"Arch is the best!");
```

OCaml：Caml 编程语言的主要实现。

```
print_endline "Arch is the best!"
```

Octave：高级解释语言，主要用于数值计算。

```
printf("Arch is the best!\n")
```

Ook!：将 brainfuck 翻译成了猩猩的语言。

```
Ook. Ook. Ook. Ook. Ook. Ook? Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook? Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook? Ook. Ook. Ook.
Ook! Ook? Ook. Ook? Ook! Ook? Ook! Ook! Ook. Ook? Ook. Ook. Ook? Ook. Ook? Ook! Ook?
Ook. Ook! Ook! Ook. Ook? Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook? Ook. Ook? Ook! Ook. Ook? Ook. Ook? Ook!
Ook. Ook? Ook. Ook! Ook? Ook! Ook! Ook? Ook! Ook. Ook? Ook! Ook? Ook! Ook! Ook? Ook.
Ook. Ook. Ook. Ook. Ook. Ook? Ook? Ook! Ook? Ook. Ook! Ook! Ook! Ook! Ook! Ook! Ook!
Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook!
Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook. Ook! Ook! Ook! Ook! Ook! Ook! Ook!
Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook!
Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook! Ook. Ook? Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook! Ook? Ook! Ook! Ook? Ook.
Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook. Ook. Ook? Ook? Ook! Ook? Ook. Ook! Ook. Ook. Ook? Ook. Ook? Ook. Ook.
Ook! Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook. Ook. Ook. Ook! Ook. Ook? Ook. Ook? Ook. Ook! Ook. Ook. Ook? Ook. Ook?
Ook. Ook. Ook! Ook. Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook!
Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook! Ook. Ook! Ook! Ook! Ook! Ook!
Ook! Ook! Ook! Ook. Ook? Ook. Ook? Ook. Ook! Ook. Ook. Ook? Ook. Ook? Ook! Ook! Ook!
Ook! Ook! Ook! Ook! Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook! Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook! Ook. Ook. Ook. Ook! Ook. Ook? Ook. Ook? Ook.
Ook. Ook. Ook! Ook. Ook! Ook? Ook! Ook! Ook? Ook! Ook. Ook. Ook. Ook. Ook. Ook. Ook.
Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook. Ook! Ook.
```

Pascal：一种有影响力的命令式和过程式编程语言。

```
program ArchIsTheBest;
begin
  writeln('Arch is the best!');
end.
```

Pepe：一种使您想大吼大叫的编程语言。在线运行\[失效链接 2022-09-17 ⓘ\]

```
# Arch
reeEeeeeeE reeEEEeeEe reeEEeeeEE REeEEeEeee Reee rEeeEeeeee reee

# is t
reeEEeEeeE RrEeEEEeeEE rEeEEEeEee reee

# he
Reee RrEeEEeeEeE rreEe rEEeerEEee reEe

# BEST!
reeEEeeeEe reee reeeEeeeeE
```

Perl：一种高级通用通用解释型动态编程语言。

```
#!/usr/bin/env perl
print "Arch is the best!\n";
```

Perl 6 : Perl 家族的最新成员。

```
#!/usr/bin/env perl6
say 'Arch is the best!';
```

PHP : 通用脚本语言。

```
<?php
echo "Arch is the best!\n";
```

Pixilang : 让我像素化。

```
print("Arch is the best!",0,0,#1897D1)
frame
```

Pony : 面向对象，参与者模型，功能安全的高性能编程语言。

```
actor Main
  new create(env: Env) =>
    env.out.print("Arch is the best!")
```

Portable GNU assembler : `as -o arch.o arch.s && ld -o arch -O0 arch.o`

```
   .section .data
archIsBest:
   .ascii  "Arch is the best!\n"
archIsBest_len:
   .long   . - archIsBest
   .section .text
   .globl _start
_start:
   xorl %ebx, %ebx
   movl $4, %eax
   xorl %ebx, %ebx
   incl %ebx
   leal archIsBest, %ecx
   movl archIsBest_len, %edx
   int $0x80
   xorl %eax, %eax
   incl %eax
   xorl %ebx, %ebx
   int $0x80
```

PostScript : 与打印机通话的语言。

```
%!PS
/monospace 60 selectfont
10 420 moveto
(Arch is the best!) show
showpage
```

Powershell : 基于 .NET 的基于任务的命令行 shell 和脚本语言。

```
Write-Output "Arch is the best!"
```

Processing : 专为电子艺术和视觉设计而构建的开源编程语言和 IDE。

```
println("Arch is the best!");
```

Prolog : 与人工智能和计算语言学相关的通用逻辑编程语言。

```
format('Arch is the best~n',[]).
```

Python : 一种通用的高级编程语言。

```
print('Arch is the best!')
```

QBASIC : 基于 QuickBASIC 的 BASIC 编程语言变体的解释器。

```
PRINT "Arch is the best!"
```

R : 统计计算的语言（还有更多！）。

```
archIsBest <- function() { cat("Arch is the best!\n") }
archIsBest()
```

Racket : Lisp-Scheme 系列中的通用多范式编程语言。

```
#lang racket

(let ([str "Arch is the best!\n"])
    (write-string str)
    (values))
```

Ruby : 一种动态的，反射性的，通用的面向对象的编程语言。

```
#!/usr/bin/ruby -w
puts 'Arch is the best!'
```

Rust : Rust 是一种系统编程语言，运行速度极快，几乎可以防止所有崩溃，并消除了数据争用。

```
fn main() {
    println!("Arch is the best!");
}
```

Salt : Salt 是自动化框架

```
 salt '*' event.fire '{"data":"Arch Is the best!"}' 'arch/best'
```

Scala : 在 JVM 上运行的一种多范例语言。

```
 object ArchIsBest extends App {
     println("Arch is the best!")
 }
```

Scheme : Lisp 的方言。

```
(display "Arch is the best!\n")
```

Seed : 一个库和解释器，通过 GNOME 平台动态地桥接 WebKit JavaScriptCore 引擎。

```
#!/usr/bin/env seed
print ('Arch is the best');
```

Shakespeare Programming Language : 旨在"*make a language with beautiful source code* \[...\]"

```
Arch is the Best.

Arthur, a young man who is the best.
Isabella, a likewise young woman who be.
The Ghost, an undead who is the article.
Beatrice, a young woman who is an adjective.

                    Act I: Setting of the Variables.

                    Scene I: Setting of Isabella and Arthur.

[Enter Arthur and Isabella]

Arthur:
 You are as dirty as the square of the sum of a rotten smelly foul devil
 and a vile lie! You are as bold as the sum of yourself and an evil hog!

Isabella:
```

Thou art as big as the square of a cute fair sweet flower! You are as proud as the sum of thyself and a cow.

[Exit Arthur]

Scene II: Setting of The Ghost.

[Enter The Ghost]

Isabella:
You art as loving as myself. Thou are as huge as the sum of yourself and twice a red old hair. You are as cowardly as the sum of yourself and a fat goat!

[Exit Isabella]

Scene III: Setting of Beatrice.

[Enter Beatrice]

The Ghost:
Thou are as good as the sum of Isabella and a mighty fine rich noble King.

[Exeunt The Ghost and Beatrice]

Act II: Printing Arch is the Best.

Scene I: Arch.

[Enter Arthur and Beatrice]

Beatrice:
Speak thy mind! You art as peaceful as the quotient between thyself and the clearest Lord.

Arthur:
Speak your mind!

[Exeunt Arthur and Beatrice]

[Enter Isabella and The Ghost]

The Ghost:
Thou are as normal as the sum of thyself and a town. Speak thy mind!

Isabella:
Speak your mind!

[Exit Isabella]

[Enter Arthur]

The Ghost:
Speak thy mind!

[Exit Arthur]

Scene II: is.

[Enter Beatrice]

Beatrice:
Thou are as old as the sum of yourself and a nose. Speak your mind!

The Ghost:
You art as pretty as the sum of thyself and a face. Speak your mind.

[Exit The Ghost]

```
[Enter Arthur]

Beatrice:
 Speak thy mind.

                    Scene III: the.

Arthur:
 You are as blue as the sum of yourself and a hamster. Speak thy mind!

[Exit Arthur]

[Enter The Ghost]

Beatrice:
 You are as prompt as the sum of thyself and a lie! Speak your mind.

[Exit Beatrice]

[Enter Isabella]

The Ghost:
 You art as happy as the sum of thyself and a large moon. Speak thy mind!

[Exit The Ghost]

[Enter Arthur]

Isabella:
 Speak thy mind.

[Exit Arthur]

                    Scene IV: Best.

[Enter Beatrice]

Beatrice:
 Thou are as blue as the sum of the sum of a curse and thyself and a bad
 codpiece. Speak your mind! You art as lovely as the sum of the sum of the
 happiness and yourself and a blossoming flower. Speak thy mind.

Isabella:
 You are as healthy as the sum of yourself and a plague! Speak thy mind.
 Thou art as huge as the sum of thyself and a hero. Speak your mind!

[Exit Beatrice]

[Enter Arthur]

Isabella:
 You art as green as the sum of thyself and a mother. Speak your mind.

[Exeunt]
```

Shoes：使用 Shoes 作为 GUI 的 Ruby 版本。

```
Shoes.app :width => 135, :height => 30 do
    para "Arch is the Best!"
end
```

Smalltalk：Smalltalk 是一种面向对象的动态类型的反射式编程语言。

```
Transcript show: 'Arch is the best!'.
```

Solidity：以太坊智能合约的面向对象编程语言。

```
pragma solidity ^0.6.0;

contract ArchIsTheBest {
  function archIsTheBest() external pure returns (string memory) {
    return "Arch is the best!";
  }
}
```

SQL : 结构化查询语言，关系数据库的查询语言

```
SELECT 'Arch is the best!';
SELECT 'Arch is the best!' from dual; -- for Oracle DB
```

Standard ML : 一种具有编译时类型检查和类型推断功能的通用，模块化，功能性编程语言。

```
print "Arch is the best!\n"
```

Swift : Apple Inc. 开发的一种通用，多范式，已编译的编程语言。

```
print("Arch is the best!")
```

Tcl/Tk : 一种脚本语言，通常用于快速原型制作，脚本化应用程序，GUI 和测试。

```
#!/usr/bin/env tclsh
puts "Arch is the best!"
```

TrumpScript : 一种基于特朗普总统话语的脚本语言。

```
say it with me, "Arch is the best!";
america is great.
```

UEFI : 可扩展的固件框架

```
#include <Uefi.h>
EFI_STATUS EFIAPI
ArchIsTheBest (
               IN EFI_HANDLE         ImageHandle,
               IN EFI_SYSTEM_TABLE   *SystemTable
               )
{
   SystemTable -> ConOut-> OutputString(SystemTable->ConOut, L"Arch is the best!\n");
   return EFI_SUCCESS;
}
```

V : 简单，快速，安全，可编译的语言，用于开发可维护的软件。

```
fn main() {
        println('Arch is the best!')
}
```

Vala : Vala 是一种新的编程语言，旨在将现代编程语言功能带给 GNOME 开发人员，而无需施加任何其他运行时要求，并且与使用 C 编写的应用程序和库相比，无需使用其他 ABI。

```
void main(string[] args) {
stdout.printf("\nArch is the best!\n\n");
}
```

var'aq : 战士的编程语言

```
"Arch is the best!" cha'
```

Verilog : 一种硬件描述语言，标准化为 IEEE 1364

```
module top;
  initial $display("Arch is the best!");
endmodule
```

VHDL : VHSIC 硬件描述语言

```
use std.textio.all;

entity top is
end top;

architecture behaviour of top is begin
  process begin
    write (output, String'("Arch is the best!"));
    wait;
  end process;
end behaviour;
```

VimScript : Vim 文本编辑器的脚本语言。

```
echo "Arch is the best!"
```

Visual Basic : Microsoft 为其组件对象模型（COM）编程模型提供的第三代事件驱动编程语言和集成开发环境（IDE）。

```
Module Arch
  Sub Main()
      MsgBox("Arch is the best!")
  End Sub
End Module
```

wenyan-lang : 古代汉语的编程语言。

```
吾有一言。曰「「阿祺，盡善矣。」」。書之。
```

Wiring（Arduino） : 麻省理工学院开发的开源编程语言"处理"为基础。

```
void setup()
{
   Serial.begin(9600);
}
void loop()
{
   Serial.print("Arch is the best!");
}
```

X11 : X11 是独立于体系结构的系统，用于显示图形用户界面。

```
cc -lX11 arch.c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <X11/Xlib.h>

int main()
{
       Display *d;
       Window w;
       XEvent e;
       int s;

       if (!(d = XOpenDisplay(NULL))) {
               fprintf(stderr, "Couldn't open display, but Arch is the best!\n");
               exit(1);
       }

       s = DefaultScreen(d);
       w = XCreateSimpleWindow(d, RootWindow(d,s), 0, 0, 110, 20, 0,
                               0, WhitePixel(d,s));
       XSelectInput(d, w, ExposureMask | KeyPressMask);
```

```
        XMapWindow(d,w);

        while (1) {
                XNextEvent(d, &e);
                if (e.type == Expose) {
                        XDrawString(d, w, DefaultGC(d, s), 5, 15, "Arch is the best!",
17);
                }
        }

        XCloseDisplay(d);
        return 0;
}
```

Z3 : Microsoft Research 的定理证明器

```
(define-const arch String "Arch is the best")
(simplify (str.++ arch))
```

Zig : 一种通用的编程语言和工具链，用于维护健壮，最佳和可重用的软件。打算弃用 C。

```
const std = @import("std");

pub fn main() !void {
    std.debug.warn("Arch is the best!\n", .{});
}
```

Zimbu : 一种快速，易学且类似于 JS 的编程语言。

```
FUNC Main() int
  IO.write("Arch is the best!")
  RETURN 0
}
```

Zsh : UNIX 命令解释器（shell），与 ksh 极为相似，但包含许多增强功能。

```
#!/bin/zsh -f
setopt extendedglob
print -- $(tput setaf 2) ${$(<<<${${${(@j: :)${(@s:_:)${:-What_Linux_is_the_best?}}}/*
(#b)([A-Z]i)/Arch $match}} tr '?' '!')} $(tput sgr0)
```

Arch 项目

这一章处理几件读完第零册才碰得到的周边事：旧版本软件包去哪找、官方仓库的包信息怎么在网页上查、打包结果用什么查、以及没有启动介质时怎么靠网络引导装系统。

《官方仓库 web 接口》与《网络引导》必读：前者管日常查包，后者按 BIOS、UEFI 对号入座，故障排除等卡住再看。Namcap 读安装与用法就能上手，创建模块留到写规则时。开头讲存档库的两节是同一页中英文两版，读一份就够，别当两份资料。最常踩的坑是把存档页的 mirrorlist 抄进配置——它指向带日期的旧仓库，抄下去等于钉在旧快照上。

2.1　Arch Linux 存档库

难度 进阶 | **篇幅** 10.6 KB | **所属** Arch 项目

本节将讲到

- 位置
- 目录
- 常见问题
- ## Arch Linux repository mirrorlist
- Generated on 2042-01-01

Arch Linux 存档库（**A**rch **L**inux **Archive**，*简称 ala*），以前称为 *Arch Linux 回滚机（Arch Linux Rollback Machine，简称 ARM）*，保存了*官方仓库快照*、*iso 镜像*和 *引导（Bootstrap）程序包*的历史版本。

用途

- 将某个包降级到某个早期版本（最新版本不能用，我需要之前的版本）
- 将所有包恢复到某个指定的历史时刻（所有包都不能用，我要恢复到两个月之前的状态）
- 查找某个历史版本的 ISO 镜像

软件包仅会保存几年时间，之后会被移动到 archive.org 的 Arch Linux Historical Archive.

位置

Arch Linux 存档库目前位于 https://archive.archlinux.org/ 和全球镜像.

这里的源代码可以帮助您架设自己的存档库服务器。

目录

存档库分为下列三个主目录：

```
├── iso
├── packages
└── repos
```

/repos

repos 这个目录包含官方仓库镜像的每日快照，按下例结构组织：

```
repos
├── 2013
│   ├── 08
│   │   └── 31
│   │       ├── community
│   │       ├── community-staging
│   │       ├── community-testing
│   │       ├── core
│   │       ├── extra
│   │       ├── gnome-unstable
│   │       ├── kde-unstable
│   │       ├── lastsync
│   │       ├── multilib
│   │       ├── multilib-staging
│   │       ├── multilib-testing
│   │       ├── pool
│   │       ├── staging
│   │       └── testing
│   ├── 09
│   │   ├── 01
│   │   ├── 02
│   │   ├── ...
│   │   ├── 21
│   │   └── 22
│   ├── 10
│   │   ├── 01
│   │   ├── 02
│   │   ├── ...
│   │
│   ├── 11
│   └── 12
├── 2014
│   ├── 01
│   │   ├── 01
│   │   ├── 02
│   │   ├── ...
│   │
│   ├── 02
│   ├── 03
│   ├── ...
│   └── 09
│       ├── 01
│       ├── ...
│       └── 28
├── last
├── month
└── week
```

注意: 最下面的三个特定目录（**last**、**week** 和 **month**）分别链接到**已同步的最新仓库版本**、**本周星期一版本**和**本月一日版本**。

/packages

packages 这个目录包含每个包的所有版本及其相应的数字签名。每个包一个目录，按首字母排序。

```
├── packages
│   ├── a
│   │   ├── awesome
│   │   │   ├── awesome-3.5.0-1-i686.pkg.tar.xz
│   │   │   ├── awesome-3.5.0-1-i686.pkg.tar.xz.sig
│   │   │   ├── awesome-3.5.0-1-x86_64.pkg.tar.xz
│   │   │   ├── awesome-3.5.0-1-x86_64.pkg.tar.xz.sig
│   │   │   ├── awesome-3.5.1-1-i686.pkg.tar.xz
│   │   │   ├── awesome-3.5.1-1-i686.pkg.tar.xz.sig
│   │   │   ├── ...
│   │   │
│   │   ├── ...
│   │   ├── awstats
│   │   └── axel
│   │
│   ├── b
│   ├── ...
│   └── z
```

你可以使用“魔法目录”.all 按包名访问所有包。这是一个没有子目录的结构。

```
├── packages
│   ├── .all
│   │   ├── awesome-3.5.1-1-i686.pkg.tar.xz
│   │   ├── ...
│   │   ├── zsh-5.0.2-3-i686.pkg.tar.xz
│   │   ├── zsh-5.0.2-4-i686.pkg.tar.xz
│   │   └── ...
```

可以下载一个压缩的索引文件，包含完整的软件包列表 index.0.xz.

```
$ curl https://archive.archlinux.org/packages/.all/index.0.xz | unxz
0ad-a14-1-i686
0ad-a14-1-x86_64
0ad-a14-2-i686
...
zziplib-0.13.62-1-x86_64
zziplib-0.13.62-2-i686
zziplib-0.13.62-2-x86_64
```

/iso

iso 目录按发布日期，保存官方 ISO 镜像和启动压缩包。

```
├── 2014.09.03
├── 2014.10.01
├── 2014.11.01
├── 2014.12.01
├── 2015.07.01
├── 2015.08.01
├── 2015.09.01
└── 2015.10.01
    ├── arch
    ├── archlinux-2015.10.01-dual.iso
    ├── archlinux-2015.10.01-dual.iso.sig
    ├── archlinux-2015.10.01-dual.iso.torrent
    ├── archlinux-bootstrap-2015.10.01-i686.tar.gz
    ├── archlinux-bootstrap-2015.10.01-i686.tar.gz.sig
    ├── archlinux-bootstrap-2015.10.01-x86_64.tar.gz
    ├── archlinux-bootstrap-2015.10.01-x86_64.tar.gz.sig
    ├── md5sums.txt
    └── sha1sums.txt
```

提示：从 2022.10.01 开始 Arch Linux Archive 加入了 BT 种子(非磁力链). 所有种子文件可以从发布页面下载。

常见问题

如何降级某个包

在 /packages 中找到需要的软件包，用 pacman 获取并安装:

```
# pacman -U https://archive.archlinux.org/packages/path/packagename.pkg.tar.xz
```

pacman 会自动下载 *.sig* 文件并根据 `/etc/pacman.conf` 设置进行校验。

或者手动下载并通过 `pacman -U` 安装.

软件包降级#自动化包含了可以简化这个过程的工具。

如何恢复所有包到指定日期

如果想恢复所有包到指定日期（比如2014年3月30日），你必须如下例所示编辑 `/etc/pacman.conf`，从而让 pacman 保持在这个时间点并且直接使用指定的服务器：

```
/etc/pacman.conf
[core]
SigLevel = PackageRequired
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch

[extra]
SigLevel = PackageRequired
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch

[community]
SigLevel = PackageRequired
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch
```

或者如下例编辑 `/etc/pacman.d/mirrorlist`：

```
/etc/pacman.d/mirrorlist
##
## Arch Linux repository mirrorlist
## Generated on 2042-01-01
##
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch
```

然后同步包数据库以强制降级：

```
# pacman -Syyuu
```

如果抱 PGP 签名错误，软件包损坏(corrupted/invalid packages),请先尝试更新 archlinux-keyring包 和 ca-certificates包. Alternatively, you can decide to temporarily disable signature checking altogether.

注意：混用归档和更新镜像很不安全。万一降级失败，可能使用的是上游软件包，会出现软件包的 epoch 和系统其它软件不一致的现象。

历史存档

维护 Arch Linux Archive 需要很多资源，所以需要定期删除老软件包。

删除前软件包会被上传到 archive.org 上的 "Arch Linux Historical Archive".

此仓库并没有提供某个时间的快照，而是提供了 `archive.archlinux.org` 重定向，老软件包会被重定向到 `archive.org`。除了下载速度慢，用户侧使用上差别不大。

从 Historical Archive 查找软件包

Arch Linux Historical Archive 包含所有软件包的索引:
https://archive.org/details/archlinuxarchive

可以通过 **identifier** 获取软件包，格式是 `archlinux_pkg_*sanitized_package_name*`.

要得到一个**规范化**的软件包名，将软件包名称中的 `@`, `+` 和 `.` 都替换为下划线 `_`.

例如 lucene++包 的软件包名称是 `archlinux_pkg_lucene__`。

通过如下页面查找软件包： https://archive.org/details/archlinux_pkg_lucene__.

通过 archive.org Python 客户端查找:

```
$ ia search subject:"archlinux package" subject:'mysql'
{"identifier": "archlinux_pkg_ejabberd-mod_mysql"}
{"identifier": "archlinux_pkg_ejabberd-mod_mysql-svn"}
{"identifier": "archlinux_pkg_gambas3-gb-db-mysql"}
{"identifier": "archlinux_pkg_gambas3-gb-mysql"}
{"identifier": "archlinux_pkg_libgda-mysql"}
```

从 Historical Archive 下载软件包

所有软件包的版本可以通过如下下载页面获取:
https://archive.org/download/archlinux_pkg_lucene__.

要通过 pacman 下载安装:

```
# pacman -U https://archive.org/download/archlinux_pkg_cjdns/cjdns-16.1-3-x86_64.pkg.tar.xz
```

pacman 的 `RemoteFileSigLevel` 选项控制校验方式，注意依赖关系需要自己解决。

通过 \[https://gi

- Moved to archive.archlinux.org on 2015-12-19.
- Old packages from 2013-2016 uploaded to archive.org on 2018-06-05.thub.com/jjjake/internetarchive archive.org Python 客户端\]也能进行下载。

下载某个指定版本：

```
$ ia download archlinux_pkg_cjdns cjdns-16.1-3-x86_64.pkg.tar.xz{,.sig}
```

下载某个软件包的所有 x86_64 版本：

```
$ ia download archlinux_pkg_cjdns --glob="*x86_64.pkg.tar.xz*"
```

历史

- 最早的 ARM （*Archlinux 回滚机*） 已于 2013-08-18 关闭。
- seblu.net 新站点已于 2013-08-31 上线。
- 2015-10-13 旧站关闭，同时启用新 URL 并导入一个新软件 agetpkg-gitAUR 。
- 2015-12-19 迁移至 archive.archlinux.org。
- 从 2018-06-05 开始 2013-2016 的老软件包上传到了 archive.org on .

2.2 Arch Linux Archive

难度 进阶 | **篇幅** 10.8 KB | **所属** Arch 项目

本节将讲到

- 位置
- 目录
- 常见问题
- ## Arch Linux repository mirrorlist
- Generated on 2042-01-01

Arch Linux 存档库（**A**rch **L**inux **Archive**，*简称 ala*），以前称为 *Arch Linux 回滚机*（*Arch Linux Rollback Machine*，*简称 ARM*），保存了*官方仓库快照*、*iso 镜像*和*引导程序包*的历史版本。

用途

- 将某个包降级到某个早期版本（最新版本不能用，我需要之前的版本）
- 将所有包恢复到某个指定的历史时刻（所有包都不能用，我要恢复到两个月之前的状态）
- 查找某个历史版本的 ISO 镜像

软件包仅会保存几年时间，之后会被移动到 archive.org 的 Arch Linux Historical Archive.

位置

Arch Linux 存档库目前位于 https://archive.archlinux.org/ 和全球镜像.

这里的源代码可以帮助您架设自己的存档库服务器。

目录

存档库分为下列三个主目录：

```
├── iso
├── packages
```

```
└── repos
```

/repos

repos 这个目录包含官方仓库镜像的每日快照，按下例结构组织：

```
repos
├── 2013
│   ├── 08
│   │   └── 31
│   │       ├── community
│   │       ├── community-staging
│   │       ├── community-testing
│   │       ├── core
│   │       ├── extra
│   │       ├── gnome-unstable
│   │       ├── kde-unstable
│   │       ├── lastsync
│   │       ├── multilib
│   │       ├── multilib-staging
│   │       ├── multilib-testing
│   │       ├── pool
│   │       ├── staging
│   │       └── testing
│   ├── 09
│   │   ├── 01
│   │   ├── 02
│   │   ├── ...
│   │   ├── 21
│   │   └── 22
│   ├── 10
│   │   ├── 01
│   │   ├── 02
│   │   ├── ...
│   │
│   ├── 11
│   └── 12
├── 2014
│   ├── 01
│   │   ├── 01
│   │   ├── 02
│   │   ├── ...
│   │
│   ├── 02
│   ├── 03
│   ├── ...
│   └── 09
│       ├── 01
│       ├── ...
│       └── 28
├── last
├── month
└── week
```

注意: 最下面的三个特定目录（**last**、**week** 和 **month**）分别链接到**已同步的最新仓库版本**、**本周星期一版本**和**本月一日版本**。

/packages

packages 这个目录包含每个包的所有版本及其相应的数字签名。每个包一个目录，按首字母排序。

```
├── packages
│   ├── a
│   │   ├── awesome
│   │   │   ├── awesome-3.5.0-1-i686.pkg.tar.xz
│   │   │   ├── awesome-3.5.0-1-i686.pkg.tar.xz.sig
│   │   │   ├── awesome-3.5.0-1-x86_64.pkg.tar.xz
```

```
│   │   │   ├── awesome-3.5.0-1-x86_64.pkg.tar.xz.sig
│   │   │   ├── awesome-3.5.1-1-i686.pkg.tar.xz
│   │   │   ├── awesome-3.5.1-1-i686.pkg.tar.xz.sig
│   │   │   ├── ...
│   │   ├── ...
│   │   ├── awstats
│   │   └── axel
│   ├── b
│   ├── ...
│   └── z
```

你可以使用“魔法目录”.all 按包名访问所有包。这是一个没有子目录的结构。

```
├── packages
│   ├── .all
│   │   ├── awesome-3.5.1-1-i686.pkg.tar.xz
│   │   ├── ...
│   │   ├── zsh-5.0.2-3-i686.pkg.tar.xz
│   │   ├── zsh-5.0.2-4-i686.pkg.tar.xz
│   │   └── ...
```

可以下载一个压缩的索引文件，包含完整的软件包列表 index.0.xz.

```
$ curl https://archive.archlinux.org/packages/.all/index.0.xz | unxz
0ad-a14-1-i686
0ad-a14-1-x86_64
0ad-a14-2-i686
...
zziplib-0.13.62-1-x86_64
zziplib-0.13.62-2-i686
zziplib-0.13.62-2-x86_64
```

/iso

iso 目录按发布日期，保存官方 ISO 镜像和启动压缩包。

```
├── 2014.09.03
├── 2014.10.01
├── 2014.11.01
├── 2014.12.01
├── 2015.07.01
├── 2015.08.01
├── 2015.09.01
└── 2015.10.01
    ├── arch
    ├── archlinux-2015.10.01-dual.iso
    ├── archlinux-2015.10.01-dual.iso.sig
    ├── archlinux-2015.10.01-dual.iso.torrent
    ├── archlinux-bootstrap-2015.10.01-i686.tar.gz
    ├── archlinux-bootstrap-2015.10.01-i686.tar.gz.sig
    ├── archlinux-bootstrap-2015.10.01-x86_64.tar.gz
    ├── archlinux-bootstrap-2015.10.01-x86_64.tar.gz.sig
    ├── md5sums.txt
    └── sha1sums.txt
```

提示：从 2022.10.01 开始 Arch Linux Archive 加入了 BT 种子(非磁力链). 所有种子文件可以从发布页面下载。

常见问题

如何降级某个包

在 /packages 中找到需要的软件包，用 pacman 获取并安装:

```
# pacman -U https://archive.archlinux.org/packages/path/packagename.pkg.tar.xz
```

pacman 会自动下载 *.sig* 文件并根据 `/etc/pacman.conf` 设置进行校验。

或者手动下载并通过 `pacman -U` 安装.

软件包降级#自动化包含了可以简化这个过程的工具。

如何恢复所有包到指定日期

如果想恢复所有包到指定日期（比如2014年3月30日），你必须如下例所示编辑 `/etc/pacman.conf`，从而让 pacman 保持在这个时间点并且直接使用指定的服务器：

```
/etc/pacman.conf
[core]
SigLevel = PackageRequired
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch

[extra]
SigLevel = PackageRequired
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch

[community]
SigLevel = PackageRequired
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch
```

或者如下例编辑 `/etc/pacman.d/mirrorlist`：

```
/etc/pacman.d/mirrorlist
##
## Arch Linux repository mirrorlist
## Generated on 2042-01-01
##
Server=https://archive.archlinux.org/repos/2014/03/30/$repo/os/$arch
```

然后同步包数据库以强制降级：

```
# pacman -Syyuu
```

如果报 PGP 签名错误，软件包损坏（corrupted/invalid packages），请先尝试更新 archlinux-keyring包 和 ca-certificates包。或者，您可以决定临时完全禁用签名检查。

注意：混用归档和更新镜像很不安全。万一降级失败，可能使用的是上游软件包，会出现软件包的 epoch 和系统其它软件不一致的现象。

历史存档

维护 Arch Linux Archive 需要很多资源，所以需要定期删除老软件包。

删除前软件包会被上传到 archive.org 上的 "Arch Linux Historical Archive".

此仓库并没有提供某个时间的快照，而是提供了 `archive.archlinux.org` 重定向，老软件包会被重定向到 `archive.org`。除了下载速度慢，用户侧使用上差别不大。

从 Historical Archive 查找软件包

Arch Linux Historical Archive 包含所有软件包的索引:
https://archive.org/details/archlinuxarchive

可以通过 **identifier** 获取软件包，格式是 `archlinux_pkg_*sanitized_package_name*`.

要得到一个**规范化**的软件包名，将软件包名称中的 `@`, `+` 和 `.` 都替换为下划线 `_`.

例如 lucene++包 的软件包名称是 `archlinux_pkg_lucene__`。

通过如下页面查找软件包： https://archive.org/details/archlinux_pkg_lucene__.

通过 archive.org Python 客户端查找:

```
$ ia search subject:"archlinux package" subject:'mysql'
{"identifier": "archlinux_pkg_ejabberd-mod_mysql"}
{"identifier": "archlinux_pkg_ejabberd-mod_mysql-svn"}
{"identifier": "archlinux_pkg_gambas3-gb-db-mysql"}
{"identifier": "archlinux_pkg_gambas3-gb-mysql"}
{"identifier": "archlinux_pkg_libgda-mysql"}
```

从 Historical Archive 下载软件包

所有软件包的版本可以通过如下下载页面获取: https://archive.org/download/archlinux_pkg_lucene__.

要通过 pacman 下载安装:

```
# pacman -U https://archive.org/download/archlinux_pkg_cjdns/cjdns-16.1-3-x86_64.pkg.tar.xz
```

pacman 的 `RemoteFileSigLevel` 选项控制校验方式，注意依赖关系需要自己解决。

通过 archive.org Python 客户端也能进行下载。

下载某个指定版本：

```
$ ia download archlinux_pkg_cjdns cjdns-16.1-3-x86_64.pkg.tar.xz{,.sig}
```

下载某个软件包的所有 x86_64 版本：

```
$ ia download archlinux_pkg_cjdns --glob="*x86_64.pkg.tar.xz*"
```

历史

- 最早的 ARM （*Archlinux 回滚机*） 已于 2013-08-18 关闭。
- seblu.net 新站点已于 2013-08-31 上线。
- 2015-10-13 旧站关闭，同时启用新 URL 并导入一个新软件 agetpkg-gitAUR 。
- 2015-12-19 迁移至 archive.archlinux.org。
- 从 2018-06-05 开始 2013-2016 的老软件包上传到了 archive.org。

2.3 官方仓库 web 接口

难度 入门 | **篇幅** 3.6 KB | **所属** Arch 项目

本节将讲到

- 软件包信息
- 软件包搜索
- 参见

本文提供了官方仓库网页界面的文档，通过该界面可以查询官方仓库并获取 JSON 格式的结果。

软件包信息

基础链接: `https://www.archlinux.org/packages/`

软件包细节

链接格式: `/*仓库*/*架构*/*软件包*/json`

例子: https://archlinux.org/packages/core/x86_64/coreutils/json/

文件列表

链接格式: `/*仓库*/*架构*/*软件包*/files/json`

例子: https://archlinux.org/packages/core/x86_64/coreutils/files/json/

软件包搜索

该界面支持除了`sort`以外的和HTML Search Form一样的查询格式。

基础链接: `https://www.archlinux.org/packages/search/json`

通过名称或描述搜索

参数: `q`

例子: https://archlinux.org/packages/search/json/?q=pacman

通过准确的名称搜索

参数: `name`

例子: https://archlinux.org/packages/search/json/?name=coreutils

通过描述搜索

参数: `desc`

例子: https://archlinux.org/packages/search/json/?desc=pacman

通过仓库筛选

可以多次使用这个参数以在多个仓库中搜索(但是请注意,如果完全省略它会在所有仓库中搜索)。

参数: `repo`

可用值: `Core`, `Core-Testing`, `Extra`, `Extra-Testing`, `Multilib`, `Multilib-Testing`

例子: https://archlinux.org/packages/search/json/?q=cursor&repo=Core&repo=Extra

通过架构筛选

可以多次使用这个参数以在多个架构的软件包中搜索(但是请注意,如果完全省略它会在所有架构的软件包中搜索)。

参数: `arch`

可用值: `any`, `x86_64`

例子: https://archlinux.org/packages/search/json/?q=cursor&arch=any&arch=x86_64

通过维护者搜索

参数: `maintainer`

例子: https://archlinux.org/packages/search/json/?repo=Extra&maintainer=orphan

通过打包者搜索

参数: `packager`

通过标记状态搜索

参数: `flagged`

可用值: `Flagged`, `Not+Flagged`

例子: https://archlinux.org/packages/search/json/?arch=x86_64&flagged=Flagged

参见

- Forum thread
- Initial feature request: FS#13026
- Kittypack: A silly little tool to poke archlinux.org/packages for info

2.4 Namcap

难度 入门 | **篇幅** 4.7 KB | **所属** Arch 项目

本节将讲到

- 安装
- 用法
- 创建模块

namcap 是用于检查二进制软件包和源代码 PKGBUILD 中常见的打包错误的一个 Arch Linux 工具。

安装

安装 namcap包。

用法

对某一文件（可为 `PKGBUILD` 或包二进制文件如 `*包名*.tar.zst`）运行 namcap：

```
$ namcap 文件路径
```

使用 `--info`（`-i`）标志运行 namcap 以输出额外信息：

```
$ namcap --info 文件路径
```

namcap 使用一种“标签”系统来分类输出。标签分为三种类型：“错误”（error，表示为 `E`）、“警告”（warning，表示为 `W`）和“信息”（informational，表示为 `I`）。错误十分重要，必须立即修复，其通常有关安全性不充分、缺少许可证或权限问题。

通常，namcap 会打印出一个人类可读的说明（可能包含修复问题的建议）。要使输出更容易由程序解析，向 namcap 传递 `--machine-readable`（`-m`）标志。

“标签文件” `/usr/share/namcap/namcap-tags`（在线源）指定了与代码中使用的带连字符标签相对应的人类可读描述（以 `#` 开头的行被视为注释）：

```
# 注释
machine-parsable-tag %s :: 标签 %s 的人类可读描述
```

注意：

- `::`（双冒号）用于分隔机器可读的标签和人类可读的描述。
- `%s` 是一个格式说明符（format specifier），详见#创建模块。

更多信息请参阅 namcap(1)、README 和 NEWS。

创建模块

namcap 主程序（`namcap.py`）接受一个包文件或 `PKGBUILD` 作为参数，并创建一个 pkginfo 对象，然后将其传递给在 `__tarball__` 和 `__pkgbuild__` 中定义的一系列规则。自定义的模块需要添加到适当的数组中：

- `__tarball__` 定义处理二进制包的规则，

- `__pkgbuild__` 定义处理 `PKGBUILD` 的规则。

一个示例的 namcap 模块如下所示：

```
namcap/url.py
import pacman

class package:
    def short_name(self):
        return "url"

    def long_name(self):
        return "Verifies url is included in a PKGBUILD"

    def prereq(self):
        return ""

    def analyze(self, pkginfo, tar):
        ret = [[], [], []]
        if not hasattr(pkginfo, 'url'):
            ret.append(("missing-url", ()))
        return ret

    def type(self):
        return "pkgbuild"
```

每个 namcap 模块必须具有以下方法：

- `short_name(self)`——返回一个模块短名称的字符串。通常，这与模块文件的 basename(1)（去除扩展名的文件名）相同。
- `long_name(self)`——返回一个包含模块简明描述的字符串。此描述会在使用 `namcap --rules=*`规则列表`*` 列出所有规则时使用。
- `prereq(self)`——返回一个包含模块正常运行所需先决条件的字符串。对于处理 `PKGBUILD` 的模块，通常是空字符串（`""`）；对于处理包文件的模块，则是`"tar"`。如果在进一步处理之前需要将包内容提取到临时目录，则应指定`"extract"`。
- `analyze(self, pkginfo, tar)`——返回一个包含三个子列表的列表，三个子列表分别对应“错误”、“警告”和“信息”标签。每个标签列表的成员应该是一个元组，包含两个部分：标签的短形式（上文提到的带连字符标签，可带有适当格式说明符（format specifier，如 `%s`））以及格式说明符所对应的参数元组。
- `type(self)`——对于处理 `PKGBUILD` 的模块，返回 `"pkgbuild"`；对于处理二进制包文件的模块，返回 `"tarball"`。

2.5 网络引导

难度 进阶 | **篇幅** 7.2 KB | **所属** Arch 项目

本节将讲到

- BIOS引导
- UEFI引导
- 故障排除

网络引导映像是小型 （< 1 MiB） 映像，可用于在系统启动时即时下载最新的 Arch Linux 版本。无需更新网络引导映像，最新版本将自动可用。网络引导镜像可以从 Arch Linux Netboot 页面下载。

注意：您需要足够的内存（可能是 1.5GiB 甚至更多）来存储和运行实时系统，否则您可能会在启动时遇到内核崩溃。

BIOS引导

若要在基于 BIOS 引导的计算机上使用网络引导，需要 `ipxe.lkrn` 或 `ipxe.pxe` 映像。

使用 ipxe.lkrn

`ipxe.lkrn` 映像可以像 Linux 内核一样引导。任何 Linux 引导加载程序（如 GRUB 或 Syslinux）都可用于从硬盘驱动器、CD 或 USB 驱动器加载它。例如，Syslinux wiki 提供了在可引导介质上安装 和配置 Syslinux 的说明。

您可以通过以下步骤制作启动 `ipxe.lkrn`的闪存驱动器：

- 使用 lsblk 找出您的设备路径。让我们假设它是 `/dev/sdc` 。
- 在设备上创建MBR分区表。
- 使用 FAT32文件系统创建主分区并将其标记为活动。
- 挂载分区，在那里创建 `boot/syslinux` 目录并复制 `ipxe.lkrn` 到该 `boot` 目录。

```
# mount /dev/sdc /mnt
# mkdir -p /mnt/boot/syslinux
# cp ipxe.lkrn /mnt/boot
```

- 创建 syslinux 配置

```
/mnt/boot/syslinux/syslinux.cfg
DEFAULT arch_netboot
   SAY Booting Arch over the network.
LABEL arch_netboot
   KERNEL /boot/ipxe.lkrn
```

- 卸载分区

```
# umount /mnt
```

- 安装 Syslinux MBR 和 Syslinux 本身

```
# dd bs=440 count=1 conv=notrunc if=/usr/lib/syslinux/bios/mbr.bin of=/dev/sdc
# syslinux --directory /boot/syslinux/ --install /dev/sdc1
```

- 现在，您应该能够使用ipxe.lkrn启动USB盘。

或者，您也可以通过运行以下命令来尝试使用 QEMU 的映像：

```
$ qemu-system-x86_64 -enable-kvm -m 2G -kernel ipxe.lkrn
```

使用 ipxe.pxe

`ipxe.pxe` 映像是 PXE 映像。它可以从现有的 PXE 环境进行链式加载。这允许配置一个 DHCP 服务器，以便从网络引导将始终引导到 Arch Linux 网络引导。 或者，您也可以从现有的 pxe 加载器（如 pxelinux）链式加载它。这是一个菜单条目示例：

```
LABEL arch_netboot_chain
  COM32 pxechn.c32
  APPEND ipxe.a56af4e6a9a9.pxe
```

要使此示例正常工作，您必须将 pxechn.c32 复制到 pxelinux.0 所在的目录中。

UEFI引导

ipxe-arch.efi 镜像可用于在 UEFI 模式下启动 Arch Linux 网络引导。仅支持 64 位 UEFI。可以通过 efibootmgr 将映像添加为引导选项，从引导管理器（如 systemd-boot 或 rEFInd）启动，或直接从 UEFI shell 启动。您也可以从 UEFI 系统上的独立 U 盘启动它。

使用 efibootmgr 安装

首先安装 efibootmgr包 软件包。假设您的 EFI 系统分区（ESP） 位于 `*/dev/sdd1*` 并挂载在 `*esp*` 下，您应该按如下方式移动它 - 让我们也给它一个更友好的名称：

```
# mkdir esp/EFI/arch_netboot
# mv ipxe.*.efi esp/EFI/arch_netboot/arch_netboot.efi
```

然后，您可以按如下方式创建启动项：

```
# efibootmgr --create --disk /dev/sdd --part 1 --
loader /EFI/arch_netboot/arch_netboot.efi --label "Arch Linux Netboot" --unicode
```

从U盘启动

如果要从 USB 闪存驱动器启动 Netboot EFI 二进制文件，请将其复制到FAT格式分区上的默认/回退启动路径 (`/EFI/BOOT/BOOTx64.EFI`)。然后，UEFI 系统应自动加载映像。

在GNU/Linux里

例如，假设 `/dev/sd*X*` 作为闪存驱动器，按如下方式准备 USB 闪存驱动器：

1. 如果尚未完成，请在设备上 `/dev/sd*X*` 创建分区表，并在设备上创建分区 (`/dev/sd*Xn*`)。

> **提示**：提示：最兼容的设置是将 MBR 分区表与类型 `0c` 为“W95 FAT32 （LBA）”的单个活动（可引导）主分区一起使用

1. 如果尚未完成，请将分区格式化为 FAT32：

```
# mkfs.fat -F 32 /dev/sdXn
```

1. 挂载文件系统：

```
# mount /dev/sdXn /mnt
```

1. 在挂载的文件系统上创建一个 `/EFI/BOOT` 目录：

```
# mkdir /mnt/EFI /mnt/EFI/BOOT
```

1. 复制 `pxe-arch.efi` 到 x64 UEFI 的默认引导路径：

```
# cp pxe-arch.efi /mnt/EFI/BOOT/BOOTx64.EFI
```

1. 卸载文件系统。

注意：您需要禁用安全启动才能启动。即使使用自定义密钥对 Netboot EFI 二进制文件进行签名，也不会对从镜像下载的内核进行签名。

故障排除

引导 EFI 二进制文件给出“device error”

如果引导 EFI 二进制文件导致 `Failed to execute ... device error`，请确保在 UEFI 设置中启用了网络堆栈。它可能需要初始化网络接口控制器 （NIC） 选项 ROM，因此此外还要查找“板载 LAN 引导 ROM”、“启动 PXE OpROM 策略”或类似设置。

Arch 开发

本章要把"想帮忙"变成"能动手"：Bug Day 与 AUR 扫除日说清何时何地与谁协作，创建一个干净的 chroot 解决本地验证环境，漏洞报告准则则管住从发现到追踪全程。建议先读漏洞报告准则和创建一个干净的 chroot：前者决定你提交的内容能否被接手，后者决定结论是否可信，两节都是必读。Bug Day 篇幅最短，可略过；AUR 扫除日多是模板与名单分工，暂不参与可跳过。

最易误解的是 chroot，以为装好能跑就算干净，其实它隔离的是本机已有的依赖与设置；其次是发现问题就直奔新建报告，开始之前那一步最常被跳过。

3.1　Bug Day

难度 入门 | **篇幅** 1.2 KB | **所属** Arch 开发

本节将讲到

- 什么时候?
- 哪里？
- 谁?

Bug Day 是致力于提高对我们溢出的 bug tracker 认识的一天。今天的目标是使社区参与尝试帮助我们消除尽可能多的错误。

什么时候?

每当开发人员感到无聊时，就会发起 Bug Day。

哪里？

错误本身记录在 bug tracker 中。

有关错误的讨论应在 IRC 上的 #archlinux-bugs 中进行。我们将设法在 Bug Day 期间至少让一位开发人员活跃。

谁?

允许，邀请和鼓励所有人参加 Bug Day。

3.2　AUR 扫除日

难度 入门 | **篇幅** 4.6 KB | **所属** Arch 开发

本节将讲到

- 关于
- 模板

- About
- Template (Package list)
- Template (TU)

关于

AUR 扫除日（AUR Cleanup Day）是隔年（奇数年）9 月 20 号进行一次的活动。

AUR 上存在大量可清理的过时软件包。你可以使用以下模板，将建议清理的软件包提交到公告邮件中提供的协作文档中，也可以将它们提交到 aur-general 邮件列表，或是直接发起请求。软件包维护者们会共同决定要被移除的软件包清单。

请加入 #archlinux-aur 进行协作和探讨。

模板

```
# AUR Cleanup Day September 20th 20XY

## About

AUR Cleanup Day is a bi-yearly (odd years) event on the 20th of September.

The [AUR](https://wiki.archlinux.org/title/Arch_User_Repository) has a large number of obsolete packages which could use cleaning up. Post suggestions of packages below, submit them to the [aur-general mailing list](https://lists.archlinux.org/listinfo/aur-general), or file [requests](https://wiki.archlinux.org/title/AUR_submission_guidelines#Requests) directly. [Package Maintainers](https://wiki.archlinux.org/title/Package_Maintainers) will get together and confirm which packages should be removed.

Join [#archlinux-aur](https://wiki.archlinux.org/title/Arch_IRC_channels) to collaborate and chat.

## Template (Package list)

### Candidates

Check for the package in the sorted lists below before adding.

- [PKGNAME](https://aur.archlinux.org/packages/PKGNAME) - Reason

#### Informative heading

- [PKGBASE](https://aur.archlinux.org/pkgbase/PKGBASE) - Reason

### Possible reasons

- Does not work anymore

- Deprecated by [PKGNAME](https://aur.archlinux.org/packages/PKGNAME)

- Obsoleted by [PKGNAME](https://aur.archlinux.org/packages/PKGNAME)

- Replaced by [PKGNAME](https://aur.archlinux.org/packages/PKGNAME)

- Duplicate of [PKGNAME](https://aur.archlinux.org/packages/PKGNAME)

- Project page and sources are not available
```

```
- "Dead" project; it's very old and does not work with the latest versions of its dependencies

- "Dead" project and too old to be useful

- Project page and sources are not available

- Not needed anymore (it's for old PKGNAME), broken source link

- Old dev-version (CVS/SVN/etc), PKGNAME project uses Git now

- Is it needed in Arch Linux?

- Broken links (see comment by USERNAME)

- Outdated for a long time

- This one should be renamed

- Deprecated by upstream

- Outdated, orphaned

- Broken links, too old, not maintained

- Included in extra/PKGNAME

- Already included in core/PKGNAME

- Old beta version, broken source link, replaced by [PKGNAME](https://aur.archlinux.org/packages/PKGNAME)

- Maintainer wrote that it may be deleted

- Dropped upstream

- Depends on [PKGNAME](https://aur.archlinux.org/packages/PKGNAME) and has not been updated for N years...

- Development discontinued, project page deleted and current version does not work

- Too old to be useful, broken source link -- hm, what is USERNAME's Opinion?

- This package is no longer needed (see comments)

## Template (TU)

**For editing by TUs only!**

### Packages to Remove

In extra:

- [PKGNAME](https://aur.archlinux.org/packages/PKGNAME) - Replaced by PKGNAME

### Packages to Keep

- [PKGNAME](https://aur.archlinux.org/packages/PKGNAME) - Reason

#### Possible reasons

- It is for compiling old stuff, let's keep it, does not harm

- Seems to be actively maintained

- Still useful, because ...
```

```
- So this package should maybe be orphaned, no obvious reason to delete it

- Needed by PKGNAME

- Package has new and active maintainer

- Seems to be down sometimes, but sometimes it works again. Package is maintained and has votes

- Should be rewritten, not dropped

- I orphaned the package

- It's not dead, it only needs some changes to work again

- Package builds with little tweaking

- Builds fine with a little patch in pkgrel N
```

3.3 创建一个干净的 chroot

难度 进阶 | **篇幅** 8.7 KB | **所属** Arch 开发

本节将讲到

- 使用 clean chroot 的意义
- 便捷方式
- 传统方法
- 处理大规模重构
- 小提示和小技巧

使用 clean chroot 的意义

clean chroot 可以直译为 干净/洁净的 chroot。在 clean chroot 环境中构建可以避免软件包依赖缺失问题。不然可能会出现意外的动态链接或是在 PKGBUILD 依赖数组 `depends` 中出现未声明的依赖。此外，它还允许用户参与 core-testing 或 extra-testing 测试仓库情况下，为稳定仓库 (core、extra) 构建软件包。

便捷方式

为了快速在 clean chroot 环境中构建软件包而无需复杂配置，你可以使用 devtools包中的辅助脚本来帮助你。

这些辅助脚本应在 PKGBUILD 的同一目录中调用，就像使用 makepkg 时一样。例如，`extra-x86_64-build` 将会自动从 `/var/lib/archbuild` 的 clean chroot 模板中创建 chroot 环境，更新它，并为 extra 仓库构建软件包。对于 multilib 构建则使用无架构参数的 `multilib-build`。请参考下表了解不同目标仓库和架构对应的构建脚本。

`-c`参数用于重置chroot模板，这在环境损坏时非常有用。在建立全新的 clean chroot 时不需要此参数。

注意：

- *core* 仓库被省略，因为这些软件包必须先通过 *core-testing* 仓库测试才能进入正式仓库
- 如果目标是为本地使用构建 *core* 仓库的软件包，建议直接使用稳定仓库而非测试仓库。此时可以直接使用`extra`构建脚本

目标仓库	架构	构建脚本	使用的 Pacman 配置文件
extra	x86_64	extra-x86_64-build	/usr/share/devtools/pacman.conf.d/extra.conf
core-testing / extra-testing	x86_64	extra-testing-x86_64-build	/usr/share/devtools/pacman.conf.d/extra-testing.conf
core-staging / extra-staging	x86_64	extra-staging-x86_64-build	/usr/share/devtools/pacman.conf.d/extra-staging.conf
multilib	x86_64	multilib-build	/usr/share/devtools/pacman.conf.d/multilib.conf
multilib-testing	x86_64	multilib-testing-build	/usr/share/devtools/pacman.conf.d/multilib-testing.conf
multilib-staging	x86_64	multilib-staging-build	/usr/share/devtools/pacman.conf.d/multilib-staging.conf

提示：pkgctl-build(1)会自动选择正确的构建脚本在 clean chroot 中构建

传统方法

设置 chroot 环境

devtools包 提供了用于创建 clean chroot 并在其中构建软件包的功能，请确保您已安装该软件包。

之后，为创建 clean chroot，请创建一个新文件夹用于存放 chroot，比如 `$HOME/chroot/`：

```
$ mkdir ~/chroot
```

之后定义 `CHROOT` 变量:

```
$ CHROOT=$HOME/chroot
```

现在就可以开始创建 clean chroot 环境了：

注意：chroot 需要 `root` 文件夹的访问权限，因为 `$CHROOT` 目录需要复制其中的必要文件来创建一个干净的工作环境

```
$ mkarchroot $CHROOT/root base-devel
```

注意：

- 若您经常使用 clean chroot 构建软件包，在 `$HOME/.bashrc` 里添加 export 命令以引入 `CHROOT` 变量是一个不错的选择
- 在 btrfs 文件系统中，chroot 会以分卷（subvolume）的形式被创建，因此您必须以 root 用户的身份运行 `btrfs subvolume delete $CHROOT/root` 才能删除这个分卷。

所有相关信息，比如打包者名称、makeflags 等都在 `~/.makepkg.conf` 中，毋庸置疑，您需要编辑 `~/.makepkg.conf` 以确保其被正常构建。别忘了修改 `$CHROOT/root/etc/pacman.d/mirrorlist` 以更换镜像源。

如果需要，您也可以在 `$CHROOT/root/etc/pacman.conf` 中启用 testing 软件仓库以在 chroot 中获取测试版本的软件包。

注意：`~` 和 `$HOME` 变量在"makechrootpkg"构建脚本中会被自动定义为 `/root/`

自定义 pacman.conf

或者，提供一个自定义的 `pacman.conf` 和 `makepkg.conf`，其中包含以下内容：

```
$ mkarchroot -C <pacman.conf> -M <makepkg.conf> $CHROOT/root base-devel
```

警告：在首次创建 clean chroot 时使用自定义的 `pacman.conf` 或 `makepkg.conf` 可能会导致对 chroot 环境进行预期外的自定义调整。*请谨慎使用。*

在 chroot 里构建

首先，请确保 base chroot (`$CHROOT/root`) 是最新的：

```
$ arch-nspawn $CHROOT/root pacman -Syu
```

其次，在包含其 PKGBUILD 的目录中调用 `makechrootpkg` 来构建软件包：

```
$ makechrootpkg -c -r $CHROOT
```

注意：传递 `-c` 参数给 `makechrootpkg` 可确保在构建前清理工作 chroot (`$CHROOT/$USER`)。

预装必要依赖包

要在 `$CHROOT/root/pacman.conf` 中启用的仓库无法提供所需依赖的情况下构建软件包，请使用 `-I *package*` 预先将它们安装到工作 chroot 中：

```
$ makechrootpkg -c -r $CHROOT -I build-dependency-1.0-1-x86_64.pkg.tar.xz -I required-package-2.0-2-x86_64.pkg.tar.xz
```

向 makepkg 传递参数

要向 makepkg 传递参数，请在 end-of-options marker (选项结束符) 之后列出它们；例如，若要强制执行 `check()`：

```
$ makechrootpkg -c -r $CHROOT -- --check
```

处理大规模重构

处理大型重构最简洁的方法是使用 *staging* 仓库。先针对 *extra* 构建第一个软件包并推送到 *staging* 仓库。然后将后续所有软件包都基于 *staging* 重新构建，并一同推送到那里。

如果无法使用 *staging* 仓库，也可以通过如下命令，使用自定义软件包进行构建：

```
# extra-x86_64-build -- -I ~/packages/foobar/foobar-2-1-any.pkg.tar.xz
```

你可以通过多次使用 `-I` 参数来指定安装多个软件包。

处理大规模重构 (Major Rebuild) 有一种更简单但也相对“不够干净 (dirtier)”的方法：将所有已构建的软件包直接安装到 chroot 环境中，且全程不对其进行清理。使用以下命令构建第一个软件包：

```
# extra-x86_64-build
```

使用以下命令构建后续的所有软件包：

```
# makechrootpkg -n -r /var/lib/archbuild/extra-x86_64
```

运行 namcap (即 `-n`参数) 意味着会将软件包安装到 chroot 中。`*-build`脚本默认也会执行此操作。

小提示和小技巧

在tmpfs中构建

若系统拥有足够内存，可为 devtools包 构建脚本指定 tmpfs 文件系统。

```
# mount --mkdir -t tmpfs -o defaults,size=20G tmpfs /mnt/chroots/arch
# extra-x86_64-build -c -r /mnt/chroots/arch
```

3.4 漏洞报告准则

难度 进阶 | **篇幅** 14.4 KB | **所属** Arch 开发

本节将讲到

- 开始之前
- 新建一个 Bug 报告
- 追踪 Bug

在 Arch Linux 问题跟踪系统报告问题是帮助社区的一种方式。然而，质量不高的问题报告却会起反作用，浪费开发者的时间。此文档将像那些愿意帮助社区的人给出有效报告问题和解决问题的指南。

参见 Simon Tatham 的如何有效的报告问题。

开始之前

准备详细规范的问题报告并不难，但是需要报告者付出精力。**报告前的工作才是最重要的。**不幸的是，很少有人花时间做好这个工作。

下面的步骤将指导您准备问题报告。

查找重复问题

当你遇到问题或要求一个新功能时，极有可能有人也遇到了同样的问题，产生同样的想法。一个问题报告可能已经存在。这时，不要创建重复问题，请跟踪原有问题。

全面查找已有信息，包括：

- Arch Linux 论坛: 论坛通常是用户寻求帮助和分享解决方案的第一处。就算当时问题可能没有解决，各位用户追加的信息和讨论能帮助你找到正确的方向。
- Arch Linux Bugtracker （Arch Linux Bug 管理系统）: 问题可能已经由其他的用户提交给了开发者。重复的问题没有什么帮助，会被直接关闭。在高级中选择全部状态,可以同时搜索已解决和未解决的问题；问题标题可能不带要搜索的关键字，请在高级中选择'搜索详情' 和 '搜索评论' 。
- 在Google 或者您常用的搜索引擎中搜索: 试着搜索程序名称、版本和相关的错误消息（如果有的话）。
- 上游论坛,邮件列表和 Bug 回报系统: 如果 Arch Linux 不是造成 Bug 的原因，这个 Bug 应该提交到上游（而不是 Arch Linux）。试着搜索最近关闭的 Bug 报告和其它正在进行中的报告，你所遇到的 Bug 可能已经在上游被修复了。
- **其它发行版的论坛**: 自由软件社区很宏大，有想法的不只有 Archer ! 考虑一下搜索其它发行版的论坛，例如 Gentoo Forums, FedoraForum.org, 和 Ubuntu Forums。

上游还是 Arch?

Arch Linux 是一个 GNU/Linux *发行版* . Arch 开发者们和 Trusted User 们负责从各处编译,打包并分发软件.**上游**意味着*来源* – 就是 Arch Linux 所分发的软件的来源.例如 FireFox 浏览器是由 Mozilla 开发的.

如果 Arch 不是造成 Bug 的主要原因,把 Bug 报告提交到 Arch 开发者并不能解决问题.

通过向上游提交 Bug 报告,你不只帮助到了 Arch Linux 用户和开发者,同时也是在帮助自由软件社区的其他用户们.这样其它发行版的用户也能从中获得解决方案.

如果你从上游获得了相关的信息,也许可以把它们发送到 Arch Linux Bugtracker ,这样 Arch 开发者和用户们便能从中了解一些信息.

所以 Arch Linux 负责什么?

- **Arch Linux 项目**: pacman, AUR, mkinitcpio, Arch Linux 网站. 不知道这个包是不是 Arch 的项目? 查找项目信息 (`pacman -Qi *package_name*` 或者通过网站) 并从中寻找上游网址 .

注意：Report issues for Arch Linux software projects in the project's respective issue tracker on gitlab.archlinux.org or GitHub instead of bugs.archlinux.org.

- **打包**: 打包包含从上游获取源代码,用相关的选项编译,确认它能在 Arch 上正确安装与运行等环节. Arch 的 打包环节不包含增加新功能或是修复 Bug ,这些是上游开发者的工作.

如果某个 Bug/Feature 不是 Arch 的原因,请把它提交给上游.另参阅有些不是 Bug 的原因.

Bug 或者 Feature ?

bug : 某些该工作的地方没有按开发者的预期工作. **feature** : something which software does or would do if somebody coded it.

有些不该提交 Bug 报告的原因

- 你想要一个还没有的功能（这是功能请求）。
- 已经提交给上游的 Bug
- 上游已经修复但是 Arch 还没更新软件包。
- **某个软件包过期了，**这种情况请使用 Arch Linux 网站上的 **标记这个包已过期**按钮.
- 某个软件包没使用来自其它发行版或其它社区的补丁,**补丁应该提交到上游**.
- 某个软件包中的**非核心功能**没有启用.这种情况应该提交功能请求.
- 某个软件包没有包含 freedesktop 相关文件,例如 **.desktop 文件**或是**图标**.只有在上游提供了这些文件但软件包中没有使用的情况下才是个 Bug .如果该上游没有提供这些文件,那么应该向上游提交功能请求.

有些不是 Feature 的原因

- 这是个 Bug .....
- **某个软件包过期了，**这种情况请使用 Arch Linux 网站上的 **标记这个包已过期**按钮.
- 某个软件包没使用来自其它发行版或其它社区的补丁,**补丁应该提交到上游**.

收集有用的信息

这是一个关于你的 Bug 报告中应该包含的信息的列表:

- **软件包的具体版本**,类似于"最新的","今天的","\[Extra\]中的"这样的描述毫无意义.
- 软件包所使用的库 (可以在 PKGBUILD 中的 *depends* 中找到),它们可能和问题相关.如果你不知道确切的信息,那就等别人来问.....
- 如果遇到了硬件问题,记得提供内核版本和硬件生产厂家。
- 这个功能**是否有正常工作过** .如果有的话,什么时间停止工作的.

- 如果有的话,提供其它相关的日志这些信息可能会帮助找到问题所在:
- Systemd 日志.如果使用syslog-ng包, `/var/log/messages` 包含内核和硬件相关问题的信息.
- `/var/log/Xorg.0.log` , `/var/log/Xorg.2.log` 或其他 Xorg 日志包含显示相关的问题的信息.
- 通过 **verbose** 或是 **debug** 模式在终端运行你的程序并收集其输出 (如果有的话,请参阅你的程序提供的文档) . 对于母语不是英语的用户,你也许需要在终端中设置相应的环境变量 (例如`export LC_ALL="C"`) 以使程序以英文输出.这样更多的人便可以读懂它.例如在终端下以 `--verbose` 选项运行 *foobar*:

```
$ export LC_ALL="C"
$ foobar --verbose
```

这样改变环境变量只对当前终端生效.

如果需要提供大量的文本,例如 dmesg 的输出或是 Xorg 日志,最好把它们保存到文件中并随着 Bug 报告提供. **但是请保证提供的文件和 Bug 相关**.

- **复现 Bug 的方法**.这很重要,因为这样可以帮助其他人测试 Bug 是否修复和寻找潜在的修复方案.
- **堆栈跟踪** .包含程序调用过程的列表,这可以帮助定位问题的位置(特别是那些导致程序崩溃的问题).调试/获取跟踪数据包含了使用gdb包 (The GNU Debugger) 生成堆栈跟踪的详细信息.

新建一个 Bug 报告

在你收集到需要的信息并且确定是个 Bug 或者功能请求以后,你就可以提交 Bug 报告或者功能请求了.

创建账户

首先在 Arch 的 Bug 回报系统上注册一个账户,像 Wiki 和论坛一样简单.

各个项目

如果认为某个 Bug 或功能请求确实和 Arch 相关,应该将其放置在正确的项目里.在创建报告时从左侧选择一个正确的项目:

- **Arch Linux** - *testing*, *core*, 或 *extra* 中的软件包的问题. 也包含 Arch Linux 的文档,网站(除了 AUR) 和安全问题.
- **Arch User Repository (AUR)** - AUR 网站代码和服务器相关.记住并不包含访问AUR中软件包的第三方程序.

- **Community Packages** - *community* 中的软件包 . 这并不包含 *unsupported* (AUR) 中的软件包.
- **Release Engineering** - 所有和发行相关的问题 (iso等.....)

对于 AUR 上软件包的问题,请使用 AUR 网站上的评论功能提交给 AUR 上该软件包的维护者.

摘要

请编写一个简洁且有描述性的摘要,下面是一些建议:

- 不要写成 "*pkgname* is broken after the last update" 这样的,这不光没描述性而且没人知道 Arch 中的 "after last update" 是啥时候.
- 用方括号表示软件包名称,例如 "**\[*pkgname*\]** 3.0.5-1 breaks copy-paste feature" .合理的命名可以帮助开发者搜索和排列报告.
- 别太长,因为太长的内容会在列表中被截断.

提示：Flyspray will try to parse URLs containing `@` (e.g. links to mailing list archives) as email address links and fail miserably. To post such URLs, make sure to URL-encode `@` as `%40`.

严重性

选择一个*严重性等级*不会帮助更快的修复 Bug .这只是让关键的问题易于搜索和被开发者标记.

Here is a general usage of severities: 常见的严重等级有这几种:

- **Critical**(严重),导致系统崩溃或启动失败,可能影响到其他用户的问题.或是对外服务的安全漏洞.
- **High**(高) - 程序的主要功能不工作,或是不那么严重的安全问题等等.
- **Medium**(中等) - 程序的非主要功能不工作,不遵循 UNIX 规范等等.
- **Low**(低) - 程序的可选功能(插件或是编译时选项)不工作.
- **Very Low**(很低) - 翻译或是文档错误,偶尔会是功能请求.

包括相关信息

这也许是 Bug 报告中最困难的一个阶段,前面的收集有用的信息一节应该包含了需要的信息;如果不知道要提供什么的话,记得"多多益善".

参见 Simon Tatham 的如何有效的报告问题。

不过开发者可能会要求你提供更多信息,幸运的是在交谈过几次以后你应该会知道该提供哪些信息了.

短小的信息可以包含在你的报告里,长的信息请使用附件功能随附在报告里.

追踪 Bug

不要认为你提交完 Bug 报告就结束了! 开发者和其他用户可能会询问你细节或是提供一些方法.没有持续的反馈会让 Bug 报告有始无终,同时也会惹恼用户和开发者们.

投票与监视

你可以为同样影响到你的 Bug **vote** (投票).投票会让开发者察觉有多少人受到了这个 Bug 的影响.但是如果你遇到了同样的问题,提交相关的信息是比投票更快的解决方法.

Watching (监视/跟踪) 一个 Bug 很重要,你会收到关于你跟踪的 Bug 的进展的邮件.如果你提交了一个报告,请确保选中了 "Notify me whenever this task changes" (在 Task 改变时通知我) 选项.

回答追加的问题

其它人花时间阅读报告并尝试帮助你修复问题,你也应该尽所能向其他人提供你能做到的帮助.收不到回应不仅可能会让你的 Bug 难以修复,还会让帮助你的人失去信心.

如果其他人需要更多信息,请尝试提供给他们.也请你尝试一下可能的解决方案。.

开发者会关闭那些有新回复而提交 Bug 的人超过几周或是一个月没有回复的 Bug 报告。

软件新版本发布时更新问题报告

有时某个 Bug 之在某些版本中出现，在新版本中已经修复，遇到这种情况的话记得在 Bug 报告中提出来然后提交一个关闭请求.

关闭已经解决的问题

有些时候用户可能自己解决了自己遇到的 Bug ,遇到这种情况的话记得在 Bug 报告中提出来,**附上解决方案的参考链接**,然后提交一个关闭请求.

问题的状态

一个 Bug 可能有下面这些状态:

- **Unconfirmed**(未确认) - 这是默认状态,表示还没人确认或是复现这个 Bug.
- **New**(新的/已确认) - 表示这个 Bug 已经确认,但还没关联到相关软件的负责人.通常是因为还没确认哪个软件引发的 Bug 的缘故.
- **Assigned**(已分配) - 这个 Bug 已经分配到与之关联的开发者.不过这不意味着只有这个开发者会帮忙解决这个 Bug .这只是表示开发者会 关注这个Bug,例如

检查任何的修补,发表解决方案和在需要时关闭 Bug 报告.所以不要直接和这个被分配的开发者联系.....

- **Researching**(研究中) - 有人正在寻求解决方案. 这通常表示这个 Bug 可能呢个需要更多经验丰富的用户帮忙解决.
- **Waiting on Response**(等待回复) 和 **Requires testing**(需要测试) - 有人提供了更多信息或是提供了一个可能的解决方法,但是需要反馈.**Watching** (监视/跟踪) 一个 Bug 很重要,因为你可能会被开发者要求提供更详细的信息.
- **A task closure has been requested**(已提交关闭请求) - 这不是最终状态,但你也许会发现某些 Bug 报告是这样的状态.和它关联的开发者会决定是否关闭它.
- **Closed**(已关闭) - 可能因为这不是个 Bug 有些不该提交 Bug 报告的原因或者已经被解决.

像开发者和 TU 这样的人负责管理和更新各个 Bug 的状态.

团队

读完第零册，「谁在维护 Arch」就成了具体问题：本章交代背后有哪些团队、各自负责什么。四节份量差得远：《访问级别和角色》只留了几句线索，可跳读；其余三节按《软件包维护者》《Arch 测试团队》《Arch 安全团队》的顺序连读，先看成为维护者的路径与活跃、不活跃维护者的区分，再看贡献方式、准则与协调，最后是篇幅最长、信息最密的安全团队一节。

最容易踩的坑是把标题当内容：角色一节看着像权限清单，实际只是一页索引；维护者名单里活跃与不活跃并列，挂着名字不等于此刻在履职；团队之间不是上下级，而是靠协调。

4.1　访问级别和角色

难度 入门 | **篇幅** 1.5 KB | **所属** 团队

本节将讲到

- 另请参见

Arch Linux 由具有以下角色（包括其他角色）的个人维护：

开发者 : 开发者维护 core 和 extra 的存储库，开发 Arch 项目并维护 `*.archlinux.org` 服务器。

受信用户 : 受信用户维护 AUR 和 community 存储库。

Arch 测试员 : Arch 测试团队确保提交到 testing 存储库的包正常。

Arch 安全追踪员 : Arch 安全团队跟踪打包上的安全问题 security.archlinux.org。

论坛主持人 : 论坛主持人主持 Arch Linux 论坛。

IRC 主持人 : IRC 频道运营商负责管理主要 IRC 频道.

ArchWiki 维护者 : ArchWiki 维护团队监督和修复 ArchWiki 上的编辑 .

ArchWiki 译者 : ArchWiki 翻译团队翻译 ArchWiki 到其他语言。

另请参见

- Getting involved
- Arch Linux 开发人员
- Arch Linux 受信用户
- Arch Linux 支持人员

4.2 软件包维护者

难度 入门 | **篇幅** 3.0 KB | **所属** 团队

本节将讲到

- 我如何成为软件包维护者?
- 活跃的软件包维护者
- 不活跃的软件包维护者

软件包维护者(之前被称为受信用户)是 Arch Linux 的正式成员，有以下任务：

1. 维护官方软件仓库中的软件包(除 *core* 之外).
2. 维护，管理和监督 AUR 的运行。

"软件包维护者" 也可以指代维护软件包的个人，请参考Arch 术语#软件包维护者.

我如何成为软件包维护者?

成为软件包维护者的*最低*要求如下：

- 知道基本的 shell 脚本
- 使用干净的高质量 PKGBUILD 在 AUR 中维护一些软件包
- 基本的社区参与（邮件列表，论坛，IRC）
- 熟练使用 Web 搜索
- 有关您要维护的软件包类型的一般想法（基本上，为什么要成为软件包维护者？）

即使仅满足这些最低要求也可以成为软件包维护者，但是在标准投票程序中对您进行判断的人们可能会对您有更多的期望。如：

- 参与错误跟踪器（报告，研究，信息）
- Arch 项目的补丁
- 参与一些开源项目（即使它们是您自己的）

如果您在阅读了这些内容之后仍然觉得自己能够胜任软件包维护者，那么第一步就是找到两个愿意支持您的软件包维护者。获得支持之后，您应该撰写一封风趣的申请，并用您的 OpenPGP 密钥签名，然后发送到aur-general 邮件列表中。

注意：如果您联系的某个软件包维护者拒绝支持您的申请，那么若您打算继续寻求其他软件包维护者的支持，则应该告知这一事实。

有关更多信息，请参阅软件包维护者章程和软件包维护者指导原则.

活跃的软件包维护者

请查看 https://archlinux.org/people/package-maintainers/

不活跃的软件包维护者

请查看 https://archlinux.org/people/package-maintainer-fellows/

4.3 Arch 测试团队

难度 入门 | **篇幅** 3.6 KB | **所属** 团队

本节将讲到

- 贡献
- 准则
- 协调

Arch 测试团队是 Arch 社区中的一个小组，负责确保提交到 testing 存储库的软件包正常运行。这包括确保软件包安装正确，不会导致所依赖的软件包损坏等。

Arch 测试员在证明其正确性之后签名软件包，以便可以将它们从 *testing* 存储库移至 *core* 存储库或 *extra* 存储库。

贡献

您可以通过电子邮件与 arch-testing-accounts@archlinux.org 联系并申请测试者帐户，从而申请成为正式的 Arch 测试者。并至少需要在邮件中指明你想使用的用户名。

如果您将得到一个测试帐户，您应该能够登录到 archweb，看到 *signoffs* 标签就可以了。该 *signoffs* 标签将包含当前处于测试仓库包的至少两个列表，并需要 *signoffs*（即，证实一个包的正确性）。

然后，您可以在本地测试列出的软件包，并通过单击每个软件包的 *signoff* 按钮对它们进行签名（如果正确）。

提示：您可以通过使用 arch-signoff包 软件包中的 signoff(1) 从命令行签名软件包来简化过程。

准则

为了测试 Arch 软件包，请牢记以下几个方面：

- 如果要测试依赖于内核模块的内核或软件包，**则应重新启动计算机并确保其正确启动**
- 尽管不禁止在虚拟化软件上进行测试，但是它可能不如在裸机安装中测试软件包有用。这特别适用于易受不同硬件类型影响的软件包，例如内核软件包。

- 如果要测试库，则可能要执行使用该库的二进制文件。使用 ldd 确保加载了共享库文件。
- 同样，如果有一个附带可执行软件包的软件包，则鼓励测试其基本功能。
- 如果在测试软件包时发现错误，请在 bugtracker 上添加详细的错误报告：
- 软件包名称，版本和 pkgrel
- 软件包的哪个组件是错误的组件（例如，二进制文件之一或配置文件）
- 错误的根源（例如，在安装或使用过程中等）
- 任何相关的错误消息/日志
- 确保该错误使用 *Packages: Testing* 类别提交

协调

注意：希望使用 *testing* 存储库的人能经常检查 arch-dev-public 邮件列表，以了解影响测试用户的任何已宣布的更改或注意事项。

您可以在 #archlinux-testing IRC 频道上与其他测试人员协调。

您可以在 arch-commits 邮件列表上查看打包程序活动的更新（高流量）。

4.4 Arch 安全团队

难度 进阶 | **篇幅** 8.7 KB | **所属** 团队

本节将讲到

- 任务
- 贡献
- 程序
- 资源
- 团队成员

Arch 安全团队是一组志愿者，其目标是使用Arch Linux软件包跟踪安全问题。所有问题都在 Arch Linux security tracker上进行跟踪。该团队以前称为 *Arch CVE Monitoring Team*.

任务

Arch 安全团队的任务是为提高 Arch Linux 的安全性做出贡献。

团队最重要的职责是查找和跟踪分配了常见漏洞和披露（CVE）的问题。CVE 是公共的，由 *CVE-YYYY-number* 形式的唯一 ID 标识。

他们发布了 ASA（*Arch Linux Security Advisory*），这是向 Arch 用户分发的特定于 Arch 的警告。ASA 已在跟踪器中安排进行同行评审，并且在发布之前需要团队成员的两次确认。

在 Arch Linux security tracker 使用的是安全团队跟踪包，增加的 CVE 和生成文本咨询的平台。

注意：

- 一个 *Arch Linux Vulnerability Group* (AVG) 是一组涉及相同内的一组包的 CVE 的 *pkgbase*。
- 符合咨询条件的软件包必须是 *core*，*extra*，*community* 或 *multilib* 存储库的一部分。

贡献

为了参与漏洞的识别，建议：

- 遵循 #archlinux-security IRC 频道。它是报告和讨论 CVE，受影响的软件包以及第一个固定软件包版本的主要沟通媒介。
- 为了及早收到有关新问题的警告，可以监视建议的 #邮件列表以获取新的 CVE，并根据需要监视其他来源。
- 我们鼓励志愿者查看咨询中的错误，问题或评论，并在 IRC 频道中进行报告。
- 订阅邮件列表 arch-security 和 oss-security。
- 将代码提交给 arch-security-tracker (GitHub) 项目是为团队做出贡献的好方法。
- 鼓励依赖 Arch Linux 软件包系统信息库的派生发行版做出贡献。这有助于所有用户的安全。

程序

在 Arch Linux 官方存储库中的软件包中发现安全漏洞时，应遵循以下步骤：

信息共享和调查阶段

- 通过您首选的渠道与 Arch 安全团队成员接触，以确保该问题已引起团队注意。
- 为了证实该漏洞，请针对当前程序包版本（包括可能的补丁程序）验证 CVE 报告，并通过搜索引擎收集有关此问题的尽可能多的信息。如果您需要帮助调查安全问题，请在 IRC 频道上寻求建议或支持。

上游情况和错误报告

可能出现两种情况：

- 如果上游发布了可解决此问题的新版本，则安全团队成员应将该软件包标记为过期。
- 如果经过长时间的延迟仍未更新软件包，则应提交有关该漏洞的错误报告。
- 如果这是一个关键的安全问题，则必须在将软件包标记为过期后立即提交错误报告。
- 如果没有上游发行，则必须提交错误报告，其中包括用于缓解问题的补丁。错误报告中必须提供以下信息：
- 有关安全问题及其影响的描述
- 链接到 CVE-ID 和（上游）报告
- 如果没有可用的版本，请链接到可缓解此问题的上游修补程序（或附件）

跟踪和发布

团队成员必须执行以下任务：

- 团队成员将在 security tracker 上创建建议，并添加 CVE 进行跟踪。
- 具有访问 arch-security 权限的团队成员将从跟踪器生成 ASA 并将其发布。

注意：如果要报告私有错误，请联系 security@archlinux.org。请注意，私有错误报告的地址是 *security*，而不是 *arch-security*。私有错误是一个过于敏感的问题，无法发布到任何人都可以阅读和利用的地方，例如 Arch Linux 基础架构中的漏洞。

资源

RSS

National Vulnerability Database (NVD)：所有 CVE 漏洞：

https://nvd.nist.gov/download/nvd-rss.xml：所有经过全面分析的 CVE 漏洞：
https://nvd.nist.gov/download/nvd-rss-analyzed.xml

邮件列表

oss-sec：有关自由软件安全性的主要列表，如果您想了解安全新闻，则此处会包含很多 CVE 属性。：信息：https://oss-security.openwall.org/wiki/mailing-lists/oss-security：订阅：oss-security-subscribe(at)lists.openwall.com：存档：
https://www.openwall.com/lists/oss-security/

完整的披露审核邮件列表（杂乱）。：信息：

https://www.securityfocus.com/archive/1/description：订阅：bugtraq-subscribe(at)securityfocus.com

Full Disclosure：另一个全披露的邮件列表（杂乱）。：信息：

https://nmap.org/mailman/listinfo/fulldisclosure：订阅：full-disclosure-request(at)seclists.org

还可以考虑遵循特定软件包的邮件列表，例如 LibreOffice, X.org, Puppetlabs, ISC 等。

其他发行版

其他发行版的资源（查找CVE，补丁，注释等）：

Red Hat 和 Fedora

: 咨询提要：https://bodhi.fedoraproject.org/rss/updates/?type=security : CVE 跟踪器：https://access.redhat.com/security/cve/<CVE-ID> : 错误跟踪器：https://bugzilla.redhat.com/show_bug.cgi?id=<CVE-ID>

Ubuntu

: 咨询供稿：https://usn.ubuntu.com/usn/atom.xml : CVE 跟踪器：https://people.canonical.com/~ubuntu-security/cve/?cve=<CVE-ID> : 数据库：https://code.launchpad.net/~ubuntu-security/ubuntu-cve-tracker/master

Debian

: CVE 跟踪器：https://security-tracker.debian.org/tracker/<CVE-ID>/ : 补丁追踪器：https://tracker.debian.org/pkg/patch : 数据库：https://salsa.debian.org/security-tracker-team/security-tracker/tree/master/data

OpenSUSE

: CVE 跟踪器：https://www.suse.com/security/cve/<CVE-ID>/

其他

CVE 的 Mitre 和 NVD 链接

: https://cve.mitre.org/cgi-bin/cvename.cgi?name=<CVE-ID> : https://web.nvd.nist.gov/view/vuln/detail?vulnId=<CVE-ID>

NVD 和 Mitre 不一定在归属后立即填写其 CVE 条目，因此与 Arch 并不总相关。CVE-ID 和“创建日期条目”字段没有特殊含义。CVE 由 CVE 编号颁发机构（CNA）归属，每个 CNA 在需要/提出要求时从 Mitre 获取 CVE 块，因此 CVE ID 未链接到归属日期。“创建日期条目”字段通常仅指示将 CVE 块分配给 CNA 的时间，仅此而已。

Linux Weekly News：LWN 每天为各种发行版提供安全更新通知。 : https://lwn.net/headlines/newrss

更多

有关更多资源，请参见 OpenWall 的开源软件安全性 Wiki.

团队成员

Arch 安全团队的当前成员是：

- Levente Polyak
- Remi Gacogne
- Christian Rebischke
- Jelle van der Waa
- Santiago Torres-Arias
- Jonathan Roemer
- Morten Linderud

注意：在 IRC 频道中运行 `!pingsec <msg>`，以突出显示当前所有安全团队成员。

第 5 章　共 4 节　85 KB

建议阅读

本章收拢三类经验：平时怎么用不出事、在国内怎么配得顺手、架服务器时看什么。

《给新用户的关于如何不去弄坏 Arch Linux 系统的建议》人人必读——分区前先备份、不混用测试仓库、不把显卡驱动当成可以"下载"的安装包、慎用 make install；国内读者再读《中国用户的推荐解决方案》，镜像、仓库、中文化与代理一并解决；不架服务器可跳《服务器部署建议》；《建议阅读》是清单，按系统管理、软件包管理、启动、图形界面、电源管理取用。最常踩的坑，是把教程里的命令、配置和脚本当成通用答案照抄。

5.1　建议阅读/给新用户的关于如何不去弄坏 Arch Linux 系统的建议

难度 进阶 | **篇幅** 15.8 KB | **所属** 建议阅读

本节将讲到

- 数据无价，分区操作需谨慎
- 不要混用测试仓库
- 不要“下载”显卡驱动
- 小心 `make install` 和其他类似命令
- 不要盲从教程

原始页面来自 Debian Wiki。

Arch Linux 是滚动更新的系统。从单独的时刻上去看，其即使可能会在更新后出现问题，但由于滚动更新，这些问题也可以被快速修复，因而总体上依然稳定可靠。除去上游引入的问题，用户手上持有的系统最高权限——root 权限——也通常是导致系统损坏的导火索。背离通常的操作手段导致系统故障是正常的，因而我们在此处列出常见的导致系统故障的操作，以期帮助大家避免这些问题。不过，若有充分的知识和谨慎的操作，即使出了问题，通常也是可以妥当解决的。

在此之前，需要知道一点的是，来自官方仓库的软件包受到官方支持，且来自 `core` 仓库的软件包更是会先进行严格的审核。而在其他系统中，你**可能会从各个网站下载安装包，但无论在哪个系统上，这通常都伴随风险**。在 Arch Linux 上，还有很多第三方软件的安装脚本来自 AUR。这些脚本相较于用户手动使用命令编译安装来说，可以让通过脚本安装的软件受到本系统的包管理软件，pacman 管理，这可以极大的减小自行安装与包管理冲突的风险，但这些脚本亦可能包含恶意代码（这曾经发生过，虽然不常见）。因此，请尽量选用来自官方仓库的软件包，并在使用来自 AUR 的安装脚本前对它们做仔细检查（比如检查包含的网址，可能包含的破坏性命令等）。

中文用户可能会选用来自非官方用户仓库，[\[archlinuxcn\]](../../01_安装与包管理/非官方用户仓库/Arch%20Linux%20中文社区仓库.md) 的软件包。同样，你也应该谨慎选用这些软件包，在非官方软件包出问题时也请向相应的人员报告问题。

注意：Arch Linux 32、Arch Linux ARM 等基于 Arch 的发行版不被官方支持。

现在，让我们开始吧。

数据无价，分区操作需谨慎

根据安装指南#分区方案示例，您可能会将除 `/boot` 和 `swap` 之外的所有系统所需内容（这包括 `/home`，您的很多个人数据会存储在这里）放在一个单独的分区。这样的配置对新用户相对友好，但是在您进行某些可能会损害数据的操作（如调整磁盘分区大小）时，由于个人数据和系统文件存放在同一个分区内，若是因为相关操作导致分区损毁，则您很可能会同时失去珍贵的个人数据。

因此，分区操作需谨慎，在安装系统的时候就应当通盘考虑如何分配磁盘，因为稍后再修改所要花费的精力会更大。比如，参考“不要把鸡蛋放在同一个篮子里”的道理，您可以在安装时，将一些目录（如 `/home`）作为单独的挂载点，将这些目录下的内容放在其他分区。这样，即使系统分区出了问题，您的数据也在其他分区内完好无损，并且您可以快速恢复您的工作环境。具体请参阅 fstab。

不要混用测试仓库

如果你使用桌面环境，你有可能想提前尝试测试版本的桌面环境，比如 KDE 和 GNOME，并相应启用 `gnome-unstable` 和 `kde-unstable`。需要注意的是，这些仓库需要同时与测试仓库，也就是 `core-testing` 和 `extra-testing` 同时启用（如果有启用更多的官方仓库也要启用相应的测试仓库），否则可能导致部分升级，简单来说就是新旧软件包不兼容的情况。要提前尝试，就都要提前尝试。一旦产生错误，则必须尽快解决问题。

通常，只要 pacman 可以运行，完整地启用测试仓库后，进行 `pacman -Syu` 就能修复这些问题。

不要“下载”显卡驱动

显卡制造商的官网可能会提供显卡驱动的安装脚本，**但是也请用来自软件仓库的驱动软件包**。Arch Linux 的软件仓库已经（基本上）提供了这些驱动，并且可以伴随 Linux 内核升级而升级，这比单独的安装脚本要可靠和安全得多。通常情况下，查阅 OpenGL、Vulkan 以及 VA-API 这三个 wiki 页面足以引导你安装并配置好运行桌面所需的显卡驱动。

即使是臭名昭著的 NVIDIA，近年来的表现也在逐渐稳定。要查阅与具体硬件品牌相关的更多信息，请参考 NVIDIA、AMDGPU、Xorg#AMD 或 Intel 图形处理器等文。本 wiki 同时提供其他软件在 Arch Linux 上的使用说明，其中也可能包含相关软件在与特定显卡配合使用时需要单独采取的措施。

小心 `make install` 和其他类似命令

pacman 包管理通过统一的方式管理系统软件及其文件，但 `make install`、`ninja install` 等安装的文件不受 pacman 管理，且可能与 pacman 管理的文件相冲突。同样，如果需要卸载由这些命令安装的文件，也需要花费一番功夫。

同样，直接运行这些命令会需要你自行管理这些软件的升级等，事实上十分不便。来自 AUR 的安装脚本实际上采用与软件仓库打包相一致的过程，因而这样安装的软件受到pacman管理。在了解相应风险后，您也可以使用 AUR 助手帮助你升级来自 AUR 的软件包。如果AUR也实在没有，也可以考虑自己创建软件包（当然这并不轻松），让 pacman 管理。还有一些常见的类似建议列在下方。

不要直接 `pip install`

Python 有相当多的库并不在 Arch Linux 仓库内，如果你不想使用 Arch 构建系统亲自打包 Python 程序的话，使用 `pip install` 是个折中的办法。但是，**请一定在 venv（或类似的虚拟环境）中运行**。如果你 `# pip install`，那么你安装的文件会与 pacman 管理的文件产生冲突。然而，即使使用 `pip install --user` 将程序安装到 `~/.local` 下，也可能安装了与系统软件包重复但版本不同的库，从而导致系统上安装的某些依赖该 Python 库的程序因库版本不正确而出错。

如果你需要用的 Python 程序支持用命令行启动运行，而无需在 Python 代码中 `import` 调用的话，推荐使用 python-pipx包 来安装它。pipx 会自动为你创建 venv，并自动在 venv 中安装你需要的程序。

有时候一些 Python 程序会依赖一些重量级库，比如 PyQt/PySide/PyGObject 等，如果在 venv 这样的隔离环境中安装则需要在 venv 中安装所需的 Qt/glib 库，这通常无法与系统上的其它软件良好集成。这种时候，你可以用 `python -m venv --system-site-packages` 来创建 venv 并在其中安装你需要的软件包。这样创建的 venv 不是与系统环境完全隔离的，venv 之内的 Python 能够导入系统范围安装的包，而系统环境的包则看不到 venv 内的包，因此不会干扰系统软件包正常工作，相当于获得了 `pip install --user` 的优点而规避了它的缺点。

不要默认启用 anaconda 环境

anaconda 是个不错的管理 Python 软件的方式。然而，它自带了包括 curl 和 ncurses 在内的许多库文件，其版本可能与系统所需要的版本冲突。因此，请不要默认启用 anaconda 环境，仅在需要使用的时候启用。

不要盲从教程

标题不是在说不要看任何教程，而是说，应该同时比对几份同主题的教程（对于 ArchWiki，则也可以是中英文，甚至其他语言），以及参考说明文档。同样，教程也有时效性，如果是五年前的教程，可能现在已经完全不适用。

但，**无论是在跟随教程之前，还是在对比几份教程之前，都应该搞清楚教程中的命令到底会做出哪些操作**。这样可能会帮助你及时发现错误，当然，即使未能发现，也能更迅速地帮助你恢复系统正常。

除了 ArchWiki 外，man 命令等是已经存在于你系统上的文档。这些也是参考来源。

不要盲目照抄配置文件与脚本

盲目照抄配置文件和脚本对于系统的危害是深远且多方面的。

每个系统或应用都有其独特的运行环境和需求。盲目照抄配置文件、脚本，往往忽略了这些差异，可能导致配置不兼容，进而引发系统错误或崩溃。

当您使用别人提供的脚本时，应当知道它是用来做什么的、它依赖什么软件包、它应当被何 shell 执行等。

点文件一文中提到的其他用户提供的配置文件、各大代码托管平台上的用户点文件仓库以及 AUR 中类似于 ml4w-hyprlandAUR 的包，都是针对那些用户自身，而非您的习惯或设备来设计的。比如说，假如您喜欢某人的终端增强配置，就草率地将其 `~/.bashrc` 直接抄进了您的 `~/.config/fish/config.fish`，这样的话，出现问题时，无论是您要向社区询问，还是他人试图解决您的问题，都会因为这其中积累的无数个“未知”，给您自己和社区带来莫大的麻烦。

移除软件包需谨慎

软件包之间有依赖关系，也就是一个软件包必须和另一些软件包一起存在。因此，移除一个软件包可能会需要相关的其他软件包也被一同移除。

移除软件包前，pacman 会显示要移除的包的列表。**一定要仔细查看每个即将被删除的包**，它们有的可能是你仍需要使用的软件包。如果不清楚，除了询问其他人以外，你还可以通过 pacman 的查询命令来查看每个软件包的简介与相关信息。

记得自己做过什么

记得自己做过什么在出问题的时候可以帮助你或者是让其他人帮你快速解决问题。比如，改动配置文件时，也许可以原地留一下注释和日期，标注原因，之类的。本文先前所提及的“不要照抄其他配置文件”也是同理，充分把握自己的系统这件事对解决之后可能发生的问题而言十分重要。

总的来说，要维持 Arch Linux 系统的稳定，你自然要付出一定的精力。

谨慎对待 LLM 给出的内容

LLM，大型语言模型（也就是 OpenAI 的 ChatGPT，Anthropic 的 Claude，Google 的 Gemini，深度求索的 DeepSeek 等 “AI”），看似是人类可以向 AI 询问任何问题，AI 就会给出看着非常可信的答案。有人可能会觉得如获至宝，看到有人问问题就把问题丢给它们问，再把回答随便贴上去。

但是，现阶段的通用大型语言模型，即便是在有联网工具能力的情况下（比如 ChatGPT 的搜索模式），也只能基本确保它们组出的是**语法语气没有问题**的句子，而**不是事实正确**的句子。即便是现在推理用的大模型，也并**没有对现实的认知能力**，自然没有其他社区成员的经验，询问它们是不大可能获得诸如“你电脑在更新的时候大写锁定键忽然闪灯，是因为闭源 NVIDIA 驱动和内核互作导致更新时内核崩溃，根据你的显卡型号，你应该换用 nvidia-open包 这个包”之类的详细结果的。

好吧，LLM 似乎确实不适合询问莫名其妙的疑难杂症——那如果是让它帮我写一个特定用途的小工具呢？不是说它们很擅长写电脑程序吗？

在你准备问出更多问题之前，再次强调一遍：**现阶段的 AI 没有对现实的认知能力**，它不能保证自己写的符合你的要求，甚至可能因为阴差阳错写出有害的东西出来（虽然更多时候是前者）。就算是用所谓的推理模型也没有用，因为 AI 的推理过程也照样会说谎。

因此，总结一下：如果一段文字看着是 AI 生成的，或者你打算向 AI 问问题，**请先认为你看到的内容一个字都不能信**。也请不要把“它说的对不对啊”的工作推给别人，因为比起对 LLM 缝缝补补，人类自己早就能把问题解决掉了。

寻求他人帮助

Arch Linux 用户通常对 Arch Linux 上的问题更为熟悉。在遵循行为准则的前提下，向官方社区和非官方的国际社区（中文用户可以看看 Arch Linux 中文社区）通常是个好主意。

行为准则#常识有言：

- 使用 Arch Linux，先要接受 Arch 之道
- 先看文档，搜索网站，做好功课，再行提问
- 寻求帮助，耐心委婉
- 乐于奉献，止于损害

遵循基本前提，提供尽可能多的细节，保持耐心多等待，不要宣泄情绪。

当然，每个社区都会有自己的建议。但通常，使用即时通讯软件时，不要频繁换行发言，尽可能一次发完。上传日志建议使用外部网站（比如 Pastebin 和每个群组自行设置的类 Pastebin 网站），并且注意删掉私人信息。询问问题时待在有问题的设备前，不要“离线提问”，避免在他人给出建议的时候你却“暂时给不出日志”又或是“暂时摸不到电脑”而消耗其他社区用户的耐心。

最后，祝你的 Arch Linux 使用之旅顺利且愉快！

5.2 建议阅读/中国用户的推荐解决方案

难度 高级 | **篇幅** 19.6 KB | **所属** 建议阅读

本节将讲到

- 软件镜像
- [社区](https://www.archlinuxcn.org/)支持
- 系统中文化
- 软件仓库
- 应用程序

编者按：

: 众所周知，中国用户有别于国际上的特殊需求，本文旨在提供可能的解决方案。

软件镜像

中国用户使用开源软件镜像可加速下载，提高访问稳定性，同时减轻国外服务器压力，对国内网络环境和开发者社区均有积极影响。

校园网联合镜像站提供了对校园网镜像站的索引和跳转服务。

本节列举了常用软件镜像。

官方仓库镜像

参见镜像源、非官方镜像源、校园网联合镜像站有关信息。

Arch Linux 中文社区仓库镜像

参见 Arch Linux 中文社区仓库、校园网联合镜像站有关信息。

其他非官方用户仓库镜像

对于 `arch4edu`、BlackArch Linux、ALHP 等非官方用户仓库，请自行搜索其镜像及其说明。

ELPA 镜像

ELPA 是 Emacs 内建包管理器 *package.el* 的软件源，许多镜像站支持了常见的 ELPA。

参见 Emacs#软件镜像源。

GNU 软件镜像

对于官方仓库里已打包或未打包的 GNU 软件，也存在一部分镜像。

参见校园网联合镜像站 GNU 软件仓库镜像使用帮助。

Flathub

参见 Flatpak#官方仓库的镜像站。

对于 Flathub，已有 USTC 镜像站和 SJTUG 镜像站的缓存。

CTAN

参见校园网联合镜像站有关内容。

参见 TeX Live。

crates.io

参见 Rust 及 Rust crates.io 索引镜像使用帮助。

社区支持

: 谅解、支援和友谊，比什么都重要。

Arch Linux 是一个社区驱动的发行版。Arch Linux 中文社区是比较活跃的 GNU/Linux 社区，用户们交流频繁，氛围热烈而融洽。Project:Arch Linux 中文社区交流群指引一文可以指引您加入社区群聊，您需要遵守 Arch Linux 中文社区的行为准则。

系统中文化

简体中文本地化、繁体中文本地化、中文本地化以及字体配置/中文等页面包含了较为详尽的 Arch Linux 系统中文化指南。

部分软件的页面亦提供了实现其中文本地化的方法。

软件仓库

Arch Linux 中文社区维护了 Arch Linux 中文社区仓库。

应用程序

办公

常用办公应用程序/套件：

- **腾讯会议** — 腾讯会议官方 Linux 版本。

: https://meeting.tencent.com/ || wemeet-binAUR 或 wemeet-bwrapAUR

- **WPS Office** — 金山公司推出的、运行于 Linux 平台上的全功能办公软件。

: https://linux.wps.cn/ || 参见 WPS Office

- **LibreOffice** — LibreOffice 是一款功能强大且自由的开源办公软件。

: https://zh-cn.libreoffice.org/ || libreoffice-fresh包 或 libreoffice-still包

- **Onlyoffice** — Only Office 是一款免费且开源的 Office 协作办公套件，支持桌面端和移动端等多平台。

: https://www.onlyoffice.com/zh/ || onlyoffice-binCNRepo 或 onlyoffice-binAUR

一些在线办公套件网站可以提供基础的办公功能:

- Office Online: 微软提供的 Office 办公套件的网页版。
- Google Docs, Sheets and Slides: Google 提供的在线文字处理、电子制表和演示程序。
- 金山文档（KDocs）：金山公司出品的可多人实时协作的在线 Office，注意它和 WPS 是不同的产品。
- 腾讯文档（QQDocs）：腾讯公司出品的在线 Office，支持多人在线编辑 Word、Excel 和 PPT 文档。

中文输入法

参见 Fcitx5 、Fcitx、 IBus 或 Scim。

参见 Rime。

在线音乐

- **网易云音乐** — 网易云音乐，从 .deb 包转换而来

: https://music.163.com/ || netease-cloud-musicAUR

- **网易云音乐（第三方）** — Linux 平台下基于 Rust + GTK4 开发的网易云音乐播放器

: https://github.com/gmg137/netease-cloud-music-gtk || netease-cloud-music-gtk4AUR 或 netease-cloud-music-gtk4CNRepo

- **SPlayer（第三方网易云音乐）** — **（现已停止更新，请使用 SPlayer-next）**一个简约的音乐播放器，支持逐字歌词，展示评论区等几乎所有基础功能，拥有极致美学设计

: https://github.com/imsyy/SPlayer || splayerAUR

- **SPlayer-next（第三方网易云音乐）** — 跨平台桌面音乐播放器，支持丰富歌词与多种音频格式

: https://github.com/SPlayer-Dev/SPlayer-Next || splayer-next-binAUR

- **YesPlayMusic（第三方网易云音乐）** — 网易云音乐的第三方应用程序。但维护频次偏低，出现问题后可能不能及时修复和更新。

: https://github.com/qier222/YesPlayMusic || yesplaymusicAUR

- **网易云音乐（第三方）** — 网易云音乐网页版的 Electron 封装。

: https://music.163.com/st/webplayer || netease-cloud-music-web-playerAUR

- **Open Orpheus（第三方网易云音乐）** — 一个对网易云音乐 Orpheus 浏览器宿主的开源实现，为网易云音乐的原版客户端提供运行环境。支持绝大部分原版客户端功能。

: https://github.com/YUCLing/open-orpheus || open-orpheusAUR

- **酷狗音乐（Spark Store ）** — 星火商店的酷狗音乐客户端

: https://www.kugou.com/ || com.kugou.sparkAUR

- **酷我音乐（第三方）** — 一款优雅的音乐播放器，可从 kuwo.cn 获取歌曲。

: https://github.com/LiuLang/kwplayer || kwplayerAUR

- **QQ 音乐** — 腾讯 QQ 音乐。

: https://y.qq.com/ || qqmusic-binAUR

- **Spotify** — 一款专有的音乐流媒体服务。

: https://www.spotify.com/ || spotifyAUR

- **listen1-desktop** — 中国所有免费音乐一网打尽，支持本地音乐、网易云、QQ、酷狗、酷我、哔哩哔哩、咪咕、千千。

: https://github.com/listen1/listen1_desktop || listen1-desktopAUR 或 listen1-desktop-appimageAUR

- **feeluown** — FeelUOwn 音乐播放器，支持本地音乐、网易云、酷我、QQ、哔哩哔哩。

: https://github.com/feeluown/FeelUOwn || feeluown-full包组

- **TTKMusicPlayer** — 模仿酷狗音乐的 TTKMusicPlayer，该音乐播放器使用基于 Qt 的 qmmp 核心库（预编译版）支持本地音乐、网易云音乐、酷我音乐、酷狗音乐。

: https://github.com/Greedysky/TTKMusicPlayer || ttkmusicplayer-binAUR

- **lx-music-desktop** — 一个基于 Electron + Vue 开发的音乐软件。

: https://github.com/lyswhut/lx-music-desktop || lx-music-desktop-binAUR

在线视频

- **腾讯视频** — 中国领先的在线视频媒体平台（预编译版，使用系统 Electron 版）。

: https://v.qq.com/ || tenvideoAUR

- **哔哩哔哩** — 哔哩哔哩官方客户端 linux 移植版。

: https://github.com/msojocs/bilibili-linux || bilibili-binAUR

- **抖音** — 抖音，记录美好生活。

: https://www.douyin.com/ || deepin-wine-douyinAUR

- **VLC** — 多平台 MPEG、VCD/DVD 和 DivX 播放器。

: https://www.videolan.org/vlc/ || vlc包

实用工具

- **bookget** — bookget 数字图书馆下载工具，目前支持约 50+ 个数字图书馆。

: https://github.com/deweizhu/bookget || bookget-gitAUR

- **ccal** — 控制台程序，将日历和中文日历一起写入标准输出。

: https://ccal.chinesebay.com/ccal/ccal.htm || ccalAUR 或 ccalCNRepo

即时通讯工具 (IM)

部分即时通讯工具的打包情况比较复杂（如微信）。

- Telegram：Telegram Messenger 是一个跨平台的实时通信软件。详情请参见 Telegram 页面。
- 腾讯 QQ：请参见腾讯 QQ 页面。
- 微信：请参见微信页面。
- Discord：请参见 Discord 页面。
- IRC：请参见分类:互联网中继聊天。
- XMPP：请参见分类:XMPP。
- Matrix：请参见 Matrix 页面。

代理

应用程序

可参见应用程序列表/互联网#代理服务器。

- Shadowsocks
- Trojan（英语：Trojan）
- WireGuard 使用 UDP 的简单、快速、现代 VPN，已内置于 Linux 内核（从 5.7 开始）
- V2rayA：安装 v2rayaAUR
- Mihomo：clash 系代理工具新核心，安装 mihomoAUR。
- Clash-Verge：安装 clash-vergeAUR（已不再更新，建议使用 flclash 或 Clash Verge Rev）。
- Clash Verge Rev：安装 clash-verge-revAUR 或 clash-verge-rev-binAUR。
- Dae：安装 daeAUR 或者 daedAUR，其中 daed 是带有 WebUI 的 dae。
- v2rayN：安装 v2raynAUR 或者 v2rayn-binAUR
- flclash：安装 flclashAUR 或者 flclash-binAUR

提示：若配置有 archlinuxcn 源，上述各包均可直接安装。

- Lantern（提示：蓝灯是专有软件）：安装 lantern-binAUR 即可。
- XX-Net

更改 hosts

获取可以科学上网的 hosts 文件，修改或替换 `/etc/hosts` 即可。

示例：从 https://github.com/googlehosts/hosts 项目获取 hosts 文件，将其内容加入 `/etc/hosts`（如原 hosts 文件无需使用，也可直接覆盖）即可。也可执行更新 hosts 文件：

```
# wget https://raw.githubusercontent.com/googlehosts/hosts/master/hosts-files/hosts -O /etc/hosts
```

为方便起见，可将其使用 alias 别名方式写入 `.bashrc`，首先编辑 `~/.bashrc`，在其中添加：

```
alias hosts='sudo wget https://raw.githubusercontent.com/googlehosts/hosts/master/hosts-files/hosts -O /etc/hosts'
```

然后执行：

```
source ~/.bashrc
```

以后更新 hosts 文件只需要执行 `hosts` 即可。

警告：GoogleHosts 项目已经停止维护

提示：可以使用 crontab 定时执行脚本 (root 身份运行或 sudo 免密码)

提示：除 hosts 方法外，可能还需要进行相应的代理设置，如对程序单独设置代理或者使用工具设置临时代理（如使用 proxychains-ng包 工具，配置好代理和 Proxychains 的配置文件后，使用`proxychains *程序名*` 使该程序从代理进行联网）或者全局代理（如桌面环境的设置中可能提供该选项），可参考各工具的相应文档进行设置，或者参考代理设置一文。

电子商务

很可惜并没有现成的维基页面，不过 Acgtyrant 用户在其博客上提供了電子商務在 Arch Linux 下的簡易解決方案。

校园网连接

中国大陆众多高校采用各种客户端拨号上网，如城市热点 Drcom，锐捷。一些学校提供有网页登录或者 linux 版客户端，可参照相关说明文档安装使用。对于未提供网页登录以及客户端者：

- Drcom 用户可参考 Drcom，锐捷用户可参照 MentoHUST 指导您通过借助 MentoHUST 进行锐捷拨号。
- 借助 Wine 尝试安装使用。
- 使用虚拟机运行，可在虚拟机中登录客户端上网，虚拟机开启桥接，安装 SSH 服务端，在 Linux 下 SSH 登录虚拟机上网。

多屏协同

如果你拥有一部 Android 手机，可以考虑在电脑上安装 scrcpy包。这将允许您利用 ADB 将手机画面以窗口的形式传输到电脑上，并让您可以通过鼠标点击等方式直接发生交互。借此，您可以在电脑上间接使用 Android 应用程序，极大缓解国内 Linux 应用生态不佳的问题。

如果是 iPhone 手机，可以考虑在电脑上安装 uxplay，这将允许您利用 airplay 将手机画面、声音等传输到电脑上。

提示：如果您发现无法通过电脑操纵手机，请检查是否在手机端开启了 ADB 安全设置等。如果有，请先试着关闭它们。

特定服务

NTP 服务器

国外的 NTP 服务器会有延迟，这里提供了一些国内 NTP 服务器（含 SNTP 服务器）。

中国科学院国家授时中心：`ntp.ntsc.ac.cn`，参见关于“网络授时域名”全面试运行测试的公告 **阿里云**：`ntp.aliyun.com` **腾讯云**：`ntp.tencent.com`，参见NTP 服务概述 **ntp.org.cn**：`cn.ntp.org.cn`

5.3 建议阅读/服务器部署建议

难度 高级 | **篇幅** 19.8 KB | **所属** 建议阅读

本节将讲到

- 网页/网络服务器
- DNS 服务器
- FTP 服务器
- NTP 服务器
- 媒体服务器

将 Arch Linux 用作服务器系统是一个不错的选择，因为它提供了高度的灵活性和最新的软件包。不过，由于它主要面向桌面用户发行且是滚动更新的发行版，因此在配置为服务器时需要一些额外的步骤和考虑，以增强系统稳定性等。

本文是建议阅读一文在配置服务器软件方面的延续，以帮助你在 Arch Linux 上运行各种服务器软件（本地或网络）。

本文主要是各种服务器软件 Wiki 页面的索引及介绍。

参见服务器。

网页/网络服务器

在互联网中存放各种网站的服务器，主要用于企业或个人网站在互联网上的发布和运用。

Apache HTTP 服务器

Apache HTTP 服务器，简称 **Apache**，是非常流行的网络服务器软件。通常和脚本语言比如 PHP、数据库 MySQL 一起工作，合称为 LAMP 栈(Linux、Apache、MySQL、PHP)。

Apache HTTP 服务器一文介绍了在 Arch Linux 上安装/设置 Apache 网页服务器，以及选择安装 PHP 和 MySQL 并集成到 Apache 服务器中的方法。

GNU Artanis

GNU Artanis 是 Scheme 编程语言的第一个产品级现代网络框架。它的设计和维护都以稳健、快速和易于使用为目标，适用于专业的网络开发。

它支持的数据库（通过 guile-dbi）有 MySQL、SQLite、PostgreSQL。

GNU Artanis 主要与 Guile 一起使用。

参见 Artanis。

Caddy

Caddy 是具有 HTTP/2 功能的网页服务器，具有自动 HTTPS 配置功能。

有关 Caddy 在 Arch Linux 上的安装、配置、用法等，参见 Caddy。

Lighttpd

Lighttpd 是一个安全、快速、标准且非常灵活的网页服务器，并对高性能环境做了最佳化。

参见 Lighttpd 了解有关信息。

Nginx

Nginx（读作 "Engine X"）是一个免费、开源、高性能的 HTTP 网络服务器和反向代理，也是一个 IMAP/POP3 代理服务器，由 Igor Sysoev 于 2005 年编写。

Nginx 一文描述了如何设置 nginx 并且如何通过 FastCGI 集成 PHP。

DNS 服务器

参见域名解析#DNS 服务器。

BIND

伯克利互联网名称服务 (Berkeley Internet Name Daemon，简称 BIND) 是 DNS 协议的一个参考实现。

参见 BIND。

Cloudflared

Cloudflared 可被用于运行本地 DNS over HTTPS（DoH）服务器，也就是存根解析器。

参见 Cloudflared。

DNS-over-HTTPS

DNS-over-HTTPS是 DNS over HTTPS 的一个实现。它可以作为一个本地解析器。

参见 DNS-over-HTTPS。

Dnscrypt-proxy

dnscrypt-proxy 可以加密和认证用户和 DNS 解析服务器之间的数据传输，支持 DNS over HTTPS 和 DNSCrypt，可以避免中间人攻击和窃听。*dnscrypt-proxy* 兼容 DNSSEC。

参见 Dnscrypt-proxy。

dnsmasq

dnsmasq 提供 DNS 服务器、支持 DHCPv6 和 PXE 的 DHCP 服务器、TFTP 服务器。它设计为轻量且占用空间小，适用于资源受限的路由器和防火墙。还可以将 dnsmasq 配置为 DNS 缓存查询，以提高对以前访问过站点的 DNS 查找速度。

参见 dnsmasq。

systemd-resolved

systemd-resolved 是一个通过 D-Bus 接口、NSS (nss-resolve(8))解析服务和一个监听 `127.0.0.53` 的本地DNS解析器为本地应用提供网络名称解析的 systemd 服务。

参见 systemd-resolved。

Unbound

Unbound 是一个具有验证，递归和缓存等功能的 DNS 解析器。

参见 Unbound。

FTP 服务器

参见 分类:FTP 服务器。

Very Secure FTP Daemon

vsftpd (“Very Secure FTP Daemon“) 是一个为 UNIX 类系统开发的轻量，稳定和安全的 FTP 服务器端。

参见 Very Secure FTP Daemon。

NTP 服务器

参见分类:NTP。

chrony

chrony 是一个对漫游友好的 NTP 客户端和服务端实现，专为不常在线的系统环境设计。

chrony 一文描述了如何配置并运行 chrony。

Network Time Protocol daemon

NTP 项目提供了一个名为简单 NTP 的参考实现。

Network Time Protocol daemon 一文介绍了如何设置和运行服务器和客户端 NTP 进程。

媒体服务器

Emby

Emby 是一个个人媒体服务器，它具有许多平台的客户端。它用于组织个人家庭媒体，以及在其他设备上播放。社区支持大量的频道，甚至可以与PVR和Tuner卡一起使用以远程提供电视流。

参见 Emby。

SSH 服务器

参见安全外壳协议。

- **Dropbear)** — 轻型 SSH 服务器。命令行 ssh 客户端名为 dbclient(1)。

: https://matt.ucc.asn.au/dropbear/dropbear.html || dropbear包

- **OpenSSH** — 用于通过 SSH 协议远程登录的高级连接工具

: https://www.openssh.com/portable.html || openssh包

仅服务端

- **TinySSH** — 一种简约的 SSH 服务器，仅实现 SSHv2 功能的一部分；只依赖 glibc。

: https://tinyssh.org/ || tinyssh包

Samba 服务器

Samba 是用于 Linux 和 Unix的标准 Windows 互操作性程序套件。自 1992 年以来，Samba 为所有使用 SMB/CIFS 协议的客户端提供了安全、稳定和快速的文件和打印服务，例如所有版本的 DOS 和 Windows、OS/2、Linux 和许多其他系统。

参见 Samba#服务器。

远程服务器

TigerVNC

TigerVNC 是 Virtual Network Computing (VNC) 协议的一种实现。

TigerVNC 一文着重介绍服务端的功能。

X11vnc

x11vnc是一个 VNC 服务器，它允许人们使用任何 VNC viewer 远程查看并控制真实的X显示器（即与物理显示器、键盘和鼠标相对应的显示器）。虽然它已不再由原作者 Karl Runge 开发，但 LibVNC 和 GitHub 社区已经接管了开发工作。

参见 X11vnc。

X2Go

X2Go 使你可以通过网络访问一台计算机的图形化桌面。访问时的网络传输使用了 Secure Shell 协议，因而传输是加密的。

参见 X2Go#服务器端配置。

Xrdp

xrdp 是一个守护程序，支持 Microsoft 的 Remote Desktop Protocol (RDP)。 它使用 Xvnc 或 xorgxrdp 作为其后端。

参见 Xrdp。

WebDAV 服务器

参见 WebDAV#服务器

对于 AgenDAV，参见 AgenDAV。

打印服务器

参见 CUPS 与 Samba。

游戏服务器

Minecraft

Minecraft 是一个关于破坏和放置方块的游戏。

对于基岩版 Minecraft 服务器，参见 Minecraft#基岩版服务器。

对于 Java 版 Minecraft 服务器，参见 Minecraft/Java 版服务端。

OpenTTD

对于 OpenTTD 服务器，参见 OpenTTD#服务器。

参见 OpenTTD JGRPP 中文百科。

Mindustry

对于 mindustryAUR 服务器，有 mindustry-serverAUR 等软件包。

NFS 服务器

NFS 网络文件系统（Network File System）是由 Sun 公司 1984 年发布的分布式文件系统协议。它允许客户端上的用户像访问本地文件一样地访问网络上的文件。

参见 NFS#服务端配置。

多用户 Jupyter 笔记本服务器

JupyterHub 是用于 Jupyter 笔记本的多用户 Web 服务器。

参见 JupyterHub。

版本控制系统服务器

Git 服务器

Git 是由 Linux 内核作者 Linus Torvalds 设计并开发的版本控制系统（VCS），现在被用来维护 AUR 软件包以及数以千计的其他项目，其中包括 Linux 内核。

Git 服务器一文概述了如何托管 Git 服务器。有关更多信息，请参阅 Pro Git 书籍的 Git on the Server 一章。

网络界面

参见 Git 服务器#Web 界面。

Subversion 服务器

Apache Subversion 是一套功能全面的版本控制系统，最初被设计为 CVS 的改进版本。其后 Subversion 的发展大大超出了取代 CVS 的原始目标，但它的基本模型、设计和接口仍然受到了这一目标的深刻影响。

Subversion 一文主要介绍架设 svn 服务器的方法。有两种流行的 svn 服务器，内建的 `svnserve` 以及更高级的选择——结合了 svn 插件的 Apache HTTP 服务器。

数据库系统

参见分类:数据库管理系统、分类:关系型数据库管理系统。

MySQL

MySQL 是 Oracle 开发的，分布广泛的多线程，多用户 SQL 数据库。

Arch Linux 支持 MariaDB，这是 MySQL 的社区开发分支，旨在实现嵌入式兼容性。Oracle 的 MySQL 降到了 AUR：mysqlAUR。另一个旨在完全兼容的分支是 Percona Server，可从 percona-server包 获得。

Percona 也有 Oracle 的 InnoDB 存储引擎的分支，称为 XtraDB。MariaDB 和 Percona Server 都使用此分支。

参见 MySQL、MariaDB。

phpMyAdmin

phpMyAdmin是一个基于网页的，帮助管理MySQL数据库的Apache/PHP前端。

参见 phpMyAdmin。

PostgreSQL

PostgreSQL 是一个开源的，社区驱动的，符合标准的对象-关系型数据库系统。

参见 PostgreSQL。

phpPgAdmin

phpPgAdmin 是一个基于网络的工具，可帮助使用 PHP 前端管理 PostgreSQL 数据库。

参见 phpPgAdmin。

SQLite

SQLite 是一个实现自包含、无服务器、零配置、事务性 SQL 数据库引擎的软件库。SQLite 是世界上部署最广泛的 SQL 数据库引擎。SQLite 的源代码在公共领域。

参见 SQLite。

Adminer

Adminer 是一个用 PHP 编写的基于网络的数据库管理工具。它可以管理 MySQL、PostgreSQL、Sqlite3、MS SQL、Oracle 数据库和 Elasticsearch。

参见 Adminer。

Elasticsearch

Elasticsearch 是基于 Lucene 的搜索引擎。它提供了一个分布式、支持多租户的全文搜索引擎，具有 HTTP 网络接口和无模式 JSON 文档。Elasticsearch 采用 Java 开发，根据 Apache License 条款开源发布。

参见 Elasticsearch。

邮件服务器

参见邮件服务器。

Postfix

Postfix) 是邮件传送代理软件。据官方说，它快速、管理简单、安全，同时足够兼容 Sendmail，从而不会影响现有用户。 因此，从外面看是 sendmail-ish 风格，但内部是完全不同的。

参见 Postfix。

UW IMAP

UW IMAP 是 University of Washington 开发的 IMAP 协议的参考服务器实现。

参见 UW IMAP。

Sendmail

Sendmail 是来自 UNIX 世界的经典 SMTP 服务器。Arch Linux 还提供了 Postfix。

参见 Sendmail。

邮件列表管理系统

Mailman

Mailman 是一个邮件列表管理系统。它与邮件服务器结合使用。

参见 Mailman。

即时通讯服务器

参见应用程序列表/互联网#即时通讯服务器。

XMPP

Jabberd2

jabberd2 是一个 XMPP 服务器，用 C 语言编写，并根据 GNU 通用公共许可作为自由软件许可。它的灵感来自 jabberd14。

参见 Jabberd2。

Matrix

Matrix 的服务端软件有 Synapse 和 Dendrite，目前 ArchWiki 和本维基上仅有关于 Synapse 的内容。关于使用 Synapse 建立 Matrix 服务器，请参阅 Matrix。

应用服务器

代理服务器

参见代理服务器。

5.4 建议阅读

难度 高级 | **篇幅** 29.2 KB | **所属** 建议阅读

本节将讲到

- 系统管理
- 软件包管理
- 启动
- 服务器
- 图形界面

本文是各种重要或常用的文章的详细索引。阅读本文前，读者应该先通过官方安装指南安装基础的 Arch Linux 系统。然后理解#系统管理和#软件包管理中解释的概念，再阅读本节的其他内容和 wiki 中的其它文章。

参见给新用户的关于如何不去弄坏 Arch Linux 系统的建议。

参见服务器部署建议。

提示：中国用户可以特别留意中国用户的推荐解决方案中的内容。

系统管理

这一部分提供系统管理方面的信息。更多内容，请参考核心工具和系统管理分类。

用户和用户组

新安装的系统只有一个超级用户，即 root。使用 root 进行日常操作是不安全的。应当创建普通用户进行日常操作，仅在管理系统时使用 root。不要在服务器上给 root 开放 SSH 登录权限。普通用户的创建方法请参阅用户和用户组。

用户和用户组是 GNU/Linux 权限控制机制的基础。管理员通过调整用户组的成员、所有者，可以控制用户使用系统资源。

安全

请阅读安全，了解有关加强系统安全性的建议和技巧。

有关允许以其他用户（如 root）身份运行命令或启动交互式 shell 的应用程序列表，请参阅应用程序列表/安全#提权。

服务管理

Arch Linux 使用 systemd 管理系统服务。新用户有必要了解其基本使用方法。通常使用 `systemctl` 命令进行系统管理，参见 systemd#systemctl 基本用法。

systemd 还提供日志系统，使用 **journalctl** 命令。更多信息，请参阅 Journal。

其他服务管理器或 init 系统

您可能希望在 Arch Linux 上使用其他服务管理器或 init 系统，如 OpenRC、shepherdAUR 等。

警告：Arch Linux 只为 systemd 提供官方支持。求助时请在帮助请求中提及使用其他服务管理器或 init 系统。

系统维护

Arch 是滚动发行系统，软件包的更新速度很快，用户需要花些时间进行系统维护。

软件包管理

此部分提供了软件包管理的信息，参见：FAQ#软件包管理与 分类:包管理。

注意：Arch 的升级有时候需要手动处理。请订阅 arch-announce 邮件列表，每次升级前查看 Arch 新闻（中文：Arch 新闻）或者订阅 RSS 源（中文：RSS 源）。

Pacman

Pacman 是 Arch 的软件包管理器：强烈建议在阅读任何其他文章之前熟悉它。

关于缓存包的长期处理，参见 pacman#清理软件包缓存。

Pacman 提示和技巧中有很多方便 pacman 使用的技巧。

软件仓库

官方软件仓库包含了各个仓库的详细介绍。非官方软件仓库包含很多个人维护的软件仓库。

如果计划使用 32 位程序，建议启用 multilib 仓库。

安装 pkgstats，可以让软件开发人员统计软件包的使用情况。您可以在统计页面查看收集到的数据。

软件仓库镜像

参见镜像源一文，获取寻找更快更新pacman镜像的方法。此外，可以查看镜像状态获取最新镜像站点同步信息。这可以用 Reflector 自动完成。

Arch 编译系统（ABS）

Ports 是 BSD 发行版最初使用的一套系统，它是本地系统中包含各种软件编译脚本的目录树。

ABS 系统相当于 Arch 的 Ports，包含 Arch 官方软件包的编译脚本——PKGBUILD。编译脚本提供了哈希验证、软件主页、版本、协议、编译步骤等信息。通过 makepkg 从编译脚本生成软件包，然后用 pacman 安装。

实际上，Arch 的所有软件包（包括官方库、AUR）都是通过 makepkg 生成的。

Arch 用户软件源（AUR）

Arch 编译系统提供了编译官方库软件的脚本，而 AUR 则提供了用户提交的、非官方的软件包编译脚本。这是一个基于 web 界面或通过 AUR 助手访问的非官方软件仓库。

其它

对于 Flatpak，参见 Flatpak。

对于 Snap，参见 Snap

对于 GNU Guix，参见 GNU Guix。

对于 Nix，参见 Nix。

对于 Python，参见 Python#软件包管理。

对于 Emacs，参见 Emacs/软件包。

启动

这部分包含系统启动方面的信息。关于 Arch 启动过程，参见 Arch 的启动流程。更多信息，参见启动过程分类。

硬件自动探测

默认情况下，udev 会在开机时自动探测硬件。禁止加载某些内核模块、手动选择要使用的模块。此外，Xorg 也使用 udev 探测硬件，用户也可以调整这方面配置。

Microcode (微码)

处理器可能有错误行为，而内核可以通过更新启动时的 *Microcode*，即微码，来修正这些错误行为。参考微码获取更多细节。

保留启动信息

一旦显示登录提示符，启动信息会被清除，使得用户无法获得启动进程的反馈信息，禁用启动信息自动清除可以解决这个问题。

开机时打开数字锁定键

大多数键盘都有一个数字锁定键（即 Num Lock 键），通过它控制小键盘的开关。用户可能希望在系统启动时打开数字锁定键，参见启动时打开数字锁定键。

服务器

参见建议阅读/服务器部署建议。

图形界面

本部分提供了在系统上安装图形程序，参阅 分类:图形用户界面。

显示服务

Xorg 是 X 窗口系统（通常称为 X11 或 X）的公开开源实现。运行具有图形用户界面（GUI）的应用程序需要使用它。

Wayland 是一种较新的替代显示服务器协议，有多个混成器可供选择。与 Xorg 相比，它的优势在于增强了安全功能，能更有效地处理现代图形任务，通过 Xwayland 保持与 Xorg 的兼容性。它目前正在积极进行开发。

显卡驱动

默认的 **modesetting** 显卡驱动对于大多数显卡都是兼容的，但如果为 AMD、Intel 或 NVIDIA 产品安装适当的驱动程序，则可提高性能并使用其他功能。

桌面环境

显示服务器只提供图形环境的基本框架，完整的用户体验还需要其他组件。

KDE、GNOME、Xfce、Cinnamon、LXDE、LXQt 等桌面环境捆绑了大量集成良好的应用程序，如窗口管理器或混成器、面板/任务栏、文件管理器、终端模拟器、文本编辑器、图标和其他实用程序。经验不足的用户可能希望安装一个桌面环境，以获得更熟悉的环境。更多资源请参见 分类:桌面环境。

窗口管理器或混成器

完整的桌面环境提供了完整的用户界面，但是通常会占用不少系统资源。希望系统性能最大化的用户可以只安装窗口管理器或混成器，然后加入需要的其他软件。大部分使用 Xorg 的桌面环境也可以换用其它的窗口管理器。动态式、堆叠式和平铺式窗口管理器处理窗口的方式各不相同。

显示管理器

许多桌面环境提供了显示管理器来自动启动图形界面和管理用户登录。显示管理器介绍了启动管理器的使用方法。登录时启动 X 提供了直接从终端启动的轻量方法。

用户目录

大家熟知的用户目录，比如说下载和音乐文件夹，是使用 `xdg-user-dirs-update.service` 用户服务创建的。这个服务包含在 xdg-user-dirs包 软件包中，并且在安装过程中已经自动启用。如果您的桌面环境或窗口管理器没有安装这个软件包，您可以手动安装它，并对每一个运行 `xdg-user-dirs-update` 以创建默认目录。

电源管理

本章对笔记本用户可能更为有用。更多信息，参见: 分类:电源管理。

ACPI 事件

电源按键或者合上笔记本会发出 ACPI 事件，可以配置系统在收到这些事件时的响应。推荐的方式是使用 systemd, 参阅电源管理#ACPI 事件。老的方法是使用 acpid, 不推荐使用。

CPU 频率调节

最新的 CPU 通常都有自动调节频率的功能。通过该功能可以有效节约电能、减少发热，提升硬件寿命。详情参见 CPU 调频。

笔记本电脑

针对特定型号笔记本电脑的配置信息，参见 分类:笔记本电脑。有关笔记本电脑文章的概览，参见笔记本电脑。

待机和休眠

待机，指系统将当前状态保存于内存中，进入的低能耗状态（保持开机）。休眠，与待机有所不同，是将当前状态保存于硬盘中，然后可以完全断电。参阅待机和休眠。

多媒体

分类:多媒体包含更多多媒体方面的资源。

声音系统

ALSA 是内核声音系统（英语：Sound system）组件，开箱即可使用（只需解除各声道的静音）。PipeWire 和 PulseAudio 等声音服务器（英语：Sound#Sound server）可以提供更多的功能，并支持更复杂的音频配置。

有关高级音频要求，请参阅专业音频。

浏览器插件

用户可以安装 Adobe Acrobat Reader、Java 之类的浏览器插件，以使用更多的富媒体互联网资源。

编解码器

多媒体应用程序利用编解码器与容器编码或解码音频、视频流媒体。要播放多媒体文件，正确安装编码器是必不可少的。

硬件视频加速

硬件视频加速让显卡进行视频的解码和编码，以减少 CPU 的负载并节省电力。

网络

本文包含网络方面的配置信息。更多信息参见：网络配置、分类:网络。

时间同步

NTP，是最常用的网络同步时间的协议。

DNS 安全

当在浏览网站，在线支付，连接 SSH 服务 和类似的事情的时候，为了更安全，请考虑使用启用了 DNSSEC（英语：DNSSEC） 的 DNS 解析服务器。它可以验证 DNS

记录的签名, 并且通过 DNS over HTTPS 或 DNSCrypt 等加密协议来加密 DNS 的传输。更多信息请参考域名解析。

DNSSEC 验证

网络安全方面安全，SSH 提供加密的网络链接。而使用支持 DNSSEC（英语：DNSSEC） 的客户端，为提供 DNS 记录验证，将更进一步加强网络安全。

配置防火墙

防火墙为Linux网络访问提供额外保护。作为 Netfilter 计划的一部分，Linux 内核内置了 iptables——一种状态防火墙（Stateful firewall）。可以通过直接或间接的方式配置它。非常推荐建立一个防火墙，参考防火墙。

网络共享

可以通过 NFS 或 SSHFS 在网络间共享文件.

用户可以使用 Samba 进行 Windows 与 Arch Linux 间的网络传输。

要将 Arch Linux 系统连接到 Active Directory 认证的网络，请阅读文章 Active Directory integration（英语：Active Directory integration）。

参阅 分类:网络共享。

输入

这一部分包含常用的输入设备配置建议。更多信息，参见输入设备分类.

键盘布局

默认情况下，非英文或其他非标准键盘可能无法按预期工作。必要的配置步骤在虚拟控制台和 Xorg下有所不同，分别在 Linux 控制台/键盘配置和 Xorg/键盘配置中进行了描述。

鼠标按键配置

一些高级鼠标可能有许多按键，默认情况下系统并不能正确配置它们。这方面的信息，参见：鼠标按键。

笔记本触摸板

Synaptics 和 ALPS 是笔记本常用的两种触摸板。对于 Synaptics 和其他一些类似触摸板的用户，请参见 Touchpad Synaptics 获取配置信息。

指点杆

参见指点杆。

性能优化

这一部分包含一些实用的性能优化技巧。通过使用这些技巧，可以有效提升程序性能。

性能测试

基准测试帮助用户评估系统性能，为优化系统性能提供信息。

性能最大化

性能优化一文提供了提升 Arch 系统性能的方法。

固态硬盘

固态硬盘一文包含固态硬盘的各个方面，包括配置和提高寿命，例如使用 TRIM。

系统服务

这一节的更多信息，请参考守护程序。

文件索引和搜索

大部分发行版都提供了 `locate` 命令进行快速文件搜索，Arch Linux 提供了几种替代方案，详见 locate。

打印

CUPS 是 OpenPrinting 为 Linux 开发的、符合标准的开源打印系统。特定型号打印机的配置信息，参见：打印机分类。

扫描

参阅 SANE、扫描仪分类。

本地邮件服务

默认安装并未提供一个邮件交换的方式。请阅读邮件服务器来查看邮件交换服务列表。

外观美化

本栏讨论 ArchLinux 界面的美化。更多信息请参考 分类:小提升。

字体

在安装桌面环境/窗口管理器之前，也许你会先安装些美观的字体。目前来说开源的 Noto 系列字体（noto-fonts-cjk包）是不错的选择，既包含拉丁字母字体也包含中文字体。参见简体中文本地化#中文字体。

而在此之前，文泉驿项目制作了文泉驿微米黑 wqy-microhei包， 再在此之前，文泉驿项目制作了文泉驿正黑矢量字体 wqy-zenhei包，内嵌有 9pt-12pt 的点阵宋体。但由于该项目并非由专业设计师制作，字形可能会出现扭曲和错误，故现阶段其实不做推荐。

提示：noto-fonts-cjk包 包含各个地区的变种，若您稍后安装的桌面环境未能使用正确的语言区域设置，则可能出现显示界面语言为简体中文但汉字显示却是日本标准的情况（置的底部为折，系字笔画出头等），此时需要修正语言区域设置，或手动修改字体配置。

可能有人需要安装微软 Windows 下的字体，参见微软字体。

请访问字体配置获取配置字体渲染的详细信息，字体提供了字体选择建议和安装方法。

对于经常使用虚拟终端的用户，可以通过配置字体提高可读性，参见 Linux 控制台#字体。

GTK 和 Qt 主题

Linux 下的图形界面基本都使用 GTK 或者 Qt 工具集。这些文章和让 Qt 和 GTK 界面相统一提供了让程序更美观的方法。

控制台优化

本部分包括控制台的优化和微调方法。参阅 分类:命令行解释器和命令行解释器。

Tab 自动补全

建议参考所选 shell 的文档，立即设置增强的 Tab 自动补全。

别名

给一个命令或命令组取别名，是使用控制台时的一种节省时间的方式。这种方式对于重复的任务特别有用，这些任务的参数在多次执行期间不需要大的改变。通常使用的省时的别名可以在这里找到 Bash#别名，这些别名也能很容易地移植到 zsh 。

用户可以自定义常用命令的别名，以方便使用。

其它 shell

Bash 是 Arch 默认安装的 shell，而安装镜像里默认使用的是 zsh 并使用 grml-zsh-config包 插件。其它选择参阅命令行解释器#Shell 列表。

Bash 增强功能

Bash#提示与技巧介绍了些 Bash 的杂项设置，包括：命令补全，历史记录，宏等等。

彩色输出

参考在终端输出彩色。

压缩文件

压缩包，或称为归档，在 GNU/Linux 十分常用。Tar 是最常用的归档工具，用户应该熟悉它的语法。此外还有 Arch 软件包使用的 zstd包 压缩包。参见归档与压缩。

控制台提示符

控制台提示符（`PS1`）可以通过环境变量灵活定制。参见 Bash 提示符自定义（Zsh 用户参见：命令提示符）。

Emacs shell

Emacs 除了用作编辑器，其高级功能更为出名，其中一项就是把Emacs变成全功能 shell。参见：Emacs 打开彩色输出后的乱码问题。

鼠标支持

在控制台中，使用鼠标复制粘贴比传统 GNU Screen 操作方式方便许多。参见：Gpm。

会话管理

tmux 或 GNU Screen 之类的终端复用器提供会话管理，在其中运行的程序不会因杀死终端、关闭 X 或用户登出而终止，只要终端复用器服务保持运行。随后，用户可以重新连接会话。

POSIX shell 工具

许多常用软件包已经安装了大多数流行的 POSITION 工具作为依赖项，但安装 posix 包 元软件包可确保所有这些工具始终存在。

除了必须使用的工具，一些可选类别也有元软件包：

- posix-c-development包
- posix-software-development包
- posix-user-portability包
- posix-xsi包

注意：给定类别中的所有可选实用程序不一定都包含在相应的元软件包中。

游戏

对于在官方仓库或 AUR 中提供软件包的游戏的列表，参见游戏列表。

对于关于游戏的通用说明，参见游戏。

关于配置部分游戏服务器，参见建议阅读/服务器部署建议#游戏服务器。

对于 Steam 本身，参见 Steam。

对于 Minecraft，参见 Minecraft。

对于 OpenTTD，参见 OpenTTD。

第 6 章　共 4 节　47 KB

本地化

系统装好不等于看得见中文：字体缺失、输入法不可用、终端与 man 手册页仍是英文或一片乱码，是中文用户最先撞上的几道墙，而它们分属不同的配置层。

必读的是《Locale》与《简体中文本地化》：前者讲清区域设置的生成与相关变量，是其余几节的共同前提；后者把终端、man 手册页翻译、软件中文化配置与乱码一并处理。要配字型与输入法时再翻《中文本地化》；不用繁体环境的话，《繁体中文本地化》可跳过，它基本是简体的对照版本。最常见的误解是把中文化当成装字体：乱码多半出在区域设置或终端一侧，而非缺字型；字型、输入法、终端、文档、软件各层的配置互不代劳，改好一处不等于处处生效。

6.1　中文本地化

难度 进阶 | **篇幅** 6.8 KB | **所属** 本地化

本节将讲到

- 字型
- 輸入法

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 字型的部分需要協助。（在 Talk:中文本地化# 中讨论）

這篇文章講述如何設定一個中文環境。

字型

- 思源體
- adobe-source-han-sans-cn-fonts包——簡體中文緊湊字型格式無襯線體
- adobe-source-han-sans-tw-fonts包——正體中文緊湊字型格式無襯線體
- adobe-source-han-sans-hk-fonts包——香港中文緊湊字型格式無襯線體
- adobe-source-han-serif-cn-fonts包——簡體中文緊湊字型格式襯線體
- adobe-source-han-serif-tw-fonts包——正體中文緊湊字型格式襯線體
- adobe-source-han-serif-hk-fonts包——香港中文緊湊字型格式襯線體
- noto 中文字型
- noto-fonts-scAUR - Noto CJK-SC 簡體中文字型
- noto-fonts-tcAUR - Noto CJK-TC 正體中文字型
- 文泉驛字型

- wqy-microhei包 - WenQuanYi Micro Hei 字體（即 Hei，Gothic 或 Dotum）是一個由 Droid Sans Fallback 來的無襯線體輪廓字型，它有高質 CJK 字型且極度緊湊（～ 5 M）。
- wqy-zenhei包 - 有宋體在內的黑體輪廓字型（有韓文和部分日文的支援）。
- wqy-bitmapfont包 - 點陣宋體字型。
- arphic fonts
- ttf-arphic-ukai包 - *Kaiti* (brush stroke) Unicode font (enabling anti-aliasing is suggested)
- ttf-arphic-uming包 - *Mingti* (printed) Unicode font
- opendesktop-fonts包 - *New Sung* font, previously is ttf-fireflysung package
- ttf-hannom包 - Chinese and Vietnamese ttf fonts
- Standart fonts of the Republic of China ministry of education in Taiwan
- ttf-twAUR - Kai and Song traditional Chinese font from the Ministry of Education of Taiwan
- ttf-twcns-fontsAUR Chinese TrueType fonts by Ministry of Education of Taiwan government, support CNS11643 standard, including Kai and Sung fontface.
- Windows Chinese fonts
- ttf-ms-win8-zh_cnAUR - windows8 simple Chinese fonts.
- ttf-ms-win8-zh_twAUR - windows8 traditional Chinese fonts.
- ttf-ms-win10-zh_cnAUR - windows10 simple Chinese fonts.
- ttf-ms-win10-zh_twAUR - windows10 traditional Chinese fonts.
- ttf-i.bmingAUR - CJK serif font that emphasis on an old-style typeface.

輸入法

以下的都是中文的輸入法（IM）：

引擎	小企鵝輸入法 5	小企鵝輸入法	IBus	Uim	Scim	Nimf
Rime	fcitx5-rime包	fcitx-rime包	ibus-rime包	–	–	內置
拼音	fcitx5-chinese-addons包	內置	ibus-pinyin包	–	scim-pinyinAUR	內置
注音	fcitx5-chewing包	fcitx-chewing包	ibus-chewing包	–	scim-chewing包	內置
倉頡 速成 快倉	fcitx5-table-extra包	fcitx-table-extra包	ibus-table-chinese包	–	scim-tablesAUR	–
五筆	內置	內置	ibus-table包	內置	scim-tablesAUR	–
Libpinyin	–	fcitx-libpinyin包	ibus-libpinyin包	–	–	–
SunPinyin	–	fcitx-sunpinyin包	ibus-sunpinyin包	–	–	–

6.2 繁体中文本地化

难度 进阶 | **篇幅** 7.6 KB | **所属** 本地化

本节将讲到

- 基本中文語系環境
- 終端機的中文化
- 軟體的中文化
- 其他中文化問題

本篇很大部分參考了 Arch Linux Localization。雖然本地化的流程是相似的，但是正體中文化所需要的軟體和設定檔仍和簡體中文化有所不同，故有此篇文章的產生。

基本中文語系環境

locale 設置

安裝中文 locale

Linux 透過 locale 來設置不同語系的環境。常用的正體中文 locale 有

```
zh_TW.UTF-8
zh_TW.Big5
```

其中較推薦使用 en_US.UTF-8 和 zh_TW.UTF-8 語系。需要修改 `/etc/locale.gen` 文件，將需要的語系前面的註解 (# 符號)刪除。之後執行 locale-gen 指令即可。

如果要查詢目前系統所使用的語系環境，可以使用 locale 指令。

```
# locale
```

如果要查詢系統所有可用的語系環境，可以使用 locale -a 指令。

```
# locale -a
```

啟動中文 locale

在 Arch Linux 中，透過設置 `/etc/locale.conf` 文件，來設置語系環境。

```
LANG=en_US.UTF-8
```

注意：不推薦在這裡設置中文語系，會導致 tty 終端機亂碼。如果要在終端機裡顯示和輸入中文，需安裝 cce、zhcon 或 fbterm

對於一般使用者，還可以在 `~/.bashrc`、`~/.xinitrc` 或是 `~/.xprofile` 中設置語系，其中的不同處在於：

- .bashrc: 每次透過終端機登入時，讀取並運行其中的設定
- .xinitrc: 每次透過 startx 登入時，讀取並運行其中的設定
- .xprofile: 每次透過圖形登入器 (如 gdm) 登入時，讀取並運行其中的設定

在圖形介面中啟用中文 locale

如果是希望在圖形介面有中文環境的話，需要修改 `~/.xinitrc` 或 `~/.xprofile`，加入

```
export LANG=zh_TW.UTF-8
```

注意：該方法適用於 slim 或無圖形登入器的場合，如果是 gdm 或 kdm 等可以在 Gnome 或 KDE 中設置語系

注意：不推薦在圖形介面中設置 LC_ALL，該變數應當保留用於測試診斷目的。

中文字體

除了設定中文 locale，還要安裝中文字體，才可顯示中文。

常見的免費中文字體有:

- wqy-bitmapfont包
- wqy-zenhei包
- ttf-arphic-ukai包
- ttf-arphic-uming包
- opendesktop-fonts包
- wqy-microhei包（AUR中）
- wqy-microhei-lite包（AUR中）

系统字體預設安裝到`/usr/share/fonts`。如果没有 root 權限或只打算自己使用某些字體，可以直接復製這些字體到`~/.fonts`目錄（或其子目錄）下面，並把該目錄加入 `/etc/fonts/local.conf` 中。

中文輸入法

常用的正體中文輸入法有 gcin、scim 以及由 gcin 分支出來的 hime-gitAUR (AUR 中)。推薦使用 gcin 或 hime-gitAUR。可看該輸入法文章，其中有較詳細的介紹。

以使用 gcin 為例，安裝好後在 `~/.xinitrc` 或 `~/.xprofile` 加入以下內容：

```
export XMODIFIERS="@im=gcin"
export XIM_PROGRAM="gcin"
export GTK_IM_MODULE="gcin"
export QT_IM_MODULE="gcin"
gcin &
```

之後即可在圖形介面中輸入中文。

終端機的中文化

軟體的中文化

Firefox

正體中文用戶請安裝 firefox-i18n-zh-tw包。

簡體中文用戶請安裝 firefox-i18n-zh-cn包。

LibreOffice

正體中文用戶請安裝 libreoffice-still-zh-tw包 或者 libreoffice-fresh-zh-tw包。

簡體中文用戶請安裝 libreoffice-still-zh-cn包 或者 libreoffice-fresh-zh-cn包。

Calligra (原 KOffice)

正體中文用戶請安裝 calligra-l10n-zh_tw包\[损坏的链接：package not found\]。

簡體中文用戶請安裝 calligra-l10n-zh_cn包\[损坏的链接：package not found\]。

PDF 閱讀器

Acrobat Reader:，正體中文用戶請裝 acroread-chtAUR\[损坏的链接：package not found\]，簡體中文用戶請安裝 acroread-chsAUR\[损坏的链接：package not found\]。

xpdf: 請安裝 xpdf-chinese-traditional包\[损坏的链接：package not found\] 和 xpdf-chinese-simplified包\[损坏的链接：package not found\]。

poppler: 請安裝 poppler-data包。

Vim

如果 locale 是 utf8，打開其他編碼的中文文件會有亂碼的問題，需要在 `~/.vimrc`中設定：

```
set fileencodings=ucs-bom,utf-8,big5,gbk,latin1
```

Vim 即會依序按照設定的編碼來打開文件。詳細的設定見 vim 中的幫助文件 `:help fileencodings`。

其他中文化問題

MS Windows 底下的中文檔案名亂碼

是因為掛載的字符集和 locale 不同所致。可以修改 `/etc/fstab` 來修正這個問題。如果 locale 是 utf8 的情形下，可能修改為如下的範例 (請根據自己的實際情形略做修改)：

```
/dev/sdXX   /mnt/win   ntfs   defaults,iocharset=utf8    0 0
```
詳細的 /etc/fstab 設定可見 fstab。

6.3 Locale

难度 进阶 | **篇幅** 11.5 KB | **所属** 本地化

本节将讲到

- 生成区域设置
- 设置当前区域
- 变量
- 疑难解答
- 另见

glibc包 和应用程序、函数库库使用区域设置显示本地化的文字、货币、时间、日期、特殊字符等包含地域属性的内容。

生成区域设置

区域设置的名称通常用 [language][_TERRITORY][.CODESET][@modifier] 的格式表示，*language* 是 ISO 639 语言代码, *territory* 是 ISO 3166 国家代码, *codeset* 是字符集或 ISO-8859-1、UTF-8 这样的编码方式。请参考 setlocale(3)。

要列出所有启用的区域设置，使用：

```
$ locale -a
```
启用一个区域设置前，需要先生成它。在 /etc/locale.gen 中取消对应的注释，然后执行 *locale-gen*。注释掉某行，则会移除对应的区域设置。请启用所有用户都可能使用的区域设置及相关#变量。

例如对于使用美式英语和中国简体的用户：

```
/etc/locale.gen
...
#en_SG ISO-8859-1
en_US.UTF-8 UTF-8
#en_US ISO-8859-1
...
#zh_CN.GBK GBK
zh_CN.UTF-8 UTF-8
#zh_CN GB2312
...
```
编辑完成以后,通过下面的命令生成 Locale：

```
# locale-gen
```
注意：

- 每次更新 glibc包 时 locale-gen 会自动运行。
- 建议优先使用 UTF-8。

设置当前区域

想要显示正在使用的 Locale 和相关的环境变量，运行：

```
$ locale
```

要使用的区域设置（从前面生成的区域设置中选择）设置在 `locale.conf` 文件中。每一个 `locale.conf` 文件都必须包含一些环境变量赋值语句，其格式与 *locale* 命令输出的格式相同。

要查看已经生成的区域设置：

```
$ localedef --list-archive
```

或者使用 localectl(1)：

```
$ localectl list-locales
```

系统区域设置

要设置整个系统使用的区域设置，需要在 `/etc/locale.conf` 中写入 `LANG` 变量，以下的 *zh_CN.UTF-8* 应为 `/etc/locale.gen` 中某个未注释条目的第一列：

```
locale.conf
LANG=zh_CN.UTF-8
```

或者使用：

```
# localectl set-locale LANG=zh_CN.UTF-8
```

参阅#变量和 localectl(1) 获得更多细节。

在用户会话中覆盖系统区域设置

系统使用的区域设置可以通过用户编辑自己的 `~/$XDG_CONFIG_HOME/locale.conf` (通常为 `~/.config/locale.conf`) 来覆盖。

这些 `locale.conf` 文件的优先级定义在 `/etc/profile.d/locale.sh` 中。

提示：

- 设置用户级区域设置，能让 `/var/log/` 中的日志以英语输出而在用户环境中使用所选语言。
- 建立 `/etc/skel/.config/locale.conf` 文件，可以让所有通过 *useradd* 命令和 `-m` 选项创建的新用户自动拥有一份相应的 `~/.config/locale.conf`。

立即启用新设置

`locale.conf` 的变更会在下次登录时生效，要立刻应用新的设置的话，可以运行：

```
$ unset LANG
$ source /etc/profile.d/locale.sh
```

注意：`LANG` 变量必须先被 `unset`，否则 `locale.sh` 不会从 `locale.conf` 中更新变量。只有新增的或被修改的命令会被自动更新，从 `locale.conf` 被移除的变量在注销前依然存在。

其它用例

和区域设置相关的变量也能像其他的环境变量一样传递给其它程序。

例如在开发和测试某个程序时,可以这样运行：

```
$ LANG=C ./my_application.sh
```

类似的，也可以通过设置环境变量让当前 shell 中运行的程序使用特定的区域设置(例如安装系统时)：

```
$ export LANG=C
```

变量

`locale.conf` 文件支持以下环境变量。

- LANG
- LANGUAGE
- LC_ADDRESS
- LC_COLLATE
- LC_CTYPE
- LC_IDENTIFICATION
- LC_MEASUREMENT
- LC_MESSAGES
- LC_MONETARY
- LC_NAME
- LC_NUMERIC
- LC_PAPER
- LC_TELEPHONE
- LC_TIME

locale(7) 包含了完整 `LC_*` 列表，locale(5) 包含了详细定义。

注意：程序在查找本地化相关的变量时遵守此优先级顺序。

LANG：默认的区域设置

这个变量的值会覆盖掉所有未设置的 `LC_*` 变量的值。

提示：假设您是西班牙的英语使用者，并且您希望程序根据西班牙的惯例处理数字和日期，仅消息为英语。您可以将 `LANG` 设置为 `es_ES.UTF-8`，并将 `LC_MESSAGES`（用于消息翻译的用户界面）设置为 `en_US.UTF -8`。

LANGUAGE：后备区域设置

使用 gettext 翻译的软件会按照 `LANGUAGE` 选择使用的语言。用户通过这个变量指定一个区域设置列表，如果前面的区域设置缺少翻译，会自动使用后面的区域设置显示

界面。例如一个澳大利亚用户可能更希望在没有合适的澳大利亚英语翻译时回退到英式英语而不是美式英语：

```
locale.conf
LANG=en_AU.UTF-8
LANGUAGE=en_AU:en_GB:en
```

注意：很多软件并未将其英文 locale 设置为 `en` 或 `en_US`，而是使用默认 locale `C`。如果在 `LANGUAGE` 中将非英文 locale 设置到 English 之后，例如 `LANGUAGE=en_US:en:es_ES`，那么即使英语字符存在，应用程序可能会选择使用后备 locale. 解决方法是强制在英语 locale 后面设置 `C`，例如 `LANGUAGE=en_US:en:C:es_ES`.

LC_TIME：时间和日期格式

如果 `LC_TIME` 设置成 `en_US.UTF-8`，日期的格式为 "MM/DD/YYYY"。要使用 ISO 8601 标准的日期格式("YYYY-MM-DD") ,使用：

```
locale.conf
LC_TIME=en_DK.UTF-8
```

glibc包 2.29 修复了一个错误，`en_US.UTF-8` 开始如预期般显示 12 小时格式的时间。如果要使用 24 小时制，请使用 `LC_TIME=C.UTF-8`。

注意：使用此变量来格式化日期不是对程序的强制要求。例如，date(1) 使用其自己的参数来格式化时间，Firefox 在版本 57 到 84 之间不支持 `LC_TIME`（Bug 1429578）。

LC_COLLATE：排序格式

这个变量的值决定排序和正则表达式的格式顺序。

例如将它设置为 `C` 可以让 *ls* 命令首先列出点开头的文件，然后是大写字母开头的文件和小写字母开头的文件：

```
locale.conf
LC_COLLATE=C
```

另见 。

为了避免潜在的问题，Arch Linux 曾经在 `/etc/profile` 中设置 `LC_COLLATE=C`，这个方法已经过时了。

LC_ALL：测试和排除问题

这个变量的值会覆盖掉 `LANG` 和所有 `LC_*` 变量的值，无论它们是否设置。 如果 `LC_ALL` 设置为`C`, 此变量将覆盖 `LANGUAGE`.

只有 `LC_ALL` 不能出现在 `locale.conf` 文件中：它只能为了测试和排除问题而设置，例如在 `/etc/profile` 中。

注意：`LC_ALL=C.UTF-8` 与 `LC_ALL=C` 不同，不会覆盖 `LANGUAGE`. 请参考 glibc bug 16621 和 gettext bug 62815.

疑难解答

我的终端不支持 UTF-8

这里列出了一些（不是全部）支持 UTF-8 的终端：

- gnustep-terminal
- konsole
- mlterm
- rxvt-unicode
- st
- 基于 VTE 的终端
- xterm - 启动时使用参数 `-u8`，或者配置资源 `xterm*utf8: 2`。

Gnome-terminal / rxvt-unicode 不支持 UTF-8

你必须在 UTF-8 的区域设置下运行它们才会有作用。按照上面的方法启用 `en_US.UTF-8`（或者其它使用 UTF-8 的区域设置）并将它设置成默认区域设置后重启系统。

我的系统的语言还是不对

可能其它文件设置了本该由 `locale.conf`，详见定义环境变量。

如果使用了桌面环境，可能是被桌面环境修改。例如 GNOME 的语言设置会覆盖 `locale.conf` 中的设置。

KDE Plasma 也通过系统设置修改界面的语言，如果按本文修改后还是原来的区域设置，请删除 `~/.config/plasma-localerc` (之前是：`~/.config/plasma-locale-settings.sh`)。

如果您正在结合 accountsservice包 使用显示管理器，请遵循 Display manager#为用户会话设置语言中的说明。

LightDM 将自动使用 accountsservice包 来设配置用户的区域设置（如果已安装）。否则，LightDM 将用户会话配置存储在 `~/.dmrc` 中。也可能从那里检索到意料外的的区域设置。

解压文件的编码不正确

老的 Windows 版本 (XP, Vista 和 7) 使用不同的编码方式压缩文件。要解压这些文件，请使用：

```
$ unzip -O CP936 file.zip
```

另见

- Gentoo:Localization/Guide
- Supposedly 2008, or earlier, Gentoo wiki article
- ICU's interactive collation testing
- Free Standards Group Open Internationalisation Initiative
- *The Single UNIX Specification* definition of Locale by The Open Group
- Locale environment variables

6.4 简体中文本地化

难度 高级 | **篇幅** 20.9 KB | **所属** 本地化

本节将讲到

- 基本中文支持
- 终端中文支持
- man 手册页的简体中文翻译
- 软件中文化配置
- 乱码问题

注意：此页面不是纯粹的对 ArchWiki Localization/Simplified Chinese 一文的翻译。

依据「Arch 之道」：我们不会为你配置好一切，因为“喜好和需求，每人皆不同”，但是会尽量确保让配置时方便和简单。事实上，甚至远比使用某些 Linux 中文发行版容易。

本文尽可能提供了各种常见软件的中文化指导。但实际应用中，你可能遇到各种各样的麻烦。遇到了麻烦，不要气馁，解决问题本身就是一种乐趣。你可以通过各种渠道寻求帮助：

- Google 等搜索引擎
- Arch 官方论坛
- Arch Linux 中文社区

基本中文支持

要正确显示中文，必需设置正确的 locale 并安装合适的中文字体。

locale 设置

安装中文 locale

Linux 中通过 locale 来设置程序运行的本地化环境。根据地区和字符编码的不同，常用的中文 locale 有：

```
zh_CN.GB18030
zh_CN.UTF-8
```

```
zh_HK.UTF-8
zh_SG.UTF-8
zh_TW.BIG-5
zh_TW.UTF-8
```

推荐使用 UTF-8 的 locale 而非老旧的各地区编码。需要修改 `/etc/locale.gen` 文件来设定系统中可以使用的 locale（取消对应项前的注释符号「#」即可）：

```
en_US.UTF-8 UTF-8
zh_CN.UTF-8 UTF-8
zh_SG.UTF-8 UTF-8
```

然后执行 `locale-gen` 命令，便可以在系统中使用这些 locale。可以通过 `locale` 命令来查看当前使用的 locale；亦可通过 `locale -a` 命令来查看目前可以使用的 locale。

配置中文 locale

配置全局 locale

在 `/etc/locale.conf` 里可以设置全局的 LANG locale，根据需要可以设置为中文或英文：

```
$ cat /etc/locale.conf
LANG=zh_CN.UTF-8
```

`LANG` 这个环境变量代表默认的区域设置，具体的含义见 Locale#LANG：默认的区域设置

注意：如果在 `/etc/locale.conf` 里把全局的 LANG locale 设置成中文，例如 `LANG=zh_CN.UTF-8`，会使得 TTY 下也尝试显示中文消息。因为 TTY 下一般没有 CJK 字体，中文可能会显示为豆腐（方块）或问号等。当今大多数桌面用户不使用 TTY，所以一般没有影响。关于 TTY 下的中文字体设置见下文有关章节。

每个用户单独的 locale 可以在 `~/.bashrc`、`~/.xinitrc` 或 `~/.xprofile` 中设置：

- `.bashrc`：每次**使用终端时**会应用此处的设置。
- `.xinitrc`：每次**使用 startx 或 SLiM 来启动 X 窗口系统时**会应用此处的设置。
- `.xprofile`：每次**使用 GDM 等显示管理器时**会应用此处的设置。

为图形界面配置中文 locale

如果你选择设置了全局的中文 locale，就不需要再为图形界面专门设置。

GDM 和 SDDM 用户可以在 GNOME 或 KDE 设置中选择语言。参见显示管理器#为用户会话设置语言。

使用其他的没有自带语言设置的登录管理器的用户，又没有设置全局中文 locale 的，可以在 `~/.xinitrc` 或 `~/.xprofile` 里单独设置中文 locale，即添加下面两行到文件的最开头（如果不确定使用哪个文件，都添加）：

```
export LANG=zh_CN.UTF-8
export LANGUAGE=zh_CN:en_US
```

警告：若欲将此两行放至 `~/.xinitrc` 中，请注意将其放在 `exec *_example_WM_or_DE_*` 行之前，此为常见错误。

注意：不推荐使用 `export LC_ALL` 来覆盖所有 locale 设置，`LC_ALL` 应该仅用于诊断调试，全局设置 `LC_ALL` 会为诊断语言设置问题带来不必要的困难。

中文字体

安装字体

除了设置好 locale，还需要安装中文字体。

常用的自由（OFL、GPL或其他自由软件协议）中文字体有：

- adobe-source-han-sans-cn-fonts包
- adobe-source-han-serif-cn-fonts包
- noto-fonts-cjk包
- wqy-microhei包
- wqy-microhei-lite包
- wqy-bitmapfont包
- wqy-zenhei包
- ttf-arphic-ukai包
- ttf-arphic-uming包

系统字体将默认安装到 `/usr/share/fonts`。如果没有 root 权限或只打算自己使用某些字体，可以直接复制这些字体到 `~/.fonts` 目录（或其子目录）下面，并把该路径加入 `/etc/fonts/local.conf` 中。具体参见后面章节。

另见：

中文字体配置

Fontconfig 设置

Fontconfig 的用户配置文件是 `$XDG_CONFIG_HOME/fontconfig/fonts.conf`，或存放于 `$XDG_CONFIG_HOME/fontconfig/conf.d/`。全局配置文件是 `/etc/fonts/conf.d`。推荐修改前者。

关于中文字体设置，参见：字体、字体配置。

Font Configuration/Chinese 提供了中文字体的 Fontconfig 示范。

另见：

- 用 fontconfig 治理 Linux 中的字体
- Linux 下的字体调校指南

修正简体中文显示为异体（日文）字形

安装 noto-fonts-cjk包 或 adobe-source-han-sans-otc-fonts包（思源黑体）或 adobe-source-han-serif-otc-fonts包（思源宋体）后，在某些情况下（框架未定义地区）汉字字形与国标标准不符，例如关、复等字字形过窄，门的字形为竖点插入横折钩而非左上斜点等。

注意：此处的字形仅为日语字形示例，并非你字体真的配置错了！你需要查看其他地方和应用程序中文字的字形以检查配置情况。

这是因为每个程序中可以设置不同的默认字体，比如 Arial 或者 Tahoma，而这些字体的属性由 fontconfig 控制，其使用顺序是据地区代码以 A-Z 字母表顺序成默认排序，由于 `ja-JP` 在 `zh_{CN,HK,SG,TW}` 之前，故优先显示日文字形。

提示：Chromium/Chrome/Firefox 浏览器的设置中可单独设置字体，例如将字体选项调成 Noto xxx CJK SC。

可选用以下方法解决（以简体中文为例）：

- 确保桌面环境的语言区域设置正确。例如，KDE可能会生成错误的语言配置文件，相应修复即可。
- 只安装符合简体中文国标标准的字体，例如思源字体的简体中文变体包 adobe-source-han-sans-cn-fonts包和adobe-source-han-serif-cn-fonts包，或者 noto-fonts-scAUR。
- 在 `locale.conf` 中添加 `LANG=zh_CN.UTF-8`，以将简体中文设置为默认语言。由于对 Locale 定义了框架内地区（即 CJK 优先度），使得默认的优先级被忽略。（不建议，理由见上）
- 手动调整优先级，将中文字形调整到日文字形之前。

  在 `/etc/fonts/conf.d/` 或 `/etc/fonts/conf.avail/` 下创建文件，例如 `64-language-selector-prefer.conf`，也可以修改或创建 `~/.fonts.conf`或在`~/.config/fontconfig/conf.d/`创建后缀为.conf的文件（仅对该用户生效）。例如针对noto-fonts-cjk包的规则，写入：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
  <alias>
```

```
    <family>sans-serif</family>
    <prefer>
      <family>Noto Sans CJK SC</family>
      <family>Noto Sans CJK TC</family>
      <family>Noto Sans CJK JP</family>
    </prefer>
  </alias>
  <alias>
      <family>serif</family>
      <prefer>
        <family>Noto Serif CJK SC</family>
        <family>Noto Serif CJK TC</family>
        <family>Noto Serif CJK JP</family>
      </prefer>
  </alias>
  <alias>
    <family>monospace</family>
    <prefer>
      <family>Noto Sans Mono CJK SC</family>
      <family>Noto Sans Mono CJK TC</family>
      <family>Noto Sans Mono CJK JP</family>
    </prefer>
  </alias>
</fontconfig>
```

如果安装的是 adobe-source-han-sans-otc-fonts包，写入：

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
  <alias>
    <family>sans-serif</family>
    <prefer>
      <family>Source Han Sans SC</family>
      <family>Source Han Sans TC</family>
      <family>Source Han Sans HW</family>
      <family>Source Han Sans K</family>
    </prefer>
  </alias>
  <alias>
    <family>monospace</family>
    <prefer>
      <family>Source Han Sans SC</family>
      <family>Source Han Sans TC</family>
      <family>Source Han Sans HW</family>
      <family>Source Han Sans K</family>
    </prefer>
  </alias>
</fontconfig>
```

注意，如果你是在 `/etc/fonts/conf.avail` 目录下创建的 xml 文件，则将该 xml 文件软链接到 `/etc/fonts/conf.d` 下，例如：

```
# ln -s /etc/fonts/conf.avail/64-language-selector-prefer.conf /etc/fonts/conf.d/64-language-selector-prefer.conf
```

然后更新字体缓存即可生效：

```
# fc-cache -fv
```

执行以下命令检查，如果出现 `NotoSansCJK-Regular.ttc: "Noto Sans CJK SC" "Regular"` 则表示设置成功：

```
# fc-match -s | grep 'Noto Sans CJK'
```

中文输入法

常用的中文输入法平台有 IBus、Fcitx、Fcitx5 和 scim。具体安装配置参见各自条目。

注意：scim 现在维护滞后，不推荐使用。Fcitx 已经停止维护，请改用 Fcitx5。Fcitx5 与 Fcitx 不兼容。

终端中文支持

引导中文支持

GRUB2 支持引导选择界面的中文显示。

TTY 中文支持

当今大多数桌面用户不需要在 TTY 下显示中文。但如果你确实需要，可以考虑以下方法之一：

- 使用打了 cjktty 补丁（可绘制包括中文在内的 Unicode BMP 字符）的内核，比如linux-lilyCNRepo）。
- 使用特殊的中文字体，比如syllazhCNRepo，强行将汉字塞进512个字符的限制之内。
- 使用第三方的 TTY 软件，例如KMSCON等。

man 手册页的简体中文翻译

安装 man-pages-zh_cn包 软件包。

软件中文化配置

Firefox

安装 firefox-i18n-zh-cn包。

LibreOffice

安装 libreoffice-fresh-zh-cn包 或 libreoffice-still-zh-cn包。

PDF 阅读器

多数 PDF 查看器已经支持中文。但也有部分需要安装额外的语言包/字体：

Acrobat 的字体为 acroread-fontsAUR，或者可以安装 acroread-fonts-systemwideAUR 以使用系统范围的字体。

Okular、Evince 等 poppler 相关的阅读器及 Inkscape、Krita、MyPaint 等可以处理 pdf 的图像处理工具需要安装 poppler-data包。

Java

对于 Oracle Java 用户，在 `/opt/java/jre/lib/fonts` 中建立 `fallback` 目录，然后链接或拷贝若干中文字体到该目录就能使 java 程序正确显示中文。例如，在已经安装 jreAUR 和 opendesktop-fonts包 的情况下，使用执行下面的命令即可：

```
# ln -s /usr/share/fonts/TTF/odosung.ttc /opt/java/jre/lib/fonts/fallback/
# cd /opt/java/jre/lib/fonts/fallback/
# mkfontdir
# mkfontscale
```

vim

如果 locale 是 UTF-8 编码，用 vim 打开其他中文编码的文件可能会乱码。需要在 `~/.vimrc` 做如下设置：

```
~/.vimrc
...
set fileencodings=utf8,cp936,gb18030,big5
...
```

中文视频字幕

MPlayer

要使 MPlayer 正确显示字幕，关键是要使字幕文件的编码和 mplayer config 里使用的编码相一致。字幕文件编码为 GBK，则 `subcp=cp936`；字幕文件编码为 UTF-8，则 `subcp=utf8`。如果字幕文件编码为 UTF-8，而设置成 `subcp=cp936`，则会出现部分乱码的情况。另一种更为简单的方法是设置成 `subcp=enca:zh:ucs-2`，由 enca 负责字幕的编码显示问题。

修改 `~/.mplayer/config`：

```
~/.mplayer/config
font='Noto Sans CJK SC'
subcp=enca:zh:ucs-2
```

使用下面的命令手动加载字幕：

```
$ mplayer xxx.avi -sub xxxxx.srt
```

如果使用图形前端（比如 SMPlayer），会更简单一些，只要在设置对话框里设定缺省字幕编码和字体即可。

xine

xine 也可以显示中文字幕，但需要制作自己的中文字体。具体可以参考：。

gstreamer

由于使用 gstreamer0.10，totem 1.4.0 应该可以自动加载同名的 srt 字幕。

LaTeX

首先需要安装 CJK 包，然后需要安装合适的字体。具体可以参考：。

乱码问题

避免乱码基本原则：使用 UTF-8 代替 GBK/GB2312。

文件名乱码

安装 convmv包，使用 `convmv` 命令转换编码格式。示例：

```
$ convmv -f GBK -t UTF-8 --notest --nosmart file
```

`-f` 指定原始编码，`-t` 指定输出编码。使用 `convmv --list` 可查询所有支持的编码。`--notest` 表示非测试而是要进行转码（如果不使用该参数只会打印出转换结果而不会实际转码），`--smart` 表示如果已经是 UTF-8 则忽略。

文件内容乱码

使用 `iconv` 命令转换格式。示例：

```
$ iconv -f GBK -t UTF-8 -o new-file origin-file
```

`-f` 指定原始编码，`-t` 指定输出编码。使用 `iconv -l` 可查询所有支持的编码。`-o` 指定输出文件。

zip 压缩包中的文件名乱码

原因：创建压缩包时使用的文件名编码和解压时不同。

Arch 上一般使用 UTF-8 编码，一般造成这种现象的原因是创建者使用了 Windows 上的默认编码，如简体中文的 GB 系列编码或日文的 JIS 系列编码等。

解决方案：使用 `-O` 参数指定编码，如

```
$ unzip -O gbk file.zip
```

`file.zip` 是压缩文件，`gbk` 是猜测该文件创建时使用的编码格式。

如有可能，建议在 Windows 上不使用 zip 压缩包，而使用 7z 等格式。

MP3 文件标签乱码

对于用 GStreamer 做后端的播放器，如 Rhythmbox，totem，设置如下的环境变量后即可正确读取 mp3 中 GBK 编码的 ID3 tag：

```
export GST_ID3_TAG_ENCODING=GBK:UTF-8:GB18030
export GST_ID3V2_TAG_ENCODING=GBK:UTF-8:GB18030
```

对于 Beep media player，可以在 `pefenrence->plugins->media` 中选中 MPEG Audio plugin 然后点击下方的 Penfenrences，此时会出现一个对话框，选择 title，将 Disable ID3v2 和 Convert non-UTF8 ID3 tags to UTF8 前的选择框选中。然后在 ID3 encoding 中填入 `gbk`。这样 bmp 就能正确显示 GBK 编码的 ID3 tag。

Quod Libet 播放器支持 tag 编辑及设置 ID3v2 编码。可以在 `~/.quodlibet/config` 中设置

```
~/.quodlibet/config
...
id3encoding = gbk
...
```

注意：Quod Libet 默认支持 utf8 编码。

最为彻底的解决方法为将编码为 gbk 的 id3 tag 转化为 utf8 编码。首先安装 python-mutagen包，然后利用下面的命令转换：

```
$ mid3iconv -e gbk XXX.mp3
```

Windows 分区下的中文文件名乱码

一般是因为挂载的字符集与 locale 不同，可以修改 `/etc/fstab`（如果不了解请仔细阅读 fstab）。如果 locale 是 UTF-8，修改为：

```
/etc/fstab
...
/dev/sdxx /media/win ntfs defaults,iocharset=utf8 0 0
```

如果 locale 是 GBK，则应该是：

```
/etc/fstab
...
/dev/sdxx /media/win ntfs defaults,iocharset=cp936 0 0
...
```

Samba 乱码

用 Arch 作为 Samba 服务器时，在 `/etc/samba/smb.conf` 中加入下面一行就可以解决 Windows 客户端乱码问题：

```
/etc/samba/smb.conf
...
unix charset=gb2312
...
```

ftp 乱码

很多 ftp 站点是 GBK 编码。如果使用 UTF-8 的 locale，下载的文件名可能会乱码。对于 lftp，在 `.lftp/rc` 下做如下设置：

```
.lftp/rc
...
set ftp:charset "gbk"
set file:charset "UTF-8"
...
```

对于 gftp，可以在 `.gftp/gftprc` 中做如下设置即可：

```
.gftp/gftprc
...
remote_charsets=gb2312
...
```

但下载下来的文件名仍然是乱码，需要打补丁编译。补丁地址为:
https://www.teatime.com.tw/%7Etommy/linux/gftp_remote_charsets.patch

翻译软件

- stardict包：星际译王。
- sdcv包：命令行的星际译王。
- ydcv包：命令行的有道词典。
- youdao-dictAUR：有道词典（图形界面），屏幕取词翻译。
- GoldenDict（goldendict-gitAUR）：默认都不带字典，可下载相应字典包（支持 Babylon 的词库格式 `.BGL`，已经不再维护的 StarDict 的词库格式（`.ifo`/`.dict`/`.idx`/`.syn`），dictd 的词库格式（`.index`/`.dict`(`.dz`），ABBYY Lingvo 的词库格式（`.dsl`/`.lsa`/`.dat`），mdict 的词库格式等等。可在互联网上下载这些词典的词库文件导入的 GoldenDict 使用（可能有版权问题）。
- moedictAUR：一个跨多平台的汉语词典，除汉字、词、成语等，还包含客家话、闽南话、简单的外文翻译、笔顺书写等等，萌典在线地址。
- linedictAUR：一个通过爬取有道翻译网页得到结果的在线英汉词典,部分支持英汉翻译，模仿 dmenu 在屏幕顶端显示结果，使用方便，由于 ydcv包 使用的 api 已失效，而有道新的 api 有免费使用次数限制，linedictAUR 是一个较好的替代品。

Arch 社区

同一句提问，发在即时聊天群、官方论坛、邮件列表还是 Bug 系统，回应天差地别。本章先说清这些地方各管什么：中文社区的群组、论坛、Telegram 频道与软件仓库，官方论坛、邮件列表、AUR、官方 IRC 频道、Wiki 与 Bug 系统；再讲行为准则如何界定越界、违反后怎样执行、有异议找谁反映。

《行为准则》与《一般准则》必读，后者是提问与报错前的对照表；《Arch Linux 中文社区》可略读，要用中文交流时再翻。最易踩的坑是拿它们当免费客服：不查 Wiki 就发问、缺陷贴群里而不进 Bug 系统，或以为准则只是客套话。

7.1　Arch Linux 中文社区

难度 入门 | **篇幅** 3.8 KB | **所属** Arch 社区

本节将讲到

- 即时聊天群组
- 论坛
- Telegram 频道
- 软件仓库
- 另请参阅

提示：本文或本章节的主题是「Arch Linux 中文社区」。关于「Arch Linux 官方社区」的内容，请查看 Arch Linux 官方设施。

即时聊天群组

Arch Linux 中文社区在不同的平台均有数个即时聊天群组，其中交流 Arch Linux 相关信息的群组被称为「主群」，而闲聊话题单独前往一个群组，被称为「ot群」（ot 是 off-topic 的缩写）。不同的接入方式是由机器人互通的。历史上 IRC 用户曾经占多数，而现今绝大多数用户位于 Telegram 或 Matrix 上。另请参阅 Project:Arch Linux 中文社区交流群指引。

Telegram 群组

因我们的反垃圾机制的要求，请先关注我们的频道 https://t.me/archlinuxcn，然后访问 https://t.me/archlinuxcn_group 加入群组。成功加群之后可以不再关注频道。

Matrix 房间

同样要求反垃圾要求，需要先加入位于 #archlinux-cn-space:nichi.co 的空间，空间中包含多个不同话题的房间。然后即可加入位于 #archlinux-cn:nichi.co 的主群。

IRC 频道

频道位于 Libera.Chat 的服务器上，地址是：`irc.libera.chat` 端口 6697。请务必使用 TLS/SSL 连接。依用户 IP 的不同，有可能需要先注册并登录账户。由于 IRC 仅支持文本，不支持图片消息和回复引用，因此不推荐使用。IRC 用户的消息都需要经 Matrix 转发才能到达 Telegram 上的用户。

XMPP 群组

注意：由于疏于维护，可能时不时出现故障，不建议使用。并可能在将来完全关停此服务。

这是一个通过机器人实现的群组。使用 XMPP（Jabber）帐号添加 `talk@archlinuxcn.org`为好友即可加入。成功加入将收到欢迎信息。由于本群组仅支持文本，不支持图片消息和回复引用，因此不推荐使用。XMPP 用户的消息都需要经 IRC 转发才能到达 Matrix 和 Telegram 上的用户。

论坛

论坛的地址是 https://forum.archlinuxcn.org/。注意论坛的账户系统是独立的，与本维基并不互通。

Telegram 频道

@archlinuxcn 频道用于发布一些 Arch Linux 新闻的中文翻译、用户需要注意的事项，同时也作为部分其它 Telegram 群组进入的反垃圾手段。提示：即使没有 Telegram 账号，使用浏览器也可以浏览频道内容的。

软件仓库

[\[archlinuxcn\] 软件仓库](../../01_安装与包管理/非官方用户仓库/Arch%20Linux%20中文社区仓库.md)是由 Arch Linux 中文社区维护的非官方用户仓库，通过使用机器人，提供了许多自动化更新的软件包。

- 介绍页面：https://www.archlinuxcn.org/archlinux-cn-repo-and-mirror/ 。请参照此页面开始使用这个仓库。
- GitHub 项目页面：https://github.com/archlinuxcn/repo 。请在此反馈使用中遇到的问题。

另请参阅

- Arch 的 IRC 频道：Arch Linux 官方的 IRC 频道列表

7.2 行为准则

难度 进阶 | **篇幅** 11.0 KB | **所属** Arch 社区

本节将讲到

- 常识
- 行为准则
- 执行
- 联系工作人员
- 另请参阅

提示：本文或本章节的主题是「Arch Linux 官方社区的行为准则」。关于「Arch Linux 中文社区的行为准则」的内容，请查看Arch Linux 中文社区交流群指引。

注意：这是 Arch Linux Code of Conduct 页面的非官方翻译，在 Arch Linux 官方的任何服务和项目中（包括但不限于官方论坛、ArchWiki、AUR 和 BugTracker 等），则以原始版本为准。

Arch 技术社区的目的是支持和完善 Arch Linux。

Arch Linux 是一个社区驱动的发行版；开发者、支持人员和提供各种帮助的人都是在用自己的私人时间进行贡献。大家希望提供一个最精简的基础系统，让每一个用户都按照自己的需要进行配置和定制。本页的行为准则是经过很多年提炼和总结得出，反映了社区的特点，目的是维护一个高信噪比的支持体系，重视个人的能动性。

熟悉这些原则和指南是对社区的尊重，同时也有助于您和其他用户交流时实现互惠互利。

常识

- 如果希望使用 Arch Linux 发行版，请参阅 Arch 之道 获得相关信息.
- 寻求帮助时，请先阅读手册、做好自己的功课并提供所需帮助的详细内容。
- 给予帮助时，请尽量向需要帮助的人提供一定的耐心或鼓励，以便解决问题。
- Arch 是个由志愿者组成的社区。比起花费时间、资源和人力维护自由与约束的种种条条框框，我们选择一个简单的理念囊括全部。因此，这篇文档或许能以此箴言概之：

> ***乐于奉献，止于损害***

行为准则

尊重

尊重他人

Arch Linux 是一个尊重他人、互相包容的社区，但社区不会容忍任何反社会或攻击性的行为。简而言之，尊重他人和他们的意见。即使你不同意他们的观点，或是不同意对方观点时，请反驳论题本身而非反驳提出问题的人。

尊重其他发行版和操作系统

禁止毁谤其他 GNU/Linux 发行版，操作系统及其用户。整个 Arch 团队很乐意自愿地提供他们的时间和精力来把 Arch Linux 发行版、文档、论坛提供给你。请也友好地对其他发行版和操作系统的志愿者、用户还有社区表现出你的尊重。

尊重工作人员

支持成员是以其公正的判断力而选择出来的，他们拥有最终发言权。注意本论坛不是一个民主场所。论坛工作人员会尽力实现广泛的和平的解决问题，然而最终往往被指责没有尽到为大多数人的利益维持和平、平等的论坛秩序的责任。因此，他们无法时刻作出让每一个人都满意的决定。请作出您自己的努力来建设一个健康的社区环境。

不要钓鱼

“troll”（钓鱼）是指一个人滥用言论自由故意打乱讨论、制造扭曲、煽动争议、并/或发布挑衅内容以引起负面关注。这个词语也描述发布这种内容的动作或内容本身。

钓鱼者通常很狡猾，他们通过矛盾的信息和隐式表句来讽刺、威吓或挑唆他人。他们往往对用词精挑细选，为他们制造混乱的意图作辩护。他们经常将指责转为社区，声称社区对他们存在误解。钓鱼行为是绝对禁止的。

不要辱骂

辱骂，在通常的定义中，表示向某人传达负面、不敬或者攻击的言论。一个同样或者更加负面的回应，通常会导致互相辱骂的结果。辱骂资深会员（包括 Arch 小组）是不被容忍的。不要诉诸人身攻击，同时请避免讽刺性或居高临下的语言。***讨论可以是建设性的，而争论往往是破坏性的。***

对自己的行为负责

如果出现了一个人与人之间的交际问题，请首先自查是否因为自己有意或无意的行为导致了这一问题。社区鼓励用户培养自我意识，保持平和的态度。对自己的行为负责通常是达成和解的第一步。

帐号

请只注册一个帐号。注册多个帐号是对资源的不必要使用，亦有可能被解释成钓鱼行为。社区禁止马甲和仿冒账号。

合法性

教唆犯罪

在网站上教唆犯罪是严厉禁止的。在这里，“教唆犯罪”的意思是，“主动地或被动地提示、帮助、引诱、提议、劝说他人从事违法犯罪活动。”

因此，不要以任何形式张贴讨论、示范或链接到教唆犯罪内容的帖子。这包括但不限于以下内容的信息或链接：关于帮助非法药品使用、盗窃、网络入侵、创建恶意代码、禁止的软件拷贝、禁止使用的版权/专利材料、破解软件(warez)，以及提供这些内容的种子(torrent)或链接的网站。非法内容将会被迅速移除并按照法律处理。

垃圾帖/广告/宣传

禁止发布垃圾帖。发现请举报，版主会处理。行为严重的账号及其 IP 将被封禁。请勿回复垃圾帖，这会增加清理的工作量。

禁止一切宣传，除非其与 Arch（项目或社区）或 GNU/Linux/FOSS 有关。禁止网络邀请、博文宣传、商业宣传。不接受期刊/事业宣传，即使与 FOSS 有关，也无法接受将社区当作宣传资源。如果不确定帖子内容是否合适，请在发表*之前*与管理团队联系。注意，短链接是垃圾帖发布者的惯用手段，以混淆社区对链接合适与否的判断，因此请勿发布此类链接。

正确性

Arch Linux 社区注重技术正确性。在寻求或给予帮助时，请致力于提供准确、完整和正确的信息。

如何巧妙地提问总结了 Arch 社区期望的提问方式。

其它需要考虑的地方:

仅支持 Arch Linux

Arch 衍生发行版通常使用不一样的软件包、版本号、软件仓库，且有着部分难以察觉的定制。在 Arch 论坛对支持这些项目极其艰难。社区技术支持只限于 Arch Linux 发行版和 Arch 用户软件仓库。所有关于衍生发行版或其他操作系统的帖子都会被关闭。

个人话题/咆哮

不要咆哮和抱怨，这样的话题会被及时关闭。这些内容更适合放在博客或其他个人网站上，不适合放到 Arch 社区中。所做的贡献应该允许所有人参与。

避免论战/有争议的话题

没有一张列表完整地列出哪些话题有钓鱼意味、有争议或具煽动性。过去，有关**宗教、体育、种族、民族主义**和**政治**的帖子都会被关闭。所以，***特别注意在 Arch 论坛上避免这些会造成不合的话题！***论坛的工作人员明白这些问题深深地扎根于人类现实。但是，这里是 GNU/Linux 发行版论坛，它没打算而且没能力有效地促进相关的评论，也不想造成动乱。

无效讨论 (BIKESHED)

以“Arch 系统和方式有个问题，我们需要讨论一下”等同的开头的帖子已经被多次证明是无用的、容易使人激愤的，通常在被管理团队警告之后就会被锁定。Arch 是一个 DIY 自助社区。如果你确定了一个系统性错误，请找到一个对你有效的解决方案，实现它，然后再发贴。

任何对 Arch Linux 开发团队使用的方法的质疑或讨论将会被密切监视，而且，*一旦被认定为无益且（或）无实际作用*会被锁定。（令人不舒服的、毫无价值的批评也不欢迎。）

如果在和 Arch 开发相关方面有问题，请确定你的主题提出了一个确切且详细的问题，并且虚心对待回复。如果可能，提供一个完全或不完全的解决方案。提交代码和补丁总是比要求别人为你做来得更有用，更具实际意义。

自由

“自由(*Liberty*)”是不受权威压迫或束缚的状态，而“自由(*freedom*)”则是能够自决的能力。因此，为了社区的“自由(freedom)”氛围，需要每个人都牺牲一点“自由(liberty)”。在社区里，自由、开放的相互帮助、发言、思想和观点的交流是被高度敬重和鼓励的，但必须注意的是，授予论坛成员的自由是*相对自由*，存在于以上规范和原则的界定和约束下。所以，对'审查'和'不宽容'的抱怨是没有根据和无法成立的，因为必要的框架必然同时导致了合乎情理的限制。举例来说，尽管我们都知道物理法则支配着能量和物质，但我们不仅不会抱怨其“压迫”，还会认为其对自身的福利必不可少。同样地，为了共同利益，为了保护所有人，Arch 社区在本文档所展现出来的高标准控制着社区自由。论坛真正自由的关键在于，培养对他人的善举和不给同伴们带来损害而达成的每个人与 Arch 之道之间的和谐。提供相当的自由的同时免于更加混乱的做法将导致的有害和消极后果，因此拥护以上原则并遵守论坛规范对整个社区是有益的。

执行

如果 Arch 论坛管理团队感到某人的行为不可接受并且正当介入，经常地会发出一个私人警告（除非是明目张胆的违规，这种情况下可能会禁言）。而且警告不会被管理团队事先讨论，无论在 Arch 论坛、IRC 频道，还是邮件列表。如果警告被无视，则会采取进一步的行动。从锁定违规帐号到删除再到禁言。进一步的行动视情况而定。

如果用户有歉意，并对平和的解决方案有兴趣并希望恢复他们的帐号，管理团队需要达成一致来接受或反对这样的要求。

联系工作人员

如果你感到发生了一个很让人反感的疏忽，或需要报告不合理的行为，通过下面方式联系工作人员：

- 论坛: 使用文本框下面的 'Report' 按钮，或发邮件给 forum@archlinux.org 联系管理员.
- Wiki: 联系 ArchWiki 管理员
- IRC 频道: 联系 IRC 频道管理员

另请参阅

除了以上几点，在社区准则之外，还有一些针对每个社区服务的一般准则和指引。

所有用户同意使用社区服务*之前*阅读这些准则和指引。

对*另请参阅*部分链接和内容的修改可能会在行为准则页面自身更改的 30 天通知期前生效。

关于这份文档

此行为准则最初在 ArchWiki 上成文（编辑历史可以在这里查阅），后续更改位于 Arch Linux 服务协议仓库。

7.3 一般准则

难度 进阶 | **篇幅** 16.0 KB | **所属** Arch 社区

本节将讲到

- [官方论坛](https://bbs.archlinux.org/)
- [邮件列表](https://lists.archlinux.org/mailman3/lists/)
- [AUR](https://aur.archlinux.org/)
- [官方 IRC 频道](../../03_网络与服务/互联网中继聊天/Arch%20的%20IRC%20频道.md)
- Wiki

这里概括了 Archers 除了社区准则（中文非官方翻译）以外，在论坛、ArchWiki、IRC 和 Bugtracker 等社区服务中所需遵守的额外的规则和建议的概述。

官方论坛

此指南是论坛管理团队长时间经验的结晶，充分考虑了社区的建议。 Arch 论坛本身是一个严肃的论坛。

首先在官方论坛上请尽量使用英语，或者考虑使用 Other Languages 模块或访问中文论坛。

如何发贴

- 选择清晰明了且有意义的（informative）标题。这更有可能吸引来自那些有该特定主题知识的经验丰富的用户的回应。这也会使得该主题容易引用，并能未来能够被有类似问题的用户搜索到。此外，避免使用诸如『求助！』、『紧急』之类的无关词语句。
- 在问问题的时候，尽可能地提供信息，包括错误信息、终端输出、日志、此前试验的方法、尝试过的文档和搜索，还有相关配置文件。
- 每个帖子一个主题。不提倡在技术问题版块里发表长贴。
- 只在一个版块里发表你的问题；选择最切题的，并发表。
- 不要发布教程和指南：文档应该放到 wiki 上，这样才能更好的维护文档。
- 回复已有帖子时，先阅读原帖并尽量专注在原帖的主题上。
- 最后，一旦找到了解决方案，将你的贴子标记为已解决，只要编辑主题贴，并在你的标题之前加上\[SOLVED\]标签。
- 注意，你应该避免使用\[CLOSED\]（已关闭），这是由系统使用，用于标记不可再回复的主题贴的。
- 如果贴子被标记为\[SOLVED\]，不要回复说“我有一个类似的问题.”；发表一个新帖，并且如果有关，可以链接到\[SOLVED\]帖子。

粘帖图片和代码

在贴出控制台信息的时候，请使用\[code\]标签。如果要贴大量的代码，请使用粘贴板\[损坏的链接：无效的章节\]。不要使用pastebin.com——有些用户无法访问这个地址，而且存在其它不方便的地方(javascript，广告，格式不好看等)。对于非英语语言环境的用户，请在要贴出的代码前加上`LC_ALL=C`。这样，信息就会以英语输出。不要贴出整个屏幕的图片；而是使用缩略图。任何尺寸大于 250x250 像素或大小超过 50Kb 的图片都会被移除。不要粘贴文本输出的截图，请直接粘贴文本本身。

生活是条双行道

这是一个简单而深刻，且不可否认的事实。确保你的帖子包含对他人来说有用的细节和信息。通过社区分享你的发现。同样地分享你遇到的问题（Failures）。在你的帖子中对应地发表“别在意，我已经修复了它了”、或因类似的原因而删除你自己的帖子，对于社区来说，不仅是自私的、无益的，而且完全是浪费资源和每个人的时间。 同样的，要求帮助、或显示出对于得到帮助的急切也不应该在这里出现。Arch 是由社区的志愿者提供的。Arch 强烈建议用户去搜索、作出努力、在帖子中反馈、帮助他人、参与其中并为社区作贡献。

请不要成为一只'吸取帮助的吸血鬼（help vampire）'。

寻求产品推荐

尽量不要发布计算机产品推荐帖子。这些话题很快就会过时而且不会对更广泛的社区提供帮助。请自行搜索并得出哪个产品最适合你的结论。

旧贴/"挖坟"

做好自己的一份子，保持论坛的整洁。不应在技术问题的版块里回复旧贴，或者说"挖坟"，因为这很可能会造成杂乱的"僵尸"信息；导致新帖旧帖混杂。因为 Arch 的滚动升级特性，过时的帖子已经不再适合当前的 Arch 了。此外，技术支持的帖子应该保持简洁并尽量避免多个页面。

经验法则

- 如果有问题，另发一贴并附上相关帖子的链接。
- 如果自己的信息与旧帖类似，但要更新，且需要补充，则另发一贴。如有必要，并加上旧帖。但要避免重复劳动，不要将 Arch Wiki 上已有的信息贴出。
- 如果有一个与版本无关或相应的解决方案，则允许回复旧帖。（通常不会超过一年）

不要刷贴/发空贴

发表空贴、无价值的信息是对刷贴的最好描述。这是令人无法忍受的。人们这样做可能基于两个原因：为了毫无意义地增加发贴数，或为了支持某个想法（就像投票一样）。刷贴的例子有（但不限于）：回复以"+1"、"lol"、"我也是"、"顶"或":)"。

发贴和回复时，确保你有话可说。这些空贴只会造成主题和讨论的混乱，使得"Show New Posts（显示新帖）"功能无效，并且浪费带宽和服务器空间。

沦为一系列"+1/-1"或"我也是/我同意/我不同意"且又无其他内容的帖子会被锁定。单开的刷帖也会被删除。

顶帖

为了将注意力吸引到你的帖子上，而发表一个简单的字词或无用的信息(顶）是不允许的。自己做研究，继续解决故障，贴出原因并对社区保持耐心。如果人们看了你的帖子却没有回复或提供帮助，也许你应该提供更多的细节。或请求指点正确的方向。通常，帖子无人回复很大程度上是因为原帖的细节不够多，或者是因为在 Wiki 上、论坛上（用搜索功能可找到）、网络上有明显的可利用解决方案，从而导致社区不愿意指出这明显之处。

交叉发贴

交叉发贴是指多次在不同版块贴出相同的问题（例如，同时在 Newbie Corner 和 Installation 两个版块发表）。这是浪费资源并不被允许的行为。任何交叉发表了的主题贴会被立马锁定或删除。

话题劫持

“话题劫持”是指在一个已有的主题贴下回复不同的话题。通常是不鼓励这么做。如果你有一个和已发表的问题有关、但又有明确不同的问题，最好开一个新帖。另外，如果你有一个似乎一样的问题，最好在回复已有帖子时附上额外的信息。不应在严肃的帖子下歪楼。

删帖策略（标记为删除）

帖子会被锁定/关闭，是因为它们要么已经在论坛或 Wiki 上有记录，要么无法与 Arch Way 协调。这样的帖子会被移到 Dust/troll-bin。五天后，帖子便会被删除。管理员有责任将帖子明显地标记为“Binned”或“即将删除（For Deletion）。”

邮件列表

Top posting

不要在顶部回信.

引用

仅引用上封邮件的必要部分，大块的引用会降低可读性。请尽量删除不相关的引用。

纯文本

纯文本是 Unix 和电子邮件的标准，HTML 并不需要，而且使用命令行客户端的用户非常反感 HTML。请注意换行，默认每行最多 72 字符。

AUR

Arch User Repository 的指南位于:

- AUR submission guidelines#Rules of submission
- AUR submission guidelines#Requests

官方 IRC 频道

注意：适用于中文社区旗下交流群的规则和建议在下方“中文社区交流群”小节内。

所有的官方 IRC 频道都位于 librea.chat 上，请首先遵守它们的网络政策和频道指南。

#archlinux频道的官方语言是英语,如果你需要以其它语言获得帮助,这里有各种本地化频道的列表.

- **#archlinux 频道的主要目的是讨论和支持 Arch Linux.** 如果和正在讨论的内容相关，一般的软件和硬件话题也是允许的。如果被要求私聊或移到其它频道，请遵守这个要求。
- 加入频道以后先通过 `/topic` 读一下频道的话题,一般包括了部分重要的信息.
- `phrik!~archbot@archlinux/bot/phrik`是唯一的官方机器人，不要滥用机器人指令，仅在有用时才使用机器人。如果希望加入你自己的机器人，请联系频道管理员。
- 不要刷屏,超过三行的消息请考虑使用 pastebin `program &> program-output.txt` 和 Pastebin 客户端可以简化这一操作.
- 想尝试某个命令或者获得命令帮助请用私聊(`/query`或 `/msg`.例如 `/query phrik help <command>`).
- 除了私聊中的离开相应外，禁止自动相应。
- 不用问谁在或是在用和你一样的软件,直接提问就好.
- 别依赖帮助,自己去寻找它.重复提问前先等等,大多数问题都是由其他*用户*回答的,例如你.
- 也别害羞,如果遇到你能回答的问题，请试着给予解答,只有更多的志愿者参与回答这个频道才能蓬勃发展.
- 如果有其他人询问关于你的问题的更多信息,记得最好总是回复他/他/它.

Wiki

下面页面包含 wiki 的规范:

- ArchWiki:Contributing#The 3 fundamental rules
- Help:Style
- Help:Discussion

Bug 系统

参考页面: Reporting bug guidelines.

中文社区交流群

irc.libera.chat 的频道列表

- #archlinux-cn ，主群组，讨论和 Arch Linux 相关的技术话题和寻求支持。
- #archlinux-cn-offtopic ，Arch Linux 相关人员的 Arch Linux 无关话题讨论群（也就是所谓的水群啦）
- #archlinux-cn-game，讨论各种游戏和邀请联机的分支频道。
- #archlinux-cn-bot，IRC 和 Telegram 机器人制作、交流和测试用频道。

由于曾经遭受过来自 IRC 的大量 spam 攻击，建议通过 libera.chat 的 NickServ 服务注册一个用户名并进行验证。 未验证的用户可能会被重定向至 #archlinux-cn-unregistered 频道。

这里是 libera.chat 提供的昵称注册指南： https://libera.chat/guides/registration

Telegram 上还有两个和 Arch Linux 中文社区相关的频道:

- @archlinuxcn , 中文社区相关消息（翻译官方新闻，testing 翻车预警，滚包预警等）。
- @archlinuxcn_updates , 官方仓库和中文社区的软件包更新推送频道。

其他入口

- 如果你使用 Matrix ，可以通过加入 #freenode_#频道名称:matrix.org 组加入聊天（例如 https://matrix.to/#/#freenode_#archlinux-cn:matrix.org ）
- 如果你使用 Telegram ，可以加入除了 #archlinux-cn-fuckbot 外的其他群组。请向群中成员获取加入方式。
- 如果你使用 XMPP/Jabber ，可以添加 talk@archlinuxcn.org （#archlinux-cn）或 offtopic@archlinuxcn.org（#archlinux-cn-offtopic）为好友加入聊天。

通用规则

- 所有的 IRC 频道都位于 Libera Chat 上，请首先遵守它们的网络政策和频道指南。
- 请勿在除 #archlinux-cn-nsfw 以外其他频道发送 NSFW(Not Safe For Work) 内容。
- 加入频道以后先通过 `\/topic\` 读一下频道的话题,会包含这个频道相关的一些信息。
- 不要问能不能问，请直接问。
- 频道内的机器人不止一个，不过也请不要滥用机器人指令。如果你想加入你自己的机器人，欢迎先来 #archlinux-cn-bot 测试。
- 不要刷屏，粘贴日志请考虑使用类 Pastebin 服务，如 https://cfp.vim-cn.com/ 或 https://fars.ee/ 。
- 贴图同样可以用类图床服务，如 https://img.vim-cn.com/ 或 https://fars.ee/ 。
- 不要对着桥接机器人说话，人类的名字在它们后面，请在回复时输入桥接机器人后面的昵称。用 weechat 客户端可以使用 bot2human 插件隐藏机器人姓名。

- 几个频道的成员基本相同，在一个频道提问后短时间没有回答请不要同时转发到别的频道。过分发布可能被视为 spam 行为。

#archlinux-cn 的额外规则

- #archlinux-cn 频道的主要目的是讨论和支持 Arch Linux 。如果你正在使用某个 Arch Linux 的衍生发行版

（如 Antergos、Manjaro、Chakara、Parabola GNU/Linux Libre 等)，非 Arch Linux 的 GNU/Linux 发行版， 或者在不受官方支持的非标准环境中使用（例如 Windows Subsystem for Linux 中），请在提问前先说明。

- 推荐阅读提问的智慧，一般一个好的问题更容易在短时间内被注意并得到解决。
- 别依赖帮助,自己去寻找它.重复提问前先等等,大多数问题都是由其他用户回答的,例如你.
- 也别害羞,如果遇到你能回答的问题，请试着给予解答,只有更多的志愿者参与回答这个频道才能蓬勃发展.
- 如果有其他人询问关于你的问题的更多信息,记得最好总是回复他/他/它.
- 管理员或其他成员在话题跑题或明显不属于技术话题时可能会要求移步至 #archlinux-cn-offtopic ,如果被要求这样做或者转至私聊，请遵守这个要求。

对于 Telegram 用户

- 加入后默认会用姓名作为昵称转发，请尽量设置一个便于 IRC 用户提及的昵称。可以通过 /nick 命令查看和修改昵称。
- 尽管转发机器人会自动将超过三行的文字发送至 Pastebin 服务，但仍需注意此点。
- 由于 Bot API 的限制（Bot 之间相互不可见），由 Telegram 上的其他机器人发送的消息无法被其他侧的 IM 看到。引用 Bot 消息时请注意。
- #archlinux-cn 和 #archlinux-cn-offtopic 已经设置成不会转发 Sticker。图片、视频、语音、文档等如未超过大小限制会被正常转发到图床。
- 由于存在转发机器人，请注意发布的信息中不要包括自己和其他人的隐私，已转发到 IRC 的消息无法被删除或修改。误发包含隐私信息的图片请联系图床管理员删除服务器存档（但是无法控制浏览器缓存）。

有关中文社区交流群的其它常见问题（例如机器人的操作指南）， 请参阅 https://fars.ee/~readme.html

维基软件

「维基软件」解决知识库架在哪、怎么长期维护：三个引擎各占一节，从安装讲到配置，读完应能在 Arch 上跑起一个维基并挂上反向代理。建议先读《DokuWiki》，安装、配置与安装后处理拆得最细，是理解共性的入口，必读；再读《MediaWiki》，四段齐全，自用看安装与配置，升级和技巧备查；《XWiki》放最后，两套 Nginx 代理方案按环境二选一，另一套可跳。

误解有三：把安装当结束，漏掉配置与安装后处理，站点能开，附件和编辑体验却不对；把两套代理方案当先后步骤；等升级才翻那段，跨版本升级往往比安装更麻烦。

8.1　XWiki

难度 入门 | **篇幅** 5.0 KB | **所属** 维基软件

本节将讲到

- 安装
- Nginx 代理配置 - 解决方案 1
- Nginx 代理配置 - 解决方案 2

XWiki 是一个用 Java 编写的开源企业级维基，重点在于可扩展性。

安装

请随时查看 XWiki 安装指南。 这些说明假定你将使用 Tomcat 和 PostgreSQL。将这些指南应用于其他组合应该不难。

- 安装 PostgreSQL。
- 为方便管理 PostgreSQL，请安装 phpPgAdmin。
- 安装tomcat（不要忘记 tomcat-native包）。
- 下载 XWiki WAR 文件
- 将 WAR 文件重命名为 `xwiki` 。
- 将 WAR 文件移至 `/var/lib/tomcat*n*/webapps` 目录。
- Tomcat 应该会自动解压缩 WAR 文件。如果没有，请重启 Tomcat。
- 此时，您可能会发现 `data` 目录出现在 `/var/lib/tomcat*n*/webapps` 中。删除它。
- 作为 root:

```
# cd /var/lib/tomcatn
# mkdir data
# chown tomcatn:tomcatn data
```

- 在 `/var/lib/tomcat*n*/webapps/xwiki/WEB-INF` 目录内：

- 打开 `xwiki.properties` 文件，修改 `environment.permanentDirectory` 字段为 `/var/lib/tomcat*n*/data/xwiki`。
- 打开 `hibernate.cfg.xml` 文件：
- 注释掉题为 "Configuration for the default database" 的部分。
- 取消注释题为 "PostgreSQL Configuration" 的部分。
- 根据需要修改数据库名称（在`connection.url`）、用户名和密码。
- 在 PostgreSQL 中创建 role 和数据库，以匹配 hibernate 配置。
- 从 Arch 用户仓库安装 postgresql-jdbcAUR。
- 作为 root:

```
# cd /usr/share/java/tomcatn
# ln -s /usr/share/java/postgresql-jdbc/postgresql-jdbc41.jar
```

- 重启 `tomcat*n*.service`。
- 在 Tomcat Manager 中点击 `/xwiki` 启动 XWiki 应用程序。
- 通过 XWiki Wizard Guide 启动 XWiki，完成配置。

Nginx 代理配置 - 解决方案 1

用于 XWiki 的 Nginx 官方指南并不正确。有一个适用于 XWiki 的替代解决方案。

- 配置 nginx 网站 `xwiki` 配置文件。

```
/etc/nginx/sites-available/xwiki
server {
  listen 80 default_server;
  server_name xwiki.<domain-name>;
  return 301 https://$host$request_uri;
}

server {
  listen [::]:443 ssl;
  listen 443 ssl;

  server_name xwiki.<domain-name>;

  # SSL Certificate section
  ssl_certificate ...
  ssl_certificate_key ...

  location = / {
    return 301 https://$host/xwiki;
  }

  location /xwiki {
    proxy_set_header   X-Real-IP $remote_addr;
    proxy_set_header   Host      $host;
    proxy_http_version 1.1;
    proxy_set_header   Upgrade $http_upgrade;
    proxy_set_header   Connection 'upgrade';
    proxy_cache_bypass $http_upgrade;
    proxy_set_header   X-Forwarded-Host $host;
    proxy_set_header   X-Forwarded-Server $host;
    proxy_set_header   X-Real-IP $remote_addr;
```

```
    proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header   X-Forwarded-Proto $scheme;
    proxy_pass         http://127.0.0.1:8080/xwiki;
  }
}
```

- 激活 `sites-enabled` 目录 （`ln -s /etc/nginx/sites-available/xwiki /etc/nginx/sites-enabled/xwiki`） 中的服务器块。
- 重启 Nginx。

Nginx 代理配置 - 解决方案 2

我发现指示nginx代理到`[http://localhost:8080/xwiki/](http://localhost:8080/xwiki/)`不起作用：生成的URL不正确。与XWiki 文档\[失效链接 2024-01-13 ⓘ\] 中的说明相反，我无法通过使用 HTTP 标头使 URL 正确。

目前我所知道的唯一解决方案是在 Tomcat 的 server.xml 文件中创建一个新的 Host 元素：

- 复制现有的 `Host` 元素，并将 `name` 属性改为 `xwiki`。
- 修改 `appBase` 属性为 `/var/lib/tomcat7/webapps-xwiki`。
- 将 `xwiki` 应用程序从 `/var/lib/tomcat7/webapps/xwiki` 移至 `/var/lib/tomcat7/webapps-xwiki/ROOT` 。
- 重启 Tomcat。
- 在 `/etc/hosts` 中添加 `xwiki` 作为 localhost 的别名。(添加到 127.0.0.1 行的末尾）。
- 指示 Nginx 代理至 `[http://xwiki:8080/](http://xwiki:8080/)`。

8.2 DokuWiki

难度 进阶 | **篇幅** 9.7 KB | **所属** 维基软件

本节将讲到

- Installation
- Configuration
- Post installation
- Further reading

摘自其网站:

: DokuWiki 是一个简单易用、用途多样并且不依赖数据库的开源维基软件。它因简洁易读的语法受到用户的喜爱。而容易维护、备份方便和易于整合则使它成为管理员的

最爱。内置的访问控制和认证管理使 DokuWiki 在企业环境下特别适合，而由充满活力的社区贡献的众多插件对其功能的扩充则令它拥有了比传统维基更广阔的应用。

See upstream for the detailed list of features.

DokuWiki should work on any web server which supports PHP. As the requirements may change over time, you should consult the requirements page for DokuWiki for additional details.

It is strongly recommend to read through the appropriate sections of DokuWiki's security page for your web server. Most popular web servers are covered but there are generic instructions as well.

Installation

The package in the official repositories unpacks DokuWiki at `/usr/share/webapps/dokuwiki` with the configuration files in `/etc/webapps/dokuwiki` and the data files in `/var/lib/dokuwiki/data`. It also changes the ownership of the relevant files to the "http" user. This should work fine for most popular web servers as packaged for Arch.

1. Install your web server of choice (e.g. Apache HTTP Server, nginx or lighttpd) and configure it for PHP. As mentioned above, DokuWiki has no need for a database server so you may be able to skip those steps when setting up your web server.
2. Install the dokuwiki包 package.
3. Configure web server for dokuwiki (see section below)
4. With your web browser of choice, open http://<your-server>/dokuwiki/install.php and continue the installation from there. For nginx the URL is http://<your-server>/install.php.

Alternatively, if you would like to install from tarball, you can read from https://www.dokuwiki.org/Install. Generally the procedure is the same as above. Instead of using pacman, you will need to download the tarball, unpack it to your server's document root (e.g. `/srv/http/dokuwiki`), and chown to the appropriate user (e.g. "http").

Configuration

If you use PHP's open_basedir, you need to include the dokuwiki directories `/etc/webapps/dokuwiki/` and `/var/lib/dokuwiki/`. For example:

```
/etc/php/php.ini
open_basedir = <other paths>:/etc/webapps/dokuwiki/:/var/lib/dokuwiki/
```

Also uncomment the following line.

```
/etc/php/php.ini
extension=gd
```

Dokuwiki needs this library for resizing images.

Then check that you have php-gd包 installed and restart `php-fpm.service`.

Apache

The package should add the file `/etc/httpd/conf/extra/dokuwiki.conf` with the following contents:

```
/etc/httpd/conf/extra/dokuwiki.conf
Alias /dokuwiki /usr/share/webapps/dokuwiki
<Directory /usr/share/webapps/dokuwiki/>
    Options +FollowSymLinks
    AllowOverride All
    order allow,deny
    allow from all
    php_admin_value open_basedir
"/tmp/:/usr/share/pear/:/usr/share/webapps/:/etc/webapps/dokuwiki/:/var/lib/dokuwiki/"
</Directory>
```

If you are running Apache 2.4 or newer, you will have to change the following lines:

```
/etc/httpd/conf/extra/dokuwiki.conf
    order allow,deny
    allow from all
```

to read:

```
/etc/httpd/conf/extra/dokuwiki.conf
    Require all granted
```

Include the newly created file in the Apache configuration by placing the following line at the end of `/etc/httpd/conf/httpd.conf`:

```
/etc/httpd/conf/httpd.conf
Include conf/extra/dokuwiki.conf
```

Make sure the folders `/etc/webapps/dokuwiki` and `/var/lib/dokuwiki` are owned by user and group "http". You may relocate these directories if you like as long as you update the references in `/etc/httpd/conf/extra/dokuwiki.conf` respectively. You should keep the reference to `/var/lib/dokuwiki/` for DokuWiki to find the plugins and tpl folder.

Afterwards restart Apache.

Then finish the installation by running the *dokuwiki/install.php* script in your browser.

lighttpd

Edit the `/etc/lighttpd/lighttpd.conf` file as per the dokuwiki instructions (might contain updated information).

Make sure the modules `mod_access` and `mod_alias` are loaded. If not, load them by adding the following to `/etc/lighttpd/lighttpd.conf`:

```
server.modules += ("mod_access")
server.modules += ("mod_alias")
```

`mod_access` provides the `url.access-deny` command, which we are using from this point.

Under the line:

```
$HTTP["url"] =~ "\.pdf$" {
  server.range-requests = "disable"
}
```

add this:

```
# subdir of dokuwiki
# comprised of the subdir of the root dir where dokuwiki is installed
# in this case the root dir is the basedir plus /htdocs/
# Note: be careful with trailing slashes when uniting strings.
# all content on this example server is served from htdocs/ up.
#var.dokudir = var.basedir + "/dokuwiki"
var.dokudir = server.document-root + "/dokuwiki"

# make sure those are always served through fastcgi and never as static files
# deny access completly to these
$HTTP["url"] =~ "/(\.|_)ht" { url.access-deny = ( "" ) }
$HTTP["url"] =~ "^" + var.dokudir + "/(bin|data|inc|conf)/"  { url.access-deny =
( "" ) }
```

These entries give some basic security to DokuWiki. lighttpd does not use .htaccess files like Apache. You CAN install with out this, but I would NEVER recommend it.

Add alias somewhere in lighttpd or fastcgi conf file:

```
alias.url += ("/dokuwiki" => "/usr/share/webapps/dokuwiki/")
```

Restart lighttpd.

nginx

Ensure that php-fpm包 is installed and started.

Add the following server block, but change the server name to your own and comment out the install.php block until you are done installing DokuWiki. This block assumes you use TLS.

```
/etc/nginx/nginx.conf
    server {
        listen 443 ssl http2;
        listen [::]:443 ssl http2;
        server_name wiki.example.com;

        root /usr/share/webapps/dokuwiki;
        index doku.php;

        #Remember to comment the below out when you are installing DokuWiki, and
uncomment it when you are done.
        location ~ /(data/|conf/|bin/|inc/|install.php) { deny all; } # secure Dokuwiki

        location ~^/\.ht { deny all; } # also secure the Apache .htaccess files
        location @dokuwiki {
            #rewrites "doku.php/" out of the URLs if you set the userewrite setting
to .htaccess in dokuwiki config page
            rewrite ^/_media/(.*) /lib/exe/fetch.php?media=$1 last;
            rewrite ^/_detail/(.*) /lib/exe/detail.php?media=$1 last;
            rewrite ^/_export/([^/]+)/(.*) /doku.php?do=export_$1&id=$2 last;
            rewrite ^/(.*) /doku.php?id=$1&$args last;
        }

        location / { try_files $uri $uri/ @dokuwiki; }
        location ~ \.php$ {
            try_files $uri =404;
            fastcgi_pass unix:/run/php-fpm7/php-fpm.sock;
            fastcgi_index index.php;
            include fastcgi.conf;
        }

    }
```

Restart nginx.

Enable upload and displaying of SVG files

DokuWiki supports SVG files but has them disabled by default.

If you wish to enable them, create the following file:

```
/etc/webapps/dokuwiki/mime.local.conf
svg image/svg+xml
```

This has security implications - see here

Post installation

Cleaning up

After configuring the server either remove the install.php file or make sure it is made inaccessible in your webserver configuration!

```
 # rm /usr/share/webapps/dokuwiki/install.php
```

Installing plugins

Many community created plugins can be found here

They can be added through the web interface (as well as updated) through the Admin menu. Some plugins cannot be downloaded, if they go over ssl (e.g. git).

Backing up

It is very trivial to backup DokuWiki, since there is no database. All pages are in plain text, and require only a simple tar, or rsync.

A quick breakdown of the directories of interest in the current (20180422_a-1) version:

```
 /usr/share/webapps/dokuwiki/data/  =>  All User Created Data
 /usr/share/webapps/dokuwiki/conf/  =>  Configuration settings
```

This may change in future releases, please consult the DokuWiki Backup FAQ for verification.

Further reading

The DokuWiki main site has all of the information and help that you could possibly need.

8.3 MediaWiki

难度 进阶 | **篇幅** 10.8 KB | **所属** 维基软件

本节将讲到

- 安装
- 配置
- 升级
- 提示和技巧

MediaWiki 是一个自由、开源的维基软件，用 PHP 写成；原本是为维基百科开发的。它也给 ArchWiki 和本维基提供了帮助。（详情查看 Special:版本和 GitHub 仓库）。

安装

为了运行 MediaWiki 你需要三个组件：

- mediawiki包 包，PHP 会作为它的依赖安装
- 一个网络服务器——例如 Apache HTTP 服务器、Nginx 和 lighttpd
- 一个数据库系统—— MariaDB、PostgreSQL、SQLite 或 MySQL

如果要在 XAMPP 上安装 MediaWiki，参见 mw:Manual:Installing MediaWiki on XAMPP

配置

实现一个可用的 MediaWiki 配置的步骤包括编辑 PHP 设置和添加 MediaWiki 配置片段。

PHP

MediaWiki 需要 `iconv` 和 `intl` 插件，所以需要把 `/etc/php/php.ini` 里面的 `extension=iconv` 和 `extension=intl` 取消注释。

可选插件：

- 为了渲染缩略图，安装 ImageMagick 或者 php-gd包（二选一）。如果安装的是后者，需要取消注释 `extension=gd`。

启用你的数据库管理系统的 API：

- 如果是 MariaDB，取消注释`extension=mysqli`。
- 如果是 PostgreSQL，安装 php-pgsql包 并取消注释 `extension=pgsql`。
- 如果是 SQLite，安装 php-sqlite包 并取消注释 `extension=pdo_sqlite`。

其次，调整会话处理方式，否则可能会在寻找 `session.save_path` 路径时遇到严重错误（`PHP Fatal error: session_start(): Failed to initialize storage module[...]`）。`/var/lib/php/sessions` 或 `/tmp/` 路径是个不错的选择。

```
/etc/php/php.ini
session.save_path = "/var/lib/php/sessions"
```

如果那个目录不存在，你要手动创建它，并更改权限：

```
# mkdir -p /var/lib/php/sessions/
# chown http:http /var/lib/php/sessions
# chmod go-rwx /var/lib/php/sessions
```

如果你使用了 PHP's open_basedir 并想允许文件上传，你需要把 `/var/lib/mediawiki/` 添加到里面去 (mediawiki包 为 `images/` 创建了指向 `/var/lib/mediawiki/` 的符号链接)。

网站服务器

Apache

按照 Apache HTTP Server#PHP 配置 PHP。

复制 `/etc/webapps/mediawiki/apache.example.conf` 到 `/etc/httpd/conf/extra/mediawiki.conf` 并按需要修改它。 添加下面这一行到 `/etc/httpd/conf/httpd.conf`:

```
Include conf/extra/mediawiki.conf
```

重启 `httpd.service`。

注意：默认示例文件 `/etc/webapps/mediawiki/apache.example.conf` 会覆盖 PHP 的 open_basedir 设置，可能会和其他页面产生冲突。 可以通过把以 `php_admin_value` 开头的行移到 `<Directory>` 标签之间来避免这个问题。而如果你运行了多个依赖同一个 server 的应用，你可以把这个值添加到 `/etc/php/php.ini` 里的 open_basedir 里，而不仅仅是放在 `/etc/httpd/conf/extra/mediawiki.conf` 里。

Nginx

对于 Nginx，请创建这样的一个文件：

```
/etc/nginx/mediawiki.conf
location / {
   index index.php;
   try_files $uri $uri/ @mediawiki;
}
location @mediawiki {
   rewrite ^/(.*)$ /index.php;
}
location ~ \.php?$ {
   include /etc/nginx/fastcgi_params;
   fastcgi_pass unix:/var/run/php-fpm/php-fpm.sock;
   fastcgi_index index.php;
   fastcgi_param  SCRIPT_FILENAME  $document_root$fastcgi_script_name;
   try_files $uri @mediawiki;
}
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
   try_files $uri /index.php;
   expires max;
   log_not_found off;
}
# Restrictions based on the .htaccess files
location ~ ^/(cache|includes|maintenance|languages|serialized|tests|images/deleted)/ {
   deny all;
}
location ~ ^/(bin|docs|extensions|includes|maintenance|mw-config|resources|serialized|
tests)/ {
   internal;
}
location ^~ /images/ {
   try_files $uri /index.php;
}
location ~ /\. {
```

```
    access_log off;
    log_not_found off;
    deny all;
}
location /rest.php {
    try_files $uri $uri/ /rest.php?$args;
}
```

请确保已经安装了 php-fpm包 并启动了 `php-fpm.service`。

在`/etc/nginx/nginx.conf`添加一个 server 块，类似这样的：

```
/etc/nginx/nginx.conf
server {
  listen 80;
  server_name mediawiki;
  root /usr/share/webapps/mediawiki;
  index index.php;
  charset utf-8;
# For correct file uploads
  client_max_body_size    100m; # Equal or more than upload_max_filesize
in /etc/php/php.ini
  client_body_timeout     60;
  include mediawiki.conf;

}
```

最后，restart `nginx.service` 和 `php-fpm.service`。

Lighttpd

应当先安装并配置好 Lighttpd。lighttpd 的 server.modules array 中需要“mod_alias“和“mod_rewrite“。将以下内容添加至 lighttpd 的配置文件：

```
/etc/lighttpd/lighttpd.conf
alias.url += ("/mediawiki" => "/usr/share/webapps/mediawiki/")
url.rewrite-once += (
                "^/mediawiki/wiki/upload/(.+)" => "/mediawiki/wiki/upload/$1",
                "^/mediawiki/wiki/$" => "/mediawiki/index.php",
                "^/mediawiki/wiki/([^?]*)(?:\?(.*))?" => "/mediawiki/index.php?
title=$1&$2"
)
```

然后，重启 `lighttpd.service` 守护进程。

数据库

按照你所需要的数据库管理系统（DBMS）的文章配置数据库服务器：MariaDB、PostgreSQL、SQLite 或 MySQL。

如果数据库密码不为空，MediaWiki会自动创建数据库(设置 MariaDB 密码的方法在 MariaDB#Reset the root password) 。否则你就需要手动创建数据库，详情参考：upstream instructions。

LocalSettings.php

在浏览器里打开 wiki 的 URL (通常是 `http://*your_server*/mediawiki/index.php`) 并进行初始化配置。参考upstream instructions的步骤。

生成的 `LocalSettings.php` 文件是用来下载的，保存它到 `/usr/share/webapps/mediawiki/LocalSettings.php`。

自 1.38.0 版本起，`/usr/share/webapps/mediawiki/LocalSettings.php` 内已包含一个符号链接。

警告：LocalSettings.php 包含了数据库连接设置（例如用户名和密码）和网络更新密码。请确保仅 root 和 http 用户可以访问 `/etc/webapps/mediawiki/LocalSettings.php` 文件：

```
# chown root:http /etc/webapps/mediawiki/LocalSettings.php
# chmod 640 /etc/webapps/mediawiki/LocalSettings.php
```

该文件定义了所部署维基的特定设置。升级 mediawiki包 软件包不会替换此文件。

LDAP Auth

使用 PluggableAuth 和 LDAP Stack。请留意“Compatibility Matrix”章节。目前 LDAP 仅能与 PluggableAuth-5.7 一同使用。

请安装并在配置中添加 ldap stack 扩展和 PluggableAuth：

- Extension:PluggableAuth
- Extension:LDAPProvider
- Extension:LDAPAuthentication2
- Extension:LDAPAuthorization
- Extension:LDAPUserInfo
- Extension:LDAPGroups

接下来请设置至少 3 个变量：

- `$LDAPProviderDomainConfigProvider`——整个 ldap 配置（可以是 json 文件）
- `$wgPluggableAuth_Config`——auth 插件列表

```
$wgPluggableAuth_Config = array(
        array('plugin' => 'LDAPAuthentication2'),
        array('plugin' => 'LDAPAuthorization'),
);
```

- 以及 `$LDAPProviderDefaultDomain`

配置完成后请运行 `php maintenance/update.php`。

升级

参考 mw:Manual:Upgrading, 不要忘记运行:

```
# cd /usr/share/webapps/mediawiki
# php maintenance/update.php
```

提示和技巧

Unicode

请确保PHP, Apache HTTP Server 和 MariaDB都用的是 UTF-8 编码。否则可能遇到因为编码不匹配导致的奇怪bug。

可视化编辑器

VisualEditor（可视化编辑器）扩展为 MediaWiki 提供了一个富文本编辑器。请参照 mw:Extension:VisualEditor 安装。

Live Arch 系统

没有网络时怎么装出一套可用的 Arch，怎样把配好的系统压成一张 ISO，是本章要解决的两个问题。《离线安装》必读：从准备本地仓库、挂载与配置到 pacstrap 与软件包的离线安装，是一条不依赖镜像站的完整链路。《Archiso》篇幅更长，初读只需跟住准备自定义配置文件、构建 ISO、使用与 QEMU 测试这条主线，技巧提示可留到改包时再翻。

最容易踩的坑在本地仓库：只堆软件包却漏了依赖或仓库元数据，pacstrap 第一步就断。也有人以为有了 archiso 就不必再用 pacstrap，其实两者不在同一层。

9.1　离线安装

难度 进阶 | **篇幅** 6.5 KB | **所属** Live Arch 系统

本节将讲到

- 准备本地仓库
- 挂载和配置
- Pacstrap
- 软件包的离线安装
- 另见

此文提供在无互联网连接时安装 Arch Linux 的指南。要达到此目的，需要另一有互联网连接的系统。

首先，按照安装指南，跳过安装指南-连接到互联网部分，直接跳到安装指南#安装必需的软件包步骤。

警告：从 ISO 解压根文件系统镜像 (`airootfs.sfs`) 或复制 live 环境根文件系统不是受支持的方法。

准备本地仓库

从 Pacman/Tips and tricks#Installing packages from a CD/DVD or USB stick 获取有关在单独的主机安装上准备具有必要文件的本地仓库的说明。

至少，对于正常运行的系统，建议使用以下软件包：

```
# pacman -Syw --cachedir . --dbpath /tmp/blankdb base base-devel linux linux-firmware mkinitcpio vim
```

创建您的自定义离线仓库

```
# repo-add ./custom.db.tar.gz ./*[^sig]
```

挂载和配置

准备好仓库后，将外部媒体连接到新安装，并将其挂载到新创建的根文件系统上：

```
# mount --mkdir /dev/sdxy /mnt/repo
```

编辑您的 archiso `/etc/pacman.conf` 并添加一个新部分：

```
/etc/pacman.conf
[custom]
SigLevel = Optional
Server = file:///mnt/repo/
```

注释掉 `[core]` 和 `[extra]` 以使 pacman 在默认仓库上不失败。

Pacstrap

您现在可以继续将本地可用的软件包打包到新安装中：

```
# pacstrap -K /mnt base base-devel linux linux-firmware mkinitcpio vim
```

软件包的离线安装

从文件安装

当离线安装仅是临时的，但需要在可访问互联网前安装自定义的包，参见 pacman#其它命令以了解安装本地包的方法。

Shell 的通配符)可用于立即安装：

```
# pacman -U /package/folder/*.tar.zst
```

离线缓存

您可以把指定文件置于 `/var/lib/pacman/sync` 以及 `/var/cache/pacman/pkg` 以便 `pacman` 认为其拥有搜索、更新和安装之一切所需。下文方法基于：。

步骤：

1. 用可访问互联网的电脑（在线电脑）下载最新包的数据库；
2. 传输至离线电脑（目标电脑）；
3. 生成离线电脑所需更新的包列表；
4. 用在线电脑下载包及其签名；
5. 传输至离线电脑的 pacman 缓存；
6. 安装更新。

提示：若您的默认仓库不是 core，extra 或 multilib，您应检查您的 `/etc/pacman.conf` 文件。

下列脚本会下载最新包数据库，若需要，可修改 `MIRROR` 至镜像状态列表之任一。

```
download_databases.sh
#!/bin/sh

ARCH="x86_64"
MIRROR="https://mirrors.kernel.org/archlinux/"

wget "${MIRROR}/core/os/${ARCH}/core.db"
wget "${MIRROR}/extra/os/${ARCH}/extra.db"
wget "${MIRROR}/multilib/os/${ARCH}/multilib.db"
```

```
# and possibly -uncomment- (if customized in /etc/pacman.conf or pacman.conf.d):

#wget "${MIRROR}/core-testing/os/${ARCH}/core-testing.db"
#wget "${MIRROR}/extra-testing/os/${ARCH}/extra-testing.db"
#wget "${MIRROR}/multilib-testing/os/${ARCH}/multilib-testing.db"

# and -additionaly- debug and staging packages.
```

转化为可执行文件并运行。您将得到多个 *.db* 文件。

下列步骤将传输 *.db* 文件至离线电脑，使您可更新包列表（效果与在线的 `pacman -Sy` 相同），然后生成需更新包的列表。

```
# cp *.db /var/lib/pacman/sync/
# pacman -Sup --noconfirm > pkglist
```

注意：确保您启用了至少一个 `/etc/pacman.d/mirrorlist` 定义的服务器。否则您将得到一个误导性的错误信息：`error: no database for package: package-name`.

您还需下载相应签名，所以请准备下载签名的列表：

```
# sed -e 's/\.zst$/.zst.sig/' ../pkglist > ../siglist
```

将这两个列表传输至在线电脑，并在空目录里下载列出的包：

```
# wget -nv -i ../pkglist
# wget -nv -i ../siglist
```

提示：当使用 cygwin 或其他 Windows 环境下载包时，文件名可能会损坏，因为 Windows 默认文件名不许有一些字符，如冒号。若要避免损坏（cygwin 环境下，因为其命名限制不同），使用 `wget --restrict-file-names=unix`.

将所有 `*.pkg.tar.zst` 和 `*.pkg.tar.zst.sig` 文件带回离线电脑，置于 `/var/cache/pacman/pkg` 并最后运行：

```
# pacman -Su
```

本地仓库

如果预计新系统将保持离线或物理隔离，应配置为仅期望本地仓库。

完善仓库

在 chroot 到您的新安装后，编辑新 `/etc/pacman.conf` 使之与旧者一致（但无 `/mnt` 前缀）：

```
/etc/pacman.conf
[custom]
SigLevel = Optional
Server = file:///repo/
```

注释掉其他所有仓库，继续配置新系统。

从此对离线系统的任何更新都可以通过挂载本地仓库的最新副本至 `/repo`，再运行 pacman 命令。

另见

- 离线阅读：离线浏览 Arch Wiki.

9.2 Archiso

难度 高级 | **篇幅** 26.6 KB | **所属** Live Arch 系统

本节将讲到

- 安装
- 准备自定义配置文件
- 构建 ISO 映像
- 使用 ISO
- 在 QEMU 中测试 ISO

翻译状态：

- 本文（或部分内容）译自 archiso，最近一次同步于 2026-06-25，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Archiso 是一个高度可定制的工具，用于构建 Arch Linux live CD/USB ISO 映像、netboot 文件、bootstrap tar 包。官方映像是用 archiso 构建的，包含这些软件包。它可以用作救援系统、Linux 安装程序或其他系统的基础。这篇 Wiki 文章介绍了如何安装 archiso，以及如何配置它以控制生成的 ISO 映像的各个方面，例如随附的软件包和文件。 技术需求和构建步骤可以在官方项目文档中找到。 archiso 通过许多 bash 脚本实现。 archiso 的核心组件是 *mkarchiso* 命令。 它的选项记录在 `mkarchiso -h` 中，这里不作介绍。

安装

安装 archiso包。

准备自定义配置文件

注意：要深入了解配置文件，可参阅 archiso 项目的 README.profile.rst。

Archiso 带有两个配置文件，**releng** 和 **baseline**。

- **releng** 用于创建正式的每月安装ISO。它可以作为创建自定义ISO映像的起点。
- **baseline** 是一种最低限度的配置，它只包括从介质启动 Live 环境所需的最低限度的软件包。

要构建配置文件的未修改版本，请参阅 #构建 ISO 映像。否则，如果您希望调整或自定义 archiso 的已发布配置文件之一，请将其从 `/usr/share/archiso/configs/*profile-name*/` 复制到一个可写目录，并使用您的名称选择。例如：

```
$ cp -r /usr/share/archiso/configs/releng/ archlive
```

现在，根据所选的配置文件和您的目标进入不同的章节。

配置文件结构

Archiso 配置文件包含定义生成的 ISO 映像的配置。配置文件结构记录在 `/usr/share/doc/archiso/README.profile.rst`。

选择包

编辑 `packages.x86_64` 以选择要在 Live 系统映像上安装的软件包，逐行列出软件包。

自定义本地仓库

要添加不在标准 Arch 仓库中的包（例如来自 AUR 或采用 ABS 定制的包），请建立一个自建本地仓库并将您的自定义包添加到其中。然后将您的仓库添加到 `pacman.conf`，如下所示：

```
archlive/pacman.conf
...
[customrepo]
SigLevel = Optional TrustAll
Server = file:///path/to/customrepo
...
```

注意：

- `pacman.conf` 中的顺序很重要。要为您的自定义存储库提供最高优先级，请将其置于其他存储库条目之上。
- 这个 `pacman.conf` 仅用于在构建映像，而不会在 live 环境中有效。要在 live 环境里使用，请参考 #向映像中添加仓库
- 确保 *mkarchiso* 过程（包括 chroot）可以访问仓库，应将仓库放在合适的目录下（如 `/tmp`），以确保映像构建时正确读取仓库。
- mkarchiso 会寻找名为 `custom` 的仓库，因此在创建仓库时，应运行：

```
$ repo-add /path/to/custom.db.tar.zst /path/to/package-1.0-1-x86_64.pkg.tar.zst
```

安装 multilib 中的软件包

要从 Multilib 资源库中安装软件包，仅需在`pacman.conf`中取消注释如下的相关行以启用 *multilib*：

```
[multilib]
Include = /etc/pacman.d/mirrorlist
```

向映像里添加文件

Airootfs 目录用作映像上 Live 系统的根目录 (`/`) 的起点。在安装包之前，它的所有内容都将被复制到工作目录中。

将任何自定义文件和/或目录放在 `airootfs/` 下的所需位置。例如，如果您在当前系统上自定义了 nftables 规则，希望在 Live 映像上沿用，请按如下方式复制它们：

```
$ cp -r /etc/nftables.conf archlive/airootfs/etc
```

同样，对于驻留在层次结构中某处的特殊配置文件也需要格外小心。使用 mkdir(1) 可以轻松地创建目录结构的缺失部分。

提示：要将文件添加到安装用户的主目录，请将其放在 `*archlive*/airootfs/root/` 中。要将文件添加到所有其他用户的主目录，请将其放在 `*archlive*/airootfs/etc/skel/`。

注意：与软件包提供的冲突的自定义文件将被覆盖，除非软件包将它们指定为 备份文件。

默认情况下，权限将是 `644` （对于文件）和 `755`（对于目录）。所有这些都将归根用户所有。要为特定文件和/或文件夹设置不同的权限或所有权，请使用 `profiledef.sh` 中的 `file_permissions` 关联列表。有关详细信息，请参阅 README.profile.rst。

向映像中添加仓库

要在 live 环境中添加一个可用的仓库，请创建一个适当修改的`pacman.conf`，并将其置于`*archlive*/airootfs/etc/`下。

若仓库也使用密钥，将密钥放置在 `*archlive*/airootfs/usr/share/pacman/keyrings/` 下。密钥文件名须以 `.gpg` 结尾。并且，密钥必须被信任。可以在同目录下创建 GnuPG 导出的信任文件，来实现这一点。文件名须以 `-trusted` 结尾。文件的第一部分是密钥指纹，第二部分是信任信息。您可以参考 `/usr/share/pacman/keyrings/archlinux-trusted` 作为实例。

ArchZFS 实例

实例中文件如下：

```
airootfs
├── etc
│   ├── pacman.conf
│   └── pacman.d
│       └── archzfs_mirrorlist
└── usr
    └── share
        └── pacman
            └── keyrings
                ├── archzfs.gpg
                └── archzfs-trusted
airootfs/etc/pacman.conf
...
[archzfs]
Include = /etc/pacman.d/archzfs_mirrorlist
...
airootfs/etc/pacman.d/archzfs_mirrorlist
Server = https://github.com/archzfs/archzfs/releases/download/experimental
airootfs/usr/share/pacman/keyrings/archzfs-trusted
3A9917BF0DED5C13F69AC68FABEC0A1208037BE9:4:
```

导入并信任此密钥（在映像构建期间，或在启动之后的设置脚本中）：

```
$ curl -Lo archzfs.gpg 'https://github.com/archzfs/archzfs-keyring/raw/master/keyring/packager/archzfs/3A9917BF0DED5C13F69AC68FABEC0A1208037BE9/3A9917BF0DED5C13F69AC68FABEC0A1208037BE9.asc'
# pacman-key --add archzfs.gpg
# pacman-key --lsign-key 3A9917BF0DED5C13F69AC68FABEC0A1208037BE9
```

内核

尽管两个 Archiso 包含的配置文件都只有linux包，但可以使映像包含其他甚至多个内核。

首先，编辑 `packages.x86_64` 以包含您想要的内核包名称。当 *mkarchiso* 运行时，它将包括所有 `*work_dir*/airootfs/boot/vmlinuz-*` 和 `*work_dir*/boot/initramfs-*.img` ISO 中的文件（以及用于 UEFI 引导的 FAT 映像中）。

mkinitcpio 默认情况下预设将构建后备 initramfs 映像。对于 ISO，主 initramfs 映像通常不包含 `autodetect` 挂钩，因此不需要额外的备用映像。为防止创建后备 initramfs 映像，使其不占用空间或减慢构建过程，请在 `*archlive*/airootfs/etc/mkinitcpio.d/*pkgbase*.preset` 中放置自定义预设。例如，对于 linux-lts包：

```
archlive/airootfs/etc/mkinitcpio.d/linux-lts.preset
PRESETS=('archiso')

ALL_kver='/boot/vmlinuz-linux-lts'
ALL_config='/etc/mkinitcpio.conf'

archiso_image="/boot/initramfs-linux-lts.img"
```

最后，创建引导加载程序配置文件来启动内核。

引导加载程序

Archiso 支持 syslinux 用于 BIOS 引导和 systemd-boot 或 GRUB 用于 UEFI 引导。有关其配置语法的信息，请参阅引导加载程序的文章。

提示：

- 当使用 El Torito 刻录到光盘时，或使用 Isohybrid 写入硬盘（或 USB 闪存驱动器或类似驱动器）时，*releng'* 配置文件默认构建到支持 BIOS 和 UEFI 启动的 ISO 中。
- 由于 isolinux 的模块化特性，您可以使用许多插件，因为所有 *.c32* 文件都已复制并可供您使用。看看 官方 syslinux 站点 和 configs/releng/syslinux archiso git repo。使用所述插件，可以制作具有视觉吸引力的复杂菜单。参见 。

mkarchiso 期望 GRUB 配置位于 `grub` 目录中，systemd-boot 配置位于 `efiboot` 目录中，而 syslinux 配置位于 `syslinux` 目录中。

UEFI 安全启动

如果要使 archiso 在启用 UEFI 安全启动的环境下可启动，则必须使用签名过的引导加载程序。您可以按照 安全启动#引导安装介质 中的说明进行操作。

systemd 单元

要为 Live 环境 启用 systemd 服务/套接字/计时器，您需要像 `systemctl enable` 那样手动创建符号链接。

例如，要启用包含 `WantedBy=multi-user.target` 的 `gpm.service`，请运行：

```
$ mkdir -p archlive/airootfs/etc/systemd/system/multi-user.target.wants
$ ln -s /usr/lib/systemd/system/gpm.service archlive/airootfs/etc/systemd/system/multi-user.target.wants/
```

可以通过读取 systemd 单元找到所需的符号链接，或者如果您安装了服务，则通过 启用 它并观察 systemctl 输出。

登录管理器

通过启用登录管理器的 systemd 服务来在启动时 Start X。如果您不知道要启用哪个 *.service*，您可以轻松找出是否在您构建 ISO 的系统上使用相同的程序。只需使用：

```
$ ls -l /etc/systemd/system/display-manager.service
```

现在在 `*archlive*/airootfs/etc/systemd/system/` 中创建相同的符号链接。对于 LXDM：

```
$ ln -s /usr/lib/systemd/system/lxdm.service
archlive/airootfs/etc/systemd/system/display-manager.service
```

这将在您的 Live 系统上的系统启动时启用 LXDM。

改变自动登录

Getty 的自动登录配置位于 `airootfs/etc/systemd/system/getty@tty1.service.d/autologin.conf`。

您可以修改这个文件来更改自动登录用户：

```
[Service]
ExecStart=
ExecStart=-/sbin/agetty --noreset --noclear --autologin username - ${TERM}
```

或者干脆删除 `autologin.conf` 来禁用自动登录。

如果使用串行控制台，请创建 `airootfs/etc/systemd/system/serial-getty@ttyS0.service.d/autologin.conf` 并添加以下内容：

```
[Service]
ExecStart=
```

```
ExecStart=-/sbin/agetty --noreset --noclear --autologin root --keep-baud
115200,57600,38400,9600 - ${TERM}
```

用户和密码

要创建在 Live 环境中可用的 用户，您必须手动编辑 `*archlive*/airootfs/etc/passwd`、`*archlive*/airootfs/etc/shadow`、`*archlive*/airootfs/etc/group` 和 `*archlive*/airootfs/etc/gshadow`。

注意：如果这些文件存在，它们必须包含 root 用户和组。

例如，添加用户 `archie`。按照 passwd(5) 语法将它们添加到 `*archlive*/airootfs/etc/passwd` ：

```
archlive/airootfs/etc/passwd
root:x:0:0:root:/root:/usr/bin/zsh
archie:x:1000:1000::/home/archie:/usr/bin/zsh
```

注意：`passwd` 文件须以换行符结尾。

按 shadow(5) 的语法将用户添加到 `*archlive*/airootfs/etc/shadow`。若要为用户设置密码，请使用 `openssl passwd -6` 生成密码哈希值，并将其添加到文件中。例如：

```
archlive/airootfs/etc/shadow
root::14871::::::
archie:
$6$randomsalt$cij4/pJREFQV/NgAgh9YyBIoCRRNq2jp5l8lbnE5aLggJnzIRmNVlogAg8N6hEEecLwXHtMQI
l2NX2HlDqhCU1:14871::::::
```

您也可以将密码字段留空。这意味着，该用户无需密码即可登录。

根据 group(5) 将用户的组和他们将加入的组添加到 `*archlive*/airootfs/etc/group`。例如：

```
archlive/airootfs/etc/group
root:x:0:root
adm:x:4:archie
wheel:x:10:archie
uucp:x:14:archie
archie:x:1000:
```

根据 gshadow(5) 创建相应的 `*archlive*/airootfs/etc/gshadow`：

```
archlive/airootfs/etc/gshadow
root:!*::root
archie:!*::
```

确保 `/etc/shadow` 和 `/etc/gshadow` 具有正确的权限：

```
archlive/profiledef.sh
...
file_permissions=(
  ...
  ["/etc/shadow"]="0:0:0400"
  ["/etc/gshadow"]="0:0:0400"
)
```

软件包安装后，*mkarchiso* 将为 `*archlive*/airootfs/etc/passwd` 中列出的用户创建所有指定的主目录并复制 `*work_directory*/x86_64/airootfs/etc/skel/*` 给他们。复制的文件将具有适当的用户和组归属。

键映射

要修改 Live 系统控制台上的默认键映射，可添加含有所需键映射的 `vconsole.conf`。细节以及更多选项，参见 Linux_控制台/键盘配置 和 vconsole.conf(5)。

例如，要将控制台和 X11 的默认键映射修改为德语（`de-latin1`），创建 `*archlive*/airootfs/etc/vconsole.conf`，使其带有下列内容：

```
archlive/airootfs/etc/vconsole.conf
KEYMAP=de-latin1
XKBLAYOUT=de
```

区域设置

要修改 Live 系统的 Locale，您须先启用并生成区域化文件。为此，须添加含有所需区域设置的自定义 `locale.gen`，并在 Live 系统中运行 `locale-gen`。由此，也可同理修改系统默认区域设置。

例如，要启用 `de_DE.UTF-8 UTF-8` 和 `en_US.UTF-8 UTF-8`，可创建 `*archlive*/airootfs/etc/locale.gen`，使其带有下列内容（细节参阅 Locale#生成区域设置）：

```
archlive/airootfs/etc/locale.gen
de_DE.UTF-8 UTF-8
en_US.UTF-8 UTF-8
```

随后，您需要在 Live 系统中运行 `locale-gen`。要达成这点，可使用 Pacman 挂钩，并随后借另一挂钩将其自动移除。创建 `*archlive*/airootfs/etc/pacman.d/hooks/locale-gen.hook`，使其带有下列内容（细节参见 Pacman#Hooks and alpm-hooks(5)）：

```
archlive/airootfs/etc/pacman.d/hooks/locale-gen.hook
# remove from airootfs!
[Trigger]
Operation = Install
Operation = Upgrade
Type = Package
Target = glibc

[Action]
Description = Generating localisation files...
When = PostTransaction
Depends = glibc
Exec = /usr/bin/locale-gen
```

之后是可选操作：要修改 Live 系统的默认区域设置，可添加自定义的 `locale.conf`。例如，要将默认区域设置改为 `de_DE.UTF-8`，创建

`*archlive*/airootfs/etc/locale.conf`，使其带有下列内容（细节参见 Locale#变量 和 locale.conf(5)）：

更改 ISO 中使用的分发名称

首先将文件 `/etc/os-release` 复制到 rootfs 的 `etc/` 文件夹中。然后，相应地编辑文件。您还可以更改 GRUB 和 syslinux 内部的名称。

构建 ISO 映像

通过运行以下命令构建一个 ISO 映像，然后您可以将其刻录到 CD 或 USB：

```
# mkarchiso -v -r -w /tmp/archiso-tmp -o /path/to/out_dir /path/to/profile/
```

- `-w` 指定工作目录。如果未指定该选项，则默认为当前目录下的 `work`。若内存允许，更好的选择是将工作目录安置在 tmpfs 下（如上所示）以加速此过程。
- `-r` 在 ISO 构建成功之后删除工作目录（若工作目录由 *mkarchiso* 创建）。
- `-o` 指定将放置构建的 ISO 映像的目录。如果未指定该选项，则默认为当前目录中的 `out`。
- 需要注意的是配置文件 `profiledef.sh` 在运行 mkarchiso 时不能指定，只能指定文件的路径。

将 `*/path/to/profile/*` 替换为您的自定义配置文件的路径，或者如果您正在构建一个 `/usr/share/archiso/configs/releng/`未修改的配置文件。

运行时，脚本将下载并安装您指定到 `*work_directory*/x86_64/airootfs` 的包，创建内核和初始化映像，应用您的自定义选项，最后将 ISO 构建到输出目录中。

提示：若遇到“无法下载*.db.sig文件”的错误，尝试添加“--skip-signature”参数。

删除工作目录

警告：如果 *mkarchiso* 被中断，运行 findmnt(8) 以确保在删除之前没有挂载绑定 - 否则，**你可能会丢失数据**（例如安装在 `/run/media/*user*/*label*` 的外部设备绑定在 `work/x86_64/airootfs/run/media/*user*/*label*` 在构建过程中）。

临时文件被复制到工作目录中。如果 *mkarchiso* 运行带有 `-r` 选项，成功构建 ISO 后，它会删除工作目录（先前未存在的话）。或者，也可手动删除：

```
# rm -rf /path/to/work_dir
```

使用 ISO

有关各种选项，请参见 Installation guide#准备安装介质。

在 QEMU 中测试 ISO

安装 可选依赖项 qemu-desktop包 和 edk2-ovmf包。

使用便捷脚本 `run_archiso` 使用 QEMU 运行构建的映像。

```
$ run_archiso -i /path/to/archlinux-yyyy.mm.dd-x86_64.iso
```

虚拟机也可以使用 UEFI 仿真运行：

```
$ run_archiso -u -i /path/to/archlinux-yyyy.mm.dd-x86_64.iso
```

提示：除了 VNC/SDL 屏幕之外，脚本还会将来自主机端口 60022 的 TCP 连接传达给虚拟机端口 22，即虚拟机 SSH 守护进程所监听的端口。可与#准备 ISO 以通过 SSH 进行安装搭配使用。

技巧提示

ISO 构建过程中运行设置命令

创建自定义配置文件时，对于部分改动，你需要在创建 ISO 之前，在 Live 系统环境中运行设置命令。例如，如果你要启用额外的区域设置，则须在 Live 系统中运行 `locale-gen`，以生成必要的区域化文件。

要在创建 ISO 之前运行设置命令，可在 `*archlive*/airootfs/etc/pacman.d/hooks/` 下，添加一个 Pacman 挂钩 。如何创建自定义 Pacman 挂钩的细节，参见 Pacman#钩子 与 alpm-hooks(5)。示例又见 #区域设置。

设置挂钩往往只需要在 ISO 构建过程中运行一次，故而在 **releng** 配置文件中包含一个 Pacman 挂钩，用于移除带有 "`remove from airootfs`" 标记的所有 Pacman 挂钩。因此，如果你不想让你的 Pacman 挂钩同样作用于正在运行的 Live 系统，可在挂钩后追加：

```
# remove from airootfs!
```

准备 ISO 以通过 SSH 进行安装

注意：自从 `archlinux-2021.02.01-x86_64.iso`，提供cloud-init 支持，`sshd.service`是默认启用的。

要 通过 SSH 安装 Arch Linux 而不与系统进行任何交互，必须将 SSH 公钥放在 `authorized_keys` 中。

添加 SSH 密钥可以手动完成（在此处解释），也可以通过 cloud-init。

要手动添加密钥，首先 复制 archiso 的 releng 配置文件 到可写目录。以下示例使用 `archlive`。

```
$ cp -r /usr/share/archiso/configs/profile/ archlive
```

在将用于登录的用户的主目录中创建一个 `.ssh` 目录。以下示例将使用 root 用户。

```
$ mkdir archlive/airootfs/root/.ssh
```

将用于登录的 SSH 公钥添加到 `authorized_keys`：

```
$ cat ~/.ssh/key1.pub >> archlive/airootfs/root/.ssh/authorized_keys
$ cat ~/.ssh/key2.pub >> archlive/airootfs/root/.ssh/authorized_keys
```

为 `.ssh` 目录和 `authorized_keys` 文件设置正确的 权限 和所有权：

```
archlive/profiledef.sh
...
file_permissions=(
  ...
  ["/root"]="0:0:0750"
  ["/root/.ssh"]="0:0:0700"
  ["/root/.ssh/authorized_keys"]="0:0:0600"
)
```

最终 构建 ISO 映像。启动 ISO 后，OpenSSH 将启动，并且可以使用相应的 SSH 私钥登录。

使用 iwd 自动连接到 Wi-Fi 网络

在配置文件的 `airootfs` 目录中创建 `/var/lib/iwd/` 并设置正确的权限：

```
$ mkdir -p archlive/airootfs/var/lib/iwd
archlive/profiledef.sh
...
file_permissions=(
  ...
  ["/var/lib/iwd"]="0:0:0700"
)
```

按照 iwd#Network configuration 和 iwd.network(5) 中的说明为您的 Wi-Fi 网络创建网络配置文件。

将配置文件保存在 `*archlive*/airootfs/var/lib/iwd/` 中。

调整根文件系统的大小

在 live 环境中，例如在需要使用 DKMS 模块的硬件上安装软件包时，根文件系统的默认大小可能不允许包的下载和安装。

提示：选中大小的理由请参见 BBS#210389，历史细节请参见 FS#45618。

当在 live 环境中下载文件或安装软件包时，它将表现为以下错误信息：

```
error: partition / too full: 63256 blocks needed, 61450 blocks free
error: not enough free disk space
error: failed to commit transaction (not enough free disk space)
Errors occurred: no packages were upgraded.
```

您亦可通过运行以下命令以动态调整根分区大小：

```
# mount -o remount,size=SIZE /run/archiso/cowspace
```

关于 `*SIZE*` 可能的参数，请参见 tmpfs(5) § size。

要在引导加载程序阶段调整大小（按 `e` 或 `Tab`），请使用以下引导选项：

```
cow_spacesize=SIZE
```

要在{ic|grub/*.cfg}}

- `syslinux/*.cfg`

您可以通过运行以下命令检查文件系统的大小：

```
$ df -h
```

查看 mkinitcpio-archiso 启动参数于此

Google Compute Engine 映像

Google Compute Engine 兼容的 `releng` 压缩映像以 archlinux-gceAUR 的形式提供。

Libvirt 虚拟机配置

运行 `releng` 映像的 libvirt 配置可用作 archlinux-libvirtAUR。

疑难解答

窗口管理器冻结

如果您想在 Live CD 中使用窗口管理器，则必须添加必要且正确的显卡驱动，否则 WM 可能会在加载时冻结。

参见

- Archiso 项目主页
- 官方档案
- Arch Linux 发布工程邮件列表
- #archlinux-releng——Arch Linux 发布工程 IRC 频道
- archiso-manager——用于构建官方每月 ISO 的工具

构建映像阶段调整大小，请将引导选项添加至：

- `efiboot/loader/entries/*.cfg`
- `grub/*.cfg`
- `syslinux/*.cfg`

您可以通过运行以下命令检查文件系统的大小：

```
$ df -h
```

查看 mkinitcpio-archiso 启动参数于此

Configuration management (简体中文)

系统装好之后，真正耗时的往往不是安装，而是让散落在各处的配置在多台机器上保持一致、随时可回退。本章只处理点文件：用 Git 跟踪点文件，把与特定设备相关的配置单独剥离，以及围绕它可用的工具与可以沿用的用户仓库。本章只有一节，没有可选读的内容，《Dotfiles》须整篇读完，它是之后自己动手整理配置的唯一依据。

最容易犯的错是把它当成一次性的搬家：点文件里既有能公开共享的部分，也有只对某台机器成立的设置，若不先分层就整体提交，换台设备轻则冲突，重则把私密内容一并推出去。

10.1　Dotfiles

难度 高级 | **篇幅** 20.1 KB | **所属** Configuration management (简体中文)

本节将讲到

- 使用 Git 跟踪点文件
- 特定设备相关的配置
- 工具
- 用户仓库
- 另见

用户对于程序的配置通常存储在所谓的点文件（以 . 开头的文件）中。通常的做法是用版本控制系统（如 Git ）来管理点文件，以跟踪其变化，并在不同的主机上同步。有许多管理点文件的方法。例如，直接跟踪主目录中的点文件；或者将其放在子文件夹中，用 shell 脚本或专用工具进行管理。除了说明如何管理你的点文件之外，本文章还包含了一个来自 Arch Linux 用户的点文件配置库列表。

使用 Git 跟踪点文件

直接使用 Git 跟踪点文件的好处是它只涉及Git而不涉及符号链接。坏处则是对于特定主机的配置通常需要多个分支来合并。

实现这一需求最简单的方式是直接在 home 目录初始化一个 Git 仓库，并在 gitignore(5)文件中使用*匹配所有文件来忽视它们。 但是两个弊端也随之而来：一是当你的 home 目录下有其他 Git 仓库时，就会难以应对（比如说你如果你忘记初始化而直接对点文件仓库进行操作的时候）。二是你很难确定当前目录下的那些文件是没有被跟踪的（因为他们已经被忽略了)。

为避免上述两个弊端，一个很受欢迎的替代方法是“裸仓库别名法”，它的原文链接在此：Ask Hacker News: What do you use to manage your dotfiles?，它只要3个命令就可轻松完成设置：

```
$ git init --bare ~/.dotfiles
$ alias dotfiles='/usr/bin/git --git-dir="$HOME/.dotfiles/" --work-tree="$HOME"'
$ dotfiles config status.showUntrackedFiles no
```

注意：通常情况下， 点文件都是默认权限，但如果某些配置一定要配置特定权限，请另寻他法。（git 不支持存储权限信息）(Discussion)

你可以采用下面的命令在新系统中复制点文件：

```
$ git clone --bare <git-repo-url> $HOME/.dotfiles
$ alias dotfiles='/usr/bin/git --git-dir="$HOME/.dotfiles/" --work-tree="$HOME"'
$ dotfiles checkout
$ dotfiles config --local status.showUntrackedFiles no
```

- 如果这时有一些点文件已经被覆盖了，你可能会遇到类似下面这样的错误：

: ``` $ dotfiles checkout error: The following untracked working tree files would be overwritten by checkout: .bashrc .gitignore Please move or remove them before you can switch branches. Aborting

: 你应该使用 `$ dotfiles checkout -f`重写已经存在的文件，或是用下面更安全的脚本方法备份所有文件，记得使用`checkout`：

```
mkdir -p .dotfiles-backup && \ dotfiles checkout 2>&1 | egrep "\s+\." | awk {'print $1'} | \ xargs -I{} mv {} .dotfiles-backup/{}
```

在这之后你可以用 alias命令 去管理点文件。如果你使用的是 Bash，只需要安装 [bash-complete-alias](https://aur.archlinux.org/packages/bash-complete-alias/)AUR 后就可以使用补全功能，最后将别名和下面的一行命令放在你的`~/.bashrc`中。

```
$ complete -F _complete_alias dotfiles
```

另一种在bash中使用补全功能的方法，在你的`~/.bashrc`中加入下面的命令（擅自）：

```
source /usr/share/bash-completion/completions/git __git_complete config __git_main
```

> **提示：**关于避免以外提交敏感信息的方式，请看 [Git#Filtering confidential information](../../07_开发与工具/版本控制系统/Git.md#Filtering_confidential_information).

特定设备相关的配置

你可能有很多台设备， 在这些设备之间同步点文件是一个很常见的问题，我们将其称之为特定设备相关的配置。

有了 [Git](../../07_开发与工具/版本控制系统/Git.md) 工具的帮助，我们可以通过以下方式解决这个问题：我们可以维护一个包含通用配置文件的主分支，同时为设备间不同的特定配置文件创建单独的分支。维护时将通用配置文件的变动提交到前者，而将与设备相关的配置文件提交到后者。也就是说，每个设备的专用分支都是在保持通用分支更新的基础上单独建立的。

某些配置文件中，比如在 [shell configuration files](../../07_开发与工具/命令行解释器/命令行解释器.md#Configuration_files) 中，可以使用条件逻辑判断不同的设备。比如，[Bash](../../07_开发与工具/命令行解释器/Bash.md) 脚本 （即`.bashrc`） 中可以根据不同的主机名（或者其他特征如类型、自定义变量等等）应用不同的配置。

```
if [[ "$(hostname)" == "archlaptop" ]]; then

else
```

fi

同样也可以用 .Xresources. 实现类似功能。

如果你觉得设置 Git 分支太麻烦，也可以使用支持文件分组的[工具](#工具)，或者使用处理工具使管理更加灵活。

工具

文件分组
：也就是将配置文件归纳成组的方法。
处理
：一些工具对配置文件进行处理，使其能够根据主机情况进行定制。

名称	包	语言	文件分组	处理
[dotbot](https://github.com/anishathalye/dotbot)	[dotbot](https://aur.archlinux.org/packages/dotbot/)AUR	Python	配置文件	无
[chezmoi](https://github.com/twpayne/chezmoi)	[chezmoi](https://archlinux.org/packages/?name=chezmoi)包	Go	基于目录	Go 模板
[dot-templater](https://github.com/kesslern/dot-templater)	[dot-templater-git](https://aur.archlinux.org/packages/dot-templater-git/)AUR	Rust	基于目录	特定语法
[toml-bombadil](https://github.com/oknozor/toml-bombadil)	[toml-bombadil](https://archlinux.org/packages/?name=toml-bombadil)	Rust	基于目录	tera
[dotdrop](https://deadc0de.re/dotdrop/)	[dotdrop](https://aur.archlinux.org/packages/dotdrop/)AUR	Python	配置文件	Jinja2
[dotfiles](https://github.com/jbernard/dotfiles)	[dotfiles](https://aur.archlinux.org/packages/dotfiles/)AUR	Python	[无](https://github.com/jbernard/dotfiles/pull/24)	无
[dotter](https://github.com/SuperCuber/dotter)	[dotter-rs](https://aur.archlinux.org/packages/dotter-rs/)AUR	Rust	配置文件	Handlebars
[dt-cli](https://dt.cli.rs)	[dt-cli](https://aur.archlinux.org/packages/dt-cli/)AUR	Rust	配置文件	Handlebars
[GNU Stow](https://www.gnu.org/software/stow/)	[stow](https://archlinux.org/packages/?name=stow)包	Perl	基于目录	无
[Mackup](https://github.com/lra/mackup)	[mackup](https://aur.archlinux.org/packages/mackup/)AUR	Python	自动按应用	无
[mir.qualia](https://github.com/darkfeline/mir.qualia)	[mir.qualia](https://aur.archlinux.org/packages/mir.qualia/)AUR	Python	无	自定义块
[rcm](https://github.com/thoughtbot/rcm)	[rcm](https://aur.archlinux.org/packages/rcm/)AUR	Shell	基于目录（按主机或标签）	无
[yas-bdsm](https://github.com/sebastiancarlos/yas-bdsm)	-	Shell	基于目录	无

Git 的替代品

如果你不喜欢 [Git](../../07_开发与工具/版本控制系统/Git.md) ，可以尝试下列工具，它们或多或少可以看作是版本控制系统。

名称	包	语言	文件分组	处理
[dotbare](https://github.com/kazhala/dotbare)	[dotbare](https://aur.archlinux.org/packages/dotbare/)AUR	Shell ([fzf](https://archlinux.org/packages/?name=fzf)包)	repository-wise	无
[dotgit](https://github.com/kobus-v-schoor/dotgit)	[dotgit](https://aur.archlinux.org/packages/dotgit/)AUR	Python	基于文件名称	无
[homeshick](https://github.com/andsens/homeshick)	[homeshick-git](https://aur.archlinux.org/packages/homeshick-git/)AUR	Bash	repository-wise	无
[homesick](https://github.com/technicalpickles/homesick)	-	Ruby	repository-wise	无
[Pearl](https://github.com/pearl-core/pearl)	[pearl-git](https://aur.archlinux.org/packages/pearl-git/)AUR	Python	repository-wise	无

[vcsh](https://github.com/RichiH/vcsh)	[vcsh](https://archlinux.org/packages/?name=vcsh)包	Shell	repository-wise	无
[yadm](https://yadm.io)(1)	[yadm](https://archlinux.org/packages/?name=yadm)包	Python	基于文件名称 (按类/操作系统/发行版/主机名称/用户)	内置模板/Jinja2/ESH (可选)
[dfm](https://github.com/justone/dotfiles)	[dfm](https://aur.archlinux.org/packages/dfm/)AUR	Perl	repository-wise	无

1. 支持以 [GPG](../../03_网络与服务/邮件/GnuPG.md) 或 OpenSSL 对文件加密。

用户仓库

> **提示：**中文社区有人整理了来自中文社区用户的 dotfiles 集锦：[archlinuxcn-dotfiles-collection](https://github.com/zjuyk/archlinuxcn-dotfiles-collection)。

> **注意：**这个表格用于提供点文件的参考和例子，如果你想将自己的点文件提交到此，请确保它们整洁、注释完善且不过时。

> **警告：**这些点文件未经Arch Linux维护团队验证，请自行承担使用风险。

作者	Shell	WM / DE	编辑器	终端	终端复用	音频	资源监视	邮件	IRC	文件管理	RSS 阅读
[adamperkowski](https://github.com/adamperkowski/dwm)	Zsh	dwm	Neovim	Kitty	tmux	mpv	自定义		[weechat](https://github.com/benmezger/dotfiles/blob/main/dot_weechat/)		
[alfunx](https://github.com/alfunx/.dotfiles)	zsh	awesome	vim	kitty	tmux	ncmpcpp/mpd	htop/lain	thunderbird			
[Ambrevar](https://gitlab.com/Ambrevar/dotfiles)	Eshell	EXWM	Emacs	Emacs (Eshell)	Emacs TRAMP + dtach	EMMS	conky/dzen	mu4e	Circe		
[ananthu](https://github.com/ask1234560/dotfiles_bspwm)	zsh	bspwm	neovim	alacritty		mpv	htop, polybar	neomutt	weechat	ranger	
[awal](https://github.com/awalGarg/dotfiles)	fish	i3	vim	st	tmux		i3status		The Lounge		
[ayekat](https://github.com/ayekat/localdir)	zsh	karuiwm	vim	rxvt-unicode	tmux	ncmpcpp/mpd	karuibar	mutt	irssi		
[bachoseven](https://github.com/BachoSeven/dotfiles)	zsh	[dwm](https://github.com/BachoSeven/dwm)	neovim	[st](https://github.com/BachoSeven/st)	ncmpcpp	bottom	neomutt	weechat	lf		
[bamos](https://github.com/bamos/dotfiles)	zsh	i3/xmonad	vim/emacs	rxvt-unicode	tmux	mpv/cmus	conky/xmobar	mutt	ERC		
[brisbin33](https://github.com/pbrisbin/dotfiles)	[zsh](https://github.com/pbrisbin/oh-my-zsh)	[xmonad](https://github.com/pbrisbin/xmonad-config)	[vim](https://github.com/pbrisbin/vim-config)	rxvt-unicode	screen		dzen	[mutt](https://github.com/pbrisbin/mutt-config)	[irssi](https://github.com/pbrisbin/irssi-config)		
[BVollmerhaus](https://gitlab.com/BVollmerhaus/dotfiles)	[fish](https://gitlab.com/BVollmerhaus/dotfiles/-/tree/master/config/fish)	[i3-gaps](https://gitlab.com/BVollmerhaus/dotfiles/blob/master/config/i3/config)	[kakoune](https://gitlab.com/BVollmerhaus/dotfiles/blob/master/config/kak/kakrc)	[kitty](https://gitlab.com/BVollmerhaus/dotfiles/-/blob/master/config/kitty/kitty.conf)				[polybar](https://gitlab.com/BVollmerhaus/dotfiles/blob/master/config/polybar/config)			[ranger](https://gitlab.com/BVollmerhaus/dotfiles/-/tree/master/config/ranger)	
[christian-heusel](https://github.com/christian-heusel/dotfiles)	[Zsh](https://github.com/pbrisbin/oh-my-zsh)	i3	Neovim	st / terminator	byobu / tmux		htop	neomutt/thunderbird	weechat	nemo / rangern	
[cinelli](https://github.com/cinelli/dotfiles)	zsh	dwm	vim	termite-git		pianobar	htop	mutt-kz	weechat		
[dikiaap](https://github.com/dikiaap/dotfiles)	zsh	i3-gaps	neovim	alacritty	tmux		i3blocks		nnn		
[Earnestly](https://github.com/Earnestly/dotfiles)	zsh	i3/orbment	vim/emacs	termite	tmux	mpd	conky	mutt	weechat		
[ErikBjare](https://github.com/ErikBjare/dotfiles)	zsh	xmonad/xfce4	vim	terminator	tmux		xfce4-panel		weechat		
[falconindy](https://github.com/falconindy/dotfiles)	bash	i3	vim	rxvt-unicode		ncmpcpp	conky	mutt			

[filiparag](https://github.com/filiparag/dotfiles)	fish	bspwm	vim	alacritty	tmux	mpv, [playerctl](https://github.com/altdesktop/playerctl)	htop, polybar	[mail-notification](https://www.nongnu.org/mailnotify/)		[pcmanfm](https://wiki.lxde.org/en/PCManFM)\[[失效链接](https://zh.wikipedia.org/wiki/Wikipedia:%E5%A4%B1%E6%95%88%E9%93%BE%E6%8E%A5) 2022-09-17 ⓘ\]	
[gardenapple](https://git.sr.ht/~gardenapple/dotfiles)	zsh	sway	neovim	kitty			htop	[aerc](https://aerc-mail.org/)			
[graysky](https://github.com/graysky2/configs/tree/master/dotfiles)	zsh	xfce4	vim	terminal		ncmpcpp	custom	thunderbird			
[hugdru](https://github.com/hugdru/dotfiles)	zsh	awesome	neovim	rxvt-unicode	tmux			thunderbird	weechat		
[insanum](https://github.com/insanum/dotfiles)	bash	herbstluftwm	vim	evilvte	tmux		dzen	mutt-kz			
[isti115](https://github.com/isti115/dotfiles)	[pwsh](https://github.com/Isti115/dotfiles/blob/master/.config/powershell/Microsoft.PowerShell_profile.ps1)	[sway](https://github.com/Isti115/dotfiles/blob/master/.config/sway/config)	[neovim](https://github.com/Isti115/dotfiles/tree/master/.config/nvim)	[alacritty](https://github.com/Isti115/dotfiles/blob/master/.config/alacritty/alacritty.yml)	tmux	[mpv](https://github.com/Isti115/dotfiles/tree/master/.config/mpv) / playerctl	[waybar](https://github.com/Isti115/dotfiles/tree/master/.config/waybar) / htop / ytop			[ranger](https://github.com/Isti115/dotfiles/tree/master/.config/ranger)		
[jasonwryan](https://hg.sr.ht/~jasonwryan/shiv)	bash/zsh	dwm	vim	rxvt-unicode	tmux	ncmpcpp	custom	mutt	irssi		
[jdevlieghere](https://github.com/JDevlieghere/dotfiles/)	zsh	xmonad	vim	terminal	tmux		htop	mutt	weechat		
[jelly](https://github.com/jelly/Dotfiles)	zsh	i3	vim	termite	tmux	ncmpcpp		mutt-kz-git	weechat		
[JonasDe](https://github.com/JonasDe/dotfiles)\[[失效链接](https://zh.wikipedia.org/wiki/Wikipedia:%E5%A4%B1%E6%95%88%E9%93%BE%E6%8E%A5) 2022-09-17 ⓘ\]	zsh	i3	vim	rxvt-unicode	tmux						
[Jorengarenar](https://github.com/Jorengarenar/dotfiles)	[sh](https://github.com/Jorengarenar/dotfiles/tree/master/shell)/[Bash](https://github.com/Jorengarenar/dotfiles/blob/master/bashrc)	[i3](https://github.com/Jorengarenar/dotfiles/tree/master/i3)	[Vim](https://github.com/Jorengarenar/dotfiles/tree/master/vim)	[xterm](https://github.com/Jorengarenar/dotfiles/blob/master/X11/Xresources)	[tmux](https://github.com/Jorengarenar/dotfiles/blob/master/tmux.conf)\[[失效链接](https://zh.wikipedia.org/wiki/Wikipedia:%E5%A4%B1%E6%95%88%E9%93%BE%E6%8E%A5) 2022-09-17 ⓘ\]	[mpv](https://github.com/Jorengarenar/dotfiles/tree/master/mpv)	[htop](https://github.com/Jorengarenar/dotfiles/blob/master/htoprc)	[aerc](https://github.com/Jorengarenar/dotfiles/tree/master/aerc)	[WeeChat](https://github.com/Jorengarenar/dotfiles/tree/master/weechat)	[ranger](https://github.com/Jorengarenar/dotfiles/blob/master/ranger.conf)	QuiteRSS
[MarkusZoppelt](https://github.com/markuszoppelt/dotfiles)	zsh	gnome	vim	terminal	tmux						
[maximbaz](https://github.com/maximbaz/dotfiles)	zsh	sway	kakoune	kitty		waybar	neomutt		nnn		
[mehalter](https://git.mehalter.com/mehalter/dotfiles)	zsh	i3-gaps	neovim	termite	tmux	cmus	gotop	neomutt	weechat	ranger	
[meskarune](https://github.com/meskarune/.dotfiles)	bash	herbstluftwm	vim	rxvt-unicode	screen		conky		weechat		
[neersighted](https://github.com/neersighted/dotfiles)	fish	i3	neovim	alacritty	tmux	ncmpcpp					
[nimaipatel](https://github.com/nimaipatel/dotfiles)	fish	awesome	neovim	alacritty		ncmpcpp					
[oibind](https://github.com/oibind/dotfiles)	fish	awesome	neovim	st	tmux		htop-vim		weechat	lf	
[OK100](https://github.com/ok100/configs)	bash	dwm	vim	rxvt-unicode		cmus	conky, dzen	mutt	weechat		
[orhun](https://github.com/orhun/dotfiles)	bash	i3-gaps	vsc	alacritty			i3status		weechat	tere	
[pablox-cl](https://github.com/pablox-cl/dotfiles)	zsh (zplug)	gnome3	neovim	kitty							
[peterzuger](https://gitlab.com/peterzuger/dotfiles)	zsh	i3-gaps	emacs	rxvt-unicode	screen	moc	htop				
[potamides](https://github.com/potamides/dotfiles)	bash	awesome	neovim	termite	tmux	ncmpcpp	conky,htop	mutt	weechat	ranger	
[reisub0](https://github.com/reisub0/dot)	fish	qtile	neovim	kitty		mpd	conky				

| [Scrumplex](https://git.sr.ht/~scrumplex/dotfiles) | fish | sway | neovim | kitty |
| ncmpcpp / pipewire | waybar | aerc | | ranger | |
| [sistematico](https://github.com/sistematico/majestic) | zsh/fish/bash | [i3-gaps]
(https://github.com/Airblader/i3) | vim/nano | termite | tmux | ncmpcpp | polybar |
mutt | weechat | | |
| [sitilge](https://git.sitilge.id.lv/sitilge/dotfiles) | zsh | sway | neovim |
alacritty | | | htop | thunderbird | | | |
| [thecashewtrader](https://git.sr.ht/~thecashewtrader/dotfiles) | Eshell | EXWM |
Emacs | Emacs (VTerm) | Emacs | Bongo | htop | mu4e | ERC | Dired | Elfeed |
| [thiagowfx](https://github.com/thiagowfx/.dotfiles) | bash/zsh | i3 | vim | alacritty
| tmux | playerctl | i3status | | | ranger | |
| [tuurep](https://github.com/tuurep/dotfiles) | bash | openbox | neovim | alacritty |
tmux | | polybar | | | | |
| [vodik](https://github.com/vodik/dotfiles) | zsh | xmonad | vim | termite-git | tmux
| ncmpcpp | custom | mutt | weechat | | |
| [w0ng](https://github.com/w0ng/dotfiles) | zsh | dwm | vim | rxvt-unicode | tmux |
ncmpcpp | custom | mutt | irssi | | |
[whitelynx](https://github.com/whitelynx/dotfiles)	fish	i3	neovim	kitty	
i3pystatus				
[whynothugo](https://git.sr.ht/~whynothugo/dotfiles)	zsh	sway	neovim
alacritty		mpv	waybar, top	neomutt		nemo	
[wryonik](https://github.com/wryonik/dotfiles)	zsh	i3-gaps-rounded	vim
terminator | | cmus | htop, i3blocks, gotop | | | ranger, nautilus | |

另见

- [gregswiki:DotFiles](https://mywiki.wooledge.org/DotFiles)
- [XMonad Config Archive](https://wiki.haskell.org/Xmonad/Config_archive)
- dotshare.it
- dotfiles.github.io
- terminal.sexy - Terminal color scheme designer